

Programación en Java

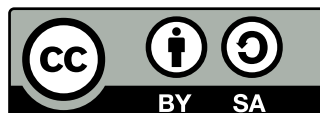
**Libro de texto para el módulo
“Programación (0485)”
de Formación Profesional**

José J. Durán

Para acceder a los contenidos de este libro, y otros títulos, visita:
<https://github.com/cavefish-dev/libros>

Edición: 7 de agosto de 2026

Esta obra está bajo una licencia Creative Commons
“Atribución-CompartirIgual 4.0 Internacional”.



Índice general

A Introducción

1	Presentación del libro	13
1.1	Estructura del libro	13
1.2	Convenciones de este libro	14
2	¿Qué es un programa?	17
2.1	¿Qué es el código fuente?	17
2.2	Tipos de lenguajes de programación	18
2.2.1	Lenguajes imperativos	18
2.2.2	Lenguajes declarativos	18
3	Cómo programar los ejemplos y ejercicios	21
3.1	Uso de JShell	22
3.2	Programar en JAVA sin un IDE	22
3.2.1	Compilar y ejecutar paquetes en Java	23
4	Resultados de Aprendizaje y contenidos del libro	25

B Programación en Java

5	Identificación de los elementos de un programa informático	29
5.1	Estructura y bloques fundamentales	30
5.1.1	Modificadores de acceso	31
5.1.2	Método main	31
5.2	Variables, tipos de datos y constantes	32
5.2.1	Literales	33
5.2.2	Comentarios	33

6	Utilización de tipos primitivos	39
6.1	Tipos de datos primitivos	39
6.1.1	Tipos numéricos	39
6.1.2	Tipos no numéricos	41
6.2	Operadores y expresiones	42
6.3	Conversiones de tipo	44
6.3.1	Conversión implícita	44
6.3.2	Conversión explícita	44
7	Funciones	51
7.1	Introducción	51
7.2	Definición y sintaxis	52
7.3	Parámetros y argumentos	53
7.3.1	Paso por valor	53
7.3.2	Sobrecarga de métodos	53
7.3.3	Parámetros de longitud variable (varargs)	53
7.4	Valor de retorno	54
7.4.1	Retorno temprano (<i>early return</i>)	54
7.5	Ámbito de las variables	55
7.6	Recursividad	55
7.6.1	Recursión vs. iteración	56
8	Utilización de objetos	61
8.1	Características de los objetos	61
8.2	Instanciación de objetos	62
8.3	Utilización de métodos y el uso de parámetros	62
8.4	Utilización de propiedades	63
8.5	Utilización de métodos estáticos	63
8.6	Constructores	64
8.7	Destrucción de objetos y liberación de memoria	66
8.8	La clase String	66
8.8.1	Inmutabilidad de los objetos String	66
8.8.2	Creación de cadenas	67
8.8.3	Concatenación y métodos útiles	67
8.8.4	Comparación de cadenas	68
9	Uso de estructuras de control	73
9.1	Estructuras de selección	73
9.1.1	Sentencias condicionales: if , else if , y else	73
9.1.2	Sentencias de selección: Switch	75
9.1.3	El operador ternario	77

9.2	Estructuras de repetición	78
9.2.1	Bucle <code>for</code>	78
9.2.2	Bucle <code>while</code> , y <code>do-while</code>	78
9.3	Estructuras de salto	79
9.3.1	Sentencia <code>return</code>	80
9.3.2	Sentencia <code>break</code> y <code>continue</code>	81
9.4	Control de excepciones	81
9.5	Recursividad	81
10	Desarrollo de clases	87
10.1	Concepto de clase	88
10.2	Estructura y miembros de una clase. Visibilidad	88
10.3	Creación de propiedades	89
10.4	Creación de métodos	91
10.5	Creación de constructores	92
10.6	Utilización de clases y objetos	94
10.7	Utilización de clases heredadas	95
10.7.1	La clase <code>Object</code>	98
11	Depuración y pruebas unitarias	109
11.1	Aserciones en Java	109
11.2	Pruebas unitarias con JUnit	110
11.3	Depuración en Java	112
11.4	Buenas prácticas de documentación	113
12	Aplicación de las estructuras de almacenamiento	119
12.1	Estructuras estáticas y dinámicas	120
12.1.1	Estructuras estáticas	120
12.1.2	Estructuras dinámicas	120
12.2	Creación de arreglos (<i>arrays</i>)	121
12.3	Matrices multidimensionales (<i>arrays</i> multidimensionales)	122
12.4	Genericidad	124
12.5	Cadenas de caracteres. Expresiones regulares	125
12.5.1	Manipulación de cadenas	125
12.5.2	Expresiones regulares	126
12.5.3	La clase <code>StringBuilder</code>	127
12.6	Colecciones	128
13	Excepciones	137
13.1	Introducción	137
13.2	Jerarquía de excepciones	137

13.3	Manejo de excepciones	138
13.3.1	<code>try-catch-finally</code>	138
13.3.2	Múltiples <code>catch</code>	139
13.3.3	Multi-catch	139
13.3.4	<code>try-with-resources</code>	139
13.4	Lanzar excepciones	140
13.4.1	<code>throw</code>	140
13.4.2	<code>throws</code>	140
13.5	Crear excepciones propias	140
13.6	Aserciones	141
14	Colecciones	147
14.1	Introducción	147
14.2	Jerarquía de colecciones	148
14.3	Listas	148
14.3.1	<code>ArrayList</code>	148
14.3.2	<code>LinkedList</code>	149
14.4	Conjuntos	149
14.4.1	<code>HashSet</code> y <code>TreeSet</code>	149
14.5	Pilas y colas	150
14.5.1	Cola FIFO con <code>ArrayDeque</code>	150
14.5.2	Pila LIFO con <code>ArrayDeque</code>	150
14.6	Mapas	150
14.7	Recorridos e iteradores	151
14.8	Programación funcional con colecciones	151
15	Lectura y escritura de información	157
15.1	Salida a pantalla y entrada desde teclado	158
15.1.1	Salida a pantalla	158
15.1.2	Entrada desde teclado	159
15.2	Flujos de datos	161
15.2.1	Lectura y escritura de bytes	161
15.2.2	Lectura y escritura de caracteres	164
15.2.3	Lectura y escritura de ficheros de acceso aleatorio	166
15.3	Ficheros de datos y registros	168
15.3.1	Ficheros tabulados (CSV)	168
15.3.2	Ficheros de marcas (XML)	169
15.4	Utilización de los sistemas de ficheros	174
15.4.1	Navegación por directorios	174
15.4.2	Creación y eliminación de ficheros y directorios	176

16 Utilización avanzada de clases	181
16.1 Sobrecarga de métodos	182
16.2 Composición de clases	182
16.3 Herencia y polimorfismo	184
16.3.1 Sobreescritura de métodos	186
16.4 Jerarquía de clases: Superclases y subclases	186
16.5 Constructores y herencia	188
16.6 Clases y métodos abstractos y finales	189
16.6.1 Clases <code>sealed</code>	190
16.7 Interfaces	191
16.7.1 La interfaz <code>Comparable</code>	192
16.7.2 Métodos por defecto en interfaces	194
16.8 Enumerados	195
16.8.1 Enumerados con métodos y atributos	195
16.8.2 Iteración sobre enumerados	196
16.8.3 Conversión de <code>String</code> a un valor de enumerado	197
16.8.4 Enumerados y <code>switch</code>	198
16.8.5 Enumerados como <code>Singleton</code>	198
16.9 El tipo <code>record</code>	199
17 Gestión de bases de datos	203
17.1 Acceso a bases de datos	203
17.2 Establecimiento de conexiones	205
17.2.1 Parámetros de conexión	206
17.2.2 Ejecución de consultas SQL	207
17.3 CRUD en bases de datos	207
17.3.1 Insertar datos	208
17.3.2 Consultar datos	208
17.3.3 Actualizar datos	208
17.3.4 Eliminar datos	209
17.4 Gestión de transacciones	210
17.4.1 Gestión de transacciones en PostgreSQL	210
18 Mantenimiento de la persistencia de los objetos	219
18.1 Bases de datos orientadas a objetos	220
18.2 Características de las bases de datos orientadas a objetos	220
18.3 Instalación del gestor de bases de datos	221
18.3.1 Instalación y configuración de <i>MySQL</i> e <i>Hibernate</i>	222
18.4 Creación de bases de datos	222
18.5 Tipos de datos objeto, atributos y métodos	223
18.6 Tipos de datos colección	224

18.7	Mecanismos de consulta	224
18.7.1	Consultas mediante el API de Hibernate	225
18.7.2	Lenguaje de consultas orientado a objetos (HQL)	225
18.7.3	Consultas nativas SQL	226
18.7.4	Criteria API	226
18.8	El lenguaje de consultas: sintaxis, expresiones, operadores	227
18.9	Recuperación, modificación y borrado de información	229
18.9.1	El concepto de repositorio	230
18.10	Ejemplo práctico: gestión de contactos	231
18.11	Resumen	237
19	Interfaces gráficas	245
19.1	Interfaces gráficas	245
19.1.1	Ejemplo básico de interfaz gráfica con Swing	246
19.2	Concepto de evento	247
19.3	Creación de controladores de eventos	247
19.3.1	Ejemplo práctico: Calculadora simple	248
 C Programación avanzada en Java		
20	¿Cómo escribir código limpio?	257
20.1	Principios de Clean Code	257
20.2	Principios SOLID	258
20.2.1	Single Responsibility Principle (SRP)	258
20.2.2	Open/Closed Principle (OCP)	259
20.2.3	Liskov Substitution Principle (LSP)	259
20.2.4	Interface Segregation Principle (ISP)	259
20.2.5	Dependency Inversion Principle (DIP)	259
20.3	<i>Code Calisthenics</i>	259
21	Principios de programación funcional en programación orientada a objetos	267
21.1	Transparencia referencial	267
21.2	Inmutabilidad	268
21.3	Métodos como elementos de primer orden	269
21.4	Expresiones lambda e interfaces funcionales	271
21.5	La clase <code>Optional</code>	271
21.6	La clase <code>Stream</code>	272

22 Patrones de diseño software	283
22.1 Patrones creacionales	283
22.1.1 El patrón <i>Abstract Factory</i>	284
22.2 Patrones estructurales	286
22.2.1 El patrón <i>Adapter</i>	287
22.3 Patrones de comportamiento	289
22.3.1 El patrón <i>Visitor</i>	290
23 Programación paralela	299
23.1 Introducción a la programación paralela	299
23.2 Las clases <code>Thread</code> , y <code>Runnable</code>	299
23.3 Concurrencia	301
23.3.1 Las clases <code>Mutex</code> , y <code>Semaphore</code>	302
23.4 Atomicidad y consistencia	304
23.4.1 La palabra clave <code>synchronized</code>	304
23.4.2 Las clases atómicas	305
23.4.3 Colecciones concurrentes	306
24 Clases de utilidad	313
24.1 La clase <code>Math</code>	313
24.2 La clase <code>Arrays</code>	314
24.3 La clase <code>Collections</code>	315
24.4 La clase <code>Objects</code>	316
24.5 La clase <code>Random</code> , <code>ThreadLocalRandom</code> y <code>SecureRandom</code>	317
24.6 Las clases <code>BigInteger</code> , y <code>BigDecimal</code>	318
24.7 Clases para fechas y horas: <code>LocalDate</code> , <code>LocalTime</code> , <code>LocalDateTime</code> , <code>Duration</code> , <code>Period</code>	320
24.8 La clase <code>UUID</code>	321
24.9 La clase <code>Base64</code>	321

D Ejercicios

25 Ejercicios de refuerzo	329
25.1 Variables y operadores	329
25.1.1 Operaciones aritméticas	329
25.1.2 Operadores de asignación compuesta	330
25.1.3 Ejercicios avanzados	331
25.2 Funciones	332
25.2.1 Funciones sin parámetros	332
25.2.2 Funciones con parámetros y valor de retorno	332

25.2.3	Recursividad	333
25.3	Estructuras de selección	333
25.3.1	Sentencia if-else	333
25.3.2	Sentencia switch-case	334
25.3.3	Operador ternario	335
25.4	Estructuras de repetición	336
25.4.1	Bucle for	336
25.4.2	Bucle while	337
25.4.3	Sentencias break y continue	338
25.5	Introducción a la POO	338
25.6	Estructuras de almacenamiento	342
25.7	Colecciones	344
25.8	Ficheros	346
25.9	POO avanzada	349
26	Ejercicios de Programación	351
26.1	Criba de Eratóstenes	351
26.2	Balanceo de paréntesis	351
26.3	Piratero de contraseñas MD5	352
26.4	Agenda personal	353
26.5	Juego de adivinanza de números	354
26.6	Juego de ahorcado	355
26.7	Calculadora IRPF	355



Introducción

Presentación del libro

Este libro está diseñado para introducir al lector en el mundo de la programación, proporcionando una base sólida y práctica. A lo largo de sus capítulos, se exploran conceptos fundamentales, ejemplos ilustrativos y ejercicios que fomentan el aprendizaje activo. El objetivo principal es acompañar al estudiante en su proceso de formación, facilitando la comprensión y el desarrollo de habilidades esenciales para programar en diversos lenguajes y entornos.

En este libro, se abordan temas como la sintaxis básica de los lenguajes de programación, estructuras de control, manejo de datos y conceptos avanzados como la programación orientada a objetos. Cada sección incluye explicaciones claras, ejemplos prácticos y ejercicios diseñados para reforzar el aprendizaje.

1.1. Estructura del libro

Este libro se divide en varias partes, cada una de las cuales aborda un aspecto diferente de la programación. Se recomienda seguir el orden de los capítulos para construir una comprensión progresiva de los conceptos.

La parte A (*Introducción*) proporciona una visión general de los contenidos de este libro, y como poder ejecutar los ejemplos y ejercicios propuestos.

La parte B (*Programación en Java*) se centra en el lenguaje Java, proporcionando una introducción a sus características y sintaxis. Se incluyen ejemplos prácticos y ejercicios que permiten al lector familiarizarse con el lenguaje y desarrollar habilidades de programación. Los puntos desarrollados en esta parte, buscan satisfacer los contenidos básicos del módulo *Programación* (código 0485) del ciclo formativo de grado superior *Desarrollo de Aplicaciones Multiplataforma* y *Desarrollo de Aplicaciones Web*.

La parte C (*Programación avanzada en Java*) se centra en conceptos más avanzados de programación, incluyendo patrones de diseño, programación funcional y técnicas de depuración. Esta sección está diseñada para aquellos que ya tienen una base en programación y desean profundizar en temas más complejos. Los contenidos de esta parte son extracurriculares y no forman parte del mó-

dulo *Programación* (0485), pero son altamente recomendables para el desarrollo profesional del alumnado.

La parte D (*Ejercicios*) contiene una serie de ejercicios prácticos que permiten al lector aplicar lo aprendido en los capítulos anteriores. Estos ejercicios están diseñados para reforzar la comprensión de los conceptos y fomentar la práctica activa.

1.2. Convenciones de este libro

A lo largo de este libro encontrarás la información organizada en capítulos y secciones, cada una dedicada a un tema específico. Dentro de cada capítulo, se presentarán ejemplos de código usando el siguiente formato:

```
1 public class Ejemplo {  
2     public static void main(String[] args) {  
3         System.out.println(";Hola, mundo!");  
4     }  
5 }
```

También se incluyen cajas de información destacada, como consejos, advertencias y ejercicios, para resaltar puntos importantes y facilitar la comprensión de los conceptos. Estas cajas están diseñadas para ser visualmente atractivas y fáciles de identificar.

Consejo



A lo largo del libro, encontrarás consejos útiles para mejorar tu comprensión y habilidades de programación. Estos consejos están diseñados para ayudarte a evitar errores comunes y optimizar tu proceso de aprendizaje.

Importante



Es fundamental prestar atención a las advertencias y notas importantes que se presentan en el libro. Estas secciones destacan aspectos críticos que pueden afectar tu comprensión o implementación de los conceptos, o que incluyan información adicional relevante para el tema tratado, pero que sean demasiado avanzados para el nivel del lector.

Ejercicio 1.1



Se incluirán ejercicios prácticos al final de cada capítulo para que puedas aplicar lo aprendido. Estos ejercicios están diseñados para ser desafiantes y fomentar la práctica activa. Los ejercicios se presentan en un formato claro

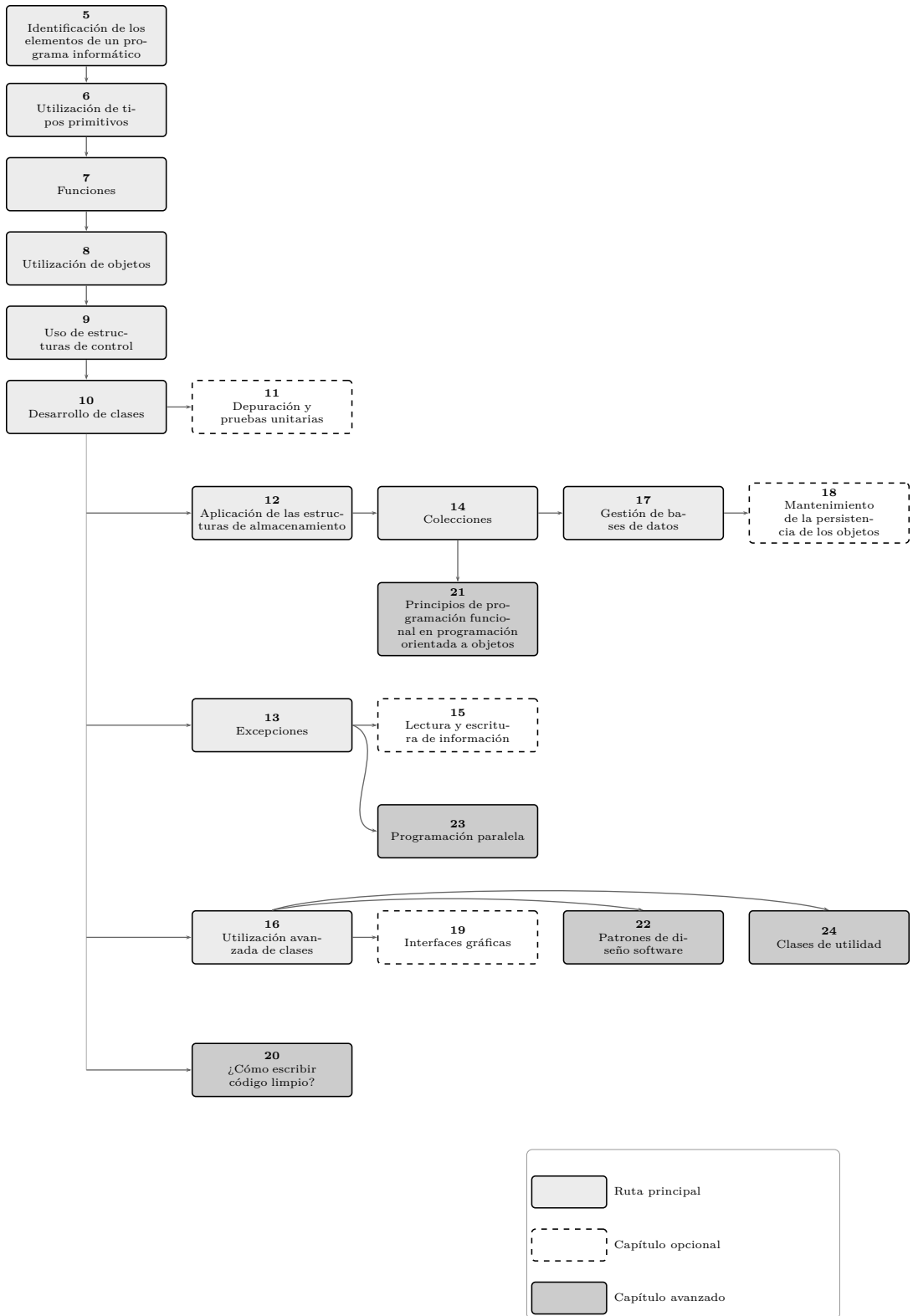


Figura 1.1: Orden de lectura recomendado.

y conciso, permitiendo al lector enfocarse en la resolución de problemas específicos.

¿Qué es un programa?

Un **programa** es un conjunto de instrucciones escritas en un lenguaje de programación que una computadora puede interpretar y ejecutar para realizar una tarea específica.

- Resuelve problemas o automatiza procesos.
- Puede ser tan simple como sumar dos números, o tan complejo como controlar un satélite.
- Los programas se escriben siguiendo reglas sintácticas y semánticas del lenguaje elegido.

Algunos ejemplos de programas incluyen: una calculadora que suma y multiplica números, un navegador web que permite visitar páginas de Internet, un videojuego que responde a las acciones del jugador, o un sistema de control de semáforos en una ciudad.

2.1. ¿Qué es el código fuente?

El **código fuente** es el conjunto de instrucciones y declaraciones escritas por un programador en un lenguaje de programación específico. Es la forma legible por humanos del programa antes de ser compilado o interpretado por una computadora.

- Contiene la lógica y las funcionalidades que el programa debe realizar.
- Generalmente está escrito en un lenguaje de alto nivel, como Python, Java o C++, aunque también puede escribirse en lenguajes de bajo nivel como ensamblador.
- Debe seguir reglas sintácticas y semánticas del lenguaje elegido.
- Puede incluir comentarios para explicar partes del código.

2.2. Tipos de lenguajes de programación

Existen diferentes paradigmas de programación, cada uno con sus propias características y aplicaciones:

2.2.1. Lenguajes imperativos

Los lenguajes imperativos indican paso a paso cómo realizar una tarea. Dentro de esta categoría encontramos:

- **Lenguaje ensamblador:** Lenguaje de bajo nivel que se comunica directamente con el hardware. Cada instrucción corresponde casi directamente a una instrucción de la CPU. Es específico para cada tipo de procesador (x86, ARM, etc.) y se utiliza principalmente en sistemas embebidos y optimización de código crítico.
- **Lenguajes estructurados:** Surgieron para facilitar la programación y reducir errores, promoviendo el uso de estructuras de control como secuencias, decisiones (`if`, `switch`) y bucles (`for`, `while`). Permiten dividir el código en funciones y procedimientos. Ejemplos: C, Pascal, Algol.
- **Lenguajes orientados a objetos:** Organizan el código en torno a objetos, que combinan datos y funciones relacionadas. Promueven conceptos como herencia, encapsulamiento y polimorfismo. Ejemplos: Java, Python, C++, C#, Ruby.

2.2.2. Lenguajes declarativos

Los lenguajes declarativos describen *qué* se quiere lograr, no *cómo* hacerlo:

- **Lenguajes funcionales:** Se basan en el concepto de funciones matemáticas y evitan modificar el estado o los datos (evitan efectos secundarios). Promueven la inmutabilidad y facilitan la concurrencia. Ejemplos: Haskell, Lisp, Erlang, OCaml.
- **Lenguajes lógicos:** Permiten expresar el conocimiento y las reglas de un problema en forma de hechos y relaciones lógicas. Son útiles en inteligencia artificial y sistemas expertos. Ejemplo principal: Prolog.

Consejo



Cada paradigma tiene ventajas y aplicaciones específicas. Elegir el adecuado depende del problema a resolver. Muchos lenguajes modernos, como

Java, combinan varios paradigmas. Nota: la clasificación anterior es una simplificación pedagógica; en la práctica los paradigmas se solapan y un mismo lenguaje puede pertenecer a varios.

Cómo programar los ejemplos y ejercicios

Para programar los ejemplos y ejercicios propuestos en este libro, se recomienda seguir los siguientes pasos:

1. **Configurar el entorno de desarrollo:** Asegúrate de tener instalado un entorno de desarrollo adecuado para el lenguaje de programación que estés utilizando. Para Java, se recomienda utilizar un IDE como IntelliJ IDEA o Eclipse.
2. **Leer atentamente el enunciado:** Antes de comenzar a programar, lee cuidadosamente el enunciado del ejercicio o ejemplo. Asegúrate de entender todos los requisitos y restricciones.
3. **Planificar la solución:** Antes de escribir código, es útil planificar la solución. Esto puede incluir la elaboración de diagramas de flujo, pseudocódigo o simplemente una lista de pasos a seguir.
4. **Implementar el código:** Comienza a escribir el código siguiendo el plan que has elaborado. No dudes en consultar la documentación oficial del lenguaje o las bibliotecas que estés utilizando.
5. **Probar y depurar:** Una vez que hayas implementado la solución, es fundamental probarla con diferentes casos de prueba. Si encuentras errores, utiliza herramientas de depuración para identificar y corregir los problemas.
6. **Refactorizar:** Después de que tu solución funcione correctamente, considera si hay oportunidades para mejorar el código. Esto puede incluir la eliminación de duplicaciones, la mejora de la legibilidad o la optimización del rendimiento.

Siguiendo estos pasos, podrás abordar de manera efectiva los ejemplos y ejercicios propuestos en este libro, desarrollando tus habilidades de programación y tu comprensión de los conceptos tratados.

3.1. Uso de JShell

JShell es una herramienta interactiva de Java que permite ejecutar fragmentos de código Java de manera rápida y sencilla. Es especialmente útil para probar pequeñas porciones de código, experimentar con nuevas ideas o aprender el lenguaje sin necesidad de crear un proyecto completo.

Para utilizar JShell, sigue estos pasos:

1. **Abrir JShell:** Si tienes Java instalado, puedes abrir JShell ejecutando el comando `jshell` en la terminal o consola de comandos.
2. **Escribir código:** Una vez abierto JShell, puedes escribir y ejecutar instrucciones Java directamente. Por ejemplo, prueba con `System.out.println("Hola, mundo");`.
3. **Explorar funcionalidades:** JShell permite definir variables, métodos y clases de forma interactiva. Puedes experimentar con fragmentos de código sin necesidad de compilar archivos completos.
4. **Salir de JShell:** Para salir, escribe `/exit` y presiona Enter.

JShell es ideal para aprender, probar ejemplos y depurar pequeñas partes de código Java de manera rápida.

3.2. Programar en JAVA sin un IDE

Si no dispones de un entorno de desarrollo integrado (IDE), puedes programar en Java utilizando únicamente la terminal y un editor de texto sencillo (como Notepad, VS Code, Vim, o Nano). Los pasos básicos son los siguientes:

1. **Escribir el código fuente:** Crea un archivo de texto con extensión `.java`. Por ejemplo, `HolaMundo.java`. Escribe el código Java en este archivo.
2. **Compilar el programa:** Abre la terminal y navega hasta la carpeta donde guardaste el archivo. Ejecuta el comando `javac HolaMundo.java` para compilar el código. Si no hay errores, se generará un archivo `HolaMundo.class`.

3. **Ejecutar el programa:** En la terminal, ejecuta el comando `java HolaMundo` para ejecutar el programa.

Este método es útil para comprender el proceso de compilación y ejecución de programas Java, y no requiere instalar software adicional más allá del JDK (Java Development Kit).

3.2.1. Compilar y ejecutar paquetes en Java

En Java, los paquetes permiten organizar el código en módulos y evitar conflictos de nombres. Para compilar y ejecutar programas que utilizan paquetes, sigue estos pasos:

1. **Estructura de directorios:** Crea una estructura de carpetas que refleje el nombre del paquete. Por ejemplo, si tu paquete se llama `com.ejemplo`, crea una carpeta `com/ejemplo/`.
2. **Escribir el código fuente:** Dentro de la carpeta correspondiente, crea el archivo Java y declara el paquete al inicio del archivo:

```
1 package com.ejemplo;  
2  
3 public class HolaPaquete {  
4     public static void main(String[] args) {  
5         System.out.println("Hola desde un paquete");  
6     }  
7 }
```

3. **Compilar recursivamente todos los archivos:** Desde la carpeta raíz del proyecto (donde está la carpeta `com`), ejecuta el comando:

```
1 javac com/**/*.java
```

Esto compilará todos los archivos Java dentro de la estructura de paquetes de forma recursiva y generará los archivos `.class` correspondientes.

4. **Ejecutar el programa:** Para ejecutar la clase, usa el nombre completo del paquete:

```
1 java com.ejemplo.HolaPaquete
```

Este procedimiento es fundamental para proyectos Java más grandes y facilita la organización y reutilización del código.

Resultados de Aprendizaje y contenidos del libro

El módulo de **Programación (0485)** del ciclo de Formación Profesional en DAM/DAW define 9 Resultados de Aprendizaje (RA) en el Real Decreto 450/2010. La siguiente tabla muestra qué capítulo de este libro desarrolla cada RA.

Consejo



Si eres docente, puedes usar esta tabla para planificar la temporalización del módulo asignando cada capítulo a una evaluación o unidad didáctica. Cada capítulo incluye ejercicios evaluables alineados con los criterios de evaluación del RA correspondiente.

RA	Descripción	Capítulo
RA1	Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.	Caps. 5–6
RA2	Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.	Caps. 7–8
RA3	Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.	Caps. 9 y 11
RA4	Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.	Caps. 10 y 16
RA5	Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases.	Cap. 15
RA6	Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos.	Caps. 12 y 14
RA7	Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.	Caps. 13 y 16
RA8	Utiliza bases de datos orientadas a objetos, identificando sus características y aplicando técnicas para almacenar y recuperar la información en ellas.	Cap. 18
RA9	Gestiona información almacenada en bases de datos relacionales manteniendo la integridad y consistencia de los datos.	Cap. 17

Cuadro 4.1: Mapeado de Resultados de Aprendizaje del módulo 0485 a los capítulos del libro.



Programación en Java

Identificación de los elementos de un programa informático

En este capítulo se presentan los elementos esenciales que componen un programa informático. Comprender estos conceptos es fundamental para desarrollar software de manera eficiente y estructurada. A continuación, se describen los principales componentes:

- **Estructura y bloques fundamentales:** Todo programa está organizado en bloques o secciones que definen su funcionamiento general, como la declaración de variables, funciones y el flujo principal de ejecución.
- **Variables:** Son espacios de memoria que permiten almacenar y manipular datos durante la ejecución del programa. Cada variable tiene un nombre identificador y puede cambiar su valor a lo largo del tiempo.
- **Tipos de datos:** Definen la naturaleza de los valores que pueden almacenar las variables, como números enteros, números reales, caracteres, cadenas de texto, entre otros.
- **Literales:** Son valores constantes escritos directamente en el código fuente, como 5, 3.14 u "Hola mundo".
- **Constantes:** Similares a las variables, pero su valor no puede modificarse una vez definido. Se utilizan para representar valores fijos a lo largo del programa.
- **Comentarios:** Son anotaciones incluidas en el código para explicar su funcionamiento o facilitar su comprensión. Los comentarios no afectan la ejecución del programa.

Estos conceptos constituyen la base sobre la que se construyen programas más complejos y permiten al programador expresar instrucciones de manera clara y precisa.

Para ilustrar estos conceptos, a lo largo de este capítulo, usaremos el siguiente fichero de ejemplo, el cual es un programa Java simple que imprime un mensaje en la consola:

```
1 package programacion.ejemplos;
2
3 public class HelloWorld {
4
5     private static final String DESPEDIDA = "Adiós mundo";
6
7     /**
8      * Método principal que se ejecuta al iniciar el programa.
9      * Imprime un mensaje de saludo y una despedida.
10     *
11     * @param args Argumentos de la línea de comandos (no se ↩
12     * utilizan en este ejemplo).
13     */
14     public static void main(String[] args) {
15         String texto = "Hola mundo";
16         System.out.println(texto);
17         // Modificación del texto
18         texto = "Hola mundo modificado";
19         System.out.println(texto);
20         System.out.println(DESPEDIDA);
21     }
22     /**
23      * Salida del programa:
24      * Hola mundo
25      * Hola mundo modificado
26      * Adiós mundo
27     */
28 }
```

Ejemplo 5.1: Programa Hola Mundo

5.1. Estructura y bloques fundamentales

Los programas en Java se organizan en distintos ficheros, los cuales pueden representar clases, interfaces, o enumerados. Cada fichero debe tener un nombre que coincida con el nombre de la clase pública que contiene, y la extensión del fichero debe ser `.java`. Por ejemplo, si tenemos una clase llamada `HelloWorld`, el fichero debe llamarse `HelloWorld.java`. Además, cada fichero debe contener una única clase pública, aunque puede contener otras clases no públicas.

Estos ficheros se organizan mediante el uso de paquetes, que son una forma de agrupar clases relacionadas. Un paquete se define al inicio del fichero con la palabra clave `package` seguida del nombre del paquete. En el ejemplo, nuestra

clase `HelloWorld` forma parte de un paquete llamado `programacion.ejemplos`, y estará dentro de la carpeta `ejemplos`, que a su vez estará dentro de la carpeta `programacion`.

En nuestro ejemplo, definimos la clase `HelloWorld` usando la palabra clave `class` seguida del nombre de la clase, y el cuerpo de la clase se define entre llaves. Dentro de la clase, podemos declarar variables, métodos y otros elementos que componen su funcionalidad. Delante de la declaración de la clase podemos incluir modificadores de acceso como `public` o `private` para controlar la visibilidad de la clase desde otros ficheros.

Las clases nos permiten organizar el código de manera modular y reutilizable. Para una definición más detallada de clases, se recomienda consultar el capítulo 10.

Importante

Recuerda que en Java, cada fichero está formado por bloques delimitados por llaves `{}`, y que cada bloque puede contener a su vez otros bloques, o instrucciones, las cuales deben delimitarse con punto y coma `;`. Esta estructura jerárquica es fundamental para entender cómo se organiza el código en Java.

5.1.1. Modificadores de acceso

Los modificadores de acceso en Java determinan la visibilidad de clases, métodos y variables. Los principales son:

- **public**: El elemento es accesible desde cualquier otra clase.
- **private**: El elemento solo es accesible dentro de la propia clase.
- **protected**: El elemento es accesible dentro del mismo paquete y por las subclases.
- **(sin modificador)**: También llamado *package-private*, el elemento es accesible solo dentro del mismo paquete.

5.1.2. Método main

El método `main` es el punto de entrada de un programa Java. Al ejecutar un programa, la máquina virtual de Java busca este método para iniciar la ejecución. Recibe como parámetro de entrada un array¹ de cadenas de texto (

¹Para más información sobre el uso de arrays, consulta el capítulo 12

`String[] args)`, que permite pasar argumentos al programa desde la línea de comandos.

El resultado de ejecutar este programa es la siguiente salida en la consola:

```
1| Hola mundo
2| Hola mundo modificado
3| Adiós mundo
```

5.2. Variables, tipos de datos y constantes

Las variables son espacios de memoria que permiten almacenar datos temporales durante la ejecución de un programa. Cada variable tiene un nombre identificador y un tipo de dato asociado, que define la naturaleza de los valores que puede almacenar. En Java, las variables pueden hacer referencia a tipos primitivos (capítulo 6), o a objetos (capítulo 8). la definición de una variable se realiza mediante la siguiente sintaxis:

```
1| tipo nombreVariable = valorInicial;
```

Donde **tipo** es el tipo de dato de la variable, **nombreVariable** es el identificador de la variable y **valorInicial** es el valor que se asigna a la variable al momento de su declaración. Es posible declarar una variable sin asignarle un valor inicial, pero en ese caso, no se podrá utilizar hasta que se le asigne un valor.

Es posible declarar varias variables del mismo tipo en una sola línea, separándolas por comas:

```
1| tipo nombreVariable1=valorInicial1, nombreVariable2=↔
   valorInicial2, ...;
```

Es importante mencionar que las variables en Java son sensibles a mayúsculas y minúsculas, por lo que `miVariable` y `mivariable` serían consideradas dos variables diferentes.

Para modificar el valor de una variable ya declarada, simplemente se asigna un nuevo valor utilizando la misma sintaxis de asignación:

```
1| nombreVariable = nuevoValor;
```

Por último, las variables pueden ser declaradas como **final**, lo que indica que su valor no puede ser modificado una vez asignado. Esto es útil para definir constantes, que son valores fijos a lo largo del programa. La sintaxis para declarar una constante es similar a la de una variable, pero se utiliza la palabra clave **final** antes del tipo de dato.

5.2.1. Literales

Los literales son valores constantes que se escriben directamente en el código fuente. En Java, existen varios tipos de literales, entre ellos:

- **Literales enteros:** Representan números enteros, como `5`, `-10` o `0`.
- **Literales de punto flotante:** Representan números reales, como `3.14`, `-2.5` o `0.0`.
- **Literales de caracteres:** Representan un único carácter, encerrado entre comillas simples, como `'a'`, `'1'` o `'\n'`.
- **Literales de cadena de texto:** Representan secuencias de caracteres, encerradas entre comillas dobles, como `"Hola mundo"` o `"123"`.
- **Literales booleanos:** Los valores `true` o `false`.
- **Literales nulos:** Representan la ausencia de un valor, utilizando la palabra clave `null`.

5.2.2. Comentarios

Los comentarios son anotaciones en el código que no afectan su ejecución, pero sirven para explicar su funcionamiento o facilitar su comprensión. En Java, existen dos tipos de comentarios:

- **Comentarios de una línea:** Se inician con dos barras inclinadas (`//`) y se extienden hasta el final de la línea.
- **Comentarios de varias líneas:** Se encierran entre los símbolos `/** */`, permitiendo incluir texto en varias líneas.
- **Comentarios de documentación:** Se utilizan para generar documentación del código y se escriben con `/** */`. Estos comentarios pueden incluir etiquetas especiales como `@param` o `@return` para documentar los parámetros y el valor de retorno de un método.

Los comentarios son una herramienta esencial para mejorar la legibilidad del código y facilitar su mantenimiento a largo plazo.

Importante

En este ejemplo, llamamos al método `System.out.println` para imprimir mensajes en la consola. Este método es parte de la clase `System`,

que proporciona acceso a funcionalidades del sistema, como la entrada y salida estándar. En este caso, `out` es un objeto de tipo `PrintStream` que permite enviar texto a la consola.

Más adelante, en el capítulo 10, se explicará en detalle cómo funcionan las clases y objetos en Java, incluyendo la clase `System` y sus métodos. Por ahora, considéralo una herramienta que te ayudará a escribir tus programas y a ver los resultados de tu código en la consola.

Resumen

En este capítulo se han presentado los elementos fundamentales que conforman un programa informático, utilizando Java como lenguaje de referencia. Se ha explicado la estructura básica de un programa, el uso de clases y paquetes, así como la importancia del método `main` como punto de entrada. Además, se han detallado los conceptos de variables, tipos de datos, literales y constantes, junto con la sintaxis para su declaración y uso. Finalmente, se ha destacado el papel de los comentarios para mejorar la legibilidad y el mantenimiento del código. Estos conocimientos son esenciales para comprender y desarrollar programas más complejos en capítulos posteriores.

Ejercicios de refuerzo

Ejercicio 5.1



Dado el siguiente fragmento de código, identifica todos sus elementos e indica el tipo y valor de cada variable:

```
1 package programacion.ejemplos;
2
3 public class Identificacion {
4     // Punto de entrada del programa
5     public static void main(String[] args) {
6         final int MAX_INTENTOS = 3;
7         int contador = 0;
8         double precio = 19.99;
9         char inicial = 'P';
10        String mensaje = "Bienvenido";
11        boolean activo = true;
12
13        contador = contador + 1;
14        System.out.println(mensaje);
```

```

15  /* Fin del programa */
16  }
17  }

```

Para cada variable y constante, indica: nombre, tipo, valor inicial y si puede modificarse. *Ver solución en la página 36.*

Ejercicio 5.2



El siguiente código contiene varios errores relacionados con la nomenclatura, el uso de **final** y los comentarios. Identifica y corrige cada error:

```

1  public class errores {
2      public static void main(String[] args) {
3          final int numero-maximo = 100;
4          int 2contador = 0;
5          String NombreUsuario = "Ana";
6          numero-maximo = 200; // intentamos modificar la
           constante
7          / Esto es un comentario de una línea
8          /* Este comentario no está cerrado
9      }
10 }

```

Ver solución en la página 36.

Ejercicio 5.3



Crea un programa que tenga en cuenta los siguientes puntos:

- Declara una constante con tu nombre.
- Declara una variable con un saludo.
- Imprime en la consola el valor de ambas variables utilizando el método `System.out.println`.
- Modifica el valor de la variable, con un texto de despedida, y vuelve a imprimirla.
- Utiliza comentarios para explicar cada parte del código.

Ver solución en la página 37.

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 5.1



Este ejercicio trabaja la identificación de los elementos básicos de un programa Java (secciones 1.2–1.4):

- **Constante:** `MAX_INTENTOS` — tipo `int`, valor `3`, declarada con `final` (no modificable).
- **Variables:** `contador` (`int`, `0`), `precio` (`double`, `19.99`), `inicial` (`char`, `'P'`), `mensaje` (`String`, `"Bienvenido"`), `activo` (`boolean`, `true`). Todas modificables.
- **Literales:** `3`, `0`, `19.99`, `'P'`, `"Bienvenido"`, `true`, `1`.
- **Comentarios:** una línea (`// Punto de entrada`) y de bloque (`/* Fin del programa */`).

Solución del ejercicio 5.2



Los errores presentes en el código son:

- `errores` debe empezar en mayúscula: `Errores` (convención de nombre de clase).
- `numero-maximo` usa un guión, no permitido; debe ser `numeroMaximo` (camelCase).
- `2contador` empieza por dígito; debe ser, por ejemplo, `contador`.
- `NombreUsuario` debería ser `nombreUsuario` (las variables usan camelCase en minúscula).
- `numero-maximo = 200` intenta modificar una constante `final`, lo que provoca un error de compilación.
- `/ Esto es un comentario` usa solo una barra; debe ser `// Esto es un comentario`.
- El comentario de bloque `/* Este comentario...` no está cerrado con `*/`.

Solución del ejercicio 5.3



```
1 // Programa completo: compilar y ejecutar ↩
  independientemente
2 public class SaludoBienvenida {
3     public static void main(String[] args) {
4         // Constante con el nombre (no puede modificarse)
5         final String NOMBRE = "Lucía";
6
7         // Variable con el saludo inicial
8         String mensaje = "¡Hola, " + NOMBRE + "! Bienvenida al ↩
          programa.";
9         System.out.println(mensaje);
10
11        // Modificamos la variable con un texto de despedida
12        mensaje = "¡Hasta pronto, " + NOMBRE + "!";
13        System.out.println(mensaje);
14    }
15 }
```


Utilización de tipos primitivos

En este capítulo, exploraremos los tipos primitivos en Java, que son los bloques de construcción fundamentales para almacenar datos. Estos tipos son esenciales para la manipulación de información y la realización de cálculos en un programa. En particular, veremos los siguientes aspectos:

- **Tipos de datos primitivos:** Analizaremos los diferentes tipos de datos primitivos que existen en Java, como `int`, `double`, `char`, `boolean`, entre otros. Explicaremos sus características, rangos de valores y cuándo es apropiado utilizar cada uno.
- **Operadores y expresiones:** Estudiaremos los operadores básicos (aritméticos, relacionales, lógicos) y cómo se utilizan para construir expresiones que manipulan los valores de los tipos primitivos. Veremos ejemplos prácticos de su uso en código.
- **Conversiones de tipo:** Aprenderemos cómo convertir valores entre diferentes tipos primitivos, tanto de manera implícita (conversión automática) como explícita (casting). Discutiremos las posibles pérdidas de información y buenas prácticas al realizar conversiones.

6.1. Tipos de datos primitivos

En Java, podemos dividir los tipos primitivos en dos categorías principales: tipos numéricos y tipos no numéricos. Los tipos numéricos se utilizan para representar valores numéricos, mientras que los tipos no numéricos se utilizan para representar valores lógicos o caracteres.

6.1.1. Tipos numéricos

Los tipos numéricos se dividen en enteros y de punto flotante. Los tipos enteros son aquellos que representan números enteros, mientras que los de punto flotante representan números con decimales.

Tipos enteros

Los tipos enteros en Java son:

- **byte**: Ocupa 1 byte (8 bits) y puede almacenar valores entre -128 y 127.
- **short**: Ocupa 2 bytes (16 bits) y puede almacenar valores entre -32,768 y 32,767.
- **int**: Ocupa 4 bytes (32 bits) y puede almacenar valores entre -2,147,483,648 y 2,147,483,647.
- **long**: Ocupa 8 bytes (64 bits) y puede almacenar valores entre 2^{63} y $2^{63}-1$. Para definir un valor de tipo **long**, se debe añadir una **L** al final del número.

```
1 byte numeroByte = 100; // Tipo byte
2 short numeroShort = 30000; // Tipo short
3 int numeroEntero = 42; // Tipo int
4 long numeroLargo = 1234567890123L; // Tipo long
```

Consejo



Es posible separar los dígitos de un número entero con guiones bajos `_` para mejorar la legibilidad. Por ejemplo, `1_000_000` es equivalente a `1000000`.

Si necesitamos realizar operaciones con números enteros que superen el rango de los tipos primitivos, podemos utilizar la clase `BigInteger` de la biblioteca estándar de Java, que permite trabajar con enteros de tamaño arbitrario.

Debido a que estos números son representados en binario, es posible asignarles valores usando distintas bases numéricas, como la base decimal, octal, hexadecimal o binaria. Para ello, se utilizan los siguientes prefijos:

- **Decimal**: No requiere prefijo, por ejemplo, `42`.
- **Octal**: Se indica con un `0` al inicio, por ejemplo, `052` (equivalente a `42` en decimal).
- **Hexadecimal**: Se indica con un `0x` o `0X` al inicio, por ejemplo, `0x2A` (equivalente a `42` en decimal).
- **Binario**: Se indica con un `0b` o `0B` al inicio, por ejemplo, `0b101010` (equivalente a `42` en decimal).

Tipos de punto flotante

Los tipos de punto flotante en Java son:

- **float**: Ocupa 4 bytes (32 bits) y puede almacenar números con decimales. Para definir un valor de tipo **float**, se debe añadir una **F** al final del número.
- **double**: Ocupa 8 bytes (64 bits) y puede almacenar números con mayor precisión que **float**. Es el tipo de punto flotante más utilizado por defecto.

```
1 float numeroFloat = 3.14F; // Tipo float
2 double numeroDouble = 2.718281828459045; // Tipo double
```

Importante

Al representar números flotantes, Java utiliza la notación de coma flotante, lo que puede llevar a errores de precisión en algunos casos. Por ejemplo, el número 0,1 no se puede representar sin introducir errores de redondeo. Esto se debe a la forma en que los números de punto flotante se almacenan en binario, utilizando el estándar IEEE 754, el cual representa los números como fracciones con divisor un número potencia de 2. Para evitar estos problemas, es recomendable utilizar **BigDecimal** para cálculos financieros o cuando se requiera alta precisión.

6.1.2. Tipos no numéricos

Los tipos no numéricos en Java son **char** y **boolean**.

Tipo carácter

El tipo **char** se utiliza para representar un único carácter Unicode. Ocupa 2 bytes (16 bits) y puede almacenar cualquier carácter, incluyendo letras, números y símbolos. Los caracteres se definen entre comillas simples.

```
1 char letra = 'A'; // Tipo char
2 char simbolo = '#'; // Tipo char
3 char digito = '5'; // Tipo char
4 char espacio = ' '; // Tipo char (espacio en blanco)
5 char saltoLinea = '\n'; // Tipo char (salto de línea)
```

Tipo booleano

El tipo `boolean` se utiliza para representar valores lógicos, es decir, verdadero (`true`) o falso (`false`). Este tipo es fundamental para el control de flujo en los programas, ya que se utiliza en estructuras de control como condicionales y bucles.

```
1 boolean esVerdadero = true; // Tipo boolean
2 boolean esFalso = false; // Tipo boolean
```

6.2. Operadores y expresiones

Los operadores son símbolos que permiten realizar operaciones sobre los valores de los tipos primitivos. En Java, existen varios tipos de operadores, entre ellos:

- **Operadores aritméticos:** Permiten realizar operaciones matemáticas básicas como suma (+), resta (-), multiplicación (*), división (/) y módulo (%).
- **Operadores de asignación:** Permiten asignar valores a variables. El operador de asignación más común es el igual (=), pero también existen operadores compuestos como +=, -=, *=, /= y %= que combinan una operación aritmética con la asignación.
- **Operadores de incremento y decremento:** Permiten aumentar o disminuir el valor de una variable en 1. Se utilizan los operadores ++ (incremento) y -- (decremento). Estos operadores pueden ser prefijos (++x o --x) o sufijos (x++ o x--), lo que afecta el orden de evaluación en expresiones más complejas.
- **Operadores de comparación:** Permiten comparar dos valores y devuelven un valor booleano. Los operadores de comparación incluyen igual a (==), diferente de (!=), mayor que (>), menor que (<), mayor o igual que (>=) y menor o igual que (<=).
- **Operadores lógicos:** Permiten combinar expresiones booleanas. Los operadores lógicos incluyen AND lógico (&&), OR lógico (||), XOR lógico (^) y NOT lógico (!).
- **Operadores de bits:** Permiten realizar operaciones a nivel de bits sobre valores enteros. Los operadores de bits incluyen AND bit a bit (&), OR bit a bit (|), XOR bit a bit (^), desplazamiento a la izquierda (<<),

desplazamiento a la derecha (>>) y desplazamiento a la derecha sin signo (>>>).

Los operadores lógicos solo pueden aplicarse a valores de tipo **boolean**, mientras que el resto de operadores solo pueden aplicarse a los otros tipos primitivos. El resultado de cada operación será del mismo tipo que los operandos, excepto en el caso de los operadores de comparación, que siempre devuelven un valor booleano.

Todos estos operadores se pueden combinar para formar expresiones más complejas. Es importante tener en cuenta el orden de precedencia de los operadores, que determina el orden en que se evalúan las expresiones. Por ejemplo, las multiplicaciones y divisiones tienen mayor precedencia que las sumas y restas, por lo que se evalúan primero. Además, se pueden utilizar paréntesis para forzar un orden específico de evaluación en las expresiones.

```

1 int a = 10;
2 int b = 5;
3 int suma = a + b; // Suma
4 int resta = a - b; // Resta
5 int multiplicacion = a * b; // Multiplicación
6 int division = a / b; // División
7 int modulo = a % b; // Módulo
8 int incremento = a++; // Incremento (postfijo). Devuelve el ↩
    valor de a antes de incrementar, y luego incrementa a en 1.
9 int decremento = --b; // Decremento (prefijo). Decrementa el ↩
    valor de b en 1 y luego devuelve el nuevo valor.
10 boolean esIgual = (a == b); // Comparación de igualdad
11 boolean esMayor = (a > b); // Comparación de mayor que
12 boolean resultadoLogico = (a > 0 && b < 10); // Operador lógico ↩
    AND
13 int desplazamientoIzquierda = a << 5; // Desplazamiento a la ↩
    izquierda. El valor de a se multiplica por 2, 5 veces.
14 int desplazamientoDerecha = a >> 3; // Desplazamiento a la ↩
    derecha. El valor de a se divide por 2, 3 veces.
15 int andBit = a & b; // AND bit a bit

```

Ejercicio 6.1



Crea un programa que calcule el área de un rectángulo y el perímetro de un cuadrado. Utiliza variables de tipo **int** para almacenar las dimensiones y muestra los resultados en la consola. Ver solución en la página 48.

6.3. Conversiones de tipo

En Java, es común que necesitemos convertir valores entre diferentes tipos primitivos. Existen dos tipos de conversiones: implícitas y explícitas.

6.3.1. Conversión implícita

La conversión implícita, también conocida como *promoción*, ocurre automáticamente cuando asignamos un valor de un tipo más pequeño a una variable de un tipo más grande. Por ejemplo, al asignar un valor de tipo `int` a una variable de tipo `long`, Java realiza la conversión automáticamente sin necesidad de especificarlo explícitamente.

```

1 int numeroEntero = 42;
2 long numeroLargo = numeroEntero; // Conversión implícita de int ↪
   a long
3 float numeroFloat = 3.14;
4 double numeroDecimal = numeroFloat; // Conversión implícita de ↪
   float a double
5 float numeroFloat = (float) numeroDecimal; // Conversión ↪
   implícita de double a float
6 float numeroFloat2 = numeroEntero; // Conversión implícita de ↪
   int a float
7 char letra = 'A';
8 int codigoAscii = letra; // Conversión implícita de char a int (↪
   código ASCII)

```

Importante

Al usar dos operandos de distintos tipos en una operación, Java promoverá automáticamente el tipo más pequeño al tipo más grande para evitar pérdida de información. Por ejemplo, si sumamos un `int` y un `double`, el `int` se convertirá a `double` antes de realizar la suma. Igualmente, si realizamos una operación entre un `float` y un `int`, el `int` se convertirá a `float`.

Podemos hacer uso de esta propiedad para realizar divisiones entre enteros y obtener un resultado de tipo `double` o `float`. Por ejemplo: `10 / (float)3` dará como resultado `3.3333333` en lugar de `3`, ya que el `int` se convierte a `float` antes de realizar la división.

6.3.2. Conversión explícita

La conversión explícita, también conocida como *casting*, se utiliza cuando queremos convertir un valor de un tipo más grande a un tipo más pequeño. En este

caso, debemos indicar explícitamente que queremos realizar la conversión utilizando paréntesis. Es importante tener en cuenta que al realizar una conversión explícita, puede haber pérdida de información si el valor original no cabe en el nuevo tipo.

```

1 long numeroLargo = 1234567890123L;
2 int numeroEntero = (int) numeroLargo; // Conversión explícita de
    long a int (puede perder información)
3 double numeroDecimal = 3.14;
4 float numeroFloat = (float) numeroDecimal; // Conversión
    explícita de double a float (puede perder precisión)
5 int numeroEntero2 = numeroDecimal; // Conversión explícita de
    double a int (pierde la parte decimal)
6 int codigoAscii = 20;
7 char letra = codigoAscii; // Conversión explícita de int a char

```

Consejo



Para realizar conversiones entre números de punto flotante y enteros, es recomendable utilizar la clase `Math`, en particular los métodos `Math.round()`, `Math.floor()` y `Math.ceil()` para redondear, truncar o redondear hacia arriba respectivamente.

Resumen

En este capítulo hemos aprendido los conceptos fundamentales sobre los tipos primitivos en Java. Los puntos clave son:

- Java ofrece varios tipos primitivos para representar datos simples: enteros (`byte`, `short`, `int`, `long`), punto flotante (`float`, `double`), caracteres (`char`) y valores lógicos (`boolean`).
- Cada tipo primitivo tiene un tamaño y un rango de valores específico, lo que influye en el uso de memoria y la precisión de los cálculos.
- Los operadores permiten manipular los valores de los tipos primitivos mediante operaciones aritméticas, lógicas, de comparación, de asignación y a nivel de bits.
- Las conversiones de tipo pueden ser implícitas (cuando Java realiza la conversión automáticamente) o explícitas (cuando el programador indica la conversión mediante *casting*), y es importante tener en cuenta la posible pérdida de información.

- Para cálculos que requieren alta precisión o valores fuera del rango de los tipos primitivos, existen clases como `BigInteger` y `BigDecimal`.

Comprender y utilizar correctamente los tipos primitivos es esencial para escribir programas eficientes y seguros en Java.

Ejercicios de refuerzo

Ejercicio 6.2



Para cada uno de los siguientes cinco escenarios, indica qué tipo primitivo de Java es el más adecuado y justifica tu elección:

1. La edad de una persona (entre 0 y 150).
2. El precio de un producto en euros (con decimales).
3. Un indicador de si un usuario está activo o no.
4. La inicial del primer nombre de un usuario.
5. El código postal de una dirección española (cinco dígitos).

Ver solución en la página 48.

Ejercicio 6.3



Dado el siguiente fragmento de código, traza el resultado de cada expresión indicando el tipo y valor resultante:

```
1 int a = 5;  
2 double b = 2.0;  
3  
4 int r1 = a / 2;  
5 double r2 = a / b;  
6 double r3 = (double) a / 2;  
7 double r4 = a / 2 + b;
```

Indica qué conversiones de tipo (implícitas o explícitas) se producen en cada caso y por qué. *Ver solución en la página 48.*

Ejercicio 6.4



Realiza las siguientes conversiones de tipo y muestra el resultado en la consola:

1. Convierte un valor de tipo `int` a `long` y muestra el resultado.
2. Convierte un valor de tipo `double` a `float` y muestra el resultado.
3. Convierte un valor de tipo `long` a `int` y muestra el resultado (ten en cuenta la posible pérdida de información).
4. Convierte un carácter a su código ASCII (tipo `int`) y muestra el resultado.

Ver solución en la página 49.

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 6.1



```

1 // Fragmento: incluir dentro de una clase
2 int base = 8;
3 int altura = 5;
4 int areaRectangulo = base * altura;
5 System.out.println("Área del rectángulo: " + areaRectangulo↵
6 );
7
8 int lado = 6;
9 int perimetroCuadrado = 4 * lado;
10 System.out.println("Perímetro del cuadrado: " + ↵
11 perimetroCuadrado);

```

Solución del ejercicio 6.2



La elección del tipo primitivo debe tener en cuenta el rango de valores, la precisión necesaria y el uso eficiente de memoria:

1. **byte** (rango 0–127 suficiente) o **int** (más habitual en la práctica).
2. **double** para mayor precisión; **float** si la memoria es crítica, aunque se prefiere **BigDecimal** en contextos financieros.
3. **boolean** — solo puede ser **true** o **false**.
4. **char** — almacena un único carácter Unicode.
5. **int** — los cinco dígitos caben ampliamente en su rango; **String** si se quiere preservar el cero inicial.

Solución del ejercicio 6.3



- $r1 = a / 2 \rightarrow$ división entera, resultado **int**: 2 (se trunca la parte decimal).
- $r2 = a / b \rightarrow$ a se promueve implícitamente a **double** (widening), resultado **double**: 2.5.
- $r3 = (\text{double})a / 2 \rightarrow$ casting explícito de a a **double**; luego 2 se promueve implícitamente, resultado **double**: 2.5.
- $r4 = a / 2 + b \rightarrow$ primero $a/2$ es división entera (2), luego 2 se

promueve a **double** para sumarlo con **b**, resultado **double**: 4.0.

El ejercicio refuerza la precedencia de operadores y la diferencia entre widening (automático) y casting (explícito).

Solución del ejercicio 6.4



```

1 // Fragmento: incluir dentro de una clase
2 // 1. int → long (conversión implícita, sin pérdida)
3 int entero = 123456789;
4 long largo = entero;
5 System.out.println("int a long: " + largo);
6
7 // 2. double → float (conversión explícita, puede perder ↪
  //   precisión)
8 double decimal = 3.141592653589793;
9 float flotante = (float) decimal;
10 System.out.println("double a float: " + flotante);
11
12 // 3. long → int (conversión explícita, puede desbordar)
13 long numeroGrande = 9876543210L;
14 int reducido = (int) numeroGrande;
15 System.out.println("long a int: " + reducido + " (↪
  //   incorrecto si supera Integer.MAX_VALUE)");
16
17 // 4. char → int (código ASCII/Unicode)
18 char letra = 'A';
19 int ascii = letra;
20 System.out.println("char 'A' a int: " + ascii); // 65

```


Funciones

En Java, una función se denomina **método**: un bloque de código nombrado que realiza una tarea concreta y pertenece a una clase. Los métodos son la unidad básica de reutilización de código y permiten estructurar los programas en partes manejables. Este capítulo se divide de la siguiente manera:

- **Definición y sintaxis:** cómo se declara un método en Java.
- **Parámetros y argumentos:** paso de datos a los métodos, sobrecarga y varargs.
- **Valor de retorno:** cómo un método devuelve resultados.
- **Ámbito de las variables:** dónde vive cada variable dentro y fuera de un método.
- **Recursividad:** métodos que se llaman a sí mismos para resolver problemas.

7.1. Introducción

Los métodos permiten:

- **Reutilizar código:** centralizar la lógica en un lugar evita duplicaciones y reduce errores.
- **Encapsular comportamiento:** ocultar detalles de implementación y exponer una interfaz clara.
- **Mejorar la legibilidad:** métodos con responsabilidad única son más fáciles de entender y mantener.
- **Facilitar las pruebas:** cada método puede probarse de forma aislada.

En Java existen dos tipos principales de métodos: los **estáticos** (pertenecen a la clase y se invocan sin crear un objeto) y los **de instancia** (pertenecen a un objeto concreto). En este capítulo nos centraremos en los métodos estáticos. Los métodos de instancia se estudiarán en el capítulo 10.

7.2. Definición y sintaxis

Un método en Java tiene la siguiente estructura:

```

1 [modificadores] tipoRetorno nombreMetodo([parámetros]) [throws ↩
    Excepcion] {
2     // cuerpo del método
3     return valor; // si tipoRetorno != void
4 }
```

Ejemplo 7.1: Estructura general de un método

Los componentes son:

- **Modificadores:** `public`, `private`, `protected`, `static`, `final`, etc.
- **Tipo de retorno:** el tipo del valor que devuelve (`int`, `String`, `void` si no devuelve nada).
- **Nombre:** identificador del método, por convención en *camelCase*.
- **Parámetros:** lista de variables de entrada (puede estar vacía).
- **throws:** opcional; indica qué excepciones puede lanzar (véase capítulo 13).

```

1 public class Calculadora {
2
3     public static int sumar(int a, int b) {
4         return a + b;
5     }
6
7     public static void main(String[] args) {
8         int resultado = Calculadora.sumar(3, 4);
9         System.out.println("Suma: " + resultado); // Suma: 7
10    }
11 }
```

Ejemplo 7.2: Método estático y llamada

7.3. Parámetros y argumentos

7.3.1. Paso por valor

Java siempre pasa los argumentos **por valor**: el método recibe una copia del dato. Para tipos primitivos esto significa que el método no puede modificar la variable original.

```

1 public static void duplicar(int x) {
2     x = x * 2; // solo afecta a la copia local
3 }
4
5 public static void main(String[] args) {
6     int n = 5;
7     duplicar(n);
8     System.out.println(n); // sigue siendo 5
9 }

```

Ejemplo 7.3: Paso por valor con primitivos

Para objetos, se pasa por valor la **referencia**: el método puede modificar el estado del objeto, pero no puede reasignar la variable original para que apunte a otro objeto.

7.3.2. Sobrecarga de métodos

Java permite definir varios métodos con el mismo nombre siempre que difieran en el número o tipo de parámetros.

```

1 public static int sumar(int a, int b) { return a + b; }
2 public static double sumar(double a, double b) { return a + b; }
3 public static int sumar(int a, int b, int c) { return a + b + c; }

```

Ejemplo 7.4: Sobrecarga de métodos

El compilador selecciona el método correcto según los argumentos en cada llamada.

7.3.3. Parámetros de longitud variable (varargs)

Con `...` se puede definir un método que acepte cualquier número de argumentos del mismo tipo.

```

1 public static int sumarTodos(int... numeros) {
2     int total = 0;
3     for (int n : numeros) total += n;
4     return total;
5 }

```

```

6
7 // Llamadas válidas:
8 sumarTodos(1, 2);
9 sumarTodos(1, 2, 3, 4, 5);
10 sumarTodos();

```

Ejemplo 7.5: Varargs

Consejo

Solo puede haber un parámetro varargs por método, y debe ser el último.

7.4. Valor de retorno

Un método puede devolver un valor con **return**. Cuando el tipo de retorno es **void**, el **return** es opcional y sirve solo para salir del método antes de su fin.

7.4.1. Retorno temprano (*early return*)

El retorno temprano simplifica la lógica evitando anidaciones innecesarias.

```

1 // Sin early return (más anidado)
2 public static String clasificar(int n) {
3     String resultado;
4     if (n > 0) {
5         resultado = "positivo";
6     } else if (n < 0) {
7         resultado = "negativo";
8     } else {
9         resultado = "cero";
10    }
11    return resultado;
12 }
13
14 // Con early return (más claro)
15 public static String clasificar(int n) {
16     if (n > 0) return "positivo";
17     if (n < 0) return "negativo";
18     return "cero";
19 }

```

Ejemplo 7.6: Early return

7.5. Ámbito de las variables

El **ámbito** (*scope*) de una variable es la región del código donde es visible y puede usarse.

- **Variables locales:** declaradas dentro de un método o bloque; solo existen mientras el bloque está activo.
- **Variables de clase (estáticas):** declaradas con `static` fuera de cualquier método; pertenecen a la clase y persisten durante toda la ejecución.
- **Shadowing:** una variable local con el mismo nombre que una variable de clase oculta a esta última dentro de su ámbito.

```

1 public class Ejemplo {
2     static int x = 10; // variable de clase
3
4     public static void mostrar() {
5         int x = 20; // variable local: oculta a la de clase
6         System.out.println(x); // 20 (local)
7         System.out.println(Ejemplo.x); // 10 (de clase)
8     }
9 }

```

Ejemplo 7.7: Ámbito y shadowing

Consejo



El shadowing puede causar confusión. Conviene evitarlo usando nombres distintos para variables locales y de clase.

7.6. Recursividad

Un método es **recursivo** cuando se llama a sí mismo para resolver una versión más pequeña del mismo problema.

Todo método recursivo necesita:

1. **Caso base:** condición que detiene la recursión y devuelve un resultado directamente.
2. **Caso recursivo:** llamada al propio método con un argumento que se acerca al caso base.

```

1 public static long factorial(int n) {
2     if (n <= 1) return 1;          // caso base
3     return n * factorial(n - 1);  // caso recursivo
4 }

```

Ejemplo 7.8: Factorial recursivo

```

1 public static int fibonacci(int n) {
2     if (n <= 1) return n;          // caso base
3     return fibonacci(n - 1) + fibonacci(n - 2); // caso recursivo
4 }

```

Ejemplo 7.9: Fibonacci recursivo

7.6.1. Recursión vs. iteración

La recursión es elegante pero tiene un coste: cada llamada consume espacio en la pila (*stack*). Para valores grandes, puede producir un `StackOverflowError`. En esos casos, la solución iterativa es preferible.

Consejo



Java no optimiza la recursión de cola (*tail call optimization*), a diferencia de otros lenguajes funcionales. Para cálculos con muchas iteraciones, usa bucles.

Ejercicios de refuerzo

Ejercicio 7.1



Escribe un método estático `esPar(int n)` que devuelva `true` si `n` es par y `false` en caso contrario. Llámalo desde `main` con varios valores y muestra el resultado. *Ver solución en la página 58.*

Ejercicio 7.2



Escribe un método estático `maximo(int a, int b, int c)` que devuelva el mayor de los tres valores. No uses `Math.max`. *Ver solución en la página 58.*

Ejercicio 7.3

Sobrecarga un método `describir` que acepte: (a) un `int`, (b) un `double`, y (c) un `String`. Cada versión debe imprimir el tipo y el valor recibido. *Ver solución en la página 58.*

Ejercicio 7.4

Escribe un método recursivo `sumaHasta(int n)` que calcule la suma de todos los enteros del 1 al `n`. Por ejemplo, `sumaHasta(4)` debe devolver 10 ($1+2+3+4$). Luego escribe la versión iterativa y compara. *Ver solución en la página 58.*

Ejercicio 7.5

Escribe un método `contarVocales(String texto)` que cuente cuántas vocales (a, e, i, o, u, mayúsculas o minúsculas) contiene el texto. Usa `String.toLowerCase()` y un bucle `for`. *Ver solución en la página 59.*

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 7.1



```
1 // Fragmento: incluir dentro de una clase
2 public static boolean esPar(int n) {
3     return n % 2 == 0;
4 }
```

Solución del ejercicio 7.2



```
1 // Fragmento: incluir dentro de una clase
2 public static int maximo(int a, int b, int c) {
3     if (a >= b && a >= c) {
4         return a;
5     }
6     if (b >= c) {
7         return b;
8     }
9     return c;
10 }
```

Solución del ejercicio 7.3



```
1 // Fragmento: incluir dentro de una clase (sobrecarga de ↩
   métodos)
2 public static void describir(int n) {
3     System.out.println("int: " + n);
4 }
5
6 public static void describir(double d) {
7     System.out.println("double: " + d);
8 }
9
10 public static void describir(String s) {
11     System.out.println("String: " + s);
12 }
```

Solución del ejercicio 7.4



```
1 // Fragmento: incluir dentro de una clase
2
3 // Versión recursiva
```

```

4 public static int sumaHasta(int n) {
5     if (n <= 0) {
6         return 0;
7     }
8     return n + sumaHasta(n - 1);
9 }
10
11 // Versión iterativa
12 public static int sumaHastaIter(int n) {
13     int total = 0;
14     for (int i = 1; i <= n; i++) {
15         total += i;
16     }
17     return total;
18 }

```

Solución del ejercicio 7.5



```

1 // Fragmento: incluir dentro de una clase
2 public static int contarVocales(String texto) {
3     int contador = 0;
4     for (char c : texto.toLowerCase().toCharArray()) {
5         if ("aeiouáéíóú".indexOf(c) >= 0) {
6             contador++;
7         }
8     }
9     return contador;
10 }

```


Utilización de objetos

En este capítulo se introducirá el concepto de objetos en Java, que son instancias de clases. Los objetos permiten encapsular datos y comportamientos relacionados, facilitando la organización y reutilización del código. Este capítulo, se divide de la siguiente manera:

- **Características de los objetos:** Los objetos tienen identidad, estado (propiedades o atributos) y comportamiento (métodos).
- **Instanciación de objetos:** como son creados los objetos.
- **Utilización de métodos, y el uso de parámetros:** Los métodos definen acciones que pueden realizar los objetos y pueden recibir parámetros para modificar su comportamiento.
- **Utilización de propiedades:** Las propiedades o atributos almacenan información sobre el estado del objeto y pueden ser accedidas o modificadas mediante métodos.
- **Utilización de métodos estáticos:** Los métodos estáticos pertenecen a la clase y no a los objetos; se invocan usando el nombre de la clase.
- **Constructores:** Son métodos especiales que se ejecutan al crear un objeto y permiten inicializar sus propiedades.
- **Destrucción de objetos y liberación de memoria:** En Java, la liberación de memoria se realiza automáticamente mediante el recolector de basura (*garbage collector*), que destruye los objetos que ya no se utilizan.

8.1. Características de los objetos

Un objeto en Java es una entidad que combina estado y comportamiento. El estado se representa mediante atributos (variables de instancia), y el comportamiento mediante métodos (funciones asociadas al objeto). La Estructura de un

objeto se define en una clase, que actúa como plantilla para crear instancias de objetos.

```
1 class Persona {  
2     String nombre;  
3     int edad;  
4  
5     void saludar() {  
6         System.out.println("Hola, mi nombre es " + nombre);  
7     }  
8 }
```

Ejemplo 8.1: Ejemplo de clase y objeto

En el ejemplo anterior, `nombre` y `edad` son atributos, y `saludar()` es un método.

Importante

El concepto de clase se verá en detalle en el capítulo 10, donde se explicará cómo se definen y utilizan las clases en Java. Por ahora, es importante entender que una clase es una plantilla que define las propiedades y comportamientos de los objetos que se crearán a partir de ella.

8.2. Instanciación de objetos

Para crear un objeto, se utiliza el operador `new` seguido del constructor de la clase.

```
1 Persona persona1 = new Persona();  
2 persona1.nombre = "Ana";  
3 persona1.edad = 25;  
4 persona1.saludar(); // Imprime: Hola, mi nombre es Ana
```

Ejemplo 8.2: Instanciación de un objeto

Una vez creada una clase, se pueden crear múltiples instancias de esa clase, cada una con su propio estado.

8.3. Utilización de métodos y el uso de parámetros

Los métodos pueden recibir parámetros para modificar su comportamiento. Por ejemplo:

```

1 class Calculadora {
2     int sumar(int a, int b) {
3         return a + b;
4     }
5 }

```

Ejemplo 8.3: Método con parámetros

En este ejemplo, el método `sumar` recibe dos parámetros `a` y `b` y devuelve su suma. Para utilizar este método, se crea un objeto de la clase `Calculadora` y se llama al método:

```

1 Calculadora calc = new Calculadora();
2 int resultado = calc.sumar(3, 4); // resultado es 7

```

Ejemplo 8.4: Uso del método

8.4. Utilización de propiedades

Las propiedades de un objeto pueden ser accedidas y modificadas directamente si son públicas (ver ejemplo 8.2), aunque es recomendable usar métodos *getter* y *setter* para proteger el acceso.

```

1 class Persona {
2     private String nombre;
3
4     public String getNombre() {
5         return nombre;
6     }
7
8     public void setNombre(String nombre) {
9         this.nombre = nombre;
10    }
11 }

```

Ejemplo 8.5: Uso de getters y setters

8.5. Utilización de métodos estáticos

Los métodos estáticos pertenecen a la clase y no a los objetos. Se invocan usando el nombre de la clase.

```

1 class Utilidades {
2     static int cuadrado(int x) {
3         return x * x;
4     }

```

5 }
}

Ejemplo 8.6: Método estático

1 `int resultado = Utilidades.cuadrado(5); // resultado es 25`

Ejemplo 8.7: Uso de método estático

El uso de métodos estáticos es útil para funciones que no dependen del estado de un objeto específico, como utilidades matemáticas o funciones de ayuda.

Consejo

Son de utilidad los métodos estáticos de la clase `Math`, como `Math.sqrt()` para calcular raíces cuadradas, o `Math.random()` para generar números aleatorios. Estos métodos son ampliamente utilizados en programación y facilitan tareas comunes sin necesidad de crear instancias de objetos.

También son de gran utilidad los métodos de la clase `System`, como `System.out.println()` para imprimir en la consola, o `System.currentTimeMillis()` para obtener el tiempo actual en milisegundos. Estos métodos proporcionan funcionalidades esenciales para interactuar con el entorno de ejecución y son ampliamente utilizados en la mayoría de los programas Java.

Otros métodos estáticos que utilizarás en el futuro incluyen `String.valueOf()` para convertir otros tipos de datos a cadenas, o `Integer.parseInt()` para convertir cadenas a enteros. Estos métodos son fundamentales para la manipulación de datos y la conversión entre tipos de datos en Java.

8.6. Constructores

Un constructor es un método especial que se ejecuta al crear un objeto. Sirve para inicializar los atributos. Por defecto, Java proporciona un constructor sin parámetros si no se define ninguno. Sin embargo, es común definir constructores personalizados para establecer valores iniciales.

```

1 class Persona {
2     String nombre;
3     int edad;
4
5     Persona(String nombre, int edad) {
6         this.nombre = nombre;
7         this.edad = edad;
8     }

```



```
9 }

```

Ejemplo 8.8: Definición de constructor

```
1 Persona persona2 = new Persona("Luis", 30);

```

Ejemplo 8.9: Uso del constructor

Es posible que una clase tenga múltiples constructores, cada uno con diferentes parámetros. Esto se conoce como *sobrecarga de constructores*. Además, si no se define un constructor, Java proporciona uno por defecto que no toma parámetros y no inicializa los atributos.

```
1 class Persona {
2     String nombre;
3     int edad;
4
5     // Constructor sin parámetros
6     Persona() {
7         this.nombre = "Desconocido";
8         this.edad = 0;
9     }
10
11    // Constructor con un parámetro
12    Persona(String nombre) {
13        this.nombre = nombre;
14        this.edad = 0;
15    }
16
17    // Constructor con dos parámetros
18    Persona(String nombre, int edad) {
19        this.nombre = nombre;
20        this.edad = edad;
21    }
22
23    void mostrarInformacion() {
24        System.out.println("Nombre: " + nombre + ", Edad: " + edad);
25    }
26 }
```

Ejemplo 8.10: Clase con varios constructores

De esta forma, se pueden crear objetos de la clase `Persona` usando diferentes constructores:

```
1 Persona p1 = new Persona();
2 Persona p2 = new Persona("Ana");
3 Persona p3 = new Persona("Luis", 30);
4
5 p1.mostrarInformacion(); // Imprime: Nombre: Desconocido, Edad: ↵
6 0
```

```
6 p2.mostrarInformacion(); // Imprime: Nombre: Ana, Edad: 0
7 p3.mostrarInformacion(); // Imprime: Nombre: Luis, Edad: 30
```

Ejemplo 8.11: Uso de varios constructores

8.7. Destrucción de objetos y liberación de memoria

En Java, la gestión de memoria es automática. El recolector de basura (*garbage collector*) elimina los objetos que ya no tienen referencias. No es necesario liberar memoria manualmente, aunque se puede sugerir la recolección con `System.gc()`, pero su ejecución no está garantizada.

Importante

Aunque Java maneja automáticamente la memoria, es importante evitar referencias circulares entre objetos, ya que pueden impedir que el recolector de basura libere memoria adecuadamente.

Además, es recomendable cerrar recursos como archivos o conexiones de red utilizando bloques `try-with-resources` o asegurándose de llamar a los métodos `close()` correspondientes para liberar recursos externos.

8.8. La clase `String`

La clase `String` en Java es una de las más utilizadas y tiene características particulares que la diferencian de otros tipos de objetos.

8.8.1. Inmutabilidad de los objetos `String`

Los objetos de tipo `String` son **inmutables**, es decir, una vez creados, su valor no puede cambiar. Si se realiza una operación que parece modificar una cadena, en realidad se crea un nuevo objeto `String` con el resultado.

```
1 String saludo = "Hola";
2 saludo = saludo + " mundo"; // Se crea un nuevo objeto String
```

Ejemplo 8.12: Inmutabilidad de `String`

En el ejemplo anterior, la variable `saludo` apunta primero a una cadena y luego a otra, pero el objeto original no se modifica.

8.8.2. Creación de cadenas

Las cadenas pueden crearse de varias formas:

```
1 String s1 = "texto"; // Literal
2 String s2 = new String("texto"); // Constructor (no recomendado)
```

Ejemplo 8.13: Formas de crear cadenas

La forma recomendada es usar literales, ya que es más eficiente y permite el uso del *pool* de cadenas.

8.8.3. Concatenación y métodos útiles

La concatenación de cadenas puede hacerse con el operador `+` o usando el método `concat()`:

```
1 String nombre = "Ana";
2 String saludo = "Hola, " + nombre;
```

Ejemplo 8.14: Concatenación de cadenas

Algunos métodos útiles de la clase `String` son:

- `length()`: Devuelve la longitud de la cadena.
- `toUpperCase()`, `toLowerCase()`: Convierte la cadena a mayúsculas o minúsculas.
- `charAt(int index)`: Devuelve el carácter en la posición indicada.
- `substring(int beginIndex, int endIndex)`: Extrae una subcadena.
- `equals(String other)`: Compara el contenido de dos cadenas.
- `trim()`: Elimina espacios en blanco al inicio y al final.

```
1 String texto = " Java ";
2 int longitud = texto.length(); // 7
3 String mayusculas = texto.toUpperCase(); // " JAVA "
4 String sinEspacios = texto.trim(); // "Java"
5 boolean igual = texto.trim().equals("Java"); // true
```

Ejemplo 8.15: Ejemplo de métodos de `String`

8.8.4. Comparación de cadenas

Para comparar el contenido de dos cadenas, se debe usar el método `equals()` y no el operador `==`, ya que este último compara referencias, no contenido.

```
1 String a = "hola";
2 String b = "hola";
3 boolean iguales = a.equals(b); // true
4 boolean mismasReferencias = (a == b); // true en este caso, pero →
    no siempre
```

Ejemplo 8.16: Comparación de cadenas

Importante

Recuerda siempre usar `equals()` para comparar cadenas por su contenido. El operador `==` solo debe usarse para comparar si dos referencias apuntan al mismo objeto.

Resumen

En este capítulo se ha presentado el concepto de objeto en Java, su creación, uso de métodos y atributos, métodos estáticos, constructores y la gestión automática de memoria. Estos conceptos son fundamentales para la programación orientada a objetos y la construcción de aplicaciones robustas y reutilizables.

Ejercicios de refuerzo

Ejercicio 8.1

Observa el siguiente fragmento de código e indica, para cada llamada a un método, si se trata de un método **estático** o de **instancia**. Justifica tu respuesta en cada caso.

Pista: no necesitas conocer las clases que aparecen, solo fijarte en si el método se invoca sobre el nombre de la clase o sobre un objeto ya creado.

Scanner es la clase que lee datos del teclado y se estudia en el capítulo 15; aquí solo sirve de ejemplo de objeto instanciado.

```
1 import java.util.Scanner;
2
3 public class MetodosEstaticosInstancia {
4     public static void main(String[] args) {
5         double valor = Math.random(); // (1)
6         double raiz = Math.sqrt(16.0); // (2)
```

```

7
8     Scanner scanner = new Scanner(System.in);
9     String linea = scanner.nextLine();    // (3)
10    scanner.close();                      // (4)
11
12    String texto = "Hola mundo";
13    int longitud = texto.length();        // (5)
14    String mayusculas = texto.toUpperCase(); // (6)
15 }
16 }

```

Ver solución en la página 71.

Ejercicio 8.2



Considera el siguiente código que crea varios objetos *Persona*:

```

1 public class Persona {
2     String nombre;
3     int edad;
4
5     public Persona(String nombre, int edad) {
6         this.nombre = nombre;
7         this.edad = edad;
8     }
9 }
10
11 Persona p1 = new Persona("Ana", 25);
12 Persona p2 = new Persona("Luis", 30);
13 Persona p3 = p1;
14 p1 = null;

```

Describe cómo queda la memoria tras ejecutar estas cuatro líneas: ¿qué son *p1*, *p2* y *p3*? ¿Qué objeto queda sin referencia y qué le ocurrirá?

Ver solución en la página 71.

Ejercicio 8.3



Crea una clase *Coche* que tenga los siguientes atributos: *marca*, *modelo* y *año*. Implementa un constructor que inicialice estos atributos y un método *mostrarDetalles()* que imprima la información del coche en la consola. Luego, crea un objeto de la clase *Coche* y llama al método *mostrarDetalles()*.

El atributo *año* debe ser un número entero, mientras que *marca* y *modelo* deben ser cadenas de texto.

Al imprimir los detalles, la marca deberá aparecer en mayúsculas, el modelo en minúsculas y el año como un número entero. Por ejemplo, si el coche es un Ford Focus del año 2020, la salida debería ser "FORD focus 2020".
Ver solución en la página 72.

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 8.1



1. `Math.random()` — **estático**: se invoca sobre la clase `Math` directamente, sin instanciar ningún objeto.
2. `Math.sqrt(16.0)` — **estático**: igual que el anterior, pertenece a la clase `Math`.
3. `scanner.nextLine()` — **de instancia**: requiere un objeto `Scanner` creado previamente; el comportamiento depende del estado de ese objeto.
4. `scanner.close()` — **de instancia**: opera sobre el objeto `scanner` concreto.
5. `texto.length()` — **de instancia**: se invoca sobre el objeto `String` `texto`.
6. `texto.toUpperCase()` — **de instancia**: depende del contenido del objeto `texto`.

Los métodos estáticos pertenecen a la clase y no necesitan objeto; los de instancia operan sobre un objeto concreto y pueden acceder a su estado.

Solución del ejercicio 8.2



- `p1 = new Persona("Ana", 25)` crea un objeto en el heap y `p1` almacena su **referencia** en la pila.
- `p2 = new Persona("Luis", 30)` crea otro objeto distinto en el heap; `p2` apunta a él.
- `p3 = p1` copia la referencia de `p1`: ahora `p3` y `p1` apuntan al *mismo* objeto `.Ana`.
- `p1 = null` elimina la referencia de `p1`, pero el objeto `.Ana` sigue accesible a través de `p3`.

Ningún objeto queda sin referencia en este punto: `.Ana` sigue viva vía `p3`. Si también se hiciera `p3 = null`, el objeto `.Ana` quedaría inalcanzable y el *garbage collector* podría liberarlo.

Solución del ejercicio 8.3



```
1 // Programa completo: compilar y ejecutar ↵
  independientemente
2 public class Coche {
3     private String marca;
4     private String modelo;
5     private int año;
6
7     public Coche(String marca, String modelo, int año) {
8         this.marca = marca;
9         this.modelo = modelo;
10        this.año = año;
11    }
12
13    public void mostrarDetalles() {
14        System.out.println(marca.toUpperCase() + " " + modelo.↵
15        toLowerCase() + " " + año);
16    }
17
18    public static void main(String[] args) {
19        Coche miCoche = new Coche("Ford", "Focus", 2020);
20        miCoche.mostrarDetalles(); // FORD focus 2020
21    }
```


Uso de estructuras de control

En la programación, las estructuras de control son fundamentales para definir el flujo de ejecución de un programa. Estas estructuras permiten tomar decisiones, repetir acciones y modificar el comportamiento del software según diferentes condiciones:

- **Estructuras de selección:** Permiten que el programa elija entre diferentes caminos de ejecución, dependiendo del cumplimiento de ciertas condiciones.
- **Estructuras de repetición:** Facilitan la ejecución de un bloque de código varias veces, ya sea un número determinado de veces o mientras se cumpla una condición.
- **Estructuras de salto:** Permiten alterar el flujo normal de ejecución, transfiriendo el control a otra parte del programa.
- **Control de excepciones:** Es un mecanismo que permite manejar situaciones inesperadas o errores durante la ejecución del programa, evitando que el software falle abruptamente.

9.1. Estructuras de selección

Las estructuras de selección permiten tomar decisiones en función de condiciones lógicas.

9.1.1. Sentencias condicionales: `if`, `else if`, y `else`

Las sentencias condicionales más comunes son `if`, `else if` y `else`. Estas permiten ejecutar diferentes bloques de código según se cumplan o no ciertas condiciones.

```
1 int edad = 18;
2 if (edad >= 18) {
3     System.out.println("Eres mayor de edad");
4 } else {
5     System.out.println("Eres menor de edad");
6 }
```

Ejemplo 9.1: Ejemplo de sentencia if-else en Java

En este ejemplo, si la variable `edad` es mayor o igual a 18, se imprime **"Eres mayor de edad"**. De lo contrario, se imprime **"Eres menor de edad"**. Es posible incluir múltiples condiciones utilizando **else if** para manejar diferentes casos:

```
1 int nota = 85;
2 if (nota >= 90) {
3     System.out.println("Excelente");
4 } else if (nota >= 70) {
5     System.out.println("Aprobado");
6 } else {
7     System.out.println("Reprobado");
8 }
```

Ejemplo 9.2: Ejemplo de if-else if-else en Java

En este caso, se evalúa la variable `nota` y se imprime un mensaje diferente según el rango en el que se encuentre. Cada bloque de código se ejecuta solo si la condición correspondiente es verdadera, y si ninguna condición anterior se cumple.

Además, también es posible escribir condiciones sin bloque **else** para ejecutar una acción específica si la condición es verdadera, sin necesidad de un bloque alternativo:

```
1 int temperatura = 30;
2 if (temperatura > 25)
3     System.out.println("Hace calor");
```

Ejemplo 9.3: Ejemplo de if sin else en Java

Consejo



Recuerda que es importante utilizar llaves `()` para agrupar las condiciones y bloques de código, especialmente cuando se trabaja con múltiples líneas de código dentro de un bloque **if** o **else**. Esto mejora la legibilidad y evita errores en la ejecución del programa.

Es recomendable utilizar comentarios para explicar la lógica detrás de las condiciones, especialmente en casos más complejos.

Es posible crear condiciones compuestas utilizando operadores lógicos co-

mo `&&` (AND) y `||` (OR).

9.1.2. Sentencias de selección: Switch

Otra estructura de selección es `switch`, útil cuando se desea comparar una variable contra varios valores posibles:

```

1 int dia = 3;
2 switch (dia) {
3     case 1:
4         System.out.println("Lunes");
5         break;
6     case 2:
7         System.out.println("Martes");
8         break;
9     case 3:
10        System.out.println("Miércoles");
11        break;
12    case 6:
13    case 7:
14        System.out.println("Fin de semana");
15        break;
16    default:
17        System.out.println("Otro día");
18 }
```

Ejemplo 9.4: Ejemplo de switch en Java

En este ejemplo, la variable `dia` se compara con diferentes casos. Si coincide con alguno, se ejecuta el bloque de código correspondiente. El uso de `break` es importante para evitar que el programa continúe ejecutando los siguientes casos después de encontrar una coincidencia. Si no se encuentra ninguna coincidencia, se ejecuta el bloque `default`.

Consejo



El uso de `switch` es especialmente útil cuando se tienen múltiples condiciones basadas en el mismo valor, ya que mejora la legibilidad del código en comparación con múltiples sentencias `if-else if`.

Si varios casos comparten el mismo bloque de código, se pueden agrupar sin necesidad de repetir el código, como se muestra en el ejemplo con los casos 6 y 7.

Además, es posible usar la sentencia `switch` con cadenas de texto a partir de Java 7, lo que permite comparar una variable de tipo `String` con diferentes valores:

```
1 String dia = "Lunes";
2 switch (dia) {
3     case "Lunes":
4         System.out.println("Hoy es lunes");
5         break;
6     case "Martes":
7         System.out.println("Hoy es martes");
8         break;
9     case "Miércoles":
10        System.out.println("Hoy es miércoles");
11        break;
12    default:
13        System.out.println("Otro día");
14 }
```

Ejemplo 9.5: Ejemplo de switch con String en Java

Otra de las peculiaridades de la sentencia **switch** es que permite devolver un valor.

```
1 public String obtenerDiaSemana(int dia) {
2     return switch (dia) {
3         case 1 -> "Lunes";
4         case 2 -> "Martes";
5         case 3 -> "Miércoles";
6         case 4 -> "Jueves";
7         case 5 -> "Viernes";
8         case 6, 7 -> "Fin de semana";
9         default -> "Día inválido";
10    };
11 }
```

Ejemplo 9.6: Switch que devuelve un valor en Java

En este ejemplo, el método `obtenerDiaSemana` utiliza la sentencia **switch** para devolver directamente un valor según el caso, usando la sintaxis de expresión introducida en Java 14.

Importante

La sentencia **switch** también puede emplearse para convertir objetos a diferentes clases.

Por ejemplo, si tienes una variable de tipo `Object` y necesitas ejecutar acciones distintas según su tipo específico, puedes utilizar **switch** para realizar el casteo de manera segura:

```
1 public void procesar(Object obj) {
2     switch (obj) {
3         case String s -> System.out.println("Cadena: " + s.↵
4         toUpperCase());
5     }
```

```

4      case Integer i -> System.out.println("Entero: " + (i ↵
    * 2));
5      case Double d -> System.out.println("Decimal: " + (d ↵
    / 2));
6      default -> System.out.println("Tipo desconocido");
7  }
8  }
9

```

Ejemplo 9.7: Ejemplo de switch para convertir objetos en Java

En este caso, el método `procesar` utiliza `switch` para determinar el tipo del objeto recibido y ejecutar una acción específica para cada tipo, realizando la conversión automáticamente en cada caso.

Ejercicio 9.1



Escribe un programa que imprima por pantalla si un año es bisiesto o no. Un año es bisiesto si es divisible por 4, excepto los años que son divisibles por 100, a menos que también sean divisibles por 400. Utiliza una estructura de selección adecuada para resolver el problema. *Ver solución en la página 85.*

9.1.3. El operador ternario

El operador ternario (`? :`) es una forma compacta de escribir una estructura `if-else` simple. Su sintaxis es:

```
1 resultado = condición ? valor_si_verdadero : valor_si_falso;
```

Por ejemplo, para asignar un mensaje según la edad de una persona:

```

1 int edad = 20;
2 String mensaje = (edad >= 18) ? "Mayor de edad" : "Menor de edad"↵
    ";
3 System.out.println(mensaje);

```

Ejemplo 9.8: Ejemplo de operador ternario en Java

Consejo



El operador ternario es útil para asignaciones simples y expresiones cortas. Sin embargo, para lógicas más complejas o cuando hay múltiples instrucciones, es preferible usar `if-else` para mantener la legibilidad del código.

9.2. Estructuras de repetición

Permiten ejecutar un bloque de código varias veces.

9.2.1. Bucle for

El bucle **for** es ideal cuando se conoce de antemano el número de iteraciones que se deben realizar. Su sintaxis es la siguiente:

```
1 for (int i = 0; i < 5; i++) {
2     System.out.println("Iteración " + i);
3 }
```

Ejemplo 9.9: Ejemplo de bucle for en Java

En este ejemplo, el bucle se ejecuta 5 veces, imprimiendo el número de iteración en cada paso. La variable `i` se inicializa en 0, y se incrementa en 1 en cada iteración hasta que alcanza 5.

Este tipo de bucle es el más apropiado cuando se necesita iterar sobre un rango de números o elementos de una colección, ya que permite controlar fácilmente el inicio, la condición de continuación y el incremento.

```
1 String texto = "Hola";
2 for (int i = 0; i < texto.length(); i++) {
3     char character = texto.charAt(i);
4     System.out.println("Carácter en posición " + i + ": " + ↵
5     character);
6 }
```

Ejemplo 9.10: Iterar sobre los caracteres de un String en Java

En este ejemplo, el bucle recorre la cadena `texto` desde la posición 0 hasta la longitud de la cadena menos uno, imprimiendo cada carácter junto con su posición.

Importante

Todos los elementos del bucle **for** son opcionales, lo que significa que puedes omitir la inicialización, la condición y el incremento. Sin embargo, es importante tener cuidado al hacerlo, ya que puede llevar a bucles infinitos si no se controla adecuadamente.

9.2.2. Bucle while, y do-while

El bucle **while** se utiliza cuando no se conoce de antemano el número de iteraciones, y se ejecuta mientras una condición sea verdadera. Su sintaxis es la siguiente:

```

1 int contador = 0;
2 while (contador < 5) {
3     System.out.println("Contador: " + contador);
4     contador++;
5 }

```

Ejemplo 9.11: Ejemplo de bucle while en Java

Este bucle continuará ejecutándose mientras la variable `contador` sea menor que 5. En cada iteración, se imprime el valor del contador y luego se incrementa en 1.

También existe el bucle `do-while`, que garantiza que el bloque de código se ejecute al menos una vez antes de evaluar la condición:

```

1 int contador = 0;
2 do {
3     System.out.println("Contador: " + contador);
4     contador++;
5 } while (contador < 5);

```

Ejemplo 9.12: Ejemplo de bucle do-while en Java

Usaremos el bucle `while` cuando la condición de continuación no dependa de un contador fijo, sino de una condición que puede cambiar durante la ejecución del programa. Por ejemplo, cuando iteremos sobre una colección usando un iterador (capítulo 12).

Importante

Es posible crear bucles infinitos si la condición nunca se vuelve falsa. Por ejemplo, si olvidamos incrementar el contador en el bucle `while` anterior, el programa se quedará atascado en un bucle infinito. Siempre asegúrate de que la condición eventualmente se vuelva falsa para evitar este problema.

Ejercicio 9.2

Crea un programa que imprime por pantalla los primeros 10 números de la serie de Fibonacci.

La serie de Fibonacci comienza con 0 y 1, y cada número siguiente es la suma de los dos anteriores: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34. *Ver solución en la página 85.*

9.3. Estructuras de salto

Las estructuras de salto permiten modificar el flujo de ejecución de manera abrupta.

9.3.1. Sentencia return

La sentencia **return** se utiliza para salir de un método y, opcionalmente, devolver un valor. Cuando se ejecuta **return**, el control del programa regresa al punto donde se llamó al método.

```
1 public int sumar(int a, int b) {
2     return a + b;
3 }
```

Ejemplo 9.13: Ejemplo de return en Java

Es posible utilizar **return** para salir de un método antes de que se alcance el final del mismo, lo que puede ser útil para evitar la ejecución de código innecesario o para manejar condiciones especiales.

```
1 public boolean contiene(String texto, char caracter) {
2     if (texto == null || texto.isEmpty()) {
3         return false; // Si el texto es nulo o vacío, retorna false
4     }
5     for (int i = 0; i < texto.length(); i++) {
6         if (texto.charAt(i) == caracter) {
7             return true; // Sale del método en cuanto encuentra un ↩
8             cero
9         }
10    }
11    return false; // Si no encuentra ningún cero, retorna false
12 }
```

Ejemplo 9.14: Uso de return dentro de un bucle for en Java

En este ejemplo, el método `contiene` busca un carácter específico en una cadena de texto. Si encuentra el carácter, utiliza **return** para salir del método inmediatamente, devolviendo **true**. Si no lo encuentra, al final del bucle se devuelve **false**.

Ejercicio 9.3



Recupera el ejercicio 9.1 y modifica el programa para que retorne **true** si el año es bisiestro y **false** en caso contrario. Utiliza una estructura de control adecuada para resolver el problema.

Crea un método llamado `esBisiesto` que reciba un año como parámetro y retorne un valor booleano indicando si el año es bisiestro o no.

Después, imprime la lista de los siguientes 20 años bisiestros a partir del año actual, utilizando el método `esBisiesto` para determinar si cada año es bisiestro o no. *Ver solución en la página 85.*

9.3.2. Sentencia break y continue

La sentencia **break** se utiliza para salir de un bucle o de una estructura de selección **switch** de manera inmediata. Por ejemplo, en un bucle **for** o **while**, al ejecutar **break**, el control del programa salta al final del bucle.

```

1 for (int i = 0; i < 10; i++) {
2     if (i == 5) {
3         break; // Sale del bucle cuando i es igual a 5
4     }
5     System.out.println("Número: " + i);
6 }

```

Ejemplo 9.15: Ejemplo de break en Java

En este ejemplo, el bucle se detiene cuando *i* alcanza el valor 5, y no se imprimen los números posteriores.

La sentencia **continue** se utiliza para saltar la iteración actual de un bucle y continuar con la siguiente iteración. Esto es útil cuando se desea omitir ciertas condiciones sin salir completamente del bucle.

```

1 for (int i = 0; i < 10; i++) {
2     if (i % 2 == 0) {
3         continue; // Salta la iteración si i es par
4     }
5     System.out.println("Número impar: " + i);
6 }

```

Ejemplo 9.16: Ejemplo de continue en Java

En este ejemplo, el bucle imprime solo los números impares del 0 al 9, ya que la sentencia **continue** salta las iteraciones donde *i* es par.

9.4. Control de excepciones

El manejo de errores en tiempo de ejecución se gestiona en Java mediante excepciones. Este mecanismo se estudia en profundidad en el capítulo 13, una vez que se han introducido las clases y la jerarquía de tipos.

9.5. Recursividad

Ahora que conoces las estructuras de control básicas, es importante mencionar la recursividad, que es una técnica donde un método se llama a sí mismo para resolver un problema. Aunque no es una estructura de control en sí misma, es una forma de controlar el flujo de ejecución.

Este es un concepto avanzado, pero es útil para resolver problemas que pueden dividirse en subproblemas más pequeños. Por ejemplo, el cálculo del factorial de un número:

```
1 public int factorial(int n) {  
2     if (n == 0) {  
3         return 1; // Caso base  
4     } else {  
5         return n * factorial(n - 1); // Llamada recursiva  
6     }  
7 }
```

Ejemplo 9.17: Ejemplo de recursividad en Java

En este ejemplo, el método `factorial` se llama a sí mismo con un valor decreciente de `n` hasta llegar al caso base (`n == 0`), donde retorna 1. La recursividad es una herramienta poderosa, pero debe usarse con cuidado para evitar desbordamientos de pila (*stack overflow*) si no se define un caso base adecuado. En el ejemplo anterior, el caso base es cuando `n` es igual a 0, pero si se llama al método con un valor negativo, se generará un error de pila.

Resumen

En este capítulo hemos aprendido a utilizar las principales estructuras de control en programación: selección (`if`, `else if`, `else`, `switch`), repetición (`for`, `while`) y salto (`return`, `break`, `continue`). Estas herramientas permiten modificar el flujo de ejecución de los programas, tomar decisiones y repetir acciones. El control de excepciones (`try-catch-finally`) se trata en el capítulo 13. Dominar su uso es fundamental para desarrollar programas robustos, legibles y eficientes.

Importante

Es posible combinar diferentes estructuras de control para crear programas más complejos y adaptados a las necesidades específicas de cada situación. Por ejemplo, se pueden utilizar bucles dentro de estructuras de selección, o viceversa, para lograr un control más preciso del flujo de ejecución.

Sin embargo, es importante evitar la complejidad excesiva en el código, ya que esto puede dificultar su comprensión y mantenimiento. Utiliza comentarios y nombres descriptivos para las variables y métodos, lo que ayudará a otros desarrolladores (y a ti mismo en el futuro) a entender mejor la lógica del programa.

También es recomendable seguir las buenas prácticas de programación, como la indentación adecuada y el uso de espacios en blanco, para mejorar

la legibilidad del código. Esto facilitará la colaboración en equipo y el mantenimiento a largo plazo del software.

Una regla general es no anidar demasiadas estructuras de control, ya que esto puede hacer que el código sea difícil de seguir. En su lugar, considera dividir el código en métodos más pequeños y manejables que realicen tareas específicas, como habrás podido comprobar en el ejercicio de los años bisiestos.

Ejercicios de refuerzo

Ejercicio 9.4



Traza la ejecución del siguiente fragmento de código e indica exactamente qué se imprime por pantalla. Muestra paso a paso los valores de `i` y `j` en cada iteración.

```
1 for (int i = 1; i <= 3; i++) {
2     for (int j = 1; j <= 3; j++) {
3         if (j == 2) {
4             continue;
5         }
6         System.out.println("i=" + i + ", j=" + j);
7     }
8 }
```

Ver solución en la página 86.

Ejercicio 9.5



Reescribe la siguiente estructura `if-else if-else` utilizando `switch`. Después, indica si existen casos en que la transformación *no* sería posible y por qué.

```
1 String dia = "LUNES";
2 String tipo;
3
4 if (dia.equals("SABADO") || dia.equals("DOMINGO")) {
5     tipo = "fin de semana";
6 } else if (dia.equals("LUNES") || dia.equals("VIERNES")) {
7     tipo = "inicio o fin de semana laboral";
8 } else {
9     tipo = "día laborable";
10 }
11 System.out.println(tipo);
```

Ver solución en la página 86.

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 9.1



```

1 // Fragmento: incluir dentro de una clase
2 int año = 2024;
3 if ((año % 4 == 0 && año % 100 != 0) || año % 400 == 0) {
4     System.out.println(año + " es bisiestro.");
5 } else {
6     System.out.println(año + " no es bisiestro.");
7 }

```

Solución del ejercicio 9.2



```

1 // Fragmento: incluir dentro de una clase
2 int anterior = 0;
3 int actual = 1;
4 System.out.print(anterior + " " + actual);
5 for (int i = 2; i < 10; i++) {
6     int siguiente = anterior + actual;
7     System.out.print(" " + siguiente);
8     anterior = actual;
9     actual = siguiente;
10 }
11 System.out.println(); // 0 1 1 2 3 5 8 13 21 34

```

Solución del ejercicio 9.3



```

1 // Fragmento: incluir dentro de una clase
2 public static boolean esBisiesto(int año) {
3     return (año % 4 == 0 && año % 100 != 0) || año % 400 == 0;
4 }
5
6 // Imprime los próximos 20 años bisiestos a partir de 2025
7 public static void main(String[] args) {
8     int encontrados = 0;
9     int año = 2025;
10    while (encontrados < 20) {
11        if (esBisiesto(año)) {
12            System.out.println(año);
13            encontrados++;
14        }
15        año++;
16    }

```

```
17| }
```

Solución del ejercicio 9.4



La sentencia **continue** salta la iteración cuando `j == 2`, por lo que solo se imprimen las combinaciones con `j==1` y `j==3`: `i=1, j=1 / i=1, j=3 / i=2, j=1 / i=2, j=3 / i=3, j=1 / i=3, j=3`. En total se ejecutan 6 impresiones (9 iteraciones menos 3 en que `j==2`). El ejercicio refuerza la comprensión de bucles anidados y el efecto de **continue** en la iteración interior.

Solución del ejercicio 9.5



La versión con **switch** usa *fall-through* para agrupar los casos equivalentes: **case** "SABADO": **case** "DOMINGO": asigna "fin de semana", **case** "LUNES": **case** "VIERNES": asigna "inicio o fin de semana laboral", y **default**: asigna "día laborable". Cada rama termina con **break**.

La transformación *no* es posible cuando la condición evalúa rangos o expresiones complejas (p. ej. `edad > 18 && edad < 65`), ya que **switch** solo permite comparar valores discretos de igualdad. Tampoco se pueden combinar directamente múltiples variables distintas en un **switch**.

Desarrollo de clases

En este capítulo se abordan los conceptos fundamentales relacionados con el desarrollo de clases en programación orientada a objetos:

1. **Concepto de clase:** Una clase es una plantilla o modelo que define las características (atributos) y comportamientos (métodos) que tendrán los objetos creados a partir de ella.
2. **Estructura y miembros de una clase. Visibilidad:** Una clase está compuesta por miembros, que pueden ser atributos (variables) o métodos (funciones).
3. **Creación de propiedades:** Las propiedades son atributos que almacenan información sobre el estado de un objeto. Se definen dentro de la clase y pueden tener distintos niveles de acceso.
4. **Creación de métodos:** Los métodos son funciones que definen el comportamiento de los objetos. Se implementan dentro de la clase y pueden manipular los atributos o realizar operaciones específicas.
5. **Creación de constructores:** El constructor es un método especial que se ejecuta al crear un objeto. Se utiliza para inicializar los atributos de la clase.
6. **Utilización de clases y objetos:** Para utilizar una clase, se crean objetos (instancias) a partir de ella. Los objetos permiten acceder a los atributos y métodos definidos en la clase.
7. **Utilización de clases heredadas:** La herencia permite crear nuevas clases basadas en clases existentes, reutilizando y extendiendo su funcionalidad.

10.1. Concepto de clase

En Java, una **clase** es una plantilla para crear objetos. Define tanto los atributos (propiedades que describen el estado del objeto) como los métodos (acciones o comportamientos que puede realizar el objeto). Las clases permiten organizar y estructurar el código, facilitando la reutilización y el mantenimiento. Cada objeto creado a partir de una clase se denomina *instancia* y posee sus propios valores para los atributos definidos en la clase.

Por ejemplo, una clase **Coche** puede representar las características y comportamientos comunes a todos los coches, como el color, la marca y la velocidad, así como las acciones de acelerar o frenar. A partir de esta clase, se pueden crear múltiples objetos, cada uno con sus propios valores para los atributos.

```

1 public class Coche {
2     String color;
3     String marca;
4     int velocidad;
5
6     public void acelerar(int incremento) {
7         velocidad += incremento;
8     }
9
10    public void frenar(int decremento) {
11        velocidad -= decremento;
12    }
13 }

```

Ejemplo 10.1: Ejemplo de clase en Java

10.2. Estructura y miembros de una clase.

Visibilidad

En Java, los miembros de una clase (atributos y métodos) pueden tener distintos niveles de visibilidad, lo que determina desde dónde pueden ser accedidos. Los modificadores de acceso más comunes son:

- **public**: El miembro es accesible desde cualquier otra clase.
- **private**: El miembro solo es accesible dentro de la propia clase.
- **protected**: El miembro es accesible dentro de la propia clase, en las clases del mismo paquete y en las subclases (incluso si están en otros paquetes).
- *Sin modificador* (también llamado *package-private*): El miembro es accesible solo dentro de las clases del mismo paquete.

El uso adecuado de la visibilidad permite proteger los datos internos de la clase y controlar cómo se accede y modifica su estado. Por ejemplo, es una buena práctica declarar los atributos como **private** y proporcionar métodos públicos (**getters** y **setters**) para acceder o modificar su valor, siguiendo el principio de encapsulamiento.

```

1 public class Persona {
2     public String nombre;    // Público: accesible desde cualquier ↪
    clase
3     protected int edad;    // Protegido: accesible desde subclases ↪
    y el mismo paquete
4     String direccion;    // Sin modificador: accesible solo en el ↪
    mismo paquete
5     private String dni;    // Privado: accesible solo dentro de la ↪
    clase
6
7     public Persona(String nombre, String dni) {
8         this.nombre = nombre;
9         this.dni = dni;
10    }
11
12    // Getter y setter para el atributo privado dni
13    public String getDni() {
14        return dni;
15    }
16
17    public void setDni(String dni) {
18        this.dni = dni;
19    }
20 }

```

Ejemplo 10.2: Visibilidad de miembros en Java

En el ejemplo anterior, se observa cómo cada atributo tiene un nivel de visibilidad diferente. El atributo `dni` es privado y solo puede ser accedido mediante los métodos públicos `getDni()` y `setDni()`. Esto ayuda a mantener la integridad de los datos y a cumplir con el principio de encapsulamiento.

10.3. Creación de propiedades

En Java, las propiedades de una clase suelen implementarse como atributos privados, siguiendo el principio de encapsulamiento. Para acceder y modificar estos atributos desde fuera de la clase, se utilizan métodos públicos denominados **getters** y **setters**. Esta práctica permite controlar cómo se accede y modifica el estado interno del objeto, validando los valores si es necesario y evitando modificaciones no deseadas.

Ejemplo: Supongamos una clase `Circulo` con una propiedad `radio`. El atributo `radio` es privado y solo puede ser accedido o modificado mediante los métodos `getRadio()` y `setRadio(double radio)`. El método `setRadio` incluye una validación para asegurar que el radio sea positivo.

```

1 public class Circulo {
2     private double radio;
3
4     public Circulo(double radio) {
5         setRadio(radio); // Se usa el setter para validar el valor ↔
6         inicial
7     }
8
9     // Getter: devuelve el valor del radio
10    public double getRadio() {
11        return radio;
12    }
13
14    // Setter: permite modificar el valor del radio con validación
15    public void setRadio(double radio) {
16        if (radio > 0) {
17            this.radio = radio;
18        }
19    }
20 }

```

Ejemplo 10.3: Uso de getters y setters

Ventajas de usar getters y setters:

- Permiten validar los valores antes de asignarlos a los atributos.
- Facilitan el mantenimiento y la evolución del código, ya que se puede modificar la lógica interna sin afectar a las clases que usan la propiedad.
- Mejoran la seguridad y el control sobre el acceso a los datos internos de la clase.

Uso de propiedades:

```

1 Circulo c = new Circulo(5.0);
2 System.out.println(c.getRadio()); // Imprime 5.0
3 c.setRadio(10.0);                 // Cambia el radio a 10.0
4 System.out.println(c.getRadio()); // Imprime 10.0
5 c.setRadio(-3.0);                 // No cambia el radio, ya que el valor ↔
6                                   es inválido
7 System.out.println(c.getRadio()); // Imprime 10.0

```

Ejemplo 10.4: Acceso a propiedades mediante getters y setters

Consejo

Aunque en Java no existe un concepto de *propiedades* como en otros lenguajes (por ejemplo, C#), el uso de getters y setters es una práctica común para simular este comportamiento y mantener el encapsulamiento.

Ejercicio 10.1

Crea una clase llamada **Persona** con los siguientes atributos privados: **nombre** (**String**), **edad** (**int**) y **email** (**String**). Implementa métodos que expongan las propiedades de la clase, siendo todas de lectura, excepto el email, que deberá ser de lectura y escritura. *Ver solución en la página 102.*

10.4. Creación de métodos

En Java, los métodos son funciones que definen el comportamiento de los objetos. Se pueden clasificar en dos tipos principales:

- **Métodos de instancia:** Operan sobre los atributos de un objeto específico. Para invocarlos, es necesario crear una instancia de la clase.
- **Métodos estáticos:** Se definen con la palabra clave **static** y pertenecen a la clase en sí, no a un objeto concreto. Se pueden llamar sin crear una instancia de la clase.

Los métodos pueden recibir parámetros y devolver valores. Además, pueden tener distintos niveles de visibilidad (**public**, **private**, etc.), lo que permite controlar desde dónde pueden ser accedidos.

Ejemplo:

```

1 public class Calculadora {
2     // Método de instancia: requiere un objeto para ser llamado
3     public int suma(int a, int b) {
4         return a + b;
5     }
6
7     // Método estático: se puede llamar sin crear un objeto
8     public static int resta(int a, int b) {
9         return a - b;
10    }
11 }
```

Ejemplo 10.5: Métodos de instancia y estáticos

Uso de métodos:

```

1 Calculadora calc = new Calculadora();
2 int resultadoSuma = calc.suma(5, 3);           // Llamada a método de instancia
3 int resultadoResta = Calculadora.resta(5, 3); // Llamada a método estático

```

Ejemplo 10.6: Invocación de métodos

Ventajas de los métodos:

- Permiten reutilizar código y organizar la lógica en bloques funcionales.
- Facilitan el mantenimiento y la legibilidad del código.
- Permiten encapsular el comportamiento y proteger los datos internos de la clase.

Ejercicio 10.2

Crea una clase llamada **Rectangulo** que tenga dos atributos privados: **base** y **altura** (de tipo **double**). Implementa los siguientes métodos:

- Un constructor que reciba la base y la altura.
- Métodos **getBase()** y **getAltura()** para acceder a los atributos.
- Un método de instancia **area()** que devuelva el área del rectángulo.
- Un método estático **esCuadrado(Rectangulo r)** que devuelva **true** si la base y la altura son iguales, y **false** en caso contrario.

Prueba la clase creando un objeto y mostrando el área y si es cuadrado.
Ver solución en la página 102.

10.5. Creación de constructores

En Java, un **constructor** es un método especial que se utiliza para inicializar los objetos de una clase. El constructor tiene el mismo nombre que la clase y no tiene tipo de retorno (ni siquiera **void**). Se invoca automáticamente cuando se crea un objeto usando la palabra clave **new**.

Características principales de los constructores:

- Pueden recibir parámetros para inicializar los atributos del objeto.

- Si no se define ningún constructor, Java proporciona uno por defecto (sin parámetros).
- Se pueden definir varios constructores en una clase (sobrecarga de constructores), siempre que tengan diferentes listas de parámetros.
- Los constructores pueden llamar a otros constructores de la misma clase usando `this(...)` o al constructor de la superclase usando `super(...)`.

Ejemplo básico de constructor:

```

1 public class Animal {
2     private String especie;
3     private int edad;
4
5     // Constructor que inicializa los atributos
6     public Animal(String especie, int edad) {
7         this.especie = especie;
8         this.edad = edad;
9     }
10 }

```

Ejemplo 10.7: Constructor en una clase Java

Ejemplo de sobrecarga de constructores:

```

1 public class Animal {
2     private String especie;
3     private int edad;
4
5     // Constructor con parámetros
6     public Animal(String especie, int edad) {
7         this.especie = especie;
8         this.edad = edad;
9     }
10
11     // Constructor sin parámetros (por defecto)
12     public Animal() {
13         this.especie = "Desconocida";
14         this.edad = 0;
15     }
16 }

```

Ejemplo 10.8: Sobrecarga de constructores

Uso de constructores:

```

1 Animal a1 = new Animal("Perro", 3);
2 Animal a2 = new Animal();

```

Ejemplo 10.9: Creación de objetos usando constructores

Ejercicio 10.3

Crea una clase llamada **Libro** con los siguientes atributos privados: **titulo** (String), **autor** (String) y **paginas** (int). Implementa:

- Un constructor que reciba los tres atributos como parámetros.
- Un constructor sin parámetros que asigne valores por defecto.
- Métodos `getTitulo()`, `getAutor()` y `getPaginas()`.

Crea dos objetos **Libro**, uno usando cada constructor, y muestra sus datos por pantalla. *Ver solución en la página 103.*

10.6. Utilización de clases y objetos

Para usar una clase, se crean objetos (instancias) con la palabra clave **new**. Una vez creado el objeto, se puede acceder a sus atributos y métodos utilizando el operador punto (`.`). Cada objeto tiene su propio estado, es decir, sus propios valores para los atributos definidos en la clase.

Ejemplo básico:

```
1 Coche miCoche = new Coche();    // Se crea un objeto de la clase ↪
   Coche
2 miCoche.color = "rojo";        // Se asigna el color
3 miCoche.marca = "Toyota";      // Se asigna la marca
4 miCoche.acelerar(20);          // Se llama al método acelerar
5 System.out.println(miCoche.velocidad); // Se muestra la ↪
   velocidad actual
```

Ejemplo 10.10: Creación y uso de objetos en Java

Acceso a métodos y atributos:

Cuando los atributos son privados, se accede a ellos mediante métodos públicos (**getters** y **setters**), siguiendo el principio de encapsulamiento:

```
1 Circulo c = new Circulo(5.0);
2 System.out.println("Radio: " + c.getRadio());
3 c.setRadio(10.0);
4 System.out.println("Nuevo radio: " + c.getRadio());
```

Ejemplo 10.11: Acceso a atributos privados mediante getters y setters

Creación de múltiples objetos:

Cada objeto creado a partir de una clase es independiente de los demás:

```
1 Coche coche1 = new Coche();
2 Coche coche2 = new Coche();
```

```

3
4 coche1.color = "azul";
5 coche2.color = "verde";
6
7 System.out.println(coche1.color); // azul
8 System.out.println(coche2.color); // verde

```

Ejemplo 10.12: Instanciación de varios objetos

Resumen de pasos para utilizar una clase:

1. Declarar una variable de tipo de la clase.
2. Crear el objeto con **new** y el constructor correspondiente.
3. Acceder a los atributos y métodos mediante el operador punto.

Ejercicio 10.4

Crea una clase llamada **CuentaBancaria** con los atributos privados **titular** (String) y **saldo** (double). Implementa:

- Un constructor que reciba el titular y el saldo inicial.
- Métodos **getTitular()** y **getSaldo()**.
- Un método **depositar(double cantidad)** que aumente el saldo.
- Un método **retirar(double cantidad)** que disminuya el saldo solo si hay suficiente dinero.

Crea un objeto **CuentaBancaria**, realiza un depósito y un retiro, y muestra el saldo final. *Ver solución en la página 104.*

10.7. Utilización de clases heredadas

La herencia es un mecanismo fundamental de la programación orientada a objetos que permite crear nuevas clases basadas en clases existentes. La clase que hereda se denomina **subclase** o *clase derivada*, y la clase de la que se hereda se llama **superclase** o *clase base*. La subclase hereda los atributos y métodos de la superclase, y puede añadir nuevos miembros o redefinir (sobrescribir) los métodos heredados para modificar su comportamiento.

En Java, la herencia se implementa utilizando la palabra clave **extends**. Una subclase puede acceder a los miembros **public** y **protected** de la superclase, pero no a los miembros **private**. Además, mediante la palabra clave **super**, la subclase puede invocar el constructor o métodos de la superclase.

Ventajas de la herencia:

- Permite reutilizar código, evitando duplicidad.
- Facilita la extensión y el mantenimiento del software.
- Permite la creación de jerarquías de clases y la especialización de comportamientos.

Ejemplo básico de herencia:

```

1 public class Vehiculo {
2     protected String marca;
3
4     public Vehiculo(String marca) {
5         this.marca = marca;
6     }
7
8     public void mostrarMarca() {
9         System.out.println("Marca: " + marca);
10    }
11 }
12
13 public class Moto extends Vehiculo {
14     private String tipo;
15
16     public Moto(String marca, String tipo) {
17         super(marca); // Llama al constructor de la superclase
18         this.tipo = tipo;
19     }
20
21     public void mostrarTipo() {
22         System.out.println("Tipo de moto: " + tipo);
23     }
24
25     // Sobrescritura de método
26     @Override
27     public void mostrarMarca() {
28         System.out.println("Marca de la moto: " + marca);
29     }
30 }

```

Ejemplo 10.13: Ejemplo de herencia en Java

En este ejemplo, la clase `Moto` hereda de `Vehiculo`. Puede acceder al atributo protegido `marca` y al método `mostrarMarca()`, además de definir su propio atributo `tipo` y el método `mostrarTipo()`. También sobrescribe el método `mostrarMarca()` para personalizar su comportamiento.

Uso de clases heredadas:

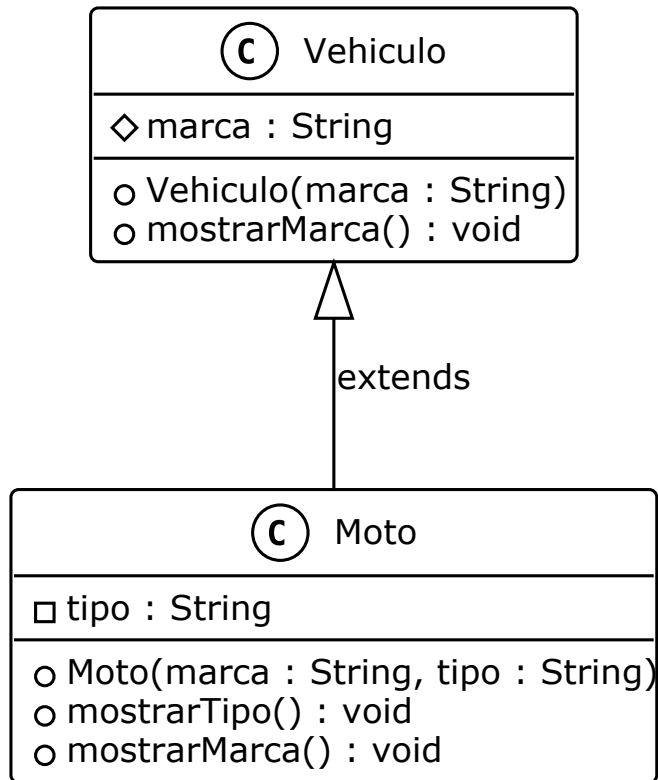


Figura 10.1: Diagrama de clases: herencia de Vehiculo a Moto

```

1 Moto miMoto = new Moto("Yamaha", "Deportiva");
2 miMoto.mostrarMarca(); // Llama al método sobrescrito en Moto
3 miMoto.mostrarTipo(); // Llama al método propio de Moto

```

Ejemplo 10.14: Uso de herencia

Importante

En Java, la herencia es simple, es decir, una clase solo puede heredar de una única superclase directa. Sin embargo, una clase puede implementar múltiples interfaces para lograr un comportamiento similar a la herencia múltiple. Veremos más sobre herencia en el capítulo 16.

Ejercicio 10.5

Crema una clase llamada **Empleado** con los atributos privados `nombre` (`String`) y `salario` (`double`), y los métodos `getNombre()`, `getSalario`

`()` y `mostrarInformacion()` que muestre los datos del empleado. Luego, crea una subclase llamada `Gerente` que herede de `Empleado` y añada el atributo privado `departamento` (`String`) y el método `getDepartamento()`. Sobrescribe el método `mostrarInformacion()` para que también muestre el departamento. Crea un objeto de tipo `Gerente` y muestra su información. *Ver solución en la página 105.*

10.7.1. La clase `Object`

En Java, todas las clases heredan, directa o indirectamente, de la clase `Object`, que se encuentra en el paquete `java.lang`. Esto significa que cualquier clase que declares, aunque no lo especifiques explícitamente, es una subclase de `Object`. La clase `Object` proporciona métodos fundamentales que están disponibles para todos los objetos en Java.

Método	Descripción
<code>toString()</code>	Devuelve una representación en forma de cadena del objeto. Es común sobrescribir este método para mostrar información relevante del objeto.
<code>equals(Object obj)</code>	Compara si dos objetos son iguales. Por defecto, compara referencias, pero se puede sobrescribir para comparar el contenido.
<code>hashCode()</code>	Devuelve un valor hash para el objeto, útil en estructuras como <code>HashMap</code> o <code>HashSet</code> .
<code>clone()</code>	Permite crear una copia del objeto (requiere implementar la interfaz <code>Cloneable</code>).
<code>getClass()</code>	Devuelve el objeto <code>Class</code> que representa la clase del objeto.

Cuadro 10.1: Principales métodos de la clase `Object`

Ejemplo de sobrescritura de `toString()` y `equals()`:

```
1 public class Persona {
2     private String nombre;
3     private int edad;
4
5     public Persona(String nombre, int edad) {
6         this.nombre = nombre;
7         this.edad = edad;
```

```

8   }
9
10  @Override
11  public String toString() {
12      return "Persona[nombre=" + nombre + ", edad=" + edad + "]";
13  }
14
15  @Override
16  public boolean equals(Object obj) {
17      if (this == obj) return true;
18      if (obj == null || getClass() != obj.getClass()) return ↩
19      false;
20      Persona otra = (Persona) obj;
21      return edad == otra.edad && nombre.equals(otra.nombre);
22  }

```

Ejemplo 10.15: Sobrescritura de métodos de Object

Uso:

```

1  Persona p1 = new Persona("Ana", 30);
2  Persona p2 = new Persona("Ana", 30);
3  System.out.println(p1); // Llama a toString()
4  System.out.println(p1.equals(p2)); // true, porque tienen los ↩
   mismos datos

```

Ejercicio 10.6

Crea una clase llamada **Producto** con los atributos privados `codigo` (String) y `precio` (double). Implementa los métodos `getCodigo()` y `getPrecio()`. Sobrescribe los métodos `toString()` y `equals(Object obj)` para que:

- `toString()` devuelva una cadena con el formato `Producto[codigo=..., precio=...]`.
- `equals(Object obj)` devuelva **true** si el código del producto es igual.

Crea dos objetos **Producto** con el mismo código y verifica si son iguales usando `equals()`. *Ver solución en la página 106.*

Resumen

En este capítulo se han presentado los conceptos fundamentales de la programación orientada a objetos relacionados con las clases. Se explicó qué es una

clase y cómo define atributos y métodos para crear objetos. Se abordaron los distintos niveles de visibilidad (`public`, `private`, `protected`) y la importancia del encapsulamiento mediante getters y setters. Se mostró cómo crear métodos de instancia y estáticos, así como el uso y sobrecarga de constructores para inicializar objetos. Además, se explicó cómo utilizar clases y objetos en Java, y se introdujo el concepto de herencia para reutilizar y extender funcionalidades. Estos conocimientos son la base para diseñar programas robustos y estructurados en Java.

Ejercicios de refuerzo

Ejercicio 10.7



El siguiente código viola el principio de encapsulación. Identifica todos los problemas y propón la versión corregida:

```

1 public class CuentaBancaria {
2     public String titular;
3     public double saldo;
4     public String numeroCuenta;
5
6     public CuentaBancaria(String titular, double saldo, ↵
7         String numeroCuenta) {
8         this.titular = titular;
9         this.saldo = saldo;
10        this.numeroCuenta = numeroCuenta;
11    }
12 }
13 // Uso:
14 CuentaBancaria cuenta = new CuentaBancaria("Ana", 1000.0, "↵
15     ES12345");
16 cuenta.saldo = -500.0; // saldo negativo sin validación
17 cuenta.numeroCuenta = "";

```

Ver solución en la página 107.

Ejercicio 10.8



Diseña una jerarquía de clases para representar formas geométricas. La jerarquía debe incluir:

- Una superclase `Forma` con atributos y métodos comunes.
- Subclases `Circulo`, `Rectangulo` y `Triangulo`.

- Cada forma debe poder calcular su área y su perímetro.

Indica los atributos, métodos y relaciones de herencia de cada clase. No es necesario escribir el código completo. *Ver solución en la página 107.*

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 10.1



```

1 // Programa completo: compilar y ejecutar ↩
  independientemente
2 public class Persona {
3     private String nombre;
4     private int edad;
5     private String email;
6
7     public Persona(String nombre, int edad, String email) {
8         this.nombre = nombre;
9         this.edad = edad;
10        this.email = email;
11    }
12
13    public String getNombre() { return nombre; }
14    public int getEdad() { return edad; }
15    public String getEmail() { return email; }
16    public void setEmail(String email) { this.email = email; }
17
18    public static void main(String[] args) {
19        Persona p = new Persona("Ana", 30, "ana@ejemplo.com");
20        System.out.println(p.getNombre() + ", " + p.getEdad() + " ↩
21        , " + p.getEmail());
22        p.setEmail("ana.nueva@ejemplo.com");
23        System.out.println("Email actualizado: " + p.getEmail());
24    }
25 }

```

Solución del ejercicio 10.2



```

1 // Programa completo: compilar y ejecutar ↩
  independientemente
2 public class Rectangulo {
3     private double base;
4     private double altura;
5
6     public Rectangulo(double base, double altura) {
7         this.base = base;
8         this.altura = altura;
9     }
10
11    public double getBase() { return base; }
12    public double getAltura() { return altura; }

```

```

13
14 public double area() {
15     return base * altura;
16 }
17
18 public static boolean esCuadrado(Rectangulo r) {
19     return r.base == r.altura;
20 }
21
22 public static void main(String[] args) {
23     Rectangulo r1 = new Rectangulo(4.0, 6.0);
24     Rectangulo r2 = new Rectangulo(5.0, 5.0);
25     System.out.println("Área r1: " + r1.area());           // ↩
26     System.out.println("¿r1 es cuadrado? " + esCuadrado(r1)); ↩
27     System.out.println("¿r2 es cuadrado? " + esCuadrado(r2)); ↩
28 }
29 }

```

Solución del ejercicio 10.3



```

1 // Programa completo: compilar y ejecutar ↩
  independiente
2 public class Libro {
3     private String titulo;
4     private String autor;
5     private int paginas;
6
7     public Libro(String titulo, String autor, int paginas) {
8         this.titulo = titulo;
9         this.autor = autor;
10        this.paginas = paginas;
11    }
12
13    public Libro() {
14        this("Sin título", "Desconocido", 0);
15    }
16
17    public String getTitulo() { return titulo; }
18    public String getAutor() { return autor; }
19    public int getPaginas() { return paginas; }
20
21    public static void main(String[] args) {
22        Libro l1 = new Libro("El Quijote", "Cervantes", 863);

```

```

23 Libro l2 = new Libro();
24 System.out.println(l1.getTitulo() + " de " + l1.getAutor()
25   + " (" + l1.getPaginas() + " págs.)");
26 System.out.println(l2.getTitulo() + " de " + l2.getAutor()
27   + " (" + l2.getPaginas() + " págs.)");
28 }
29 }

```

Solución del ejercicio 10.4



```

1 // Programa completo: compilar y ejecutar ↵
2   independientemente
3 public class CuentaBancaria {
4     private String titular;
5     private double saldo;
6
7     public CuentaBancaria(String titular, double saldoInicial) ↵
8     {
9         this.titular = titular;
10        this.saldo = saldoInicial;
11    }
12
13    public String getTitular() { return titular; }
14    public double getSaldo() { return saldo; }
15
16    public void depositar(double cantidad) {
17        if (cantidad > 0) {
18            saldo += cantidad;
19        }
20    }
21
22    public void retirar(double cantidad) {
23        if (cantidad > 0 && cantidad <= saldo) {
24            saldo -= cantidad;
25        } else {
26            System.out.println("Saldo insuficiente.");
27        }
28    }
29
30    public static void main(String[] args) {
31        CuentaBancaria cuenta = new CuentaBancaria("Luis", 500.0) ↵
32        ;
33        cuenta.depositar(200.0);
34        System.out.println("Tras depósito: " + cuenta.getSaldo()) ↵
35        ; // 700.0
36        cuenta.retirar(150.0);

```



```

33 System.out.println("Tras retirada: " + cuenta.getSaldo()) ↵
34 ; // 550.0
35 cuenta.retirar(1000.0); // Saldo insuficiente.
36 }

```

Solución del ejercicio 10.5



```

1 // Programa completo: compilar y ejecutar ↵
  independiente
2 public class Empleado {
3     private String nombre;
4     private double salario;
5
6     public Empleado(String nombre, double salario) {
7         this.nombre = nombre;
8         this.salario = salario;
9     }
10
11     public String getNombre() { return nombre; }
12     public double getSalario() { return salario; }
13
14     public void mostrarInformacion() {
15         System.out.println("Nombre: " + nombre + ", Salario: " + ↵
16         salario + " €");
17     }
18 }
19 class Gerente extends Empleado {
20     private String departamento;
21
22     public Gerente(String nombre, double salario, String ↵
23     departamento) {
24         super(nombre, salario);
25         this.departamento = departamento;
26     }
27
28     public String getDepartamento() { return departamento; }
29
30     @Override
31     public void mostrarInformacion() {
32         super.mostrarInformacion();
33         System.out.println("Departamento: " + departamento);
34     }
35
36     public static void main(String[] args) {

```

```

36 Gerente g = new Gerente("Carmen", 45000.0, "Tecnología");
37 g.mostrarInformacion();
38 }
39 }

```

Solución del ejercicio 10.6



```

1 // Programa completo: compilar y ejecutar ↵
  independiente
2 public class Producto {
3     private String codigo;
4     private double precio;
5
6     public Producto(String codigo, double precio) {
7         this.codigo = codigo;
8         this.precio = precio;
9     }
10
11     public String getCodigo() { return codigo; }
12     public double getPrecio() { return precio; }
13
14     @Override
15     public String toString() {
16         return "Producto[codigo=" + codigo + ", precio=" + precio↵
17             + "]";
18     }
19
20     @Override
21     public boolean equals(Object obj) {
22         if (this == obj) { return true; }
23         if (obj == null || getClass() != obj.getClass()) { return↵
24             false; }
25         Producto otro = (Producto) obj;
26         return codigo.equals(otro.codigo);
27     }
28
29     @Override
30     public int hashCode() {
31         return codigo.hashCode();
32     }
33
34     public static void main(String[] args) {
35         Producto p1 = new Producto("A001", 19.99);
36         Producto p2 = new Producto("A001", 25.00);
37         System.out.println(p1);           // Producto[codigo=A001↵
38         , precio=19.99]

```

```

36 | System.out.println("¿Iguales? " + p1.equals(p2)); // true →
    | (mismo código)
37 | }
38 | }

```

Solución del ejercicio 10.7



El problema principal es que los atributos son **public**, lo que permite modificarlos sin ninguna validación desde cualquier clase. La versión corregida aplica encapsulación: los tres atributos pasan a ser **private** (**numeroCuenta** además **final**), se añaden getters públicos (**getTitular()**, **getSaldo()**, **getNumeroCuenta()**) y se sustituyen los accesos directos por métodos con validación: **ingresarDinero(double)** que comprueba que la cantidad sea positiva, y **retirarDinero(double)** que comprueba que no se supere el saldo. Con atributos **private** y métodos de acceso controlado, cualquier modificación del estado pasa necesariamente por lógica de validación.

Solución del ejercicio 10.8



La jerarquía básica sería:

- Forma (superclase abstracta): atributo **color**; métodos abstractos **calcularArea()** y **calcularPerimetro()**; método concreto **getColor()**.
- Circulo **extends** Forma: atributo **radio**; implementa **calcularArea() = πr^2** y **calcularPerimetro() = $2\pi r$** .
- Rectangulo **extends** Forma: atributos **ancho** y **alto**; implementa ambos métodos.
- Triangulo **extends** Forma: atributos **base** y **altura** (más los tres lados para el perímetro); implementa ambos métodos.

Gracias al polimorfismo, se puede escribir **Forma f = new Circulo(5)** y llamar a **f.calcularArea()** sin conocer el tipo concreto en tiempo de compilación.

Depuración y pruebas unitarias

La depuración y las pruebas unitarias son etapas fundamentales en el desarrollo de software, ya que permiten identificar y corregir errores, así como garantizar que el código funcione según lo esperado. En este capítulo se abordarán conceptos y herramientas clave para mejorar la calidad y confiabilidad de las aplicaciones.

1. **Aserciones:** Son instrucciones que permiten verificar que ciertas condiciones se cumplan durante la ejecución del programa. Ayudan a detectar errores lógicos y facilitan el mantenimiento del código.
2. **Prueba, depuración y documentación de la aplicación:** Se explorarán técnicas para probar el funcionamiento del software, métodos para depurar errores y buenas prácticas para documentar el proceso, asegurando así un desarrollo más robusto y eficiente.

11.1. Aserciones en Java

Las aserciones en Java permiten comprobar supuestos en el código durante la ejecución. Si una aserción falla, el programa lanza un error `AssertionError`.

```
1 public class EjemploAsercion {  
2     public static void main(String[] args) {  
3         int edad = -5;  
4         assert edad >= 0 : "La edad no puede ser negativa";  
5         System.out.println("Edad: " + edad);  
6     }  
7 }
```

Ejemplo 11.1: Ejemplo de aserción en Java

En este ejemplo, si `edad` es negativo, la aserción fallará y mostrará el mensaje correspondiente.

Importante

Las aserciones se incluyen en Java desde la versión 1.4, lo cual significa que código anterior a esa versión podía usar la palabra clave **assert** sin problemas, pero no se ejecutaría como aserciones a menos que se habiliten explícitamente.

Para habilitar las aserciones, se debe ejecutar el programa con la opción **-ea** (enable assertions) en la línea de comandos: `java -ea`

EjemploAsercion

Si se usa JShell, se puede habilitar con: `jshell -R -ea`

Si se ejecuta sin esta opción, las aserciones no se evaluarán y el programa funcionará normalmente.

Aparte de la palabra clave **assert**, es posible usar librerías que ofrezcan funcionalidades similares, como **AssertJ**, que permiten realizar aserciones más complejas y legibles. Librerías de tests como JUnit también utilizan aserciones para validar el comportamiento del código durante las pruebas unitarias, e incluyen sus propios métodos de aserción.

Ejercicio 11.1

Crea un programa que calcule el factorial de un número entero positivo. Utiliza aserciones para verificar que el número es no negativo antes de calcular el factorial. *Ver solución en la página 116.*

11.2. Pruebas unitarias con JUnit

JUnit es un marco de pruebas muy utilizado en Java para escribir y ejecutar pruebas unitarias. Permite verificar que los métodos de una clase funcionen correctamente.

```
1 public class Calculadora {
2     public int sumar(int a, int b) {
3         return a + b;
4     }
5 }
```

Ejemplo 11.2: Clase Calculadora

```
1 import static org.junit.Assert.assertEquals;
2 import org.junit.Test;
3
4 public class CalculadoraTest {
5     @Test
```

```

6 public void testSuma() {
7     Calculadora calc = new Calculadora();
8     int resultado = calc.sumar(2, 3);
9     assertEquals(5, resultado);
10 }
11 }

```

Ejemplo 11.3: Ejemplo básico de prueba unitaria con JUnit

La clase `CalculadoraTest` contiene un método de prueba `testSuma` que verifica que el método `sumar` de la clase `Calculadora` devuelve el resultado esperado. El método `assertEquals` se utiliza para comparar el valor esperado con el valor real. Este método está anotado con `@Test`, lo que indica a JUnit que es un método de prueba. Una anotación permite añadir metadatos a las clases y métodos, y en este caso, indica que el método debe ser ejecutado como una prueba al ejecutar los tests de JUnit.

Consejo



Para ejecutar las pruebas, se recomienda usar un entorno de desarrollo (IDE), tal como *Eclipse* o *IntelliJ IDEA*, o usar la línea de comandos con *Maven* o *Gradle*. También es posible automatizar la ejecución de pruebas unitarias con herramientas de integración continua como Jenkins o GitHub Actions.

Ejercicio 11.2



Crea una clase `Calculadora` con métodos para sumar, restar, multiplicar y dividir. Escribe pruebas unitarias para cada uno de estos métodos utilizando JUnit. *Ver solución en la página 116.*

Método	Descripción
<code>assertEquals(expected, actual)</code>	Verifica que dos valores sean iguales.
<code>assertNotEquals(expected, actual)</code>	Verifica que dos valores sean diferentes.
<code>assertTrue(condition)</code>	Verifica que una condición sea verdadera.
<code>assertFalse(condition)</code>	Verifica que una condición sea falsa.
<code>assertNull(object)</code>	Verifica que un objeto sea <code>null</code> .
<code>assertNotNull(object)</code>	Verifica que un objeto no sea <code>null</code> .

Método	Descripción
<code>assertSame(expected, actual)</code>	Verifica que dos referencias apunten al mismo objeto.
<code>assertNotSame(expected, actual)</code>	Verifica que dos referencias no apunten al mismo objeto.
<code>fail()</code>	Falla la prueba de manera explícita.

Cuadro 11.1: Principales métodos de aserción en JUnit

11.3. Depuración en Java

La depuración permite ejecutar el programa paso a paso, inspeccionar variables y encontrar errores. Los entornos de desarrollo integrados (IDE) como *Eclipse* o *IntelliJ IDEA* ofrecen herramientas visuales para depurar.

Es común que cada entorno de desarrollo ofrezca características específicas para la depuración, como:

- **Puntos de interrupción (breakpoints):** Permiten detener la ejecución en una línea específica.
- **Inspección de variables:** Se pueden examinar los valores de las variables en tiempo real.
- **Ejecución paso a paso:** Permite avanzar instrucción por instrucción para analizar el flujo del programa.

Ejercicio 11.3



Utiliza el depurador de tu IDE para analizar el siguiente código y comprender cómo funciona la ejecución paso a paso. Identifica el valor de las variables en cada paso:

```
1 public class PrimeraLetraPalabra {
2     public static void main(String[] args) {
3         String texto = "Depuración y pruebas unitarias";
4         int numeroPalabras = 0;
5         for (int i = 0; i < texto.length(); i++) {
6             if (i == 0 || texto.charAt(i - 1) == ' ') {
7                 System.out.println("Letra inicial " + texto.charAt(i) + " en la posición " + i);
8                 numeroPalabras++;
9             }
10        }
```



```

10     }
11     System.out.println("Número total de palabras: " + ↵
    numeroPalabras);
12 }
13 }

```

Ejemplo 11.4: Bucle de ejemplo para depuración

11.4. Buenas prácticas de documentación

Documentar el proceso de depuración y pruebas es fundamental para el mantenimiento del software. En Java, se recomienda usar comentarios y la herramienta *Javadoc* para generar documentación automática.

```

1 /**
2  * Clase que realiza operaciones matemáticas básicas.
3  */
4 public class Calculadora {
5     /**
6      * Suma dos números enteros.
7      * @param a Primer sumando
8      * @param b Segundo sumando
9      * @return La suma de a y b
10     */
11     public int sumar(int a, int b) {
12         return a + b;
13     }
14 }

```

Ejemplo 11.5: Ejemplo de documentación con Javadoc

Importante

Es recomendable seguir un estilo de codificación consistente y documentar adecuadamente las clases, métodos y variables. Esto facilita la comprensión del código y su mantenimiento a largo plazo. Además, se pueden utilizar herramientas como *Checkstyle* o *SonarQube* para analizar la calidad del código y asegurar que se sigan las mejores prácticas.

Ejercicio 11.4

Añade comentarios y documentación *Javadoc* a la clase `Calculadora` que creaste en el ejercicio 11.2. Asegúrate de documentar cada método y sus

parámetros.

Resumen

En este capítulo se abordaron conceptos esenciales para mejorar la calidad del software: el uso de aserciones para verificar supuestos en el código, la implementación de pruebas unitarias con JUnit para asegurar el correcto funcionamiento de los métodos, las herramientas de depuración disponibles en los entornos de desarrollo y la importancia de documentar adecuadamente el proceso. Estas prácticas permiten detectar y corregir errores de manera eficiente, facilitando el mantenimiento y la confiabilidad de las aplicaciones.

Ejercicios de refuerzo

Ejercicio 11.5



Clasifica cada uno de los siguientes errores como **error de compilación**, **error en tiempo de ejecución** o **error lógico**. Para cada uno, explica cómo detectarlo y corregirlo.

1. `int x = "hola";` — asignar una cadena a un entero.
2. `int media = suma / cantidad;` cuando `cantidad` vale 0.
3. Un método que calcula el promedio de `n` notas dividiendo la suma entre `n + 1` en lugar de entre `n`.
4. `String s = null; s.length();` — llamar a un método sobre una referencia nula.
5. Un bucle `for` que itera de `i=1` hasta `i<=10` pero solo debería iterar hasta `i<10`.

Ver solución en la página 117.

Ejercicio 11.6



El siguiente programa contiene un error lógico sutil. Describe la estrategia de depuración que seguirías: qué puntos de ruptura pondrías, qué variables inspeccionarías y cómo confirmarías que el error está corregido.

```
1 public static int contarPares(int desde, int hasta) {  
2     int contador = 0;  
3     for (int i = desde; i <= hasta; i++) {  
4         if (i % 2 == 1) { // error aqui  
5             contador++;  
6         }  
7     }  
8     return contador;  
9 }
```

Ver solución en la página 118.

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 11.1



```

1 // Fragmento: incluir dentro de una clase
2 // Ejecutar con: java -ea NombreClase (para activar ↪
  aserciones)
3 public static long factorial(int n) {
4     assert n >= 0 : "El número debe ser no negativo";
5     if (n == 0) {
6         return 1;
7     }
8     return n * factorial(n - 1);
9 }
10
11 // System.out.println(factorial(5)); // 120
12 // System.out.println(factorial(-1)); // AssertionError si ↪
  se ejecuta con -ea

```

Solución del ejercicio 11.2



Clase Calculadora:

```

1 // Programa completo: clase Calculadora para usar con JUnit
2 public class Calculadora {
3     public int sumar(int a, int b) {
4         return a + b;
5     }
6
7     public int restar(int a, int b) {
8         return a - b;
9     }
10
11     public int multiplicar(int a, int b) {
12         return a * b;
13     }
14
15     public double dividir(int a, int b) {
16         if (b == 0) {
17             throw new ArithmeticException("División por cero");
18         }
19         return (double) a / b;
20     }
21 }

```

Pruebas unitarias con JUnit 5:

```

1 // Fragmento: clase de pruebas JUnit 5 para Calculadora

```

```

2 import org.junit.jupiter.api.Test;
3 import static org.junit.jupiter.api.Assertions.assertEquals ↵
4 ;
5 import static org.junit.jupiter.api.Assertions.assertThrows ↵
6 ;
7
8 class CalculadoraTest {
9     private final Calculadora calc = new Calculadora();
10
11     @Test
12     void sumarDosPositivos() {
13         assertEquals(7, calc.sumar(3, 4));
14     }
15
16     @Test
17     void restarDosNumeros() {
18         assertEquals(1, calc.restar(5, 4));
19     }
20
21     @Test
22     void multiplicarDosNumeros() {
23         assertEquals(12, calc.multiplicar(3, 4));
24     }
25
26     @Test
27     void dividirDosNumeros() {
28         assertEquals(2.5, calc.dividir(5, 2));
29     }
30
31     @Test
32     void dividirPorCeroLanzaExcepcion() {
33         assertThrows(ArithmeticException.class, () -> calc. ↵
34             dividir(5, 0));
35     }
36 }

```

Solución del ejercicio 11.5



1. **Error de compilación:** el compilador detecta la incompatibilidad de tipos y el programa ni siquiera llega a generarse. Se corrige cambiando el tipo a `String` o el valor a un entero.
2. **Error en tiempo de ejecución:** el código compila, pero la división entera entre cero interrumpe el programa al ejecutarse. Se previene comprobando que `cantidad != 0` antes de dividir.

3. **Error lógico:** el programa compila y termina sin interrumpirse, pero produce un resultado incorrecto. Solo se detecta con pruebas unitarias o comparando con un valor esperado calculado a mano.
4. **Error en tiempo de ejecución:** al no apuntar `s` a ningún objeto, la llamada al método interrumpe el programa. Se previene comprobando que `s != null` antes de invocar métodos.
5. **Error lógico:** el bucle ejecuta una iteración de más. Se detecta con una prueba que verifique el número de iteraciones o el resultado final.

Fíjate en la diferencia práctica: los errores de compilación te los señala el compilador, los de ejecución te los señala el propio programa al detenerse, y los lógicos no te los señala nadie. Por eso los últimos son los que más justifican escribir pruebas.

Solución del ejercicio 11.6



El error es `i % 2 == 1`: esa condición selecciona los números **impares**, no los pares. El método compila y se ejecuta sin fallar, pero devuelve siempre un resultado incorrecto; es por tanto un error lógico, el tipo más difícil de detectar porque nada avisa de que algo va mal. La estrategia de depuración sería:

1. Escribir primero un caso de prueba con el resultado esperado calculado a mano: para `contarPares(1, 10)` la respuesta correcta es 5, y el método devuelve 5 también, así que este caso **no** destapa el fallo.
2. Elegir un caso que distinga ambas interpretaciones: `contarPares(1, 3)` debe devolver 1 (solo el 2), pero devuelve 2 (el 1 y el 3).
3. Colocar un punto de ruptura dentro del `if` e inspeccionar el valor de `i` cada vez que se detiene. Al ver que se detiene con `i` valiendo 1 y 3, queda claro que la condición está invertida.
4. Corregir la condición a `i % 2 == 0` y volver a ejecutar ambas pruebas.

Este ejercicio ilustra por qué un único caso de prueba no basta: un caso simétrico puede dar el resultado correcto por casualidad y ocultar el error.

Aplicación de las estructuras de almacenamiento

En este capítulo se abordarán los conceptos fundamentales relacionados con las estructuras de almacenamiento en programación. Estas estructuras permiten organizar y manipular datos de manera eficiente, facilitando la resolución de problemas complejos y el desarrollo de aplicaciones robustas. A continuación, se presentan los principales temas que se tratarán:

1. **Estructuras estáticas y dinámicas:** Se explicará la diferencia entre estructuras de almacenamiento cuyo tamaño es fijo en tiempo de compilación (estáticas) y aquellas que pueden crecer o reducirse durante la ejecución del programa (dinámicas).
2. **Creación de arreglos (arrays):** Se mostrará cómo declarar, inicializar y utilizar arreglos para almacenar colecciones de datos homogéneos.
3. **Matrices multidimensionales (arrays multidimensionales):** Se abordará el uso de arreglos con más de una dimensión, útiles para representar tablas, matrices matemáticas y otras estructuras complejas.
4. **Genericidad:** Se introducirá el concepto de estructuras genéricas, que permiten definir colecciones de datos independientes del tipo de los elementos que almacenan.
5. **Cadenas de caracteres. Expresiones regulares:** Se explicará cómo manipular texto mediante cadenas de caracteres y cómo utilizar expresiones regulares para buscar y procesar patrones en cadenas.
6. **Colecciones: Listas, Conjuntos y Diccionarios:** Se presentarán las principales colecciones dinámicas, sus características y casos de uso: listas (secuencias ordenadas), conjuntos (colecciones sin elementos repetidos) y diccionarios (pares clave-valor).

7. **Operaciones agregadas: filtrado, reducción y recolección:** Se describirán operaciones comunes sobre colecciones, como filtrar elementos, reducirlos a un solo valor o recolectar resultados en nuevas estructuras.

Cada uno de estos puntos será explicado en detalle, mostrando ejemplos prácticos y aplicaciones.

12.1. Estructuras estáticas y dinámicas

En Java, las estructuras de almacenamiento pueden ser estáticas (como los *arrays*) o dinámicas (como las colecciones de la biblioteca estándar). Las estructuras estáticas tienen un tamaño fijo, mientras que las dinámicas pueden crecer o reducirse durante la ejecución.

12.1.1. Estructuras estáticas

Las estructuras estáticas, como los arreglos, reservan una cantidad fija de memoria al momento de su creación. Esto significa que no es posible cambiar su tamaño durante la ejecución del programa. Son útiles cuando se conoce de antemano la cantidad de elementos que se van a almacenar y se requiere un acceso rápido a cada elemento mediante un índice. Sin embargo, su principal limitación es la falta de flexibilidad para adaptarse a cambios en la cantidad de datos.

Más adelante, veremos cómo se crean y utilizan los arreglos en Java, ya sean de una dimensión o multidimensionales.

12.1.2. Estructuras dinámicas

Las estructuras dinámicas, a diferencia de las estáticas, permiten modificar su tamaño durante la ejecución del programa. Esto se logra mediante el uso de clases como `ArrayList`, `LinkedList`, `HashSet` y `HashMap`, que forman parte de la biblioteca estándar de Java. Estas estructuras son ideales cuando no se conoce de antemano la cantidad de elementos a almacenar o cuando se requiere agregar y eliminar elementos de manera flexible. Aunque suelen consumir más memoria y pueden ser ligeramente más lentas que los arreglos, ofrecen una gran versatilidad y facilitan la gestión de datos en aplicaciones dinámicas.

```
1 import java.util.ArrayList;
2
3 ArrayList<String> nombres = new ArrayList<>(); // La lista está ↪
   vacía inicialmente
4 nombres.add("Ana"); // Agrega un elemento
```



```

5 nombres.add("Juan"); // Agrega otro elemento
6 nombres.add("Luis"); // Agrega otro elemento
7 System.out.println(nombres.get(1)); // Imprime "Juan"
8 nombres.remove(0); // Elimina el primer elemento ("Ana")
9 System.out.println(nombres.size()); // Imprime 2, ya que quedan ↩
  dos elementos

```

Veremos más adelante en este capítulo cómo funcionan estas estructuras y cómo se pueden utilizar para resolver problemas comunes en programación.

12.2. Creación de arreglos (arrays)

Los arreglos son estructuras de datos que permiten almacenar múltiples elementos del mismo tipo en una sola variable. En Java, los arreglos son objetos y se pueden crear de forma estática o dinámica.

```

1 int[] numeros = new int[5]; // Array de 5 enteros
2 numeros[0] = 10;
3 numeros[1] = 20;
4 System.out.println(numeros[0]); // Imprime 10

```

En Java, los *arrays* se crean usando la palabra clave **new** seguida del tipo de datos y el tamaño del arreglo entre corchetes. Los elementos se acceden mediante índices, comenzando desde 0. No es posible cambiar el tamaño de un *array* una vez creado, lo que puede ser una limitación en algunos casos. Igualmente, si se intenta acceder a un índice fuera de los límites del arreglo, se lanzará una excepción `ArrayIndexOutOfBoundsException`. Por defecto, el *array* se inicializa con valores predeterminados: 0 para enteros, **false** para booleanos y **null** para objetos.

Importante

En Java, y la mayoría de lenguajes de programación, los arreglos comienzan en el número 0, debido a que representan posiciones relativas en la memoria. Por ejemplo, el primer elemento de un arreglo se encuentra en la posición 0, el segundo en la posición 1, y así sucesivamente. Esto significa que si un arreglo tiene 5 elementos, sus índices van del 0 al 4. Si intentas acceder al índice 5, obtendrás un error de índice fuera de rango.

En el caso del lenguaje de programación C, el acceso a los elementos de un arreglo se realiza mediante punteros, lo que permite una mayor flexibilidad, pero también puede llevar a errores si no se maneja correctamente. El primer elemento de un arreglo se encuentra en la dirección de memoria base del arreglo, y los siguientes elementos se encuentran en direcciones consecutivas. Por ejemplo, si tienes un arreglo de enteros de 5 elementos,

el primer elemento estará en la dirección base, el segundo en la dirección base más el tamaño de un entero, y así sucesivamente.

Es posible crear arreglos ya inicializados con valores específicos, lo cual simplifica la declaración y asignación de valores en una sola línea:

```
1 String[] diasSemana = {"Lunes", "Martes", "Miércoles", "Jueves", ↵  
    "Viernes", "Sábado", "Domingo"}; // Array de String con los ↵  
    días de la semana  
2 System.out.println(diasSemana[0]); // Imprime "Lunes"  
3 System.out.println(diasSemana.length); // Imprime 7, el tamaño ↵  
    del array
```

Ejercicio 12.1



Crea un arreglo de enteros de tamaño 10, inicialízalo con valores del 1 al 10 y muestra por pantalla el contenido del arreglo.

*Pista: Puedes usar un bucle **for** para recorrer el arreglo y mostrar sus elementos. Ver solución en la página 133.*

Consejo



El método `main`, que es el punto de entrada de un programa Java, también puede recibir un arreglo de cadenas como argumento. Esto permite pasar parámetros al ejecutar el programa desde la línea de comandos.

Por ejemplo, si ejecutas el programa con `java MiPrograma arg1 arg2`, el arreglo `args` contendrá `["arg1", "arg2"]`.

Ejercicio 12.2



Crea un programa que reciba como argumentos una serie de números enteros, y que imprima por pantalla la suma de todos ellos.

Pista: Utiliza `Integer.parseInt(texto)`, para convertir cada valor a un entero. Ver solución en la página 133.

12.3. Matrices multidimensionales (*arrays multidimensionales*)

Como los arreglos son objetos en Java, es posible crear arreglos de arreglos, lo que permite crear estructuras multidimensionales. Los arreglos multidimensionales son útiles para representar datos en forma de tablas, matrices o cualquier

otra estructura que requiera más de una dimensión.

```
1 int[][] matriz = {
2     {1, 2, 3},
3     {4, 5, 6}
4 };
5 System.out.println(matriz[1][2]); // Imprime 6
```

En el ejemplo podemos ver que el tipo del arreglo es `int[][]`, lo que indica que es un arreglo de arreglos de enteros. Cada subarreglo representa una fila de la matriz. Para acceder a un elemento específico, se utilizan dos índices: el primero para la fila y el segundo para la columna.

También es posible crear arreglos multidimensionales de forma dinámica, especificando el tamaño de cada dimensión:

```
1 int altura = 5;
2 int[][] trianguloDePascal = new int[altura][];
3 for (int i = 0; i < altura; i++) {
4     trianguloDePascal[i] = new int[i + 1];
5     trianguloDePascal[i][0] = 1;
6     for (int j = 1; j < i; j++) {
7         trianguloDePascal[i][j] = trianguloDePascal[i - 1][j - 1] + ↩
8         trianguloDePascal[i - 1][j];
9     }
10    trianguloDePascal[i][i] = 1;
11 }
```

Ejercicio 12.3



Completa el ejemplo anterior, añadiendo un bucle que imprima el contenido de la matriz `trianguloDePascal` por pantalla, mostrando cada fila en una línea separada.

Pista: Utiliza un bucle anidado para recorrer las filas y columnas de la matriz. Pista: Puedes usar `System.out.print(valor)` para un valor sin generar un salto de línea. Ver solución en la página 133.

Consejo



Para imprimir el contenido de un arreglo multidimensional de forma más legible, puedes utilizar la clase `Arrays` de Java, que proporciona un método `toString()` para arreglos unidimensionales y `deepToString()` para arreglos multidimensionales.

```
1 import java.util.Arrays;
2
3 int[] arreglo = {1, 2, 3, 4, 5};
4 System.out.println(Arrays.toString(arreglo)); // Imprime ↩
```

```

    [1, 2, 3, 4, 5]
5
6 int[][] matriz = {
7     {1, 2, 3},
8     {4, 5, 6}
9 };
10 System.out.println(Arrays.deepToString(matriz)); // Imprime →
    [[1, 2, 3], [4, 5, 6]]

```

12.4. Genericidad

Java permite definir clases y métodos genéricos para trabajar con diferentes tipos de datos sin duplicar código. Esto permite que una misma clase o método pueda operar con distintos tipos de datos, aumentando la reutilización del código y la flexibilidad.

```

1 public class Caja<T> {
2     private T contenido;
3     public void guardar(T valor) { contenido = valor; }
4     public T obtener() { return contenido; }
5 }
6
7 // Uso:
8 Caja<Integer> cajaEntero = new Caja<Integer>();
9 // Caja que almacena enteros.
10 // El tipo T se sustituye por Integer.
11 // Se puede obviar el tipo al instanciar, ya que Java lo infiere →
12 // automáticamente. P.ej:
13 // Caja<String> cajaString = new Caja<>();
14 cajaEntero.guardar(100);
15 System.out.println(cajaEntero.obtener()); // Imprime 100

```

Esta flexibilidad permite que reutilicemos el mismo código para diferentes tipos de datos, sin necesidad de crear múltiples versiones de la misma clase o método. La *genericidad* es una característica poderosa que mejora la legibilidad y mantenibilidad del código.

Ejercicio 12.4



Crea una clase genérica llamada **Cons** que almacene dos valores de cualquier tipo. Implementa métodos para establecer y obtener los valores, y muestra un ejemplo de uso con diferentes tipos de datos.

*Pista: Utiliza dos parámetros genéricos para los dos valores, p.ej. **Cons**<*

T, U>.

12.5. Cadenas de caracteres. Expresiones regulares

A lo largo de este libro hemos visto como usar cadenas de texto, o *string*, para representar y manipular texto. Sin embargo, Java ofrece una amplia variedad de métodos y clases para trabajar con cadenas de caracteres de manera más avanzada. En este apartado, exploraremos algunas de las características más útiles relacionadas con las cadenas de caracteres y las expresiones regulares.

12.5.1. Manipulación de cadenas

Las cadenas de caracteres en Java son objetos de la clase `String`, que proporciona una gran cantidad de métodos para manipular texto. Algunos de los métodos más comunes incluyen:

- `length()`: Devuelve la longitud de la cadena.
- `charAt(int index)`: Devuelve el carácter en la posición especificada.
- `substring(int beginIndex, int endIndex)`: Devuelve una subcadena entre los índices especificados.
- `toLowerCase()` y `toUpperCase()`: Convierte la cadena a minúsculas o mayúsculas.
- `trim()`: Elimina los espacios en blanco al principio y al final de la cadena.
- `replace(CharSequence target, CharSequence replacement)`: Reemplaza todas las ocurrencias de una subcadena por otra.
- `split(String regex)`: Divide la cadena en un arreglo de subcadenas utilizando una expresión regular como delimitador.

Como la clase `String` es inmutable, cada vez que se realiza una operación que modifica la cadena, se crea un nuevo objeto `String`. Esto significa que las operaciones de concatenación y modificación pueden ser costosas en términos de rendimiento si se realizan repetidamente. Para operaciones más eficientes, se recomienda utilizar la clase `StringBuilder`.

Ejercicio 12.5

Crea un programa que reciba una matrícula de coche, con el formato 1234-ABC, y realice las siguientes operaciones:

1. Comprueba si la matrícula es válida (4 dígitos seguidos de un guion y 3 letras).
2. Convierte la matrícula a mayúsculas.
3. Reemplaza el guion por un espacio.
4. Imprime la matrícula resultante.

12.5.2. Expresiones regulares

Las expresiones regulares son patrones que permiten buscar, validar y manipular cadenas de texto de manera flexible y potente. En Java, se utilizan principalmente a través de las clases `Pattern` y `Matcher` del paquete `java.util.regex`.

Por ejemplo, para comprobar si una cadena cumple con un formato específico, como una matrícula de coche, se puede usar una expresión regular:

```
1 import java.util.regex.Pattern;
2 import java.util.regex.Matcher;
3
4 String matricula = "1234-ABC";
5 Pattern patron = Pattern.compile("\\d{4}-[A-Z]{3}");
6 Matcher matcher = patron.matcher(matricula);
7
8 if (matcher.matches()) {
9     System.out.println("La matrícula es válida");
10 } else {
11     System.out.println("La matrícula no es válida");
12 }
```

En este ejemplo, el patrón `"\\d{4}-[A-Z]{3}"` significa: cuatro dígitos, un guion y tres letras mayúsculas. El método `matches()` comprueba si toda la cadena cumple el patrón.

Las expresiones regulares también se pueden usar para buscar y reemplazar partes de una cadena:

```
1 String texto = "El número es 1234-ABC";
2 String resultado = texto.replaceAll("\\d{4}-[A-Z]{3}", "↪
3     MATRÍCULA");
4 System.out.println(resultado); // Imprime: El número es ↪
5     MATRÍCULA
```

Las expresiones regulares son herramientas muy potentes para validar datos de entrada, extraer información y transformar texto en aplicaciones Java.

Consejo



Un objeto de la clase `Pattern` representa una máquina de estados finitos que reconoce cadenas de texto que coinciden con un patrón específico. Por otro lado, un objeto de la clase `Matcher` es responsable de realizar la búsqueda y coincidencia de patrones en una cadena de texto utilizando el patrón definido por el objeto `Pattern`.

Debido a esto, es recomendable compilar el patrón una sola vez, usando un objeto `static final`. Esto mejora el rendimiento, especialmente si el patrón se va a utilizar repetidamente, ya que evita recompilar la expresión regular cada vez que se necesite.

Por ejemplo:

```

1  public class ValidadorMatricula {
2      private static final Pattern PATRON_MATRICULA = Pattern
        .compile("\\d{4}-[A-Z]{3}");
3
4      public static boolean esValida(String matricula) {
5          Matcher matcher = PATRON_MATRICULA.matcher(matricula)
6          ;
7          return matcher.matches();
8      }
9
10     // Uso:
11     // if (ValidadorMatricula.esValida(matricula)) {
12     //     System.out.println("La matrícula es válida");
13     // }
14

```

Ejercicio 12.6



Visita la web <https://regexr.com/> y prueba diferentes expresiones regulares para validar formatos de texto. Crea una expresión regular que valide direcciones de correo electrónico y otra que valide números de teléfono en formato español (6XXXXXXXX o 9XXXXXXXX).

12.5.3. La clase `StringBuilder`

La clase `StringBuilder` permite construir y modificar cadenas de texto de manera eficiente, especialmente cuando se realizan muchas operaciones de con-

catenación. A diferencia de la clase `String`, que es inmutable, `StringBuilder` modifica la cadena original sin crear nuevos objetos en cada operación.

```
1 StringBuilder sb = new StringBuilder();
2 sb.append("Hola");
3 sb.append(" ");
4 sb.append("mundo");
5 System.out.println(sb.toString()); // Imprime "Hola mundo"
```

`StringBuilder` proporciona métodos como `append()`, `insert()`, `delete()`, `reverse()` y otros, que permiten manipular el contenido de la cadena de forma eficiente. Al finalizar, se puede obtener el resultado como un objeto `String` usando el método `toString()`.

Consejo



Si necesitas modificar cadenas de texto dentro de un bucle, utiliza `StringBuilder` para mejorar el rendimiento y evitar la creación innecesaria de objetos.

Ejercicio 12.7



Crea un programa que construya una frase a partir de palabras pasadas por argumento al método `main`. Utiliza `StringBuilder` para concatenar las palabras y muestra la frase resultante.

Si la entrada es "Hola", "mundo", "Java", el programa debería imprimir "Hola mundo Java.". Ten en cuenta que debes añadir un espacio entre cada palabra y un punto al final de la frase. *Ver solución en la página 134.*

12.6. Colecciones

Java proporciona un rico conjunto de estructuras de datos dinámicas a través del *Collections Framework*: listas, conjuntos, mapas, pilas y colas. Estas estructuras se estudian en detalle en el capítulo 14.

Resumen

En este capítulo hemos explorado las principales estructuras de almacenamiento en programación, desde los arreglos estáticos y dinámicos hasta las colecciones más avanzadas como listas, conjuntos, diccionarios, colas y pilas. Aprendiste a crear y manipular arreglos unidimensionales y multidimensionales, a utilizar la genericidad para escribir código reutilizable, y a trabajar con cadenas de caracteres y expresiones regulares para procesar texto de manera eficiente.

También conociste las colecciones de la biblioteca estándar de Java, sus características y casos de uso, así como el uso de iteradores para recorrer y modificar colecciones de forma segura. Finalmente, se introdujeron las operaciones agregadas y los *streams*, que permiten procesar colecciones de manera funcional y declarativa.

El dominio de estas estructuras y técnicas es fundamental para desarrollar programas robustos, eficientes y fáciles de mantener.

Cuadro 12.1: Principales métodos de la interfaz `Collection<E>`

Método	Descripción
<code>boolean add(E e)</code>	Añade el elemento especificado a la colección. Devuelve <code>true</code> si la colección cambió como resultado.
<code>boolean addAll(Collection<? extends E> c)</code>	Añade todos los elementos de la colección especificada a esta colección.
<code>void clear()</code>	Elimina todos los elementos de la colección.
<code>boolean contains(Object o)</code>	Devuelve <code>true</code> si la colección contiene el elemento especificado.
<code>boolean containsAll(Collection<?> c)</code>	Devuelve <code>true</code> si la colección contiene todos los elementos de la colección especificada.
<code>boolean isEmpty()</code>	Devuelve <code>true</code> si la colección no contiene elementos.
<code>Iterator<E> iterator()</code>	Devuelve un iterador sobre los elementos de la colección.
<code>boolean remove(Object o)</code>	Elimina una sola instancia del elemento especificado, si está presente.
<code>boolean removeAll(Collection<?> c)</code>	Elimina de la colección todos los elementos que están también en la colección especificada.
<code>boolean retainAll(Collection<?> c)</code>	Conserva solo los elementos que están también en la colección especificada.
<code>int size()</code>	Devuelve el número de elementos en la colección.
<code>Object[] toArray()</code>	Devuelve un arreglo que contiene todos los elementos de la colección.
<code><T> T[] toArray(T[] a)</code>	Devuelve un arreglo que contiene todos los elementos de la colección; el tipo del arreglo es el especificado.

Cuadro 12.2: Principales métodos de la interfaz `Map<K, V>`

Método	Descripción
<code>V put(K key, V value)</code>	Asocia el valor especificado con la clave especificada. Si la clave ya existía, reemplaza el valor anterior y devuelve el valor antiguo.
<code>V get(Object key)</code>	Devuelve el valor asociado a la clave especificada, o <code>null</code> si no existe.
<code>boolean containsKey(Object key)</code>	Devuelve <code>true</code> si el mapa contiene la clave especificada.
<code>boolean containsValue(Object value)</code>	Devuelve <code>true</code> si el mapa contiene al menos una clave asociada al valor especificado.
<code>V remove(Object key)</code>	Elimina la clave (y su valor asociado) del mapa. Devuelve el valor eliminado o <code>null</code> si no existía.
<code>void clear()</code>	Elimina todas las asociaciones del mapa.
<code>int size()</code>	Devuelve el número de pares clave-valor en el mapa.
<code>boolean isEmpty()</code>	Devuelve <code>true</code> si el mapa no contiene asociaciones.
<code>Set<K> keySet()</code>	Devuelve un conjunto con todas las claves del mapa.
<code>Collection<V> values()</code>	Devuelve una colección con todos los valores del mapa.
<code>Set<Map.Entry<K, V>> entrySet()</code>	Devuelve un conjunto con todas las asociaciones clave-valor del mapa.
<code>V putIfAbsent(K key, V value)</code>	Si la clave no está asociada a ningún valor, la asocia al valor especificado y lo devuelve; si ya existe, no modifica el valor y devuelve el valor actual.
<code>void putAll(Map<? extends K, ? extends V> m)</code>	Copia todas las asociaciones del mapa especificado a este mapa.

Ejercicios de refuerzo

Ejercicio 12.8



Para cada uno de los siguientes casos de uso, indica qué estructura o clase de las vistas en este capítulo emplearías (*array* unidimensional, *array* multidimensional, `String` o `StringBuilder`) y justifica tu elección:

1. Las doce temperaturas medias mensuales de una ciudad, que siempre son doce.
2. El tablero de un juego de tres en raya.
3. Construir un informe de texto concatenando diez mil líneas dentro de un bucle.
4. Guardar el NIF de una persona, que no cambia una vez asignado.

Ver solución en la página 134.

Ejercicio 12.9



Completa la siguiente tabla escribiendo la expresión regular que valida cada formato. Después, escribe un método `boolean valida(String texto)` que compruebe el primero de ellos usando `Pattern` y `Matcher`.

Formato	Expresión regular
NIF: 8 dígitos y una letra mayúscula	
Código postal: exactamente 5 dígitos	
Matrícula: 4 dígitos, guion y 3 letras	
Hora <code>hh:mm</code> en formato de 24 horas	

Ver solución en la página 135.

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 12.1



```

1 // Fragmento: incluir dentro de una clase
2 int[] numeros = new int[10];
3 for (int i = 0; i < numeros.length; i++) {
4     numeros[i] = i + 1;
5 }
6 for (int n : numeros) {
7     System.out.print(n + " ");
8 }
9 System.out.println(); // 1 2 3 4 5 6 7 8 9 10

```

Solución del ejercicio 12.2



```

1 // Programa completo: compilar y ejecutar →
   independientemente
2 // Uso: java SumaArgs 3 7 10 5
3 public class SumaArgs {
4     public static void main(String[] args) {
5         int suma = 0;
6         for (String arg : args) {
7             suma += Integer.parseInt(arg);
8         }
9         System.out.println("Suma: " + suma);
10    }
11 }

```

Solución del ejercicio 12.3



```

1 // Fragmento: completa el ejemplo del triángulo de Pascal →
   añadiendo la impresión
2 int altura = 5;
3 int[][] trianguloDePascal = new int[altura][];
4 for (int i = 0; i < altura; i++) {
5     trianguloDePascal[i] = new int[i + 1];
6     trianguloDePascal[i][0] = 1;
7     for (int j = 1; j < i; j++) {
8         trianguloDePascal[i][j] = trianguloDePascal[i - 1][j - 1] →
            + trianguloDePascal[i - 1][j];
9     }
10    trianguloDePascal[i][i] = 1;
11 }

```

```

12 // Impresión fila a fila
13 for (int[] fila : trianguloDePascal) {
14     for (int valor : fila) {
15         System.out.print(valor + " ");
16     }
17     System.out.println();
18 }
19 // 1
20 // 1 1
21 // 1 2 1
22 // 1 3 3 1
23 // 1 4 6 4 1

```

Solución del ejercicio 12.7



```

1 // Programa completo: compilar y ejecutar ↪
  // independientemente
2 // Uso: java FraseBuilder Hola mundo Java
3 public class FraseBuilder {
4     public static void main(String[] args) {
5         StringBuilder sb = new StringBuilder();
6         for (int i = 0; i < args.length; i++) {
7             sb.append(args[i]);
8             if (i < args.length - 1) {
9                 sb.append(" ");
10            }
11        }
12        sb.append(".");
13        System.out.println(sb.toString()); // Hola mundo Java.
14    }
15 }

```

Solución del ejercicio 12.8



1. Array unidimensional de tamaño fijo: `double[] medias = new double[12];`. El número de elementos se conoce de antemano y no varía, que es justo el caso en el que una estructura estática resulta preferible a una dinámica.
2. Array multidimensional: `char[][] tablero = new char[3][3];`. La rejilla tiene dos dimensiones de tamaño fijo y el acceso por fila y columna es directo.
3. `StringBuilder`. `String` es inmutable: cada concatenación crea un

objeto nuevo y copia todo el contenido anterior, así que el bucle acaba teniendo un coste cuadrático. `StringBuilder` modifica un búfer interno y evita esas copias.

4. `String`. El valor no se modifica nunca, de modo que la inmutabilidad no supone ningún coste y además aporta seguridad: nadie puede alterarlo por accidente.

Solución del ejercicio 12.9



Formato	Expresión regular
NIF	<code>\\d{8}[A-Z]</code>
Código postal	<code>\\d{5}</code>
Matrícula	<code>\\d{4}-[A-Z]{3}</code>
Hora hh:mm	<code>([01]\\d 2[0-3]):[0-5]\\d</code>

La hora es la única que necesita alternativas: las dos primeras cifras valen de 00 a 19 (`[01]\\d`) o de 20 a 23 (`2[0-3]`), y los minutos van de 00 a 59 (`[0-5]\\d`). Un patrón como `\\d{2}:\\d{2}` daría por buena una hora inexistente como 99:99.

El patrón se compila una sola vez en una constante `static final`, tal y como se recomienda más arriba en este mismo capítulo:

```

1 import java.util.regex.Matcher;
2 import java.util.regex.Pattern;
3
4 public class ValidadorNif {
5
6     // El patron se compila una sola vez, en una constante ↪
7     // static final.
8     private static final Pattern NIF = Pattern.compile("\\d{8}↪
9     [A-Z]");
10
11     public static boolean valida(String texto) {
12         Matcher matcher = NIF.matcher(texto);
13         return matcher.matches();
14     }
15
16     public static void main(String[] args) {
17         System.out.println(valida("12345678Z")); // true
18         System.out.println(valida("1234567Z"));  // false: solo↪
19         // 7 digitos
20         System.out.println(valida("12345678z")); // false: ↪
21         // letra minuscúla

```

```
18 | }  
19 | }
```


13

Excepciones

Las excepciones son el mecanismo de Java para gestionar situaciones anómalas durante la ejecución: divisiones entre cero, archivos inexistentes, índices fuera de rango, etc. En lugar de que el programa se interrumpa sin control, las excepciones permiten detectar el problema y reaccionar de forma ordenada. Este capítulo cubre:

- **Jerarquía de excepciones:** cómo las organiza Java.
- **Manejo de excepciones:** `try-catch-finally` y sus variantes.
- **Lanzar excepciones:** `throw` y `throws`.
- **Crear excepciones propias:** definir tipos de excepción específicos.
- **Aserciones:** verificar invariantes en tiempo de desarrollo.

13.1. Introducción

Cuando ocurre un error en tiempo de ejecución, Java crea un objeto de excepción que describe el problema y lo **lanza** (*throws*). Si nadie lo captura, el programa termina mostrando la traza de la pila (*stack trace*). El objetivo del manejo de excepciones es **capturar** ese objeto y responder apropiadamente.

```
1 int[] arr = {1, 2, 3};  
2 System.out.println(arr[5]); // lanza ↪  
   ArrayIndexOutOfBoundsException
```

Ejemplo 13.1: Excepción sin manejar

13.2. Jerarquía de excepciones

Todas las excepciones descienden de `Throwable`:

- **Error**: problemas graves del sistema (p. ej., `StackOverflowError`, `OutOfMemoryError`). No deben capturarse en el código de aplicación.
- **Exception**: situaciones recuperables. Se divide en:
 - **Checked exceptions** (comprueban en compilación): descienden de `Exception` pero no de `RuntimeException`. El compilador obliga a capturarlas o declararlas con `throws`. Ejemplo: `IOException`.
 - **Unchecked exceptions** (no se comprueban): descienden de `RuntimeException`. No es obligatorio capturarlas. Ejemplo: `NullPointerException`, `IllegalArgumentException`.

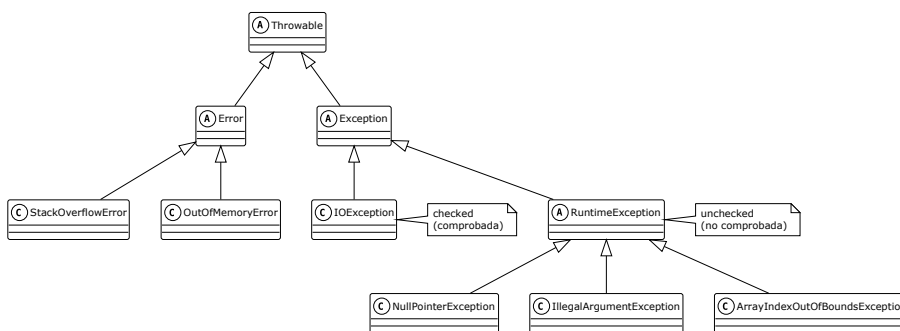


Figura 13.1: Jerarquía de excepciones en Java

Importante

Como regla general: usa **checked exceptions** para errores recuperables que el llamador puede anticipar (p. ej., archivo no encontrado), y **unchecked exceptions** para errores de programación (p. ej., argumento nulo inesperado).

13.3. Manejo de excepciones

13.3.1. try-catch-finally

```

1 try {
2     // código que puede lanzar una excepción
3     int resultado = 10 / divisor;
4 } catch (ArithmeticException e) {
5     // se ejecuta si se lanza ArithmeticException
6     System.out.println("Error: " + e.getMessage());

```

```

7 } finally {
8   // se ejecuta siempre, con o sin excepción
9   System.out.println("Operación finalizada");
10 }

```

Ejemplo 13.2: Estructura try-catch-finally

El bloque **finally** es útil para liberar recursos (cerrar ficheros, conexiones, etc.) independientemente de si hubo error.

13.3.2. Múltiples catch

Se pueden encadenar varios bloques **catch** para distintos tipos de excepción. Java comprueba los bloques en orden y ejecuta el primero que coincida. Las excepciones más específicas deben ir antes que las más generales.

```

1 try {
2   String s = null;
3   System.out.println(s.length());
4 } catch (NullPointerException e) {
5   System.out.println("Referencia nula");
6 } catch (Exception e) {
7   System.out.println("Error genérico: " + e.getMessage());
8 }

```

Ejemplo 13.3: Múltiples catch

13.3.3. Multi-catch

Desde Java 7, un solo **catch** puede manejar varios tipos separados por `|`:

```

1 try {
2   // ...
3 } catch (IOException | SQLException e) {
4   System.out.println("Error de E/S o base de datos: " + e.↵
5     getMessage());
6 }

```

Ejemplo 13.4: Multi-catch

13.3.4. try-with-resources

Para recursos que implementan **AutoCloseable** (como ficheros o conexiones), **try-with-resources** garantiza que se cierren al salir del bloque, aunque haya excepción.

```

1 try (BufferedReader reader = new BufferedReader(new FileReader("↵
    datos.txt"))) {
2     String linea = reader.readLine();
3     System.out.println(linea);
4 } catch (IOException e) {
5     System.out.println("No se pudo leer el archivo: " + e.↵
        getMessage());
6 }
7 // reader se cierra automáticamente aquí

```

Ejemplo 13.5: try-with-resources

13.4. Lanzar excepciones

13.4.1. throw

Con **throw** se lanza explícitamente una excepción desde el código:

```

1 public static double dividir(double a, double b) {
2     if (b == 0) throw new IllegalArgumentException("El divisor no ↵
        puede ser cero");
3     return a / b;
4 }

```

Ejemplo 13.6: throw

13.4.2. throws

Si un método puede lanzar una checked exception y no la captura, debe declararla con **throws** en su firma:

```

1 public static String leerPrimeraLinea(String ruta) throws ↵
    IOException {
2     try (BufferedReader reader = new BufferedReader(new FileReader↵
        (ruta))) {
3         return reader.readLine();
4     }
5 }

```

Ejemplo 13.7: throws en la firma del método

El llamador será responsable de capturar o re-declarar la excepción.

13.5. Crear excepciones propias

Se pueden definir nuevos tipos de excepción extendiendo **Exception** (checked) o **RuntimeException** (unchecked):

```

1 public class SaldoInsuficienteException extends Exception {
2     private double saldo;
3
4     public SaldoInsuficienteException(double saldo) {
5         super("Saldo insuficiente: " + saldo);
6         this.saldo = saldo;
7     }
8
9     public double getSaldo() { return saldo; }
10 }

```

Ejemplo 13.8: Excepción propia

```

1 public void retirar(double cantidad) throws ↵
    SaldoInsuficienteException {
2     if (cantidad > this.saldo) throw new ↵
        SaldoInsuficienteException(this.saldo);
3     this.saldo -= cantidad;
4 }

```

Ejemplo 13.9: Uso de la excepción propia

13.6. Aserciones

Una aserción verifica una condición que **siempre debería ser cierta** en ese punto del código. Se usan para detectar errores de lógica durante el desarrollo.

```

1 assert condicion;
2 assert condicion : "Mensaje de error";

```

Ejemplo 13.10: Sintaxis de assert

Si la condición es **false**, se lanza un **AssertionError**. Las aserciones están **desactivadas por defecto** en tiempo de ejecución; se activan con la opción **-ea** de la JVM.

Consejo



No uses aserciones para validar argumentos de métodos públicos (usa **IllegalArgumentException** en su lugar). Las aserciones son para invariantes internos que nunca deberían fallar si el código es correcto.

Ejercicios de refuerzo

Ejercicio 13.1



Escribe un método `leerEntero(Scanner sc)` que pida al usuario un número entero y repita la petición si introduce algo que no es un entero. Usa **try-catch** con `InputMismatchException`.

Pista: `Scanner` es la clase que Java usa para leer datos del teclado. Aquí te basta con `sc.nextInt()`, que devuelve el siguiente entero introducido, y `sc.next()`, que descarta lo que el usuario haya escrito cuando no es válido. La clase se estudia en detalle en el capítulo 15. Ver solución en la página 143.

Ejercicio 13.2



Dado un array de enteros, escribe un método `media(int[] arr)` que calcule la media. Lanza una `IllegalArgumentException` si el array está vacío o es `null`. Ver solución en la página 143.

Ejercicio 13.3



Define una excepción no comprobada `SaldoInsuficienteException` que reciba el saldo actual y la cantidad solicitada. Escribe una clase `CuentaBancaria` con un saldo inicial y un método `retirar(double cantidad)` que lance dicha excepción si el saldo es insuficiente. En el `main`, crea una cuenta con 100€, retira 40€ correctamente y luego intenta retirar 200€ capturando la excepción. Ver solución en la página 143.

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 13.1



```

1 // Fragmento: incluir dentro de una clase
2 import java.util.InputMismatchException;
3 import java.util.Scanner;
4
5 public static int leerEntero(Scanner sc) {
6     while (true) {
7         try {
8             System.out.print("Introduce un entero: ");
9             return sc.nextInt();
10        } catch (InputMismatchException e) {
11            System.out.println("Eso no es un entero. Inténtalo de ↵
12                nuevo.");
13            sc.nextLine(); // limpiar el buffer
14        }
15    }

```

Solución del ejercicio 13.2



```

1 // Fragmento: incluir dentro de una clase
2 public static double media(int[] arr) {
3     if (arr == null || arr.length == 0) {
4         throw new IllegalArgumentException("El array no puede ser ↵
5             nulo ni vacío");
6     }
7     int suma = 0;
8     for (int n : arr) {
9         suma += n;
10    }
11    return (double) suma / arr.length;

```

Solución del ejercicio 13.3



```

1 // Fragmento: definir estas clases en el mismo archivo o ↵
2     paquete
3 class SaldoInsuficienteException extends RuntimeException {
4     SaldoInsuficienteException(double saldo, double solicitado ↵
5         ) {

```

```

5   super(String.format("Saldo %.2f € insuficiente para ↵
6   retirar %.2f €", saldo, solicitado));
7   }
8   }
9   class CuentaBancaria {
10  private double saldo;
11
12  CuentaBancaria(double saldoInicial) {
13      if (saldoInicial < 0) {
14          throw new IllegalArgumentException("El saldo inicial no ↵
15          puede ser negativo");
16      }
17      this.saldo = saldoInicial;
18  }
19
20  void depositar(double cantidad) {
21      if (cantidad <= 0) {
22          throw new IllegalArgumentException("La cantidad a ↵
23          depositar debe ser positiva");
24      }
25      saldo += cantidad;
26  }
27
28  void retirar(double cantidad) {
29      if (cantidad <= 0) {
30          throw new IllegalArgumentException("La cantidad a ↵
31          retirar debe ser positiva");
32      }
33      if (cantidad > saldo) {
34          throw new SaldoInsuficienteException(saldo, cantidad);
35      }
36      saldo -= cantidad;
37  }
38
39  double getSaldo() {
40      return saldo;
41  }
42  }
43
44  // Uso en main:
45  class Demo {
46      public static void main(String[] args) {
47          CuentaBancaria cuenta = new CuentaBancaria(100.0);
48          System.out.println("Saldo inicial: " + cuenta.getSaldo()) ↵
49          ; // 100.0

```



```
47  cuenta.depositar(50.0);
48  System.out.println("Tras depositar 50: " + cuenta.↵
    getSaldo()); // 150.0
49
50  cuenta.retirar(40.0);
51  System.out.println("Tras retirar 40: " + cuenta.getSaldo↵
    ()); // 110.0
52
53  try {
54      cuenta.retirar(200.0); // lanza ↵
        SaldoInsuficienteException
55  } catch (SaldoInsuficienteException e) {
56      System.out.println("Error: " + e.getMessage());
57  }
58
59  System.out.println("Saldo final: " + cuenta.getSaldo()); ↵
    // 110.0
60  }
61 }
```


Colecciones

El **Java Collections Framework** proporciona un conjunto de interfaces y clases para almacenar y manipular grupos de objetos. A diferencia de los arrays, las colecciones son dinámicas: crecen y se reducen automáticamente. Este capítulo cubre:

- **Jerarquía:** las interfaces principales del framework.
- **Listas:** `ArrayList` y `LinkedList`.
- **Conjuntos:** `HashSet` y `TreeSet`.
- **Pilas y colas:** `ArrayDeque`.
- **Mapas:** `HashMap` y `TreeMap`.
- **Recorridos e iteradores.**
- **Programación funcional con colecciones:** lambdas y Streams.

14.1. Introducción

Los arrays tienen tamaño fijo y no ofrecen operaciones de búsqueda, ordenación o eliminación. Las colecciones resuelven estos problemas:

```
1 // Array: tamaño fijo
2 int[] numeros = new int[3];
3
4 // ArrayList: tamaño dinámico
5 ArrayList<Integer> lista = new ArrayList<>();
6 lista.add(1);
7 lista.add(2);
8 lista.add(3);
9 lista.remove(Integer.valueOf(2)); // elimina el elemento 2
```

Ejemplo 14.1: Array vs. ArrayList

14.2. Jerarquía de colecciones

Las principales interfaces son:

- **Collection**: raíz de listas, conjuntos y colas.
 - **List**: colección ordenada con duplicados permitidos.
 - **Set**: colección sin duplicados.
 - **Queue**: colección con semántica de cola.
- **Map**: asocia claves únicas a valores (no extiende **Collection**).

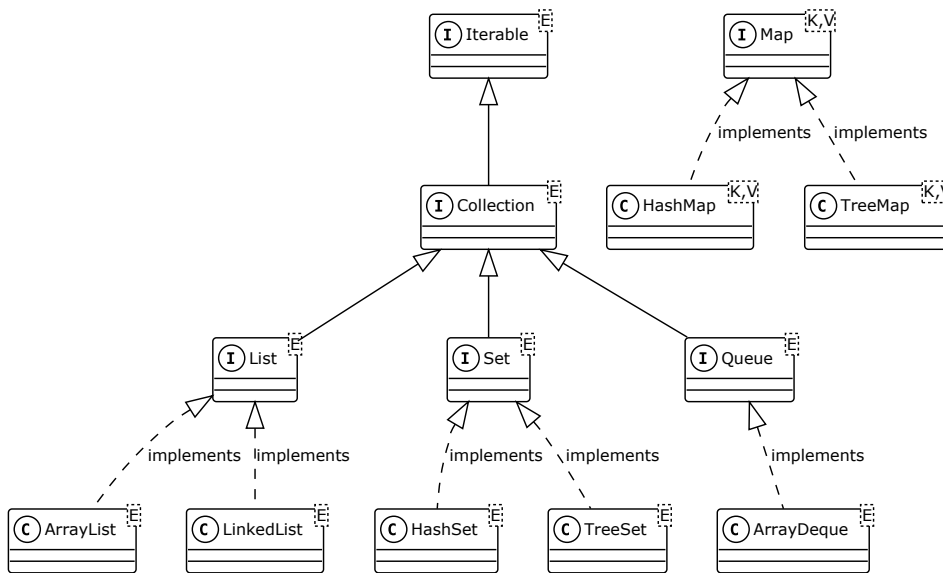


Figura 14.1: Jerarquía del Java Collections Framework

14.3. Listas

14.3.1. ArrayList

Implementación basada en array dinámico. Acceso aleatorio en $O(1)$; inserción/eliminación en el medio en $O(n)$.

```

1 import java.util.ArrayList;
2
3 ArrayList<String> nombres = new ArrayList<>();
4 nombres.add("Ana");
  
```

```

5 nombres.add("Luis");
6 nombres.add("María");
7 System.out.println(nombres.get(1)); // Luis
8 nombres.remove("Luis");
9 System.out.println(nombres.size()); // 2

```

Ejemplo 14.2: ArrayList básico

14.3.2. LinkedList

Implementación basada en lista doblemente enlazada. Inserción/eliminación en extremos en $O(1)$; acceso aleatorio en $O(n)$. Preferible cuando hay muchas inserciones/eliminaciones al principio o al final de la lista.

```

1 import java.util.LinkedList;
2
3 LinkedList<Integer> lista = new LinkedList<>();
4 lista.addFirst(1);
5 lista.addLast(3);
6 lista.add(1, 2); // inserta 2 en posición 1
7 System.out.println(lista); // [1, 2, 3]

```

Ejemplo 14.3: LinkedList

14.4. Conjuntos

Un Set no permite elementos duplicados.

14.4.1. HashSet y TreeSet

- HashSet: no garantiza orden; operaciones en $O(1)$ promedio.
- TreeSet: mantiene los elementos en orden natural (o con Comparator); operaciones en $O(\log n)$.

```

1 import java.util.HashSet;
2 import java.util.TreeSet;
3
4 HashSet<String> hash = new HashSet<>();
5 hash.add("banana"); hash.add("manzana"); hash.add("banana");
6 System.out.println(hash.size()); // 2 (sin duplicados)
7
8 TreeSet<String> tree = new TreeSet<>(hash);
9 System.out.println(tree); // [banana, manzana] (ordenado)

```

Ejemplo 14.4: HashSet y TreeSet

14.5. Pilas y colas

ArrayDeque es la implementación recomendada para ambas estructuras.

14.5.1. Cola FIFO con ArrayDeque

```
1 import java.util.ArrayDeque;
2
3 ArrayDeque<String> cola = new ArrayDeque<>();
4 cola.offer("primero");
5 cola.offer("segundo");
6 cola.offer("tercero");
7 System.out.println(cola.poll()); // primero
8 System.out.println(cola.poll()); // segundo
```

Ejemplo 14.5: Cola FIFO

14.5.2. Pila LIFO con ArrayDeque

```
1 ArrayDeque<String> pila = new ArrayDeque<>();
2 pila.push("primero");
3 pila.push("segundo");
4 pila.push("tercero");
5 System.out.println(pila.pop()); // tercero
6 System.out.println(pila.pop()); // segundo
```

Ejemplo 14.6: Pila LIFO

14.6. Mapas

Un Map asocia claves únicas a valores.

- HashMap: sin orden garantizado; $O(1)$ promedio para get/put.
- TreeMap: claves en orden natural; $O(\log n)$.

```
1 import java.util.HashMap;
2
3 HashMap<String, Integer> edades = new HashMap<>();
4 edades.put("Ana", 25);
5 edades.put("Luis", 30);
6 edades.put("Ana", 26); // sobrescribe el valor de "Ana"
7
8 System.out.println(edades.get("Ana")); // 26
```

```

9 System.out.println(edades.containsKey("Luis")); // true
10 System.out.println(edades.size()); // 2

```

Ejemplo 14.7: HashMap

14.7. Recorridos e iteradores

Todas las colecciones que implementan `Iterable` (listas, conjuntos, colas) pueden recorrerse con `for-each`:

```

1 for (String nombre : nombres) {
2     System.out.println(nombre);
3 }

```

Ejemplo 14.8: Recorrido con for-each

Para mapas, se itera sobre las entradas:

```

1 import java.util.Map;
2
3 for (Map.Entry<String, Integer> entrada : edades.entrySet()) {
4     System.out.println(entrada.getKey() + ": " + entrada.getValue()
5 );

```

Ejemplo 14.9: Recorrido de un Map

14.8. Programación funcional con colecciones

A partir de Java 8, las colecciones se pueden procesar con **lambdas** y **Streams**, lo que permite escribir código más expresivo y conciso. Este tema se desarrolla en detalle en el capítulo 21; aquí se muestra un ejemplo introductorio:

```

1 import java.util.List;
2
3 List<String> nombres = List.of("Ana", "Luis", "María", "Pedro");
4
5 // Filtrar nombres de más de 4 letras y convertir a mayúsculas
6 nombres.stream()
7     .filter(n -> n.length() > 4)
8     .map(String::toUpperCase)
9     .forEach(System.out::println);
10 // MARÍA
11 // PEDRO

```

Ejemplo 14.10: Streams: filtrar y transformar

Ejercicios de refuerzo

Ejercicio 14.1



Para cada uno de los siguientes casos de uso, indica qué colección de Java usarías y justifica tu elección:

1. Lista de estudiantes de una clase a la que se accede frecuentemente por posición (primero, segundo, etc.).
2. Conjunto de palabras únicas que aparecen en un texto, que se quiere mantener en orden alfabético.
3. Directorio telefónico donde se busca el número a partir del nombre de la persona.
4. Historial de acciones de un editor de texto para implementar la función «deshacer».

Ver solución en la página 154.

Ejercicio 14.2



Crea un `ArrayList<String>` con una lista de la compra. Añade al menos cinco productos. Después elimina uno por nombre y muestra la lista resultante por pantalla. *Ver solución en la página 154.*

Ejercicio 14.3



Dado un `ArrayList<Integer>` con números repetidos, usa un `HashSet` para eliminar los duplicados y muestra cuántos elementos únicos hay. *Ver solución en la página 154.*

Ejercicio 14.4



Crea una agenda telefónica con `HashMap<String, String>` (nombre → teléfono). Añade tres contactos, busca uno por nombre e imprime todos los contactos con su teléfono. *Ver solución en la página 155.*

Ejercicio 14.5



Simula un historial de navegación web usando `ArrayDeque` como pila. El usuario visita tres páginas; cada vez que pulsa “atrás” se desapila la

página actual. Muestra la página en la que queda al final. *Ver solución en la página 155.*

Ejercicio 14.6



Dado un texto, cuenta la frecuencia de cada palabra usando un `HashMap<String, Integer>`. Ignora mayúsculas y minúsculas. *Ver solución en la página 155.*

Ejercicio 14.7



Completa la siguiente tabla comparando la complejidad temporal de las operaciones más comunes en `ArrayList`, `LinkedList` y `HashSet`. Después, indica en qué situación elegirías cada una.

Operación	ArrayList	LinkedList	HashSet
Insertar al final			
Buscar un elemento			
Acceder por índice			

Ver solución en la página 156.

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 14.1



1. `ArrayList<String>`: acceso por índice en $O(1)$ y tamaño dinámico.
2. `TreeSet<String>`: garantiza unicidad de elementos y los mantiene ordenados (orden natural o un `Comparator`).
3. `HashMap<String, String>`: búsqueda por clave en $O(1)$ de media; si se necesita orden de inserción, `LinkedHashMap`; si se quiere orden alfabético de nombres, `TreeMap`.
4. `Deque<Accion>` (implementado con `LinkedList` o `ArrayDeque`): la operación «deshacer» retira el último elemento añadido (LIFO), lo que se corresponde con una pila.

Solución del ejercicio 14.2



```

1 // Fragmento: incluir dentro de una clase
2 import java.util.ArrayList;
3
4 ArrayList<String> lista = new ArrayList<>();
5 lista.add("leche");
6 lista.add("pan");
7 lista.add("huevos");
8 lista.add("mantequilla");
9 lista.add("zumo");
10 lista.remove("pan");
11 System.out.println(lista); // [leche, huevos, mantequilla, ↵
    zumo]
```

Solución del ejercicio 14.3



```

1 // Fragmento: incluir dentro de una clase
2 import java.util.ArrayList;
3 import java.util.HashSet;
4 import java.util.List;
5
6 ArrayList<Integer> nums = new ArrayList<>(List.of(1, 2, 2, ↵
    3, 3, 3, 4));
7 HashSet<Integer> unicos = new HashSet<>(nums);
8 System.out.println("Elementos únicos: " + unicos.size()); ↵
    // 4
```

Solución del ejercicio 14.4



```

1 // Fragmento: incluir dentro de una clase
2 import java.util.HashMap;
3 import java.util.Map;
4
5 HashMap<String, String> agenda = new HashMap<>();
6 agenda.put("Ana", "612345678");
7 agenda.put("Luis", "698765432");
8 agenda.put("María", "655111222");
9
10 System.out.println("Teléfono de Ana: " + agenda.get("Ana"))↵
11 ;
12 for (Map.Entry<String, String> entrada : agenda.entrySet())↵
13 {
14     System.out.println(entrada.getKey() + ": " + entrada.↵
15         getValue());
16 }

```

Solución del ejercicio 14.5



```

1 // Fragmento: incluir dentro de una clase
2 import java.util.ArrayDeque;
3
4 ArrayDeque<String> historial = new ArrayDeque<>();
5 historial.push("inicio.html");
6 historial.push("productos.html");
7 historial.push("detalle.html");
8
9 historial.pop(); // volver atrás desde detalle
10 historial.pop(); // volver atrás desde productos
11
12 System.out.println("Página actual: " + historial.peek()); ↵
13     // inicio.html

```

Solución del ejercicio 14.6



```

1 // Fragmento: incluir dentro de una clase
2 import java.util.HashMap;
3
4 String texto = "hola mundo hola Java mundo hola";
5 HashMap<String, Integer> frecuencia = new HashMap<>();
6 for (String palabra : texto.toLowerCase().split("\\s+")) {
7     frecuencia.put(palabra, frecuencia.getDefault(palabra, ↵

```

```
0) + 1);
8 }
9 System.out.println(frecuencia); // {hola=3, mundo=2, java=1}
```

Solución del ejercicio 14.7



Operación	ArrayList	LinkedList	HashSet
Insertar al final	O(1) amort.	O(1)	O(1) amort.
Buscar un elemento	O(n)	O(n)	O(1) media
Acceder por índice	O(1)	O(n)	N/A

- Elige **ArrayList** cuando necesites acceso aleatorio frecuente y pocas inserciones/borrados en el medio.
- Elige **LinkedList** cuando las inserciones y borrados sean frecuentes en cualquier posición, o la uses como cola/pila.
- Elige **HashSet** cuando necesites búsquedas rápidas y no importe el orden ni el acceso por posición.

Lectura y escritura de información

En el desarrollo de aplicaciones informáticas, la gestión de la entrada y salida de información es fundamental. Este capítulo aborda los conceptos y técnicas esenciales para la lectura y escritura de datos, tanto desde dispositivos externos como desde el propio sistema de archivos. Se estudiarán los diferentes tipos de flujos, el manejo de ficheros y directorios, así como las operaciones básicas de entrada desde teclado y salida a pantalla. El objetivo es proporcionar las bases necesarias para manipular información de manera eficiente y segura en programas de ordenador.

Los temas que se tratarán en este capítulo son:

1. **Salida a pantalla y entrada desde teclado:** Se estudiarán las técnicas para capturar información introducida por el usuario a través del teclado y para mostrar resultados en pantalla, incluyendo el uso de diferentes formatos de visualización.
2. **Flujos de datos:** Se explicarán los conceptos de flujo de entrada y salida, diferenciando entre flujos de bytes y de caracteres. Se presentarán las clases y estructuras más comunes para su manejo en distintos lenguajes de programación.
3. **Ficheros de datos y registros:** Se abordará la organización de la información en ficheros, el concepto de registro y las ventajas de estructurar los datos de esta manera.
4. **Utilización de los sistemas de ficheros:** Se explicará cómo interactuar con el sistema de archivos del sistema operativo, incluyendo la navegación por directorios y la obtención de información sobre los mismos. Además, se presentarán las operaciones básicas de creación, eliminación y modificación de ficheros y directorios.

15.1. Salida a pantalla y entrada desde teclado

La salida a pantalla y la entrada desde teclado son operaciones fundamentales en cualquier lenguaje de programación. En Java, se utilizan principalmente las clases `System.out` para la salida estándar (pantalla) y `Console` para la entrada estándar (teclado).

15.1.1. Salida a pantalla

Para mostrar información por pantalla, se utiliza el método `System.out.println()` para imprimir una línea de texto, o `System.out.print()` para imprimir sin salto de línea.

```
1 public class SalidaPantalla {
2     public static void main(String[] args) {
3         System.out.println("Hola, mundo!");
4         System.out.print("Introduce tu nombre: ");
5     }
6 }
```

Ejemplo 15.1: Ejemplo de salida a pantalla

Formateo de salida

Para mostrar información de manera más legible o con un formato específico, Java proporciona el método `System.out.printf()`, que permite imprimir texto utilizando especificadores de formato similares a los del lenguaje C. Por ejemplo, se pueden mostrar números con un número fijo de decimales, alinear texto o incluir variables dentro de una cadena.

```
1 public class FormateoSalida {
2     public static void main(String[] args) {
3         String nombre = "Ana";
4         int edad = 22;
5         double nota = 8.75;
6         System.out.printf("Nombre: %s, Edad: %d, Nota: %.2f\n", ↵
        nombre, edad, nota);
7         System.out.printf("%-10s %5d %7.2f\n", nombre, edad, nota); ↵
        // alineación
8     }
9 }
```

Ejemplo 15.2: Formateo de salida con printf

En este ejemplo, los especificadores `%s`, `%d` y `%.2f` se utilizan para imprimir una cadena, un entero y un número decimal con dos cifras decimales, respectivamente. Además, se puede controlar la anchura y alineación de los campos usando valores como `%-10s` (alineado a la izquierda en 10 espacios).

El método `String.format()` funciona de manera similar, pero devuelve la cadena formateada en lugar de imprimirla directamente.

15.1.2. Entrada desde teclado

Una forma de leer datos desde el teclado en Java es utilizando la clase `Console`, que proporciona métodos para leer texto y contraseñas de manera sencilla y segura. La clase `Console` es especialmente útil cuando se desea ocultar la entrada del usuario, como en el caso de contraseñas.

A continuación se muestra un ejemplo de uso de `Console` para leer una línea de texto y una contraseña:

```

1 public class EntradaConsola {
2     public static void main(String[] args) {
3         java.io.Console consola = System.console();
4         if (consola != null) {
5             String nombre = consola.readLine("Introduce tu nombre: ");
6             char[] contrasena = consola.readPassword("Introduce tu ↵
contraseña: ");
7             System.out.println("Hola, " + nombre + ". Tu contraseña ↵
tiene " + contrasena.length + " caracteres.");
8         } else {
9             System.out.println("No se puede obtener la consola. ↵
Ejecuta el programa desde una terminal.");
10        }
11    }
12 }

```

Ejemplo 15.3: Lectura desde teclado con `Console`

`Console` solo está disponible cuando el programa se ejecuta desde una terminal o consola real, no desde algunos entornos de desarrollo integrados (IDE). Si `System.console()` devuelve `null`, significa que la consola no está disponible.

Lectura de texto desde teclado, usando la clase `Scanner`

Para leer texto introducido por el usuario desde el teclado, se puede utilizar la clase `Scanner`:

```

1 import java.util.Scanner;
2
3 public class LeerTeclado {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         System.out.print("Introduce una línea de texto: ");
7         String texto = sc.nextLine();
8         System.out.println("Has escrito: " + texto);
9         sc.close();

```

```
10 }
11 }
```

Ejemplo 15.4: Lectura de texto desde teclado

Esta clase permite leer fácilmente cadenas, números y otros tipos de datos desde la entrada estándar. Al final de este capítulo, se incluye una tabla con los principales métodos de la clase **Scanner**.

Ejercicio 15.1

Crea un programa que lea por consola una serie de números enteros introducidos por el usuario, hasta que lea una línea vacía. El programa debe calcular y mostrar la suma de todos los números introducidos, así como la media aritmética.

Consejo

También puedes utilizar la clase **Scanner** para leer datos desde un archivo de texto. Para ello, simplemente crea un objeto **Scanner** pasando un objeto **File** como argumento. Por ejemplo:

```
1 import java.io.File;
2 import java.io.FileNotFoundException;
3 import java.util.Scanner;
4
5 public class LeerArchivoConScanner {
6     public static void main(String[] args) {
7         try (Scanner sc = new Scanner(new File("alumnos.txt"))) ↩
8         {
9             while (sc.hasNextLine()) {
10                 String linea = sc.nextLine();
11                 System.out.println(linea);
12             }
13         } catch (FileNotFoundException e) {
14             e.printStackTrace();
15         }
16 }
```

Ejemplo 15.5: Lectura de un archivo de texto con Scanner

En este caso, el objeto **Scanner** leerá el contenido del archivo línea a línea, permitiendo procesar los datos de manera similar a como se haría con **BufferedReader**.

15.2. Flujos de datos

En Java, los flujos de datos (*streams*)¹ permiten la entrada y salida de información. Existen dos tipos principales: flujos de bytes y flujos de caracteres.

- **Flujos de bytes:** Usan clases como `InputStream` y `OutputStream`. Se emplean para datos binarios. Pueden representar cualquier tipo de dato, como imágenes o archivos de audio. Además, permite acceder a flujos de información de red o dispositivos externos.
- **Flujos de caracteres:** Usan clases como `Reader` y `Writer`. Se emplean para texto. Estos flujos simplifican la lectura y escritura de caracteres, gestionando automáticamente la codificación de caracteres (como UTF-8 o ISO-8859-1).

15.2.1. Lectura y escritura de bytes

Para leer y escribir bytes, se utilizan las clases `FileInputStream` y `FileOutputStream`. Estas clases permiten manipular archivos binarios directamente. Es importante usar estas clases junto a clases de buffer como `BufferedInputStream` y `BufferedOutputStream` para mejorar el rendimiento al leer y escribir grandes cantidades de datos.

```

1 import java.io.FileInputStream;
2 import java.io.FileOutputStream;
3 import java.io.IOException;
4
5 public class CopiarArchivoBinario {
6     public static void main(String[] args) {
7         try (FileInputStream fis = new FileInputStream("imagen.jpg")↵
8             ;
9             FileOutputStream fos = new FileOutputStream("copia_imagen↵
10                .jpg")) {
11             byte[] buffer = new byte[1024];
12             int bytesLeidos;
13             while ((bytesLeidos = fis.read(buffer)) != -1) {
14                 fos.write(buffer, 0, bytesLeidos);
15             }
16         } catch (IOException e) {
17             e.printStackTrace();
18         }
19     }
20 }

```

Ejemplo 15.6: Lectura y escritura de archivos binarios

¹No confundir con la clase `Stream`

En este ejemplo, se copian los datos de un archivo binario a otro utilizando un búfer de bytes. El uso de `FileInputStream` y `FileOutputStream` permite trabajar con cualquier tipo de archivo, no solo texto.

Importante

Al trabajar con flujos de bytes, es fundamental cerrar los flujos después de su uso para liberar recursos del sistema. En Java, esto se puede hacer automáticamente utilizando el bloque `try-with-resources`, que asegura que los recursos se cierren correctamente al finalizar el bloque.

El uso de `BufferedInputStream` y `BufferedOutputStream` permite leer y escribir datos de archivos de manera más eficiente, ya que utilizan un búfer interno para reducir el número de accesos al disco.

```

1 import java.io.BufferedInputStream;
2 import java.io.BufferedOutputStream;
3 import java.io.FileInputStream;
4 import java.io.FileOutputStream;
5 import java.io.IOException;
6
7 public class CopiarArchivoConBuffer {
8     public static void main(String[] args) {
9         try (BufferedInputStream bis = new BufferedInputStream(new
10             FileInputStream("imagen.jpg"));
11             BufferedOutputStream bos = new BufferedOutputStream(new
12             FileOutputStream("copia_imagen.jpg"))) {
13             byte[] buffer = new byte[1024];
14             int bytesLeidos;
15             while ((bytesLeidos = bis.read(buffer)) != -1) {
16                 bos.write(buffer, 0, bytesLeidos);
17             }
18         } catch (IOException e) {
19             e.printStackTrace();
20         }
21     }
22 }

```

Ejemplo 15.7: Copia eficiente de archivos binarios con buffer

En este ejemplo, `BufferedInputStream` y `BufferedOutputStream` mejoran el rendimiento al leer y escribir bloques de datos más grandes de una sola vez.

Lectura de archivos de red

Para leer archivos desde una red, Java proporciona clases como `InputStream` y `BufferedInputStream` junto con la clase `Socket` para conectarse a un servidor remoto. Por ejemplo, para descargar datos de un servidor web:

```

1 import java.io.BufferedInputStream;
2 import java.io.FileOutputStream;
3 import java.io.IOException;
4 import java.io.InputStream;
5 import java.net.URL;
6
7 public class DescargarArchivo {
8     public static void main(String[] args) {
9         String urlArchivo = "https://ejemplo.com/archivo.pdf";
10        try (InputStream in = new BufferedInputStream(new URL(↵
urlArchivo).openStream());
11            FileOutputStream fileOut = new FileOutputStream("↵
archivo_descargado.pdf")) {
12            byte[] buffer = new byte[1024];
13            int bytesLeidos;
14            while ((bytesLeidos = in.read(buffer, 0, 1024)) != -1) {
15                fileOut.write(buffer, 0, bytesLeidos);
16            }
17        } catch (IOException e) {
18            e.printStackTrace();
19        }
20    }
21 }

```

Ejemplo 15.8: Lectura de datos desde una URL

En este ejemplo, se abre un flujo de entrada desde una URL y se escribe el contenido descargado en un archivo local. El uso de `BufferedInputStream` mejora la eficiencia de la lectura de datos desde la red.

Ejercicio 15.2



Crea un programa que descargue una imagen desde una URL y la guarde en el sistema de archivos local. Utiliza las clases `URL`, `InputStream` y `FileOutputStream` para realizar la descarga.

El nombre del fichero a descargar se pasará como argumento al programa, así como el nombre del fichero donde se guardará la imagen. Por ejemplo, si el programa se llama `DescargarImagen`, se ejecutaría así:

```

1 java DescargarImagen https://ejemplo.com/imagen.jpg ↵
    imagen_descargada.jpg

```

Una vez descargado el fichero, deberá mostrarse un mensaje indicando que la descarga se ha completado correctamente, y el tamaño del fichero descargado en bytes.

15.2.2. Lectura y escritura de caracteres

Para trabajar con texto, Java proporciona las clases `FileReader` y `FileWriter`, así como sus versiones con buffer: `BufferedReader` y `BufferedWriter`. Estas clases permiten leer y escribir archivos de texto de manera eficiente y sencilla.

Lectura de archivos de texto

```

1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4
5 public class LeerArchivoTexto {
6     public static void main(String[] args) {
7         try (BufferedReader br = new BufferedReader(new FileReader("↵
8             alumnos.txt"))) {
9             String linea;
10            while ((linea = br.readLine()) != null) {
11                System.out.println(linea);
12            }
13        } catch (IOException e) {
14            e.printStackTrace();
15        }
16    }

```

Ejemplo 15.9: Lectura de un archivo de texto línea a línea

En este ejemplo, se lee el archivo `alumnos.txt` línea a línea y se muestra su contenido por pantalla. El uso de `BufferedReader` permite leer texto de manera eficiente.

Escritura de archivos de texto

```

1 import java.io.BufferedWriter;
2 import java.io.FileWriter;
3 import java.io.IOException;
4
5 public class EscribirArchivoTexto {
6     public static void main(String[] args) {
7         try (BufferedWriter bw = new BufferedWriter(new FileWriter("↵
8             salida.txt"))) {
9             bw.write("Primera línea de texto\n");
10            bw.write("Segunda línea de texto\n");
11        } catch (IOException e) {
12            e.printStackTrace();
13        }

```

```

13 }
14 }

```

Ejemplo 15.10: Escritura de texto en un archivo

En este caso, se escribe texto en el archivo `salida.txt`. Si el archivo no existe, se crea automáticamente. Si existe, se sobrescribe su contenido.

Importante

Al igual que con los flujos de bytes, es fundamental cerrar los flujos de caracteres para liberar recursos. El uso de `try-with-resources` facilita esta tarea.

Ejercicio 15.3

Crea un programa que lea un fichero tabulado (CSV) y muestre su contenido por pantalla. El fichero contendrá varias líneas, cada una con varios campos separados por comas. Utiliza las clases `BufferedReader` y `String.split()` para procesar cada línea. El programa deberá mostrar cada campo en una línea separada, indicando el número de línea y el número de campo. Por ejemplo, si el fichero contiene:

```

1 nombre,edad,carrera
2 Juan,20,Ingeniería
3 Ana,22,Matemáticas

```

El programa deberá mostrar:

```

1 Línea 1, Campo 1: nombre
2 Línea 1, Campo 2: edad
3 Línea 1, Campo 3: carrera
4 Línea 2, Campo 1: Juan
5 Línea 2, Campo 2: 20
6 Línea 2, Campo 3: Ingeniería
7 Línea 3, Campo 1: Ana
8 Línea 3, Campo 2: 22
9 Línea 3, Campo 3: Matemáticas

```

Modos de acceso

En Java, es posible abrir un fichero en diferentes modos, como lectura, escritura o adición. Estos modos se especifican al crear el objeto correspondiente. Por ejemplo, al usar `FileWriter`, se puede indicar si se desea sobrescribir el fichero existente o añadir contenido al final del mismo:

```

1 import java.io.BufferedWriter;
2 import java.io.FileWriter;

```

```

3 import java.io.IOException;
4
5 public class AnadirArchivo {
6     public static void main(String[] args) {
7         try (BufferedWriter bw = new BufferedWriter(new FileWriter("↵
alumnos.txt", true))) {
8             bw.write("Luis,21,Física\n");
9         } catch (IOException e) {
10             e.printStackTrace();
11         }
12     }
13 }

```

Ejemplo 15.11: Apertura de un fichero en modo adición

En este caso, el segundo parámetro de `FileWriter` es `true`, lo que indica que se debe añadir el contenido al final del fichero en lugar de sobrescribirlo. Si se omite este parámetro, el fichero se sobrescribirá por defecto. Esto es útil para mantener un registro de datos sin perder la información existente.

15.2.3. Lectura y escritura de ficheros de acceso aleatorio

Los ficheros de acceso aleatorio permiten leer y escribir datos en cualquier posición del archivo, sin necesidad de procesar todo el contenido secuencialmente. En Java, esto se logra mediante la clase `RandomAccessFile`, que proporciona métodos para posicionarse en cualquier parte del fichero y realizar operaciones de lectura o escritura.

Uso de `RandomAccessFile`

La clase `RandomAccessFile` permite abrir un archivo en modo lectura (`"r"`) o lectura/escritura (`"rw"`). Se pueden leer y escribir datos primitivos (enteros, cadenas, etc.) y moverse a cualquier posición del archivo usando el método `seek()`.

```

1 import java.io.RandomAccessFile;
2 import java.io.IOException;
3
4 public class AccesoAleatorio {
5     public static void main(String[] args) {
6         try (RandomAccessFile raf = new RandomAccessFile("datos.bin"↵
, "rw")) {
7             // Escribir datos en posiciones específicas
8             raf.writeInt(100); // posición 0
9             raf.writeDouble(3.14); // posición 4
10            raf.writeUTF("Hola"); // posición 12
11        }
12    }
13 }

```

```

12 // Leer datos desde una posición concreta
13 raf.seek(0); // volver al inicio
14 int numero = raf.readInt();
15 double decimal = raf.readDouble();
16 String texto = raf.readUTF();
17
18 System.out.println("Número: " + numero);
19 System.out.println("Decimal: " + decimal);
20 System.out.println("Texto: " + texto);
21 } catch (IOException e) {
22     e.printStackTrace();
23 }
24 }
25 }

```

Ejemplo 15.12: Ejemplo básico de acceso aleatorio

En este ejemplo, se escriben y leen datos en posiciones específicas del archivo. El método `seek(long pos)` permite moverse a cualquier byte del fichero, lo que resulta útil para modificar registros concretos sin tener que leer todo el archivo.

Importante

El acceso aleatorio es especialmente útil para ficheros de registros de tamaño fijo, como bases de datos simples o índices, donde se necesita modificar o consultar rápidamente un registro concreto.

Otra ventaja es que el fichero no se carga completamente en memoria, lo que permite trabajar con archivos grandes sin consumir demasiada memoria RAM. Sin embargo, es importante tener en cuenta que el uso de `RandomAccessFile` puede ser más complejo que el de flujos secuenciales, ya que se debe gestionar manualmente la posición del cursor y el formato de los datos. Esto es especialmente relevante cuando se trabaja con ficheros que ocupen varios megabytes, o incluso gigabytes (por ejemplo, el mapa de un videojuego masivo en línea).

Consejo

Al trabajar con `RandomAccessFile`, es importante tener en cuenta el tamaño de los datos que se están leyendo o escribiendo. Por ejemplo, al usar `writeInt()` se escriben 4 bytes, y al usar `writeDouble()` se escriben 8 bytes. Asegúrate de posicionarte correctamente con `seek()` antes de realizar operaciones de lectura o escritura para evitar errores de formato o datos corruptos.

Ejercicio 15.4

Crea un programa que almacene una lista de registros de alumnos en un fichero binario usando `RandomAccessFile`. Cada registro debe tener un nombre (cadena de longitud fija, por ejemplo 20 caracteres), una edad (entero) y una nota (double). El programa debe permitir añadir nuevos registros, consultar un registro por su posición y modificar la nota de un alumno dado su número de registro.

El programa deberá tener otro método que lea todos un registro específico del fichero y lo muestre por pantalla. Utiliza `seek()` para posicionarte en el registro correspondiente al realizar las operaciones de consulta y modificación.

Para realizar este programa, deberás definir una estructura de datos que represente un alumno, y utilizar los métodos de `RandomAccessFile` para leer y escribir los datos en el fichero. Asegúrate de manejar correctamente las excepciones y cerrar el fichero al finalizar las operaciones.

Además, deberá crear un menú interactivo que permita al usuario elegir entre las diferentes operaciones (añadir, consultar, modificar) y realizar las acciones correspondientes. El menú deberá repetirse hasta que el usuario decida salir del programa.

15.3. Ficheros de datos y registros

Existen múltiples formas de organizar la información en ficheros. Una de las más comunes es el uso de registros, que permiten almacenar datos estructurados en un formato específico. En esta sección veremos como trabajar con los principales tipos de fichero usados para almacenar datos, y como representar la información de cada uno de ellos.

15.3.1. Ficheros tabulados (CSV)

En el ejercicio 15.3 se ha visto un ejemplo de fichero tabulado, que utiliza comas para separar los campos. Este formato es ampliamente utilizado para almacenar datos en forma de tablas, y es fácil de leer y escribir tanto por humanos como por programas. En él, la primera línea suele contener los nombres de los campos, y las siguientes líneas contienen los datos correspondientes.

La ventaja de este tipo de fichero es su simplicidad y compatibilidad con muchas aplicaciones, como hojas de cálculo y bases de datos. Además, es fácil de manipular con herramientas de programación, como habrás podido comprobar en el ejercicio.

Ejercicio 15.5

Modifica el programa del ejercicio 15.3 para que, en lugar de mostrar los campos por pantalla, los almacene en una lista de objetos. Cada objeto representará una fila del fichero, y tendrá atributos correspondientes a cada campo. Utiliza la clase `ArrayList` para almacenar los objetos. Cada línea deberá representarse como un objeto de la clase `Map<String, String>`, en el cual cada clave será el nombre del campo y el valor será el dato correspondiente. Al final, muestra por pantalla el contenido de la lista de objetos.

Una vez implementado el procesamiento del fichero, crea un método que convierta la lista de objetos de tipo `Map<String, String>`, en una lista de objetos de una clase personalizada, por ejemplo, `Alumno`, que tenga los atributos `nombre`, `edad` y `carrera`. El método deberá recorrer la lista de mapas y crear un objeto `Alumno` por cada línea del fichero, añadiéndolo a una nueva lista. Finalmente, muestra por pantalla el contenido de la lista de objetos `Alumno`.

En este ejercicio, se pretende que practiques la lectura de ficheros tabulados, el uso de colecciones y la creación de objetos personalizados.

Pista: Puedes implementar el método `toString()` de la clase `Alumno` para que muestre los datos del alumno de forma legible. Por ejemplo, si el objeto `Alumno` tiene los atributos `nombre`, `edad` y `carrera`, el método `toString()` podría devolver una cadena como: "Nombre: Juan, Edad: 20, Carrera: Ingeniería". Esto facilitará la visualización de los datos al imprimir los objetos en la consola.

15.3.2. Ficheros de marcas (XML)

Los ficheros XML (*eXtensible Markup Language*) son un formato de texto que permite almacenar datos estructurados de manera jerárquica. Se utilizan ampliamente para intercambiar información entre aplicaciones y sistemas, ya que son legibles tanto por humanos como por máquinas.

Un fichero XML está compuesto por etiquetas que definen la estructura de los datos. Cada etiqueta puede contener atributos y valores, lo que permite representar información compleja de forma organizada.

```

1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <alumnos xmlns="http://www.example.com/alumnos">
3   <alumno carrera="Ingeniería" edad="20" nombre="Juan"/>
4   <alumno carrera="Matemáticas" edad="22" nombre="Ana"/>
5   <alumno carrera="Física" edad="21" nombre="Luis"/>

```

```
6 </alumnos>
```

Ejemplo 15.13: Ejemplo de fichero XML

Para trabajar con ficheros XML en Java, se pueden utilizar bibliotecas como JAXB (Java Architecture for XML Binding) o DOM (Document Object Model). Estas bibliotecas permiten leer y escribir ficheros XML de manera sencilla, convirtiendo los datos en objetos Java y viceversa.

```
1 package programacion.ejemplos.xml;
2
3 import javax.xml.bind.JAXBContext;
4 import javax.xml.bind.JAXBException;
5 import javax.xml.bind.Unmarshaller;
6 import java.io.File;
7
8 public class LeerXML {
9     public static void main(String[] args) {
10         try {
11             JAXBContext context = JAXBContext.newInstance(Alumnos.class);
12             Unmarshaller unmarshaller = context.createUnmarshaller();
13             Alumnos alumnos = (Alumnos) unmarshaller.unmarshal(new File("alumnos.xml"));
14             for (Alumno a : alumnos.getAlumno()) {
15                 System.out.println(a);
16             }
17         } catch (JAXBException e) {
18             e.printStackTrace();
19         }
20     }
21 }
```

Ejemplo 15.14: Lectura de un fichero XML con JAXB

La clase `LeerXML` utiliza JAXB para leer un fichero XML y convertirlo en una lista de objetos `Alumno`. Esto se logra ya que JAXB permite mapear las etiquetas XML a clases Java, facilitando la manipulación de los datos.

```
1 package programacion.ejemplos.xml;
2
3 import java.util.List;
4 import javax.xml.bind.annotation.XmlElement;
5 import javax.xml.bind.annotation.XmlRootElement;
6
7 @XmlRootElement(name = "alumnos", namespace = "http://www.example.com/alumnos")
8 class Alumnos {
9     private List<Alumno> alumno;
10 }
```

```

11  @XmlElement(name = "alumno", namespace = "http://www.example.↵
    com/alumnos")
12  public List<Alumno> getAlumno() {
13      return alumno;
14  }
15
16  public void setAlumno(List<Alumno> alumno) {
17      this.alumno = alumno;
18  }
19 }

```

Ejemplo 15.15: Clase que representa la lista de alumnos

La clase `Alumnos` es una colección de objetos `Alumno`. Utiliza anotaciones de JAXB para indicar cómo se deben serializar y deserializar los datos XML. La anotación `@XmlRootElement` indica que esta clase es la raíz del documento XML, y la anotación `@XmlElement` indica que cada objeto `Alumno` se corresponde con un elemento XML.

```

1  package programacion.ejemplos.xml;
2
3  import javax.xml.bind.annotation.XmlAttribute;
4  import javax.xml.bind.annotation.XmlRootElement;
5
6  @XmlRootElement(name = "alumno", namespace = "http://www.example.↵
    .com/alumnos")
7  public class Alumno {
8      private String nombre;
9      private int edad;
10     private String carrera;
11
12     public Alumno() {
13         // Constructor por defecto requerido por JAXB
14     }
15
16     public Alumno(String nombre, int edad, String carrera) {
17         this.nombre = nombre;
18         this.edad = edad;
19         this.carrera = carrera;
20     }
21
22     @XmlAttribute(name = "nombre")
23     public String getNombre() {
24         return nombre;
25     }
26
27     public void setNombre(String nombre) {
28         this.nombre = nombre;
29     }
30 }

```

```

31  @XmlAttribute(name = "edad")
32  public int getEdad() {
33      return edad;
34  }
35
36  public void setEdad(int edad) {
37      this.edad = edad;
38  }
39
40  @XmlAttribute(name = "carrera")
41  public String getCarrera() {
42      return carrera;
43  }
44
45  public void setCarrera(String carrera) {
46      this.carrera = carrera;
47  }
48
49  @Override
50  public String toString() {
51      return "Nombre: " + nombre + ", Edad: " + edad + ", Carrera:↵
52      " + carrera;
53  }

```

Ejemplo 15.16: Clase que representa a un alumno

En esta clase, se definen los atributos de un alumno, como **nombre**, **edad** y **carrera**. Las anotaciones `@XmlAttribute` indican que estos atributos se corresponden con atributos XML. Además, se incluye un método `toString()` para facilitar la visualización de los datos del alumno.

Para escribir datos en un fichero XML, se puede utilizar el siguiente ejemplo:

```

1  package programacion.ejemplos.xml;
2
3  import javax.xml.bind.JAXBContext;
4  import javax.xml.bind.JAXBException;
5  import javax.xml.bind.Marshaller;
6  import java.io.File;
7  import java.util.List;
8
9  public class EscribirXML {
10     public static void main(String[] args) {
11         try {
12             JAXBContext context = JAXBContext.newInstance(Alumnos.↵
13             class);
14             Marshaller marshaller = context.createMarshaller();
15             marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, ↵
16             Boolean.TRUE);

```

```

16     Alumnos alumnos = new Alumnos();
17     alumnos.setAlumno(List.of(
18         new Alumno("Juan", 20, "Ingeniería"),
19         new Alumno("Ana", 22, "Matemáticas"),
20         new Alumno("Luis", 21, "Física")
21     ));
22
23     marshaller.marshal(alumnos, new File("alumnos.xml"));
24 } catch (JAXBException e) {
25     e.printStackTrace();
26 }
27 }
28 }

```

Ejemplo 15.17: Escritura de un fichero XML con JAXB

Importante

Para poder trabajar con ficheros XML en Java, es necesario incluir la biblioteca JAXB en el proyecto. A partir de Java 9, esta biblioteca no está incluida por defecto, por lo que se debe añadir como dependencia en el archivo `pom.xml` si se utiliza Maven, o descargarla manualmente si se trabaja sin un gestor de dependencias.

En Maven, se puede añadir las siguientes dependencias al archivo `pom.xml`:

```

1 <dependency>
2   <groupId>javax.xml.bind</groupId>
3   <artifactId>jaxb-api</artifactId>
4   <version>2.3.1</version>
5 </dependency>
6 <dependency>
7   <groupId>org.glassfish.jaxb</groupId>
8   <artifactId>jaxb-runtime</artifactId>
9   <version>2.3.1</version>
10 </dependency>
11 <dependency>
12   <groupId>org.glassfish.jaxb</groupId>
13   <artifactId>jaxb-core</artifactId>
14   <version>2.3.0.1</version>
15 </dependency>
16 <dependency>
17   <groupId>javax.activation</groupId>
18   <artifactId>activation</artifactId>
19   <version>1.1.1</version>
20 </dependency>

```

Asegúrate de que tu entorno de desarrollo esté configurado correctamente para utilizar estas bibliotecas, ya que son necesarias para trabajar con

ficheros XML en Java.

Ejercicio 15.6



Crea un programa con dos métodos estáticos, en uno creará un fichero XML, y en otro leerá el contenido de ese fichero y lo mostrará por pantalla. El fichero XML tendrá que contener un nodo raíz `calendario`, y dentro de este, varios nodos `evento` que representen eventos del calendario. Cada nodo `evento` tendrá los atributos `fecha`, `hora` y `descripcion`. Por ejemplo:

```
1 <calendario xmlns="http://www.example.com/calendario">
2   <evento fecha="2023-10-01" hora="10:00" descripcion="↵
   Reunión con el equipo"/>
3   <evento fecha="2023-10-02" hora="15:00" descripcion="↵
   Presentación del proyecto"/>
4   <evento fecha="2023-10-03" hora="09:00" descripcion="↵
   Desayuno con el cliente"/>
5 </calendario>
```

15.4. Utilización de los sistemas de ficheros

Java proporciona la clase `File` y, desde Java 7, la API `java.nio.file` para interactuar con el sistema de archivos. También se pueden utilizar las clases `Path` y `Files` para realizar operaciones más avanzadas.

15.4.1. Navegación por directorios

Para listar el contenido de un directorio y navegar por la estructura de carpetas, se puede utilizar la clase `File` o la API moderna `java.nio.file`. A continuación se muestran ejemplos con ambas opciones.

Listar archivos y subdirectorios con `File`

```
1 import java.io.File;
2
3 public class ListarDirectorio {
4     public static void main(String[] args) {
5         File directorio = new File(".");
6         File[] archivos = directorio.listFiles();
7         if (archivos != null) {
8             for (File archivo : archivos) {
```

```

9      if (archivo.isDirectory()) {
10         System.out.println("[DIR] " + archivo.getName());
11     } else {
12         System.out.println("[FILE] " + archivo.getName());
13     }
14 }
15 }
16 }
17 }

```

Ejemplo 15.18: Listar archivos y directorios con File

En este ejemplo, se lista el contenido del directorio actual. El método `isDirectory()` permite distinguir entre archivos y carpetas.

Navegación avanzada con `java.nio.file`

Desde Java 7, la API `java.nio.file` proporciona clases como `Path` y `Files` para trabajar con rutas y directorios de forma más flexible.

```

1 import java.nio.file.*;
2
3 public class ListarDirectorioNIO {
4     public static void main(String[] args) throws Exception {
5         Path ruta = Paths.get(".");
6         try (DirectoryStream<Path> stream = Files.newDirectoryStream(
7             ruta)) {
8             for (Path entry : stream) {
9                 if (Files.isDirectory(entry)) {
10                     System.out.println("[DIR] " + entry.getFileName());
11                 } else {
12                     System.out.println("[FILE] " + entry.getFileName());
13                 }
14             }
15         }
16 }

```

Ejemplo 15.19: Listar archivos usando `java.nio.file`

Esta API permite realizar operaciones más avanzadas, como filtrar archivos por extensión, recorrer subdirectorios recursivamente o acceder a atributos de los archivos.

Ejercicio 15.7



Crea un programa que recorra recursivamente un directorio y muestre por pantalla la estructura completa de carpetas y archivos, indicando el nivel de profundidad mediante sangrados o símbolos.

15.4.2. Creación y eliminación de ficheros y directorios

Para crear y eliminar archivos y directorios en Java, se pueden utilizar tanto la clase `File` como la API moderna `java.nio.file`. A continuación se muestran ejemplos de ambas formas.

Creación y eliminación con `File`

```

1 import java.io.File;
2 import java.io.IOException;
3
4 public class CrearEliminarFile {
5     public static void main(String[] args) {
6         File archivo = new File("nuevo_archivo.txt");
7         File directorio = new File("nueva_carpeta");
8
9         try {
10             if (archivo.createNewFile()) {
11                 System.out.println("Archivo creado: " + archivo.getName()
12             );
13             }
14             if (directorio.mkdir()) {
15                 System.out.println("Directorio creado: " + directorio.getName()
16             );
17             }
18             } catch (IOException e) {
19                 e.printStackTrace();
20             }
21
22             // Eliminar archivo y directorio
23             if (archivo.delete()) {
24                 System.out.println("Archivo eliminado.");
25             }
26             if (directorio.delete()) {
27                 System.out.println("Directorio eliminado.");
28             }
29         }
30     }
31 }

```

Ejemplo 15.20: Crear y eliminar archivos y directorios con `File`

El método `createNewFile()` crea un archivo vacío si no existe, y `mkdir()` crea un nuevo directorio. Para eliminar, se usa `delete()`. Ten en cuenta que un directorio solo se puede eliminar si está vacío.

Creación y eliminación con `java.nio.file`


```

1 import java.nio.file.*;
2
3 public class CrearEliminarNIO {
4     public static void main(String[] args) throws Exception {
5         Path archivo = Paths.get("archivo_nio.txt");
6         Path directorio = Paths.get("carpeta_nio");
7
8         // Crear archivo y directorio
9         Files.createFile(archivo);
10        Files.createDirectory(directorio);
11
12        // Eliminar archivo y directorio
13        Files.deleteIfExists(archivo);
14        Files.deleteIfExists(directorio);
15    }
16 }

```

Ejemplo 15.21: Crear y eliminar archivos y directorios con `java.nio.file`

La API `Files` proporciona métodos como `createFile()`, `createDirectory()`, `delete()` y `deleteIfExists()` para gestionar archivos y carpetas de forma sencilla y robusta.

Importante

Antes de eliminar un directorio, asegúrate de que esté vacío. Si necesitas eliminar un directorio con contenido, puedes usar el método `Files.walk()` para recorrer y eliminar todos los archivos y subdirectorios.

Ejercicio 15.8

Crema un programa que imprima por pantalla todos los ficheros que se encuentran en un directorio específico, y recursivamente en sus subdirectorios hasta un nivel máximo de profundidad (en este caso, 3). El programa deberá recibir como argumento el directorio a explorar. Utiliza la clase `Files` y sus métodos para listar los ficheros y directorios. El programa deberá mostrar el nombre de cada fichero, su tamaño en bytes y la ruta completa.

Resumen

En este capítulo se han presentado los conceptos y técnicas fundamentales para la gestión de entrada y salida de información en Java. Se ha explicado cómo realizar operaciones básicas de lectura desde teclado y escritura en pantalla, así como el uso de flujos de datos para trabajar con archivos binarios y de texto.

Se han mostrado ejemplos prácticos de lectura y escritura secuencial, acceso aleatorio a ficheros, y manipulación de archivos estructurados como CSV y XML. Además, se ha abordado la interacción con el sistema de archivos, incluyendo la creación, eliminación y navegación por directorios. Finalmente, se han propuesto ejercicios para afianzar los conocimientos adquiridos.

Cuadro 15.1: Principales métodos de la clase `Scanner`

Método	Descripción
<code>String</code> <code>nextLine()</code>	Lee una línea completa de texto
<code>String</code> <code>next()</code>	Lee la siguiente palabra (token)
<code>int</code> <code>nextInt()</code>	Lee el siguiente valor entero
<code>double</code> <code>nextDouble()</code>	Lee el siguiente valor de tipo <code>double</code>
<code>float</code> <code>nextFloat()</code>	Lee el siguiente valor de tipo <code>float</code>
<code>boolean</code> <code>nextBoolean()</code>	Lee el siguiente valor booleano
<code>boolean</code> <code>hasNext()</code>	Indica si hay otro token disponible
<code>boolean</code> <code>hasNextInt()</code>	Indica si el siguiente token es un entero
<code>boolean</code> <code>hasNextDouble()</code>	Indica si el siguiente token es un double
<code>void</code> <code>close()</code>	Cierra el objeto <code>Scanner</code>

Ejercicios de refuerzo

Ejercicio 15.9



El siguiente código abre un fichero para lectura pero presenta problemas en la gestión de recursos. Identifica los problemas y reescríbelo correctamente.

```
1 public static void leerFichero(String ruta) throws ↪
2     IOException {
3     FileReader fr = new FileReader(ruta);
4     BufferedReader br = new BufferedReader(fr);
5     String linea;
6     while ((linea = br.readLine()) != null) {
7         System.out.println(linea);
8     }
9     br.close();
10    // fr nunca se cierra si br.close() lanza una excepcion
11 }
```

Ver solución en la página 180.

Ejercicio 15.10



Necesitas leer línea a línea un fichero de texto de 1 GB. Se proponen tres alternativas:

- A) Usar `FileReader` directamente.
- B) Usar `BufferedReader` envolviendo a `FileReader`.
- C) Usar `Scanner` apuntando al fichero.

¿Cuál elegirías y por qué? Considera el rendimiento y el uso de memoria.
Ver solución en la página 180.

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 15.9



Los problemas son: si se produce una excepción antes de `br.close()`, el `BufferedReader` (y el `FileReader` subyacente) quedan abiertos, lo que supone una fuga de recursos. Además, `fr` nunca se cierra explícitamente si `br.close()` falla.

La solución es usar `try-with-resources`: se declara el `BufferedReader` en el paréntesis del `try` (`try (BufferedReader br = new BufferedReader(new FileReader(ruta)))`) y Java llama automáticamente a `close()` al salir del bloque, tanto en el caso normal como si se lanza una excepción. Esto funciona porque `BufferedReader` implementa `AutoCloseable`.

Solución del ejercicio 15.10



La opción recomendada es **B) `BufferedReader`**:

- `FileReader` (opción A) lee carácter a carácter del disco, lo que implica una llamada al sistema operativo por cada carácter: muy lento para ficheros grandes.
- `BufferedReader` (opción B) añade una capa de *buffering*: lee bloques del disco en memoria (por defecto 8 KB) y los sirve desde ahí, reduciendo drásticamente el número de operaciones de E/S. El método `readLine()` es directo y eficiente.
- `Scanner` (opción C) tiene más sobrecarga interna porque analiza tokens con expresiones regulares; es conveniente para entrada interactiva o ficheros pequeños, pero no está optimizado para leer ficheros grandes línea a línea.

Para ficheros muy grandes también se puede usar `Files.lines(Path)` de la API de Streams de Java, que permite procesar las líneas de forma lazy sin cargar todo el fichero en memoria.

Utilización avanzada de clases

En este capítulo profundizaremos en el uso avanzado de las clases en la programación orientada a objetos. Analizaremos conceptos fundamentales que permiten diseñar sistemas más robustos, reutilizables y mantenibles. Abordaremos temas como la composición de clases, la herencia y el polimorfismo, así como la organización de jerarquías mediante superclases y subclasses. También exploraremos el uso de clases y métodos abstractos y finales, el papel de las interfaces, la sobreescritura de métodos y el funcionamiento de los constructores en contextos de herencia. Estos conceptos son esenciales para aprovechar al máximo el paradigma orientado a objetos y desarrollar aplicaciones complejas de manera eficiente.

En este capítulo, se abordarán los siguientes temas:

1. **Sobrecarga de métodos:** Veremos cómo definir múltiples métodos con el mismo nombre, pero diferentes parámetros, lo que permite una mayor flexibilidad en la implementación de funcionalidades.
2. **Composición de clases:** Veremos cómo crear clases complejas a partir de otras, facilitando la reutilización de código y la representación de relaciones de tipo “tiene-un” entre objetos.
3. **Herencia y polimorfismo:** Estudiaremos cómo la herencia permite derivar nuevas clases a partir de existentes y cómo el polimorfismo posibilita que distintos objetos respondan de manera específica a los mismos métodos.
4. **Jerarquía de clases: Superclases y subclasses:** Analizaremos la organización jerárquica de las clases, donde las subclasses heredan características de las superclases.
5. **Constructores y herencia:** Examinaremos el funcionamiento de los constructores en jerarquías de clases y cómo asegurar la correcta inicialización de los objetos.

6. **Clases y métodos abstractos y finales:** Explicaremos el uso de clases y métodos abstractos para definir comportamientos que deben ser implementados por las subclases, y el papel de los elementos finales para restringir la herencia o la sobrescritura.
7. **Interfaces:** Abordaremos cómo las interfaces permiten definir contratos de métodos que las clases deben implementar, promoviendo la reutilización y la flexibilidad sin herencia múltiple.
8. **Enumerados:** Veremos cómo definir tipos enumerados para representar un conjunto fijo de constantes, mejorando la legibilidad y la seguridad del código.
9. **El tipo record:** Introduciremos el tipo `record` en Java, que permite definir clases inmutables de manera concisa, facilitando la creación de objetos que representan datos sin necesidad de escribir código boilerplate.

16.1. Sobrecarga de métodos

La sobrecarga de métodos (*method overloading*) se introdujo en el capítulo 7. En el contexto de clases, aplica exactamente igual: una clase puede definir varios métodos con el mismo nombre siempre que difieran en número o tipo de parámetros. El compilador selecciona el método correcto en cada llamada.

16.2. Composición de clases

La composición consiste en construir clases complejas a partir de otras, estableciendo relaciones de tipo “tiene-un”. Es preferible a la herencia cuando la relación no es estrictamente jerárquica.

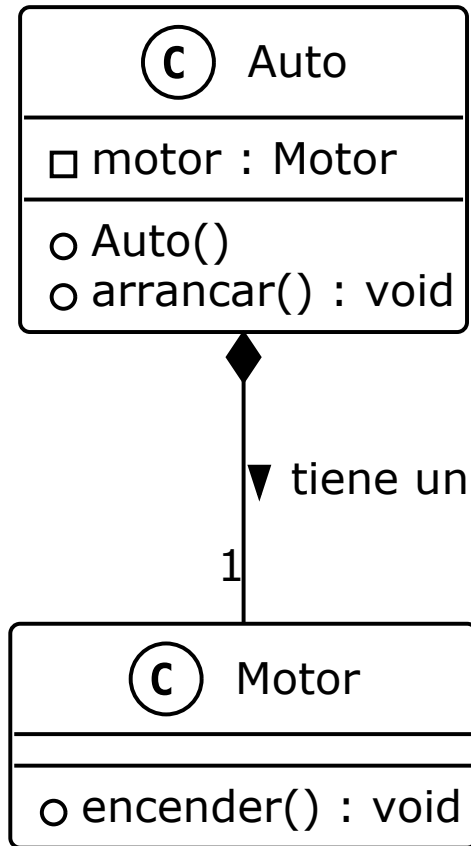
```
1 public class Motor {
2     public void encender() {
3         System.out.println("Motor encendido");
4     }
5 }
6
7 public class Auto {
8     private Motor motor;
9     public Auto() {
10         motor = new Motor();
11     }
12     public void arrancar() {
13         motor.encender();
14         System.out.println("Auto en marcha");
```

```

15 | }
16 | }

```

Ejemplo 16.1: Ejemplo de composición de clases en Java

Figura 16.1: Diagrama de composición: **Auto** contiene un **Motor**

El uso de la composición nos permite crear objetos más flexibles y reutilizables, ya que podemos combinar diferentes componentes sin necesidad de heredar de una clase base. Al usar herencia, podremos cambiar el comportamiento de la clase **Auto** sin necesidad de modificar la clase **Motor**, lo que facilita el mantenimiento y la evolución del código.

Ejercicio 16.1



Crea una clase **Libro** que contenga atributos como título, autor y número de páginas. Luego, crea una clase **Biblioteca** que tenga una lista de

libros. Implementa métodos para añadir, eliminar y listar los libros en la biblioteca. Utiliza la composición para relacionar las clases.

Crea un método `imprimirLibros` en la clase `Biblioteca` que imprima los títulos de todos los libros almacenados. Utiliza un bucle para recorrer la lista de libros y mostrar sus títulos. El formato de cada línea debe ser: **Título del libro - Autor del libro - Número de páginas**, y deberá ser generado dentro de la clase `Libro`.

16.3. Herencia y polimorfismo

La herencia permite crear nuevas clases basadas en otras existentes. El polimorfismo posibilita que objetos de diferentes clases respondan de manera específica a los mismos métodos. Esto permite diseñar sistemas más flexibles y reutilizables, donde las subclases pueden extender o modificar el comportamiento de las superclases.

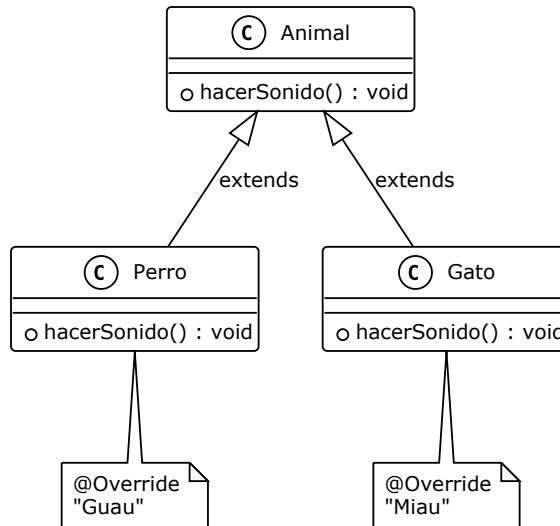
```

1 public class Animal {
2     public void hacerSonido() {
3         System.out.println("Sonido de animal");
4     }
5 }
6
7 public class Perro extends Animal {
8     @Override
9     public void hacerSonido() {
10         System.out.println("Guau");
11     }
12 }
13
14 public class Gato extends Animal {
15     @Override
16     public void hacerSonido() {
17         System.out.println("Miau");
18     }
19 }
20
21 // Uso del polimorfismo
22 Animal miAnimal = new Perro();
23 miAnimal.hacerSonido(); // Imprime "Guau"

```

Ejemplo 16.2: Ejemplo de herencia y polimorfismo en Java

Como puedes ver en el ejemplo, la clase `Animal` define un método `hacerSonido`, que es sobrescrito por las subclases `Perro` y `Gato`. Al crear una referencia de

Figura 16.2: Polimorfismo: Perro y Gato sobrescriben `hacerSonido()`

tipo `Animal` que apunta a un objeto de tipo `Perro`, podemos llamar al método `hacerSonido` y obtener el comportamiento específico del perro.

Para declarar una clase como subclase de otra, se utiliza la palabra clave **`extends`**. La subclase hereda todos los métodos y atributos de la superclase, y puede añadir nuevos o modificar los existentes. Además, las subclases pueden acceder a los métodos y atributos de la superclase utilizando la palabra clave **`super`**, en especial, podrán acceder a los miembros **`protected`**.

```

1 public class Vehiculo {
2     protected void encenderMotor() {
3         System.out.println("Motor encendido");
4     }
5 }
6
7 public class Coche extends Vehiculo {
8     public void arrancar() {
9         encenderMotor(); // Acceso permitido porque es protegido
10        System.out.println("Coche en marcha");
11    }
12 }
13
14 // Uso:
15 Coche miCoche = new Coche();
16 miCoche.arrancar(); // Imprime "Motor encendido" y "Coche en ↵
    marcha"
  
```

Ejemplo 16.3: Acceso a métodos protegidos de la superclase

16.3.1. Sobreescritura de métodos

La sobreescritura permite redefinir métodos heredados en subclases para adaptar o ampliar su funcionalidad.

```

1 public class Persona {
2     public void saludar() {
3         System.out.println("Hola");
4     }
5 }
6
7 public class Estudiante extends Persona {
8     @Override
9     public void saludar() {
10        System.out.println("Hola, soy estudiante");
11    }
12 }

```

Ejemplo 16.4: Ejemplo de sobreescritura de métodos en Java

Esto se logra utilizando la anotación `@Override` para indicar que el método está siendo sobreescrito. Al llamar al método `saludar` en un objeto de tipo `Estudiante`, se ejecutará la implementación específica de esa clase. Si queremos llamar al método de la superclase, podemos hacerlo utilizando `super.saludar()`. Esto evitará la necesidad de duplicar código y permitirá que las subclases personalicen el comportamiento de los métodos heredados.

Ejercicio 16.2



Crea una jerarquía de clases que represente diferentes tipos de vehículos: `Vehiculo`, `Coche`, `Moto` y `Camion`. Implementa un método `mostrarDetalles` en cada clase que imprima información específica del vehículo. Utiliza el polimorfismo para crear una lista de vehículos y llamar al método `mostrarDetalles` para cada uno.

Pista: Puedes utilizar una lista de tipo `List<Vehiculo>` para almacenar los diferentes tipos de vehículos. Luego, recorre la lista y llama al método `mostrarDetalles` en cada vehículo.

16.4. Jerarquía de clases: Superclases y subclases

Las superclases definen comportamientos y atributos comunes, mientras que las subclases los heredan y pueden añadir o modificar funcionalidades.

```

1 public class Garaje {
2     protected List<Vehiculo> vehiculos;
3 }

```

```

4  public void agregarVehiculo(Vehiculo vehiculo) {
5      vehiculos.add(vehiculo);
6  }
7
8  public void eliminarVehiculo(Vehiculo vehiculo) {
9      vehiculos.remove(vehiculo);
10 }
11
12 public void mostrarVehiculos() {
13     for (Vehiculo v : vehiculos) {
14         v.mostrarDetalles();
15     }
16 }
17 }
18
19 public class GarajeDeCoches extends Garaje {
20     @Override
21     public void agregarVehiculo(Vehiculo vehiculo) {
22         if (vehiculo instanceof Coche) {
23             super.agregarVehiculo(vehiculo);
24         } else {
25             System.out.println("Solo se pueden agregar coches al ↩
26 garaje de coches");
27         }
28 }

```

Ejemplo 16.5: Ejemplo de jerarquía de clases en Java

Consejo

Utiliza la palabra clave `instanceof` para verificar el tipo de un objeto antes de realizar una conversión de tipo (casting). Esto ayuda a evitar errores en tiempo de ejecución.

Es posible utilizar `instanceof` para comprobar si un objeto es una instancia de una clase específica o de una de sus subclasses, dentro de una sentencia `switch`:

```

1  switch (vehiculo) {
2      case Coche c -> System.out.println("Es un coche");
3      case Moto m -> System.out.println("Es una moto");
4      case Camion cam -> System.out.println("Es un camión");
5      default -> System.out.println("Tipo de vehículo ↩
6 desconocido");
7  }

```

Ejemplo 16.6: Uso de `instanceof` y `switch` con patrones

Ejercicio 16.3

Recupera el ejercicio 16.1 y crea una jerarquía de clases que represente diferentes tipos de libros: `Libro`, `LibroDigital` y `LibroFisico`. Implementa un método `mostrarDetalles` en cada clase que imprima información específica del libro. Utiliza el polimorfismo para crear una lista de libros y llamar al método `mostrarDetalles` para cada uno. Cada clase deberá heredar de una superclase común `Libro`.

16.5. Constructores y herencia

Los constructores de las superclases pueden ser invocados desde las subclases usando `super()`, asegurando la correcta inicialización de los objetos. Esto permite que las subclases hereden el estado inicial de la superclase y añadan su propia lógica de inicialización, así como sus propios campos y métodos.

```

1 public class Empleado {
2     private String nombre;
3     public Empleado(String nombre) {
4         this.nombre = nombre;
5     }
6 }
7
8 public class Gerente extends Empleado {
9     private String departamento;
10    public Gerente(String nombre, String departamento) {
11        super(nombre);
12        this.departamento = departamento;
13    }
14 }
```

Ejemplo 16.7: Ejemplo de constructores y herencia en Java

La llamada a `super(nombre)` en el constructor de `Gerente` asegura que el campo `nombre` de la superclase `Empleado` sea inicializado correctamente antes de que se ejecute el código específico del constructor de la subclase. Esta llamada debe ser la primera instrucción en el constructor de la subclase.

Importante

Si una superclase no tiene un constructor sin parámetros, es obligatorio llamar a uno de sus constructores desde la subclase utilizando `super()`. De lo contrario, el compilador generará un error. Además, si la superclase tiene un constructor con parámetros, la subclase debe llamar a ese constructor con los argumentos adecuados. Si no se

especifica una llamada a `super()`, el compilador intentará llamar al constructor sin parámetros de la superclase, lo que puede resultar en un error si no existe.

Ejercicio 16.4



Continúa con el ejercicio 16.1 y modifica los constructores de las clases `LibroDigital` y `LibroFisico` para que hereden de la superclase `Libro`. Asegúrate de que cada subclase inicialice correctamente los atributos heredados y los propios. Implementa un método `mostrarDetalles` en cada subclase que imprima información específica del libro, incluyendo el tipo de libro (digital o físico).

En este caso, la clase `LibroFisico` será la única con el parámetro `páginas`, mientras que la clase `LibroDigital` tendrá un parámetro adicional para el formato del libro (por ejemplo, PDF, EPUB, etc.).

16.6. Clases y métodos abstractos y finales

Las clases y métodos abstractos definen comportamientos que deben ser implementados por las subclases. Los elementos finales (`final`) no pueden ser modificados ni heredados. A diferencia de las clases normales, las clases abstractas no pueden ser instanciadas directamente. En su lugar, se utilizan como base para crear subclases que implementen los métodos abstractos definidos en la clase abstracta.

```

1 public abstract class Figura {
2     public abstract double area();
3     public final void mostrar() {
4         System.out.println("Esta figura tiene un área de: " + area()↵
5     };
6 }
7
8 public class Circulo extends Figura {
9     private double radio;
10    public Circulo(double radio) {
11        this.radio = radio;
12    }
13    @Override
14    public double area() {
15        return Math.PI * radio * radio;
16    }

```

17 }

Ejemplo 16.8: Ejemplo de clase y método abstracto y final en Java

Como puedes ver en el ejemplo, la clase **Figura** es abstracta y define un método abstracto **area()**. La subclase **Circulo** implementa este método y proporciona su propia lógica para calcular el área. Además, el método **mostrar()** es final, lo que significa que no puede ser sobrescrito por las subclases. Sin embargo, este método delega parte de su lógica al método abstracto **area()**, que sí debe ser implementado por las subclases. Esto facilita la reutilización de código y garantiza que todas las subclases proporcionen una implementación del método **area()**.

16.6.1. Clases sealed

Las clases **sealed** (selladas), introducidas en Java 17, permiten restringir qué clases pueden heredar de una clase o implementar una interfaz. Esto proporciona mayor control sobre la jerarquía de clases y mejora la seguridad y mantenibilidad del código.

Para declarar una clase como sellada, se utiliza la palabra clave **sealed** junto con la cláusula **permits**, que indica explícitamente qué subclases pueden extenderla. Las subclases deben ser **final**, **sealed** o **non-sealed**.

```
1 public sealed class Figura permits Circulo, Rectangulo { }
2
3 public final class Circulo extends Figura { }
4
5 public final class Rectangulo extends Figura { }
```

Ejemplo 16.9: Ejemplo de clase sealed en Java

En este ejemplo, la clase **Figura** es sellada y solo permite que las clases **Circulo** y **Rectangulo** la extiendan. Esto significa que no se pueden crear nuevas subclases de **Figura** a menos que se declare explícitamente en la cláusula **permits**. Si se intenta crear una subclase de **Figura** que no esté en la lista de permitidas, el compilador generará un error. Esto permite controlar la jerarquía de clases y evitar extensiones no deseadas.

También es posible declarar una subclase como **non-sealed** para permitir que otras clases la extiendan libremente:

```
1 public sealed class Figura permits Circulo, Rectangulo, Poligono →
   { }
2
3 public final class Circulo extends Figura { }
4
5 public final class Rectangulo extends Figura { }
6
```

```
7 | public non-sealed class Poligono extends Figura { }
```

Ejemplo 16.10: Subclase non-sealed

En este caso, la clase `Poligono` es `non-sealed`, lo que significa que cualquier clase puede extenderla, mientras que las clases `Circulo` y `Rectangulo` siguen siendo `final` y no pueden ser extendidas. Esto permite que en el futuro se puedan añadir nuevas subclases, pero con un control más estricto sobre la jerarquía de clases.

16.7. Interfaces

Las interfaces permiten definir contratos de métodos que las clases deben implementar. A diferencia de con las clases abstractas, una clase puede implementar múltiples interfaces. Las interfaces son útiles para definir comportamientos comunes que pueden ser implementados por diferentes clases, independientemente de su jerarquía de herencia.

```
1 | interface Imprimible {
2 |     void imprimir();
3 | }
4 |
5 | interface Guardable {
6 |     void guardar();
7 | }
8 |
9 | public class Documento implements Imprimible, Guardable {
10 |     @Override
11 |     public void imprimir() {
12 |         System.out.println("Imprimiendo documento...");
13 |     }
14 |
15 |     @Override
16 |     public void guardar() {
17 |         System.out.println("Guardando documento...");
18 |     }
19 | }
20 |
21 | // Uso:
22 | Documento doc = new Documento();
23 | doc.imprimir(); // Imprime "Imprimiendo documento..."
24 | doc.guardar();  // Imprime "Guardando documento..."
```

Ejemplo 16.11: Ejemplo de clase que implementa múltiples interfaces

En este ejemplo, la clase `Documento` implementa las interfaces `Imprimible` y `Guardable`, por lo que debe proporcionar una implementación para los métodos definidos en ambas interfaces. A diferencia de con la herencia, se usa la palabra

clave **implements** para indicar que una clase implementa una o más interfaces. Es posible implementar una o varias interfaces y extender de una superclase al mismo tiempo, lo que permite combinar comportamientos de diferentes fuentes.

```

1 class Animal {
2     public void comer() {
3         System.out.println("El animal come");
4     }
5 }
6
7 interface Volador {
8     void volar();
9 }
10
11 interface Nadador {
12     void nadar();
13 }
14
15 public class Pato extends Animal implements Volador, Nadador {
16     @Override
17     public void volar() {
18         System.out.println("El pato vuela");
19     }
20
21     @Override
22     public void nadar() {
23         System.out.println("El pato nada");
24     }
25 }
26
27 // Uso:
28 Pato pato = new Pato();
29 pato.comer(); // Imprime "El animal come"
30 pato.volar(); // Imprime "El pato vuela"
31 pato.nadar(); // Imprime "El pato nada"

```

Ejemplo 16.12: Ejemplo de extends e implements a la vez

En este ejemplo, la clase **Pato** hereda de **Animal** y también implementa las interfaces **Volador** y **Nadador**, combinando herencia e implementación de interfaces.

16.7.1. La interfaz Comparable

La interfaz **Comparable<T>** es una interfaz genérica de Java que permite definir un criterio de comparación natural entre objetos de una clase. Al implementar esta interfaz, una clase debe proporcionar la implementación del método **compareTo**, que se utiliza para comparar el objeto actual con otro objeto del mismo

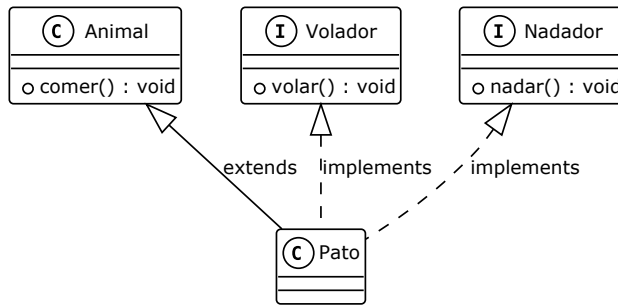


Figura 16.3: Herencia e implementación de interfaces: Pato extiende Animal e implementa Volador y Nadador

tipo.

```

1 public class Persona implements Comparable<Persona> {
2     private String nombre;
3     private int edad;
4
5     public Persona(String nombre, int edad) {
6         this.nombre = nombre;
7         this.edad = edad;
8     }
9
10    @Override
11    public int compareTo(Persona otra) {
12        return Integer.compare(this.edad, otra.edad);
13    }
14 }

```

Ejemplo 16.13: Implementación de la interfaz Comparable

En este ejemplo, la clase `Persona` implementa `Comparable<Persona>` y define el método `compareTo` para comparar personas por su edad. Si el resultado es negativo, el objeto actual es menor; si es cero, son iguales; si es positivo, es mayor.

Esto permite ordenar colecciones de objetos usando métodos como `Collections.sort()`:

```

1 List<Persona> personas = new ArrayList<>();
2 personas.add(new Persona("Ana", 30));
3 personas.add(new Persona("Luis", 25));
4 personas.add(new Persona("Marta", 35));
5
6 Collections.sort(personas);
7 // Ahora la lista está ordenada por edad ascendente

```

Ejemplo 16.14: Ordenar una lista de objetos con Comparable

Implementar `Comparable` es útil cuando se necesita un orden natural para los objetos de una clase, como al almacenar elementos en estructuras ordenadas (`TreeSet`, `TreeMap`) o al ordenar listas.

16.7.2. Métodos por defecto en interfaces

A partir de Java 8, las interfaces pueden incluir métodos por defecto (default methods), que proporcionan una implementación predeterminada. Esto permite añadir nuevas funcionalidades a las interfaces sin romper el código existente que las implementa.

Para definir un método por defecto, se utiliza la palabra clave `default`:

```

1 interface Saludo {
2     default void saludar() {
3         System.out.println("¡Hola!");
4     }
5 }
6
7 class Persona implements Saludo {
8     // Puede sobrescribir saludar() si lo desea
9 }
10
11 // Uso:
12 Persona p = new Persona();
13 p.saludar(); // Imprime "¡Hola!"

```

Ejemplo 16.15: Método por defecto en una interfaz

Las clases que implementan la interfaz pueden sobrescribir el método por defecto si necesitan un comportamiento específico. Los métodos por defecto son útiles para evolucionar interfaces y evitar la duplicación de código.

Ejercicio 16.5



Continúa con el ejercicio 16.1 y crea una interfaz `ItemBiblioteca` que defina métodos como `mostrarDetalles` (sin implementación), e `imprimirDetalles` (que imprima por pantalla el resultado de `mostrarDetalles()`). Implementa esta interfaz en las clases `Libro`, `LibroDigital` y `LibroFisico`. Asegúrate de que cada clase proporcione su propia implementación de los métodos definidos en la interfaz.

Pista: Comprueba que ocurre si solo declaras la clase abstracta `Libro` como extensión de la interfaz `ItemBiblioteca` y no implementas la interfaz en ella.

16.8. Enumerados

Los enumerados permiten definir un conjunto fijo de constantes, mejorando la legibilidad y seguridad del código.

```

1 public enum DiaSemana {
2     LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO
3 }
4
5 // Uso del enumerado
6 DiaSemana hoy = DiaSemana.LUNES;
7 System.out.println(hoy); // Imprime "LUNES"

```

Ejemplo 16.16: Ejemplo de enumerado en Java

Usaremos enumerados para representar valores fijos y conocidos, y cuando queramos que no se puedan crear instancias arbitrarias de una clase. Los enumerados pueden tener métodos, constructores y atributos, lo que los hace muy versátiles.

16.8.1. Enumerados con métodos y atributos

Los enumerados pueden tener constructores, atributos y métodos, igual que una clase normal, pero sus instancias son fijas y predefinidas. Esto permite asociar información adicional y comportamientos a cada constante del enumerado.

```

1 public enum NivelPrioridad {
2     BAJA(1), MEDIA(2), ALTA(3);
3
4     private final int valor;
5
6     NivelPrioridad(int valor) {
7         this.valor = valor;
8     }
9
10    public int getValor() {
11        return valor;
12    }
13
14    public void mostrarMensaje() {
15        switch (this) {
16            case BAJA -> System.out.println("Prioridad baja");
17            case MEDIA -> System.out.println("Prioridad media");
18            case ALTA -> System.out.println("¡Prioridad alta!");
19        }
20    }
21 }
22
23 // Uso:

```

```

24 NivelPrioridad prioridad = NivelPrioridad.ALTA;
25 System.out.println(prioridad.getValor()); // Imprime 3
26 prioridad.mostrarMensaje(); // Imprime "¡Prioridad alta!"

```

Ejemplo 16.17: Enumerado con atributos y métodos

Además, los enumerados pueden implementar interfaces, lo que permite definir comportamientos comunes para todas las constantes del enumerado.

```

1 interface Operacion {
2     int aplicar(int a, int b);
3 }
4
5 public enum TipoOperacion implements Operacion {
6     SUMA {
7         @Override
8         public int aplicar(int a, int b) {
9             return a + b;
10        }
11    },
12    RESTA {
13        @Override
14        public int aplicar(int a, int b) {
15            return a - b;
16        }
17    },
18    MULTIPLICACION {
19        @Override
20        public int aplicar(int a, int b) {
21            return a * b;
22        }
23    }
24 }
25
26 // Uso:
27 TipoOperacion op = TipoOperacion.SUMA;
28 System.out.println(op.aplicar(3, 4)); // Imprime 7

```

Ejemplo 16.18: Enumerado que implementa una interfaz

16.8.2. Iteración sobre enumerados

Para iterar sobre todas las constantes de un enumerado, se puede utilizar el método estático `values()`, que devuelve un arreglo con todas las instancias del enumerado en el orden en que fueron declaradas.

```

1 for (DiaSemana dia : DiaSemana.values()) {
2     System.out.println(dia);

```

3 | }

Ejemplo 16.19: Iterar sobre los valores de un enumerado

Esto imprimirá todos los días de la semana uno por línea. Este patrón es útil para mostrar menús, validar entradas o realizar operaciones sobre todos los valores posibles de un enumerado.

16.8.3. Conversión de String a un valor de enumerado

Para convertir una cadena de texto (`String`) en un valor de un enumerado, se puede utilizar el método estático `valueOf` que proporciona Java. Este método lanza una excepción si la cadena no coincide exactamente con el nombre de una constante del enumerado. Es común combinarlo con un bucle para validar la entrada del usuario:

```

1 String entrada = "VIERNES";
2 try {
3     DiaSemana dia = DiaSemana.valueOf(entrada);
4     System.out.println("Día seleccionado: " + dia);
5 } catch (IllegalArgumentException e) {
6     System.out.println("Valor no válido para el día de la semana."↵
7     );
8 }

```

Ejemplo 16.20: Conversión de String a enumerado

Si se desea hacer la conversión de forma insensible a mayúsculas/minúsculas, se puede iterar sobre los valores del enumerado:

```

1 String entrada = "viernes";
2 DiaSemana diaSeleccionado = null;
3 for (DiaSemana dia : DiaSemana.values()) {
4     if (dia.name().equalsIgnoreCase(entrada)) {
5         diaSeleccionado = dia;
6         break;
7     }
8 }
9 if (diaSeleccionado != null) {
10     System.out.println("Día seleccionado: " + diaSeleccionado);
11 } else {
12     System.out.println("Valor no válido para el día de la semana."↵
13     );
14 }

```

Ejemplo 16.21: Conversión insensible a mayúsculas/minúsculas

Este patrón es útil para validar y convertir entradas de usuario a valores de enumerados de forma segura.

16.8.4. Enumerados y switch

Los enumerados pueden utilizarse en sentencias **switch**, lo que permite ejecutar diferentes bloques de código según el valor del enumerado. A partir de Java 12, se puede usar la sintaxis mejorada de **switch** con flechas (->) y bloques de código.

```

1 DiaSemana dia = DiaSemana.LUNES;
2
3 switch (dia) {
4     case LUNES, MARTES, MIERCOLES, JUEVES, VIERNES ->
5         System.out.println("Es un día laborable");
6     case SABADO, DOMINGO ->
7         System.out.println("Es fin de semana");
8 }

```

Ejemplo 16.22: Uso de switch con enumerados

Esto facilita la escritura de código claro y seguro, ya que el compilador verifica que se contemplen todos los valores posibles del enumerado. Si se omite algún valor, el compilador puede advertirlo, ayudando a evitar errores.

Además, se pueden usar los enumerados en expresiones **switch** para devolver valores:

```

1 String mensaje = switch (dia) {
2     case LUNES -> "Comienza la semana";
3     case VIERNES -> ";Ya es viernes!";
4     case SABADO, DOMINGO -> "Disfruta el fin de semana";
5     default -> "Día normal";
6 };
7 System.out.println(mensaje);

```

Ejemplo 16.23: Switch como expresión con enumerados

El uso de enumerados con **switch** mejora la legibilidad y robustez del código, especialmente cuando se manejan conjuntos fijos de valores.

16.8.5. Enumerados como Singleton

Los enumerados en Java pueden utilizarse para implementar el patrón *Singleton* de forma sencilla y segura. Un *singleton* es una clase de la que solo puede existir una única instancia en toda la aplicación. Utilizar un enumerado para este propósito garantiza que la instancia sea única incluso en presencia de serialización o reflexión.

```

1 public enum GestorConfiguracion {
2     INSTANCIA;
3
4     private String configuracion;

```

```

5
6 public void cargar(String valor) {
7     this.configuracion = valor;
8 }
9
10 public String obtener() {
11     return configuracion;
12 }
13 }
14
15 // Uso:
16 GestorConfiguracion.INSTANCE.cargar("modo=produccion");
17 System.out.println(GestorConfiguracion.INSTANCE.obtener()); // ↩
    Imprime "modo=produccion"

```

Ejemplo 16.24: Singleton usando un enumerado

Este enfoque es recomendado por los expertos de Java, ya que es simple, seguro frente a ataques de serialización y garantiza que solo exista una instancia del *singleton*.

Ejercicio 16.6



Continúa el ejercicio 16.1 y crea un enumerado `GeneroLibro` que represente diferentes géneros literarios por ejemplo: `FICCION`, `NO_FICCION`, `CIENCIA_FICCION`, etc. Modifica las clases `Libro`, `LibroDigital` y `LibroFisico` para incluir un atributo de tipo `GeneroLibro`. Modifica el método `mostrarDetalles` en cada clase para incluir el género del libro. Utiliza un `switch` para imprimir un mensaje diferente según el género del libro.

16.9. El tipo *record*

A partir de Java 16, se introduce el tipo especial *record*, que permite definir clases inmutables de manera concisa. Los *records* son útiles para representar datos simples, como estructuras o tuplas, sin necesidad de escribir código repetitivo para constructores, métodos `equals`, `hashCode` y `toString`.

```

1 public record Punto(int x, int y) { }
2
3 // Uso del record
4 Punto p = new Punto(3, 5);
5 System.out.println(p.x()); // Imprime 3
6 System.out.println(p.y()); // Imprime 5
7 System.out.println(p);     // Imprime "Punto[x=3, y=5]"

```

Ejemplo 16.25: Ejemplo de record en Java

Los *records* son inmutables: sus campos no pueden cambiarse después de la creación del objeto. Además, los *records* pueden implementar interfaces y contener métodos adicionales si es necesario.

```

1 public record Persona(String nombre, int edad) {
2     public boolean esMayorDeEdad() {
3         return edad >= 18;
4     }
5 }
6
7 // Uso:
8 Persona persona = new Persona("Ana", 20);
9 System.out.println(persona.esMayorDeEdad()); // Imprime true

```

Ejemplo 16.26: Record con método adicional

El uso de *records* mejora la legibilidad y reduce errores al trabajar con datos inmutables.

Consejo



Utiliza *records* cuando necesites representar datos inmutables de forma sencilla y clara. Son ideales para DTOs (*Data Transfer Objects*) y otras estructuras de datos simples.

Puedes usar *records* para definir clases que representen entidades con atributos inmutables, como coordenadas, fechas, o cualquier otro tipo de dato que no necesite ser modificado una vez creado. Además, son útiles para agrupar parámetros de un método o función, evitando la necesidad de crear clases adicionales para representar esos datos.

Ejercicio 16.7



Crea un **record** llamado **Coordenadas** que represente un punto en un plano 2D con atributos **x** e **y** de tipo **int**, e implementa un método que calcule la distancia entre dos puntos. Dentro de este *record*, crea una constante que represente el punto de origen (0, 0).

Después, crea un método *angulo()* que calcule el ángulo entre dos puntos respecto al eje X. Utiliza la fórmula del ángulo en coordenadas polares: $\text{angulo} = \text{Math.atan2}(y, x)$. Asegúrate de que el método retorne el ángulo en grados. Una vez creado ese método, haz que los puntos sean ordenables, de forma que puedas ordenarlos por su ángulo respecto al origen.

Resumen

En este capítulo hemos explorado conceptos avanzados de la programación orientada a objetos en Java, como la sobrecarga y sobreescritura de métodos, la composición de clases, la herencia y el polimorfismo. Hemos visto cómo organizar jerarquías de clases mediante superclases y subclases, el uso de constructores en contextos de herencia y la importancia de las clases y métodos abstractos y finales. También se abordaron las interfaces, los enumerados y el tipo especial *record*, que permiten diseñar sistemas más robustos, flexibles y mantenibles. Estos conceptos son fundamentales para desarrollar aplicaciones complejas y aprovechar al máximo las capacidades del lenguaje Java.

Ejercicios de refuerzo

Ejercicio 16.8



Para cada uno de los siguientes tres casos, indica si usarías una **interfaz** o una **clase abstracta** y justifica tu respuesta:

1. Quieres definir un contrato de serialización que cualquier clase (sin relación entre sí) pueda implementar.
2. Tienes un grupo de clases relacionadas (`Animal` → `Perro`, `Gato`) y quieres compartir código de implementación común.
3. Una clase necesita asumir múltiples «roles» independientes (p. ej., un objeto que sea a la vez `Comparable` y `Serializable`).

Ver solución en la página 202.

Ejercicio 16.9



Diseña una jerarquía de excepciones para una aplicación bancaria que necesita representar los siguientes errores: fondos insuficientes, cuenta bloqueada y límite de transferencia excedido. Para cada excepción, indica si sería **checked** o **unchecked** y justifica la elección. *Ver solución en la página 202.*

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 16.8



1. **Interfaz:** no existe relación jerárquica entre las clases que la implementan, y solo se necesita el contrato (la firma de los métodos), no código compartido.
2. **Clase abstracta:** las clases comparten una relación «es-un» y se puede reutilizar código común en la superclase. Permite definir métodos concretos que las subclases heredan sin reescribirlos.
3. **Interfaces:** Java no permite herencia múltiple de clases, pero sí la implementación de múltiples interfaces. Cada «rol» se modela como una interfaz independiente, aplicando el principio de segregación de interfaces.

Solución del ejercicio 16.9



Una posible jerarquía:

- `BancoException` **extends** `Exception` — clase base checked para el dominio bancario.
- `FondosInsuficientesException` **extends** `BancoException` — **checked**: situación recuperable (el usuario puede ingresar fondos). El constructor recibe saldo y cantidad para construir un mensaje descriptivo.
- `CuentaBloqueadaException` **extends** `BancoException` — **checked**: el usuario puede contactar con el banco para desbloquearla.
- `LimiteTransferenciaExcedidoException` **extends** `RuntimeException` — **unchecked**: indica un error de programación (el código nunca debería intentar superar el límite configurado); no se espera recuperación en tiempo de ejecución.

La distinción clave es: usa **checked** para condiciones recuperables que el llamante debe anticipar, y **unchecked** para errores de programación de los que no tiene sentido recuperarse.

Gestión de bases de datos

Las bases de datos son herramientas fundamentales en el desarrollo de aplicaciones modernas, ya que permiten almacenar, organizar y gestionar grandes volúmenes de información de manera eficiente y segura. La correcta gestión de bases de datos facilita el acceso concurrente, la integridad de los datos y la escalabilidad de los sistemas. En este capítulo se abordarán los conceptos esenciales relacionados con el acceso, manipulación y administración de bases de datos, así como los estándares y características principales que rigen su funcionamiento. Los temas a tratar incluyen:

1. **Acceso a bases de datos:** Se explicarán los conceptos fundamentales sobre cómo las aplicaciones interactúan con las bases de datos, los diferentes tipos de bases de datos (relacionales y no relacionales), así como los estándares más utilizados (por ejemplo, SQL, NoSQL, ODBC, JDBC) y las características que deben cumplir los sistemas de gestión de bases de datos (SGBD), como la integridad, seguridad, concurrencia y escalabilidad.
2. **Establecimiento de conexiones:** Se detallará el proceso para conectar una aplicación con una base de datos, incluyendo la configuración de parámetros de conexión, autenticación de usuarios, manejo de errores y buenas prácticas para mantener conexiones seguras y eficientes.
3. **Almacenamiento, recuperación, actualización y eliminación de información en bases de datos:** Se abordarán las operaciones básicas de manipulación de datos (CRUD: Create, Read, Update, Delete), el uso de sentencias SQL para realizar estas operaciones, la gestión de transacciones para asegurar la consistencia de los datos y ejemplos prácticos de cómo implementar estas acciones desde distintos lenguajes de programación.

17.1. Acceso a bases de datos

El acceso a bases de datos es un proceso fundamental en el desarrollo de aplicaciones. Existen diferentes tipos de bases de datos, siendo las más comunes

las bases de datos relacionales (como MySQL, PostgreSQL, Oracle) y las no relacionales o NoSQL (como MongoDB, Redis). Para interactuar con ellas, se utilizan estándares y tecnologías como SQL para bases de datos relacionales y APIs específicas para NoSQL.

En el caso de Java, el estándar más utilizado para acceder a bases de datos es JDBC (Java Database Connectivity), que proporciona una API para conectar y ejecutar consultas en bases de datos desde aplicaciones Java.

Cuadro 17.1: Comparativa de sistemas de gestión de bases de datos populares

Base de datos		Tipo	Lenguaje principal	Características principales
MySQL		SQL	SQL	Código abierto, ampliamente utilizada, soporte ACID, replicación, comunidad grande.
PostgreSQL		SQL	SQL	Código abierto, extensible, soporte avanzado de transacciones, tipos de datos personalizados.
Oracle Database		SQL	SQL	Comercial, alto rendimiento, escalabilidad, características empresariales avanzadas.
Microsoft Server	SQL	SQL	SQL	Integración con productos Microsoft, herramientas de administración avanzadas, soporte empresarial.
SQLite		SQL	SQL	Ligera, embebida, sin servidor, ideal para aplicaciones móviles y de escritorio.
MongoDB		NoSQL	BSON / JSON	Orientada a documentos, escalabilidad horizontal, flexible, consultas ad-hoc.
Redis		NoSQL	Clave-valor	Almacenamiento en memoria, muy rápido, soporte para estructuras de datos avanzadas.
Cassandra		NoSQL	CQL	Distribuida, alta disponibilidad, escalabilidad horizontal, tolerancia a fallos.
Neo4j		NoSQL	Cypher	Orientada a grafos, ideal para relaciones complejas, consultas eficientes de grafos.

Base de datos	Tipo	Lenguaje principal	Características principales
Elasticsearch	NoSQL	JSON	Orientada a búsquedas, indexación de texto completo, análisis en tiempo real.

En este capítulo, nos centraremos en el acceso a bases de datos relacionales utilizando JDBC, que es la forma estándar de conectar aplicaciones Java con bases de datos. JDBC proporciona una interfaz para ejecutar consultas SQL, gestionar conexiones y manejar resultados de manera eficiente.

17.2. Establecimiento de conexiones

Para que una aplicación pueda interactuar con una base de datos, primero debe establecer una conexión. En Java, esto se realiza a través de JDBC, utilizando un *driver* específico para cada tipo de base de datos. A continuación se muestra un ejemplo básico de cómo establecer una conexión a una base de datos MySQL:

```

1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.SQLException;
4
5 public class ConexionBD {
6     public static void main(String[] args) {
7         String url = "jdbc:mysql://localhost:3306/mi_base_de_datos";
8         String usuario = "root";
9         String contrasena = "contrasena";
10
11         try (Connection conexion = DriverManager.getConnection(url, ↵
12             usuario, contrasena)) {
13             System.out.println("Conexión exitosa a la base de datos.")↵
14         } catch (SQLException e) {
15             System.out.println("Error al conectar: " + e.getMessage())↵
16         }
17     }
18 }

```

Ejemplo 17.1: Conexión a una base de datos MySQL con JDBC

Es importante manejar correctamente las excepciones y cerrar las conexiones para evitar fugas de recursos.

Consejo

Para la realización de los ejercicios de este capítulo, puedes utilizar una base de datos local como MySQL o SQLite. Si dispones de *Docker*, puedes levantar un contenedor con una base de datos MySQL o PostgreSQL para practicar. Para ello, ejecuta el siguiente comando:

```
1 docker run --name mysql -e MYSQL_ROOT_PASSWORD=contrasena -e MYSQL_DATABASE=mi_base_de_datos -p 3306:3306 mysql:latest
2
```

Recuerda disponer del *driver* JDBC correspondiente en tu proyecto. Para MySQL, puedes añadir la dependencia en tu archivo `pom.xml` si usas Maven:

```
1 <dependency>
2   <groupId>mysql</groupId>
3   <artifactId>mysql-connector-java</artifactId>
4   <version>8.0.33</version>
5 </dependency>
6
```

17.2.1. Parámetros de conexión

Los parámetros de conexión son los datos necesarios para que una aplicación pueda comunicarse correctamente con una base de datos. Los parámetros más comunes incluyen:

- **URL de conexión:** Especifica la ubicación de la base de datos y el tipo de SGBD. Por ejemplo, para MySQL: `jdbc:mysql://localhost:3306/mi_base_de_datos`.
- **Usuario:** Nombre de usuario con permisos para acceder a la base de datos.
- **Contraseña:** Clave asociada al usuario.
- **Opciones adicionales:** Algunos SGBD permiten añadir parámetros extra en la URL, como el uso de SSL, el juego de caracteres, el tiempo de espera, etc.

Un ejemplo de URL de conexión para diferentes bases de datos:

- MySQL: `jdbc:mysql://localhost:3306/mi_base_de_datos`
- PostgreSQL: `jdbc:postgresql://localhost:5432/mi_base_de_datos`

- SQLite: jdbc:sqlite:mi_base_de_datos.db

Importante

Es recomendable no almacenar las credenciales directamente en el código fuente. Utiliza archivos de configuración o variables de entorno para gestionar estos datos de forma segura.

Para leer las credenciales, puedes almacenarlas en variables de entorno. Podrás acceder a ellas en tu código Java de la siguiente manera:

```
1 String usuario = System.getenv("DB_USER");
2 String contrasena = System.getenv("DB_PASSWORD");
3
```

17.2.2. Ejecución de consultas SQL

Para ejecutar consultas SQL, como la creación de tablas, puedes utilizar el método ‘executeUpdate’ de un objeto ‘Statement’ o ‘PreparedStatement’. A continuación se muestra un ejemplo de cómo crear una tabla llamada ‘usuarios’ desde Java usando JDBC:

```
1 String sql = "CREATE TABLE IF NOT EXISTS usuarios (" +
2     "id INT AUTO_INCREMENT PRIMARY KEY," +
3     "nombre VARCHAR(100) NOT NULL," +
4     "email VARCHAR(100) NOT NULL UNIQUE" +
5     ")";
6 try (Statement stmt = conexion.createStatement()) {
7     stmt.executeUpdate(sql);
8     System.out.println("Tabla 'usuarios' creada correctamente.");
9 } catch (SQLException e) {
10    System.out.println("Error al crear la tabla: " + e.getMessage()
11    );
12 }
```

Ejemplo 17.2: Crear una tabla en la base de datos

17.3. Almacenamiento, recuperación, actualización y eliminación de información en bases de datos

Las operaciones básicas sobre una base de datos se conocen como operaciones CRUD (Crear, Leer, Actualizar, Eliminar). A continuación se presentan ejemplos de cómo realizar estas operaciones en Java usando JDBC:

17.3.1. Insertar datos

Una de las operaciones más comunes es insertar datos en una tabla. Para ello, se utiliza la sentencia ‘INSERT INTO’. A continuación se muestra un ejemplo de cómo insertar un registro en la tabla ‘usuarios’:

```
1 String sql = "INSERT INTO usuarios (nombre, email) VALUES (?, ?)↵
2 ";
3 try (PreparedStatement stmt = conexion.prepareStatement(sql)) {
4     stmt.setString(1, "Juan");
5     stmt.setString(2, "juan@email.com");
6     stmt.executeUpdate();
7 }
```

Ejemplo 17.3: Insertar un registro en la base de datos

En este ejemplo, se utiliza un ‘PreparedStatement’ para evitar inyección SQL y mejorar la seguridad. Los signos de interrogación (‘?’) son marcadores de posición que se reemplazarán por los valores reales al ejecutar la consulta.

17.3.2. Consultar datos

Para recuperar datos de una base de datos, se utiliza la sentencia ‘SELECT’. A continuación se muestra un ejemplo de cómo consultar todos los registros de la tabla ‘usuarios’:

```
1 String sql = "SELECT * FROM usuarios";
2 try (Statement stmt = conexion.createStatement();
3     ResultSet rs = stmt.executeQuery(sql)) {
4     while (rs.next()) {
5         System.out.println("ID: " + rs.getInt("id"));
6         System.out.println("Nombre: " + rs.getString("nombre"));
7         System.out.println("Email: " + rs.getString("email"));
8     }
9 }
```

Ejemplo 17.4: Consultar registros de la base de datos

El resultado de la consulta se almacena en un objeto ‘ResultSet’, que permite iterar sobre los registros devueltos. Cada llamada a ‘rs.next()’ avanza al siguiente registro. Los métodos ‘getInt’, ‘getString’, etc., se utilizan para obtener los valores de las columnas del registro actual. De esta manera, se pueden procesar los datos recuperados de la base de datos.

17.3.3. Actualizar datos

Para modificar datos existentes en una tabla, se utiliza la sentencia ‘UPDATE’. Es importante especificar una condición en la cláusula ‘WHERE’ para evitar ac-

tualizar todos los registros accidentalmente. El siguiente ejemplo muestra cómo actualizar el correo electrónico de un usuario identificado por su nombre:

```

1 String sql = "UPDATE usuarios SET email = ? WHERE nombre = ?";
2 try (PreparedStatement stmt = conexion.prepareStatement(sql)) {
3     stmt.setString(1, "nuevo@email.com");
4     stmt.setString(2, "Juan");
5     stmt.executeUpdate();
6 }

```

Ejemplo 17.5: Actualizar un registro en la base de datos

17.3.4. Eliminar datos

Para eliminar datos de una tabla, se utiliza la sentencia ‘DELETE’. Es fundamental incluir una condición en la cláusula ‘WHERE’ para evitar eliminar todos los registros accidentalmente. El siguiente ejemplo muestra cómo eliminar un usuario identificado por su nombre:

```

1 String sql = "DELETE FROM usuarios WHERE nombre = ?";
2 try (PreparedStatement stmt = conexion.prepareStatement(sql)) {
3     stmt.setString(1, "Juan");
4     stmt.executeUpdate();
5 }

```

Ejemplo 17.6: Eliminar un registro de la base de datos

Ejercicio 17.1



Crea un programa en Java, que gestione un listín telefónico. Debe permitir realizar las siguientes operaciones:

- Insertar un nuevo contacto con nombre, teléfono y correo electrónico.
- Consultar todos los contactos.
- Actualizar el teléfono de un contacto dado su nombre.
- Eliminar un contacto dado su nombre.
- Buscar un contacto por nombre y mostrar sus datos.
- Buscar un contacto por teléfono y mostrar sus datos.

*Pista: para simplificar el código, crea un método que cree la tabla, si el programa recibe por parámetro la palabra **init**. Puedes hacer que este programa sea interactivo, pidiendo al usuario qué operación desea realizar y mostrando un menú por consola. Ver solución en la página 214.*

17.4. Gestión de transacciones

Las transacciones permiten agrupar varias operaciones en una sola unidad de trabajo, asegurando que todas se realicen correctamente o ninguna se aplique en caso de error. En JDBC, se pueden gestionar transacciones de la siguiente manera:

```

1 try {
2     conexion.setAutoCommit(false);
3
4     // Operaciones SQL aquí
5
6     conexion.commit(); // Confirma la transacción
7 } catch (SQLException e) {
8     conexion.rollback(); // Revierte la transacción en caso de ↩
9     error
10 }

```

Ejemplo 17.7: Gestión de transacciones en JDBC

El método `setAutoCommit(false)` desactiva el modo de autocommit, lo que significa que las operaciones no se guardarán automáticamente en la base de datos hasta que se llame a `commit()`. Si ocurre un error, se puede llamar a `rollback()` para revertir todas las operaciones realizadas desde el último `commit`.

17.4.1. Gestión de transacciones en PostgreSQL

En PostgreSQL, las transacciones se gestionan de manera similar a otros SGBD. Sin embargo, es importante destacar que PostgreSQL ofrece características avanzadas como el aislamiento de transacciones y la posibilidad de utilizar puntos de guardado (*savepoints*) para revertir parcialmente una transacción.

```

1 BEGIN; -- Inicia una transacción
2 INSERT INTO usuarios (nombre, email) VALUES ('Ana', 'ana@example↩
3     .com');
4 COMMIT; -- Confirma la transacción

```

Ejemplo 17.8: Uso de transacciones en PostgreSQL

A diferencia de otros SGBD, PostgreSQL permite crear puntos de guardado dentro de una transacción, lo que permite revertir a un estado anterior sin deshacer toda la transacción. Esto es útil para manejar errores en operaciones complejas.

```

1 BEGIN; -- Inicia una transacción
2 SAVEPOINT sp1; -- Crea un punto de guardado
3 INSERT INTO usuarios (nombre, email) VALUES ('Ana', 'ana@example↩
4     .com');

```

```
4 | ROLLBACK TO sp1; -- Revierte al punto de guardado  
5 | COMMIT; -- Confirma la transacción
```

Ejemplo 17.9: Uso de savepoints en PostgreSQL

Este mecanismo permite una mayor flexibilidad en la gestión de transacciones, permitiendo deshacer solo ciertas partes de una transacción sin perder todo el trabajo realizado hasta ese momento. Además, PostgreSQL ejecutará automáticamente un *rollback* si la conexión se cierra inesperadamente, garantizando la integridad de los datos, así como ejecutar cualquier trigger definido en la base de datos, y asegurar que las restricciones de integridad se mantengan.

Consejo



Utiliza transacciones para garantizar la integridad de los datos, especialmente en operaciones que afectan a múltiples tablas o registros. Asegúrate de manejar correctamente las excepciones y realizar un *rollback* en caso de error.

Resumen

En este capítulo hemos aprendido los conceptos fundamentales sobre la gestión de bases de datos en el desarrollo de aplicaciones. Se han presentado los diferentes tipos de bases de datos, los estándares y tecnologías más utilizados, y cómo establecer conexiones seguras desde Java utilizando JDBC. Además, se han mostrado ejemplos prácticos de las operaciones básicas CRUD (crear, leer, actualizar y eliminar datos), así como la importancia de las transacciones para mantener la integridad de la información. Estos conocimientos son esenciales para desarrollar aplicaciones robustas y seguras que requieran almacenar y manipular datos de manera eficiente.

Cuadro 17.2: Principales métodos de la clase `ResultSet` en JDBC

Método	Descripción
<code>boolean next()</code>	Avanza el cursor al siguiente registro del conjunto de resultados. Devuelve <code>true</code> si hay un registro disponible.
<code>int getInt(String columnLabel)</code>	Obtiene el valor de la columna especificada como un entero.
<code>String getString(String columnLabel)</code>	Obtiene el valor de la columna especificada como una cadena de texto.
<code>double getDouble(String columnLabel)</code>	Obtiene el valor de la columna especificada como un número de punto flotante.
<code>boolean getBoolean(String columnLabel)</code>	Obtiene el valor de la columna especificada como un valor booleano.
<code>Date getDate(String columnLabel)</code>	Obtiene el valor de la columna especificada como un objeto <code>java.sql.Date</code> .
<code>Object getObject(String columnLabel)</code>	Obtiene el valor de la columna especificada como un objeto genérico.
<code>void close()</code>	Cierra el <code>ResultSet</code> y libera los recursos asociados.
<code>boolean wasNull()</code>	Indica si la última columna leída tenía valor SQL NULL.
<code>int findColumn(String columnLabel)</code>	Devuelve el índice de la columna especificada por nombre.
<code>ResultSetMetaData getMetaData()</code>	Devuelve información sobre las columnas del conjunto de resultados.

Ejercicios de refuerzo

Ejercicio 17.2



Usando un `Statement`, consulta la tabla `usuarios` de tu base de datos y muestra por pantalla el nombre y el email de cada usuario. Usa un `try-with-resources` para cerrar el `Statement` y el `ResultSet` automáticamente. *Ver solución en la página 216.*

Ejercicio 17.3



Usando un `PreparedStatement`, inserta un nuevo usuario en la tabla `usuarios` con los datos que elijas. Muestra el número de filas afectadas que devuelve `executeUpdate()`. *Ver solución en la página 217.*

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 17.1



```

1 // Solución completa: gestión de listín telefónico con JDBC
2 import java.sql.Connection;
3 import java.sql.DriverManager;
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.sql.SQLException;
7 import java.sql.Statement;
8
9 public class ListinTelefonico {
10
11     static final String URL = System.getenv().getOrDefault("↵
12         DB_URL", "jdbc:mysql://localhost:3306/programacion");
13     static final String USUARIO = System.getenv("DB_USER"); //↵
14         definir con: export DB_USER=<usuario>
15     static final String CONTRASENA = System.getenv("DB_PASS");↵
16         // definir con: export DB_PASS=<contraseña>
17
18     public static void main(String[] args) throws SQLException↵
19     {
20         if (args.length > 0 && args[0].equals("init")) {
21             crearTabla();
22         }
23         try (Connection conn = DriverManager.getConnection(URL, ↵
24             USUARIO, CONTRASENA)) {
25             insertar(conn, "Ana García", "612345678", "ana@ejemplo.↵
26             com");
27             insertar(conn, "Luis Pérez", "698765432", "luis@ejemplo.↵
28             com");
29
30             System.out.println("--- Todos los contactos ---");
31             consultarTodos(conn);
32
33             System.out.println("\n--- Buscar 'Ana García' ---");
34             buscarPorNombre(conn, "Ana García");
35
36             actualizarTelefono(conn, "Luis Pérez", "600111222");
37             System.out.println("\n--- Tras actualizar teléfono de ↵
38             Luis ---");
39             consultarTodos(conn);
40
41             eliminar(conn, "Ana García");
42             System.out.println("\n--- Tras eliminar a Ana ---");
43             consultarTodos(conn);

```

```

36     }
37 }
38
39 static void crearTabla() throws SQLException {
40     String sql = "CREATE TABLE IF NOT EXISTS contactos ("
41         + "id INT AUTO_INCREMENT PRIMARY KEY,"
42         + "nombre VARCHAR(100) NOT NULL,"
43         + "telefono VARCHAR(20),"
44         + "correo VARCHAR(100))";
45     try (Connection conn = DriverManager.getConnection(URL, ↵
46         USUARIO, CONTRASENA);
47         Statement st = conn.createStatement()) {
48         st.executeUpdate(sql);
49         System.out.println("Tabla 'contactos' creada.");
50     }
51 }
52
53 static void insertar(Connection conn, String nombre, ↵
54     String tel, String correo)
55     throws SQLException {
56     String sql = "INSERT INTO contactos (nombre, telefono, ↵
57     correo) VALUES (?, ?, ?)";
58     try (PreparedStatement ps = conn.prepareStatement(sql)) {
59         ps.setString(1, nombre);
60         ps.setString(2, tel);
61         ps.setString(3, correo);
62         ps.executeUpdate();
63     }
64 }
65
66 static void consultarTodos(Connection conn) throws ↵
67     SQLException {
68     try (Statement st = conn.createStatement();
69         ResultSet rs = st.executeQuery("SELECT nombre, telefono ↵
70         , correo FROM contactos")) {
71         while (rs.next()) {
72             System.out.printf(" %s | %s | %s%n",
73                 rs.getString("nombre"), rs.getString("telefono"), rs.↵
74                 getString("correo"));
75         }
76     }
77 }
78
79 static void buscarPorNombre(Connection conn, String nombre ↵
80 ) throws SQLException {
81     String sql = "SELECT nombre, telefono, correo FROM ↵
82     contactos WHERE nombre = ?";

```

```

75     try (PreparedStatement ps = conn.prepareStatement(sql)) {
76         ps.setString(1, nombre);
77         try (ResultSet rs = ps.executeQuery()) {
78             if (rs.next()) {
79                 System.out.printf(" %s | %s | %s%n",
80                     rs.getString("nombre"), rs.getString("telefono"), rs
81                     .getString("correo"));
82             } else {
83                 System.out.println(" Contacto no encontrado.");
84             }
85         }
86     }
87
88     static void actualizarTelefono(Connection conn, String nombre, String nuevoTel)
89         throws SQLException {
90         String sql = "UPDATE contactos SET telefono = ? WHERE nombre = ?";
91         try (PreparedStatement ps = conn.prepareStatement(sql)) {
92             ps.setString(1, nuevoTel);
93             ps.setString(2, nombre);
94             ps.executeUpdate();
95         }
96     }
97
98     static void eliminar(Connection conn, String nombre)
99         throws SQLException {
100         try (PreparedStatement ps =
101             conn.prepareStatement("DELETE FROM contactos WHERE nombre = ?")) {
102             ps.setString(1, nombre);
103             ps.executeUpdate();
104         }
105     }

```

Solución del ejercicio 17.2



```

1 // Fragmento: usar dentro de un bloque con Connection conn ya establecida
2 String sql = "SELECT nombre, email FROM usuarios";
3 try (Statement st = conn.createStatement();
4     ResultSet rs = st.executeQuery(sql)) {
5     while (rs.next()) {

```



```
6      System.out.println(rs.getString("nombre") + " - " + rs.↵  
      getString("email"));  
7  }  
8 }
```

Solución del ejercicio 17.3



```
1 // Fragmento: usar dentro de un bloque con Connection conn ↵  
   ya establecida  
2 String sql = "INSERT INTO usuarios (nombre, email) VALUES ↵  
   (?, ?)";  
3 try (PreparedStatement ps = conn.prepareStatement(sql)) {  
4     ps.setString(1, "Marta");  
5     ps.setString(2, "marta@ejemplo.com");  
6     int filas = ps.executeUpdate();  
7     System.out.println("Filas insertadas: " + filas);  
8 }
```


Mantenimiento de la persistencia de los objetos

La persistencia de los objetos es un aspecto fundamental en el desarrollo de aplicaciones modernas, ya que permite almacenar y recuperar información de manera eficiente y segura. En este capítulo se exploran los conceptos y mecanismos relacionados con la persistencia, haciendo especial énfasis en las bases de datos orientadas a objetos. Se analizarán sus características principales, los pasos para su instalación y configuración, así como las técnicas para la creación, consulta y manipulación de datos. Además, se abordarán los distintos tipos de datos y colecciones que pueden gestionarse en estos sistemas, proporcionando una visión integral sobre el mantenimiento de la persistencia en aplicaciones orientadas a objetos. Los temas a tratar incluyen:

1. **Bases de datos orientadas a objetos:** En este apartado se explicará qué son las bases de datos orientadas a objetos, sus ventajas frente a otros modelos y su relevancia en aplicaciones modernas.
2. **Características de las bases de datos orientadas a objetos:** Se detallarán las principales propiedades de este tipo de bases de datos, como la encapsulación, herencia y polimorfismo.
3. **Instalación del gestor de bases de datos:** Se describirán los pasos necesarios para instalar y configurar un gestor de bases de datos orientado a objetos, incluyendo requisitos y recomendaciones.
4. **Creación de bases de datos:** Se explicará el proceso para crear una base de datos orientada a objetos, definiendo las estructuras y objetos necesarios.
5. **Tipos de datos objeto; atributos y métodos:** Se analizarán los distintos tipos de datos objeto, así como la definición de atributos y métodos asociados.

6. **Tipos de datos colección:** Se describirán los tipos de datos colección disponibles en bases de datos orientadas a objetos y su utilidad para gestionar conjuntos de objetos.
7. **Mecanismos de consulta:** Se abordarán las diferentes formas de consultar información en bases de datos orientadas a objetos, destacando las ventajas de cada método.
8. **El lenguaje de consultas: sintaxis, expresiones, operadores:** Se presentará el lenguaje utilizado para realizar consultas, explicando su sintaxis, expresiones y operadores disponibles.
9. **Recuperación, modificación y borrado de información:** Se explicarán las técnicas para recuperar, modificar y eliminar datos almacenados en la base de datos.

18.1. Bases de datos orientadas a objetos

Las bases de datos orientadas a objetos permiten almacenar información en forma de objetos, manteniendo las características de la programación orientada a objetos como encapsulación, herencia y polimorfismo. Aunque existen gestores específicos, en la práctica se utilizan *frameworks* como Hibernate para mapear objetos Java a tablas en bases de datos relacionales como MySQL.

18.2. Características de las bases de datos orientadas a objetos

Las bases de datos orientadas a objetos (*OODB*) se diseñan para almacenar información en forma de objetos, tal como se representan en lenguajes de programación orientados a objetos. A diferencia de las bases de datos relacionales, donde los datos se organizan en tablas y filas, en una *OODB* los datos se almacenan como instancias de clases, permitiendo una correspondencia directa entre el modelo de datos y el modelo de programación.

Entre las ventajas de las bases de datos orientadas a objetos destacan:

- **Persistencia transparente:** Los objetos pueden ser almacenados y recuperados sin necesidad de convertirlos a otro formato, lo que simplifica el desarrollo y mantenimiento del software.
- **Soporte para herencia y polimorfismo:** Las *OODB* permiten que las relaciones de herencia entre clases se reflejen directamente en la base de datos, facilitando la reutilización y extensión de modelos de datos.

- **Encapsulamiento:** Los datos y los métodos asociados a los objetos se almacenan juntos, manteniendo la integridad y coherencia del modelo de datos.
- **Gestión de relaciones complejas:** Las referencias entre objetos pueden representarse de forma natural, permitiendo la modelización de estructuras de datos complejas como grafos, árboles y colecciones anidadas.

Algunos ejemplos de sistemas de gestión de bases de datos orientadas a objetos son db4o, ObjectDB y Versant. Sin embargo, en la práctica, muchas aplicaciones utilizan *frameworks* de mapeo objeto-relacional (*ORM*) como *Hibernate*, que permiten trabajar con objetos en el código y almacenarlos en bases de datos relacionales tradicionales. Estos *frameworks* proporcionan una capa de abstracción que traduce las operaciones sobre objetos en consultas SQL, facilitando la persistencia sin perder las ventajas de la programación orientada a objetos.

La elección entre una base de datos orientada a objetos y una relacional depende de los requisitos del proyecto, la complejidad del modelo de datos y la integración con otras tecnologías. En aplicaciones donde el modelo de datos es altamente jerárquico o requiere relaciones complejas entre objetos, una *OODB* puede ofrecer ventajas significativas en términos de rendimiento y facilidad de desarrollo.

18.3. Instalación del gestor de bases de datos

En esta sección se abordará el proceso de instalación y configuración de los sistemas necesarios para gestionar la persistencia de objetos en aplicaciones Java. Aunque existen gestores de bases de datos orientadas a objetos como *db4o* y *ObjectDB*, en este capítulo se opta por utilizar *Hibernate* junto con *MySQL*. Esta elección se debe a que *Hibernate* es un *framework* ampliamente utilizado en la industria para el mapeo objeto-relacional (*ORM*), lo que permite trabajar con objetos Java y almacenarlos en bases de datos relacionales de manera eficiente y transparente. *MySQL*, por su parte, es uno de los sistemas de gestión de bases de datos más populares y robustos, con gran soporte y documentación.

El uso combinado de *Hibernate* y *MySQL* facilita el aprendizaje de técnicas modernas de persistencia, proporciona mayor flexibilidad y asegura compatibilidad con proyectos reales. A continuación se describen los pasos generales para instalar y configurar estos sistemas, permitiendo implementar la persistencia de objetos en aplicaciones Java de forma práctica y profesional.

18.3.1. Instalación y configuración de *MySQL* e *Hibernate*

Para trabajar con persistencia en Java usando *Hibernate* y *MySQL*, primero se debe instalar *MySQL* y configurar el acceso desde Java.

1. **Instalar MySQL:** Descargue e instale MySQL desde <https://dev.mysql.com/downloads/installer/>.

2. **Crear una base de datos:**

```
CREATE DATABASE ejemplo_hibernate;
```

3. **Agregar dependencias de Hibernate y MySQL Connector en su proyecto Java (pom.xml):**

```
1  <dependency>
2    <groupId>org.hibernate.orm</groupId>
3    <artifactId>hibernate-core</artifactId>
4    <version>6.4.4.Final</version>
5  </dependency>
6  <dependency>
7    <groupId>mysql</groupId>
8    <artifactId>mysql-connector-java</artifactId>
9    <version>8.0.33</version>
10 </dependency>
11
```

Al igual que en el capítulo anterior, es posible ejecutar el gestor de bases de datos en un contenedor Docker. Esto permite una instalación rápida y sencilla, sin necesidad de configurar el entorno localmente. Podrás levantar la base de datos MySQL en un contenedor Docker con el siguiente comando:

```
1 docker run --name mysql -e MYSQL_ROOT_PASSWORD=tu_contrasena -e ↪
  MYSQL_DATABASE=ejemplo_hibernate -p 3306:3306 -d mysql:latest
```

18.4. Creación de bases de datos

Para crear una base de datos en MySQL, puede utilizar la consola de comandos o una herramienta gráfica como MySQL Workbench. El siguiente comando crea una base de datos llamada `ejemplo_hibernate`:

```
CREATE DATABASE ejemplo_hibernate;
```

Una vez creada la base de datos, Hibernate se encargará de generar las tablas necesarias a partir de las entidades Java definidas en el proyecto. Esto se logra mediante la configuración de la propiedad `hibernate.hbm2ddl.auto` en el archivo `persistence.xml`, que puede tomar valores como `update`, `create` o `validate`.

Por ejemplo, con la siguiente configuración:

```
1 <property name="hibernate.hbm2ddl.auto">update</property>
```

Hibernate actualizará automáticamente la estructura de la base de datos para reflejar los cambios en las entidades Java cada vez que la aplicación se inicie.

18.5. Tipos de datos objeto, atributos y métodos

En las bases de datos orientadas a objetos y en frameworks ORM como Hibernate, los tipos de datos objeto corresponden a las clases definidas en el lenguaje de programación. Cada clase representa una entidad y sus atributos se mapean a columnas de la base de datos. Los métodos permiten encapsular la lógica de negocio asociada a la entidad.

Por ejemplo, considere la siguiente entidad Java que representa un producto:

```
1 @Entity
2 @Table(name = "producto")
3 public class Producto {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String nombre;
9     private double precio;
10
11     // Método para aplicar descuento
12     public void aplicarDescuento(double porcentaje) {
13         this.precio = this.precio * (1 - porcentaje / 100);
14     }
15
16     // Getters y setters
17 }
```

En este ejemplo, `Producto` es un tipo de dato objeto, con atributos (`id`, `nombre`, `precio`) y métodos (`aplicarDescuento`). *Hibernate* se encarga de mapear estos atributos a columnas en la tabla `producto` de la base de datos, y los métodos permiten manipular los datos de la entidad de forma encapsulada.

18.6. Tipos de datos colección

En las bases de datos orientadas a objetos y en frameworks ORM como Hibernate, los tipos de datos colección permiten almacenar conjuntos de objetos relacionados, como listas, conjuntos o mapas. Estos tipos de colección son útiles para modelar relaciones uno-a-muchos o muchos-a-muchos entre entidades.

Hibernate soporta varias colecciones estándar de Java, como `List`, `Set` y `Map`. Para mapear una colección en una entidad, se utilizan anotaciones como `@OneToMany`, `@ManyToMany` y `@ElementCollection`.

Por ejemplo, considere una entidad `Categoria` que contiene una colección de productos:

```

1 @Entity
2 @Table(name = "categoria")
3 public class Categoria {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String nombre;
9
10    @OneToMany(mappedBy = "categoria", cascade = CascadeType.ALL)
11    private List<Producto> productos = new ArrayList<>();
12
13    // Getters y setters
14 }
```

En este ejemplo, la entidad `Categoria` tiene una colección de objetos `Producto`. La anotación `@OneToMany` indica que una categoría puede tener muchos productos asociados. *Hibernate* se encarga de gestionar la relación y la persistencia de los objetos en la base de datos.

Las colecciones permiten modelar relaciones complejas y facilitan la gestión de conjuntos de datos relacionados dentro de las aplicaciones orientadas a objetos.

18.7. Mecanismos de consulta

En los sistemas de persistencia orientados a objetos y *frameworks ORM* como *Hibernate*, existen diversos mecanismos para consultar información almacenada en la base de datos. Los más comunes son:

- **Consultas mediante el API de Hibernate:** Permite recuperar objetos utilizando métodos como `find`, `get`, o `createQuery`.
- **Lenguaje de consultas orientado a objetos (HQL):** Hibernate Query Language es similar a SQL pero opera sobre entidades y sus atributos, no

sobre tablas.

- **Consultas nativas SQL:** Es posible ejecutar directamente sentencias SQL sobre la base de datos.
- **Criterio API:** Permite construir consultas de manera programática y segura, útil para consultas dinámicas.

A continuación se presentan ejemplos de cada mecanismo.

18.7.1. Consultas mediante el API de Hibernate

El API de Hibernate permite recuperar objetos directamente usando métodos como `find` y `get` del `EntityManager` o `Session`. Por ejemplo:

```
1 Producto producto = entityManager.find(Producto.class, 1L);
2 // O usando Session de Hibernate
3 Producto producto2 = session.get(Producto.class, 2L);
```

Estos métodos devuelven la instancia del objeto correspondiente al identificador proporcionado, facilitando la recuperación de datos sin necesidad de escribir consultas SQL explícitas.

18.7.2. Lenguaje de consultas orientado a objetos (HQL)

Hibernate Query Language (HQL) es un lenguaje similar a SQL, pero opera sobre entidades y sus atributos. Por ejemplo, para obtener todos los productos con precio mayor a 100:

```
1 List<Producto> productos = entityManager.createQuery(
2     "FROM Producto p WHERE p.precio > 100", Producto.class
3 ).getResultList();
```

HQL permite realizar consultas complejas utilizando la estructura de objetos definida en el modelo.

Por ejemplo, para obtener todas las categorías que tienen al menos tres productos con precio mayor a 100, se puede utilizar HQL con una consulta agregada:

```
1 List<Categoria> categorias = entityManager.createQuery(
2     "SELECT c FROM Categoria c JOIN c.productos p " +
3     "WHERE p.precio > 100 GROUP BY c.id HAVING COUNT(p) >= 3", ↵
4     Categoria.class
5 ).getResultList();
```

Esta consulta recupera las instancias de `Categoria` que cumplen con el criterio especificado, demostrando cómo HQL permite expresar consultas complejas sobre relaciones y atributos de entidades.

También podemos realizar consultas en las que es necesario pasar parámetros específicos. Por ejemplo, para obtener los productos cuyo nombre contiene una palabra específica utilizando parámetros en HQL:

```
1 String palabraClave = "laptop";
2 List<Producto> productos = entityManager.createQuery(
3     "FROM Producto p WHERE p.nombre LIKE :clave", Producto.class
4 )
5 .setParameter("clave", "%" + palabraClave + "%")
6 .getResultList();
```

En este ejemplo, el parámetro `:clave` permite construir consultas dinámicas y seguras, evitando problemas de inyección de SQL.

18.7.3. Consultas nativas SQL

Hibernate también permite ejecutar consultas SQL nativas directamente sobre la base de datos. Por ejemplo:

```
1 List<Producto> productos = entityManager.createNativeQuery(
2     "SELECT * FROM producto WHERE precio > 100", Producto.class
3 ).getResultList();
```

Las consultas nativas son útiles cuando se requiere aprovechar funciones específicas del gestor de base de datos.

También es posible realizar consultas nativas con parámetros, y entre varias tablas. Por ejemplo, para obtener los productos de una categoría específica:

```
1 Long categoriaId = 1L;
2 List<Producto> productos = entityManager.createNativeQuery(
3     "SELECT * FROM producto WHERE categoria_id = :categoriaId", ↵
4     Producto.class
5 )
6 .setParameter("categoriaId", categoriaId)
7 .getResultList();
```

18.7.4. Criterias API

La Criteria API permite construir consultas de manera programática y segura, facilitando la creación de consultas dinámicas. Por ejemplo:

```
1 CriteriaBuilder cb = entityManager.getCriteriaBuilder();
2 CriteriaQuery<Producto> cq = cb.createQuery(Producto.class);
3 Root<Producto> root = cq.from(Producto.class);
4 cq.select(root).where(cb.gt(root.get("precio"), 100));
5 List<Producto> productos = entityManager.createQuery(cq).↵
6     getResultList();
```

Este enfoque es especialmente útil cuando los criterios de consulta se definen en tiempo de ejecución.

18.8. El lenguaje de consultas: sintaxis, expresiones, operadores

El lenguaje de consultas en Hibernate, conocido como HQL (Hibernate Query Language), es similar a SQL pero opera sobre entidades y sus atributos. La sintaxis de HQL permite realizar consultas complejas utilizando expresiones y operadores específicos.

A continuación se presentan ejemplos prácticos de sintaxis, expresiones y operadores en HQL:

Selección de entidades

Para obtener todos los productos:

```
1 List<Producto> productos = entityManager.createQuery(
2     "FROM Producto", Producto.class
3 ).getResultList();
```

Filtrado con operadores de comparación

Para obtener productos con precio mayor a 500:

```
1 List<Producto> productos = entityManager.createQuery(
2     "FROM Producto p WHERE p.precio > 500", Producto.class
3 ).getResultList();
```

Uso de operadores lógicos

Para obtener productos cuyo precio esté entre 100 y 1000:

```
1 List<Producto> productos = entityManager.createQuery(
2     "FROM Producto p WHERE p.precio >= 100 AND p.precio <= 1000", ↵
3     Producto.class
4 ).getResultList();
```

Expresiones de texto

Para buscar productos cuyo nombre contenga la palabra “tablet”:

```
1 List<Producto> productos = entityManager.createQuery(
2     "FROM Producto p WHERE p.nombre LIKE :clave", Producto.class
3 )
4 .setParameter("clave", "%tablet%")
5 .getResultList();
```

Ordenamiento de resultados

Para obtener productos ordenados por precio descendente:

```
1 List<Producto> productos = entityManager.createQuery(  
2     "FROM Producto p ORDER BY p.precio DESC", Producto.class  
3 ).getResultList();
```

Consultas agregadas

Para obtener el número total de productos:

```
1 Long total = entityManager.createQuery(  
2     "SELECT COUNT(p) FROM Producto p", Long.class  
3 ).getSingleResult();
```

Para obtener el precio promedio de los productos:

```
1 Double promedio = entityManager.createQuery(  
2     "SELECT AVG(p.precio) FROM Producto p", Double.class  
3 ).getSingleResult();
```

Consultas con relaciones

Para obtener los productos de una categoría específica:

```
1 List<Producto> productos = entityManager.createQuery(  
2     "FROM Producto p WHERE p.categoria.id = :categoriaId", ↵  
3     Producto.class  
4 )  
5 .setParameter("categoriaId", 2L)  
6 .getResultList();
```

Uso de parámetros

Para obtener productos con precio menor a un valor dado:

```
1 Double limite = 300.0;  
2 List<Producto> productos = entityManager.createQuery(  
3     "FROM Producto p WHERE p.precio < :limite", Producto.class  
4 )  
5 .setParameter("limite", limite)  
6 .getResultList();
```

Estos ejemplos muestran cómo utilizar la sintaxis de HQL, expresiones y operadores para realizar consultas eficientes y seguras sobre entidades y sus relaciones en Hibernate.

18.9. Recuperación, modificación y borrado de información

En los sistemas de persistencia orientados a objetos y frameworks ORM como Hibernate, las operaciones básicas sobre los datos incluyen la recuperación (lectura), modificación (actualización) y borrado (eliminación) de información. A continuación se presentan ejemplos prácticos de cada operación utilizando el EntityManager de JPA/Hibernate.

Recuperación de información

Para recuperar una entidad por su identificador:

```
1 Producto producto = entityManager.find(Producto.class, 1L);
```

Para recuperar una lista de entidades que cumplen un criterio:

```
1 List<Producto> productos = entityManager.createQuery(  
2     "FROM Producto p WHERE p.precio > 100", Producto.class  
3 ).getResultList();
```

Modificación de información

Para modificar los atributos de una entidad, primero se recupera el objeto, se actualizan sus valores y se confirma la transacción:

```
1 entityManager.getTransaction().begin();  
2 Producto producto = entityManager.find(Producto.class, 1L);  
3 producto.setPrecio(250.0);  
4 entityManager.getTransaction().commit();
```

También se pueden realizar actualizaciones masivas mediante HQL:

```
1 entityManager.getTransaction().begin();  
2 int actualizados = entityManager.createQuery(  
3     "UPDATE Producto p SET p.precio = p.precio * 0.9 WHERE p.↵  
4     precio > 500"  
5 ).executeUpdate();  
6 entityManager.getTransaction().commit();
```

Borrado de información

Para eliminar una entidad específica:

```
1 entityManager.getTransaction().begin();  
2 Producto producto = entityManager.find(Producto.class, 2L);  
3 entityManager.remove(producto);  
4 entityManager.getTransaction().commit();
```

Para eliminar varias entidades que cumplen un criterio:

```

1 entityManager.getTransaction().begin();
2 int eliminados = entityManager.createQuery(
3     "DELETE FROM Producto p WHERE p.precio < 50"
4 ).executeUpdate();
5 entityManager.getTransaction().commit();

```

Estas operaciones permiten gestionar de manera eficiente el ciclo de vida de los objetos persistentes en la base de datos, asegurando la integridad y coherencia de la información almacenada.

18.9.1. El concepto de repositorio

El patrón repositorio es una abstracción que encapsula la lógica de acceso a datos, proporcionando una interfaz clara para realizar operaciones de persistencia sobre entidades. En aplicaciones Java con *Hibernate*, los repositorios suelen implementarse como clases que utilizan el **EntityManager** para interactuar con la base de datos.

A continuación se muestra un ejemplo básico de un repositorio para la entidad **Producto**:

```

1 public class ProductoRepository {
2
3     private EntityManager entityManager;
4
5     public ProductoRepository(EntityManager entityManager) {
6         this.entityManager = entityManager;
7     }
8
9     public Producto findById(Long id) {
10         return entityManager.find(Producto.class, id);
11     }
12
13     public List<Producto> findAll() {
14         return entityManager.createQuery("FROM Producto", Producto.class)
15             .getResultList();
16     }
17
18     public void save(Producto producto) {
19         entityManager.getTransaction().begin();
20         entityManager.persist(producto);
21         entityManager.getTransaction().commit();
22     }
23
24     public void update(Producto producto) {
25         entityManager.getTransaction().begin();

```

```

26     entityManager.merge(producto);
27     entityManager.getTransaction().commit();
28 }
29
30 public void delete(Long id) {
31     entityManager.getTransaction().begin();
32     Producto producto = entityManager.find(Producto.class, id);
33     if (producto != null) {
34         entityManager.remove(producto);
35     }
36     entityManager.getTransaction().commit();
37 }
38 }

```

Este repositorio proporciona métodos para recuperar, guardar, actualizar y eliminar productos. El uso de repositorios facilita la organización del código, promueve la reutilización y permite separar la lógica de acceso a datos de la lógica de negocio.

En proyectos más grandes, es común definir interfaces para los repositorios y utilizar *frameworks* como *Spring Data JPA*, que generan automáticamente la implementación de los métodos básicos de persistencia. Por ejemplo:

```

1 public interface ProductoRepository extends JpaRepository<Producto, Long> {
2     List<Producto> findByPrecioGreaterThan(Double precio);
3 }

```

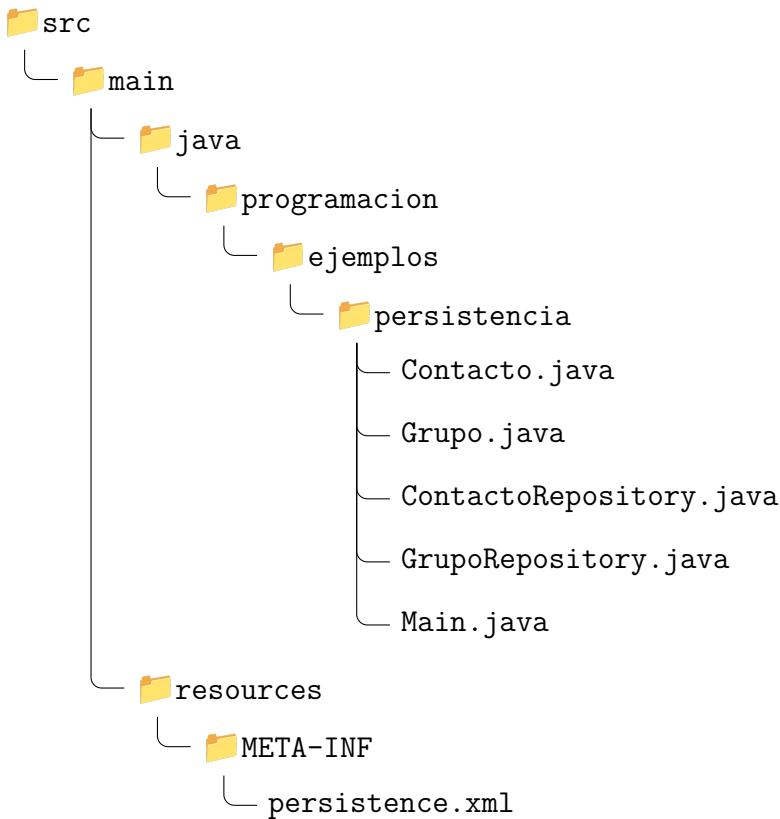
Con esta interfaz, Spring Data JPA proporciona automáticamente métodos para buscar productos por precio, sin necesidad de escribir la implementación manualmente. Además, permite personalizar consultas utilizando la convención de nombres, lo que simplifica aún más el acceso a datos.

El patrón repositorio es fundamental para mantener un diseño limpio y escalable en aplicaciones que utilizan Hibernate y JPA para la persistencia de objetos.

18.10. Ejemplo práctico: gestión de contactos

En esta sección se desarrollará una aplicación Java sencilla para la gestión de contactos, utilizando Hibernate y MySQL como sistema de persistencia. El ejemplo incluirá dos entidades relacionadas: **Contacto** y **Grupo**. Cada contacto pertenece a un grupo, y cada grupo puede tener varios contactos. Este ejemplo muestra cómo definir entidades relacionadas, configurar Hibernate, implementar repositorios y realizar operaciones básicas de persistencia en una aplicación Java para la gestión de contactos.

Estructura de archivos



Definición de entidades

```
1 package programacion.ejemplos.persistencia;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "contacto")
7 public class Contacto {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    private String nombre;
13    private String email;
14    private String telefono;
15
16    @ManyToOne
17    @JoinColumn(name = "grupo_id")
```



```

18 private Grupo grupo;
19
20 // Getters y setters
21 }

1 package programacion.ejemplos.persistencia;
2
3 import jakarta.persistence.*;
4 import java.util.ArrayList;
5 import java.util.List;
6
7 @Entity
8 @Table(name = "grupo")
9 public class Grupo {
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     private String nombre;
15
16     @OneToMany(mappedBy = "grupo", cascade = CascadeType.ALL)
17     private List<Contacto> contactos = new ArrayList<>();
18
19     // Getters y setters
20 }

```

Configuración de Hibernate

Cree el archivo `persistence.xml` con la configuración de conexión y mapeo de entidades:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence/↵
5     persistence_2_1.xsd"
6     version="2.1">
7     <persistence-unit name="ejemplo_hibernate" transaction-type="↵
8     RESOURCE_LOCAL">
9         <provider>org.hibernate.jpa.HibernatePersistenceProvider</↵
10        provider>
11        <class>programacion.ejemplos.persistencia.Contacto</class>
12        <class>programacion.ejemplos.persistencia.Grupo</class>
13        <properties>
14            <property name="javax.persistence.jdbc.driver" value="com.↵
15            mysql.cj.jdbc.Driver"/>
16            <property name="javax.persistence.jdbc.url" value="↵
17            jdbc:mysql://localhost:3306/ejemplo_hibernate"/>

```

```
13     <property name="javax.persistence.jdbc.user" value="root"/>↵
14     <property name="javax.persistence.jdbc.password" value="↵
    tu_contrasena"/>
15     <property name="hibernate.dialect" value="org.hibernate.↵
    dialect.MySQLDialect"/>
16     <property name="hibernate.hbm2ddl.auto" value="update"/>
17     <property name="hibernate.show_sql" value="true"/>
18 </properties>
19 </persistence-unit>
20 </persistence>
```

Repositorios para acceso a datos

```
1 package programacion.ejemplos.persistencia;
2
3 import jakarta.persistence.*;
4 import java.util.List;
5
6 public class ContactoRepository {
7     private EntityManager entityManager;
8
9     public ContactoRepository(EntityManager em) {
10         this.entityManager = em;
11     }
12
13     public void save(Contacto contacto) {
14         entityManager.getTransaction().begin();
15         entityManager.persist(contacto);
16         entityManager.getTransaction().commit();
17     }
18
19     public Contacto findById(Long id) {
20         return entityManager.find(Contacto.class, id);
21     }
22
23     public List<Contacto> findAll() {
24         return entityManager.createQuery("FROM Contacto", Contacto.↵
            class).getResultList();
25     }
26
27     public List<Contacto> findByGrupo(Long groupId) {
28         return entityManager.createQuery(
29             "FROM Contacto c WHERE c.grupo.id = :groupId", Contacto.↵
            class)
30             .setParameter("groupId", groupId)
31             .getResultList();
32     }
```

```

33 }

1 package programacion.ejemplos.persistencia;
2
3 import jakarta.persistence.*;
4 import java.util.List;
5
6 public class GrupoRepository {
7     private EntityManager entityManager;
8
9     public GrupoRepository(EntityManager em) {
10         this.entityManager = em;
11     }
12
13     public void save(Grupo grupo) {
14         entityManager.getTransaction().begin();
15         entityManager.persist(grupo);
16         entityManager.getTransaction().commit();
17     }
18
19     public Grupo findById(Long id) {
20         return entityManager.find(Grupo.class, id);
21     }
22
23     public List<Grupo> findAll() {
24         return entityManager.createQuery("FROM Grupo", Grupo.class).↵
25         getResultList();
26     }
27 }

```

Clase principal para pruebas

```

1 package programacion.ejemplos.persistencia;
2
3 import jakarta.persistence.*;
4 import java.util.List;
5
6 public class Main {
7     public static void main(String[] args) {
8         EntityManagerFactory emf = Persistence.↵
9         createEntityManagerFactory("ejemplo_hibernate");
10         EntityManager em = emf.createEntityManager();
11
12         GrupoRepository grupoRepo = new GrupoRepository(em);
13         ContactoRepository contactoRepo = new ContactoRepository(em)↵
14         ;
15
16         // Crear grupo
17         Grupo amigos = new Grupo();

```

```

16     amigos.setNombre("Amigos");
17     grupoRepo.save(amigos);
18
19     // Crear contactos
20     Contacto c1 = new Contacto();
21     c1.setNombre("Ana Pérez");
22     c1.setEmail("ana@email.com");
23     c1.setTelefono("6*****");
24     c1.setGrupo(amigos);
25
26     Contacto c2 = new Contacto();
27     c2.setNombre("Luis Gómez");
28     c2.setEmail("luis@email.com");
29     c2.setTelefono("9*****");
30     c2.setGrupo(amigos);
31
32     contactoRepo.save(c1);
33     contactoRepo.save(c2);
34
35     // Consultar contactos por grupo
36     List<Contacto> contactosAmigos = contactoRepo.findByGrupo(↵
amigos.getId());
37     for (Contacto c : contactosAmigos) {
38         System.out.println(c.getNombre() + " - " + c.getEmail());
39     }
40
41     em.close();
42     emf.close();
43 }
44 }

```

Ejercicio 18.1



Cree una aplicación Java que implemente un sistema de gestión de tareas utilizando Hibernate y MySQL. El sistema debe permitir crear, actualizar, eliminar y listar tareas, cada una con un título, descripción, estado (pendiente, en progreso, completada) y fecha de creación. Además, implementa una funcionalidad para filtrar tareas por estado. El sistema debe incluir las siguientes entidades:

- **Tarea:** Representa una tarea con atributos como `titulo`, `descripcion`, `estado` y `fechaCreacion`.
- **Usuario:** Representa un usuario que puede tener varias tareas asignadas.

Ver solución en la página 240.

18.11. Resumen

En este capítulo se ha explorado el concepto de persistencia de objetos en aplicaciones Java, destacando la importancia de almacenar y recuperar información de manera eficiente. Se han presentado las bases de datos orientadas a objetos y sus principales características, así como el uso de *frameworks ORM* como *Hibernate* junto con *MySQL* para implementar la persistencia. Se han detallado los pasos de instalación, la creación de bases de datos, el mapeo de entidades y colecciones, y los mecanismos de consulta mediante *HQL*, SQL nativo y *Criteria API*. Además, se han explicado las operaciones básicas de recuperación, modificación y borrado de datos, y el patrón repositorio para organizar el acceso a datos. Finalmente, se ha desarrollado un ejemplo práctico de gestión de contactos y se ha propuesto un ejercicio para implementar un sistema de gestión de tareas, consolidando los conocimientos adquiridos sobre persistencia en aplicaciones orientadas a objetos.

Cuadro 18.1: Elementos del lenguaje HQL

Elemento HQL	Descripción y Ejemplo
FROM	Selecciona entidades. Ejemplo: FROM Producto
WHERE	Filtra resultados. Ejemplo: FROM Producto p WHERE p.precio >100
ORDER BY	Ordena resultados. Ejemplo: FROM Producto p ORDER BY p.precio DESC
SELECT	Selecciona atributos específicos. Ejemplo: SELECT p.nombre FROM Producto p
JOIN	Une entidades relacionadas. Ejemplo: FROM Categoria c JOIN c.productos p
GROUP BY	Agrupar resultados. Ejemplo: SELECT c FROM Categoria c JOIN c.productos p GROUP BY c.id
HAVING	Filtra grupos. Ejemplo: HAVING COUNT(p) >3
COUNT, AVG, SUM, MIN, MAX	Funciones agregadas. Ejemplo: SELECT COUNT(p) FROM Producto p
UPDATE	Actualiza entidades. Ejemplo: UPDATE Producto p SET p.precio = p.precio * 0.9 WHERE p.precio >500
DELETE	Elimina entidades. Ejemplo: DELETE FROM Producto p WHERE p.precio <50
LIKE	Búsqueda por patrón. Ejemplo: FROM Producto p WHERE p.nombre LIKE :clave
IN	Filtra por lista de valores. Ejemplo: FROM Producto p WHERE p.estado IN ('pendiente', 'completada')
BETWEEN	Filtra por rango. Ejemplo: FROM Producto p WHERE p.precio BETWEEN 100 AND 500
IS NULL, IS NOT NULL	Filtra valores nulos. Ejemplo: FROM Producto p WHERE p.descripcion IS NULL
DISTINCT	Elimina duplicados. Ejemplo: SELECT DISTINCT p.estado FROM Producto p
SET PARAMETER	Parámetros en consultas. Ejemplo: .setParameter("clave", "%tablet%")

Ejercicios de refuerzo

Ejercicio 18.2



Usando `EntityManager`, persiste un objeto `Producto` con el nombre y precio que elijas. Tras el `commit()`, imprime el `id` asignado automáticamente por JPA y verifica que no es cero. *Ver solución en la página 242.*

Ejercicio 18.3



Escribe una consulta JPQL que devuelva todos los `Producto` cuyo precio sea superior a 50€, ordenados de menor a mayor precio. Usa un parámetro nombrado `:min`. Imprime el nombre y precio de cada resultado. *Ver solución en la página 243.*

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 18.1



Entidad Tarea:

```

1 // Entidad JPA: Tarea
2 import jakarta.persistence.Column;
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.EnumType;
5 import jakarta.persistence.Enumerated;
6 import jakarta.persistence.GeneratedValue;
7 import jakarta.persistence.GenerationType;
8 import jakarta.persistence.Id;
9 import jakarta.persistence.Table;
10 import java.time.LocalDate;
11
12 @Entity
13 @Table(name = "tareass")
14 public class Tarea {
15
16     public enum Estado { PENDIENTE, EN_PROGRESO, COMPLETADA }
17
18     @Id
19     @GeneratedValue(strategy = GenerationType.IDENTITY)
20     private Long id;
21
22     @Column(nullable = false)
23     private String titulo;
24
25     private String descripcion;
26
27     @Enumerated(EnumType.STRING)
28     private Estado estado = Estado.PENDIENTE;
29
30     private LocalDate fechaCreacion = LocalDate.now();
31
32     public Tarea() {}
33
34     public Tarea(String titulo, String descripcion) {
35         this.titulo = titulo;
36         this.descripcion = descripcion;
37     }
38
39     public Long getId() { return id; }
40     public String getTitulo() { return titulo; }
41     public Estado getEstado() { return estado; }

```



```

42 public void setEstado(Estado estado) { this.estado = estado; }
43
44 @Override
45 public String toString() {
46     return String.format("[%s] %s (%s)", estado, titulo, fechaCreacion);
47 }
48 }

```

Servicio GestorTareas:

```

1 // Fragmento: servicio de gestión de tareas con EntityManager
2 import jakarta.persistence.EntityManager;
3 import java.util.List;
4
5 public class GestorTareas {
6
7     private final EntityManager em;
8
9     public GestorTareas(EntityManager em) {
10         this.em = em;
11     }
12
13     public void guardar(Tarea tarea) {
14         em.getTransaction().begin();
15         em.persist(tarea);
16         em.getTransaction().commit();
17         System.out.println("Tarea guardada: " + tarea);
18     }
19
20     public List<Tarea> listarTodas() {
21         return em.createQuery("FROM Tarea t ORDER BY t.fechaCreacion", Tarea.class)
22             .getResultList();
23     }
24
25     public List<Tarea> filtrarPorEstado(Tarea.Estado estado) {
26         return em.createQuery(
27             "FROM Tarea t WHERE t.estado = :estado ORDER BY t.titulo", Tarea.class)
28             .setParameter("estado", estado)
29             .getResultList();
30     }
31
32     public void cambiarEstado(Long id, Tarea.Estado nuevoEstado) {

```

```

33     em.getTransaction().begin();
34     Tarea tarea = em.find(Tarea.class, id);
35     if (tarea != null) {
36         tarea.setEstado(nuevoEstado);
37     }
38     em.getTransaction().commit();
39 }
40
41 public void eliminar(Long id) {
42     em.getTransaction().begin();
43     Tarea tarea = em.find(Tarea.class, id);
44     if (tarea != null) {
45         em.remove(tarea);
46     }
47     em.getTransaction().commit();
48 }
49 }
50
51 // Uso en main (con emf = EntityManagerFactory ya creado):
52 // EntityManager em = emf.createEntityManager();
53 // GestorTareas gestor = new GestorTareas(em);
54 //
55 // gestor.guardar(new Tarea("Estudiar JPA", "Repasar ↪
56 // anotaciones"));
57 // gestor.guardar(new Tarea("Hacer tests", "JUnit 5"));
58 //
59 // gestor.listarTodas().forEach(System.out::println);
60 //
61 // gestor.cambiarEstado(1L, Tarea.Estado.EN_PROGRESO);
62 // gestor.filtrarPorEstado(Tarea.Estado.EN_PROGRESO).↪
63 //     forEach(System.out::println);

```

Solución del ejercicio 18.2



```

1 // Fragmento: usar con EntityManagerFactory emf ya creado
2 EntityManager em = emf.createEntityManager();
3 em.getTransaction().begin();
4
5 Producto p = new Producto();
6 p.setNombre("Teclado mecánico");
7 p.setPrecio(89.99);
8 em.persist(p);
9
10 em.getTransaction().commit();
11 System.out.println("ID asignado por JPA: " + p.getId());
12 em.close();

```

Solución del ejercicio 18.3



```
1 // Fragmento: usar con EntityManager em ya creado
2 double precioMinimo = 50.0;
3 List<Producto> resultados = em.createQuery(
4     "FROM Producto p WHERE p.precio > :min ORDER BY p.↵
5     precio ASC",
6     Producto.class)
7     .setParameter("min", precioMinimo)
8     .getResultList();
9 resultados.forEach(p ->
10     System.out.println(p.getNombre() + ": " + p.getPrecio() +↵
11     "€"));
```


Interfaces gráficas

En este capítulo exploraremos el desarrollo de interfaces gráficas de usuario (GUI) en Java. Aprenderás cómo crear aplicaciones visuales que permiten la interacción mediante ventanas, botones y otros componentes gráficos. Veremos los conceptos fundamentales de eventos y controladores, y construiremos ejemplos prácticos utilizando la biblioteca Swing. Al finalizar, tendrás las bases para diseñar programas interactivos y atractivos. Los temas a tratar incluyen:

1. **Interfaces gráficas:** Introducción a las interfaces gráficas de usuario (GUI), su importancia y los principales componentes visuales que permiten la interacción con el usuario. Se explicarán las bibliotecas disponibles en Java para crear GUIs, como AWT, Swing y JavaFX.
2. **Concepto de evento:** Explicación de qué son los eventos en una interfaz gráfica, cómo se generan y cómo permiten que la aplicación responda a las acciones del usuario, como clics de botones, movimientos del ratón y entradas de teclado.
3. **Creación de controladores de eventos:** Detalle sobre cómo implementar y asociar controladores (listeners) a los componentes gráficos para gestionar eventos. Se mostrarán ejemplos prácticos de cómo detectar y responder a diferentes tipos de eventos en Java.

19.1. Interfaces gráficas

Una interfaz gráfica de usuario (GUI, por sus siglas en inglés) permite a los usuarios interactuar con una aplicación mediante elementos visuales como ventanas, botones, cuadros de texto y menús. En Java, las interfaces gráficas se pueden crear utilizando bibliotecas como AWT, Swing y JavaFX.

Importante

Swing es la biblioteca más utilizada para crear interfaces gráficas en Java estándar. JavaFX es más moderna y recomendada para aplicaciones nuevas, pero Swing sigue siendo ampliamente usada en educación y proyectos existentes.

Además de Swing, existen otras bibliotecas para crear interfaces gráficas en Java:

- **AWT (Abstract Window Toolkit):** Fue la primera biblioteca gráfica de Java. Proporciona componentes básicos y depende del sistema operativo para renderizar los elementos, lo que puede causar diferencias visuales entre plataformas. Actualmente se usa principalmente para aplicaciones sencillas o compatibilidad con código antiguo.
- **JavaFX:** Es una biblioteca moderna para crear interfaces gráficas avanzadas, con soporte para gráficos 2D y 3D, animaciones, efectos visuales y estilos CSS. Permite desarrollar aplicaciones más atractivas y flexibles, y es la opción recomendada para proyectos nuevos.
- **SWT (Standard Widget Toolkit):** Utilizada principalmente en el entorno Eclipse, SWT ofrece acceso directo a los componentes nativos del sistema operativo, logrando una apariencia más integrada. Es menos común fuera de aplicaciones específicas.

Cada biblioteca tiene sus ventajas y desventajas. Swing es ampliamente usada y fácil de aprender, JavaFX ofrece más funcionalidades modernas, y AWT/SWT pueden ser útiles en casos particulares.

19.1.1. Ejemplo básico de interfaz gráfica con Swing

El siguiente ejemplo muestra cómo crear una ventana simple con un botón usando Swing:

```
1 import javax.swing.*;  
2  
3 public class VentanaBasica {  
4     public static void main(String[] args) {  
5         JFrame ventana = new JFrame("Mi primera ventana");  
6         JButton boton = new JButton("Haz clic");  
7         ventana.add(boton);  
8         ventana.setSize(300, 200);  
9         ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
10        ventana.setVisible(true);  
    }
```

```

11 }
12 }

```

Ejemplo 19.1: Ventana básica con Swing

Este programa crea una ventana con un botón. Al ejecutar el código, aparecerá una ventana titulada “Mi primera ventana”.

19.2. Concepto de evento

Un evento es una acción que ocurre en la interfaz gráfica, como hacer clic en un botón, mover el ratón o escribir en un cuadro de texto. Los eventos permiten que la aplicación responda a las acciones del usuario.

Consejo



En Java, los eventos se gestionan mediante *listeners* (escuchadores), que son objetos que detectan y responden a eventos específicos.

19.3. Creación de controladores de eventos

Para responder a eventos, se deben crear controladores (*listeners*) y asociarlos a los componentes gráficos. El siguiente ejemplo muestra cómo detectar un clic en un botón:

```

1 import javax.swing.*;
2 import java.awt.event.*;
3
4 public class EventoBoton {
5     public static void main(String[] args) {
6         JFrame ventana = new JFrame("Evento de botón");
7         JButton boton = new JButton("Haz clic");
8         JLabel etiqueta = new JLabel("Esperando clic...");
9
10        boton.addActionListener(new ActionListener() {
11            public void actionPerformed(ActionEvent e) {
12                etiqueta.setText("¡Botón pulsado!");
13            }
14        });
15
16        ventana.setLayout(new java.awt.FlowLayout());
17        ventana.add(boton);
18        ventana.add(etiqueta);
19        ventana.setSize(300, 100);
20        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
21        ventana.setVisible(true);

```

```
22 }  
23 }
```

Ejemplo 19.2: Controlador de eventos para un botón

En este ejemplo, al pulsar el botón, la etiqueta cambia su texto.

Cuadro 19.1: Componentes gráficos comunes en Swing

Componente	Descripción
JButton	Botón. Permite al usuario realizar una acción al hacer clic.
JLabel	Etiqueta de texto. Muestra información estática o dinámica.
JTextField	Campo de texto. Permite la entrada de texto en una sola línea.
JTextArea	Área de texto. Permite la entrada y visualización de texto en varias líneas.
JCheckBox	Casilla de verificación. Permite seleccionar o deseleccionar una opción.
JRadioButton	Botón de opción. Permite seleccionar una opción dentro de un grupo.
JComboBox	Lista desplegable. Permite seleccionar una opción de una lista.
JPanel	Panel contenedor. Agrupa y organiza otros componentes.
JFrame	Ventana principal. Contenedor de nivel superior para la interfaz.

19.3.1. Ejemplo práctico: Calculadora simple

A continuación se presenta un ejemplo de una calculadora simple implementada con Swing. Esta aplicación permite al usuario ingresar dos números en campos de texto, pulsar un botón para sumarlos y ver el resultado en una etiqueta. El programa también gestiona errores si los valores ingresados no son números válidos.

```
1 import javax.swing.*;  
2 import java.awt.event.*;  
3  
4 public class Calculadora {  
5     public static void main(String[] args) {  
6         JFrame ventana = new JFrame("Calculadora");
```



```

7   JTextField campo1 = new JTextField(5);
8   JTextField campo2 = new JTextField(5);
9   JButton boton = new JButton("Sumar");
10  JLabel resultado = new JLabel("Resultado: ");
11
12  boton.addActionListener(new ActionListener() {
13      public void actionPerformed(ActionEvent e) {
14          try {
15              int a = Integer.parseInt(campo1.getText());
16              int b = Integer.parseInt(campo2.getText());
17              resultado.setText("Resultado: " + (a + b));
18          } catch (NumberFormatException ex) {
19              resultado.setText("Error: ingresa números válidos");
20          }
21      }
22  });
23
24  ventana.setLayout(new java.awt.FlowLayout());
25  ventana.add(campo1);
26  ventana.add(campo2);
27  ventana.add(boton);
28  ventana.add(resultado);
29  ventana.setSize(350, 120);
30  ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31  ventana.setVisible(true);
32 }
33 }

```

Ejemplo 19.3: Calculadora simple en Swing

A continuación se explica paso a paso cómo funciona el ejemplo de la calculadora:

1. Se crea una ventana principal (JFrame) con el título “Calculadora”.
2. Se añaden dos campos de texto (JTextField) donde el usuario puede ingresar los números a sumar.
3. Se agrega un botón (JButton) con el texto “Sumar”.
4. Se coloca una etiqueta (JLabel) para mostrar el resultado de la suma o un mensaje de error.
5. Se asocia un controlador de eventos (ActionListener) al botón. Cuando el usuario hace clic en el botón:
 - El programa intenta leer los valores de los campos de texto y convertirlos a números enteros.

- Si ambos valores son válidos, se realiza la suma y se muestra el resultado en la etiqueta.
 - Si alguno de los valores no es un número válido, se muestra un mensaje de error en la etiqueta.
6. Finalmente, se organiza la ventana con un diseño de flujo (**FlowLayout**), se añaden todos los componentes y se muestra la ventana al usuario.

Ejercicio 19.1



Modifica el ejemplo de la calculadora para que también permita restar, multiplicar y dividir los dos números. Añade botones para cada operación y muestra el resultado correspondiente en la etiqueta. *Ver solución en la página 251.*

Resumen

En este capítulo aprendiste los conceptos básicos de las interfaces gráficas en Java, el manejo de eventos y la creación de controladores. Usando Swing, puedes construir aplicaciones interactivas y visuales. Practica creando tus propias ventanas y componentes para dominar la programación gráfica en Java.

Ejercicios de refuerzo

Ejercicio 19.2



Crea un programa Java con una ventana **JFrame** de 400×300 píxeles, título "Hola, mundo" y un **JLabel** centrado con el texto "¡Hola, mundo!". Al cerrar la ventana, el programa debe terminar (**EXIT_ON_CLOSE**). *Ver solución en la página 252.*

Ejercicio 19.3



Añade a la ventana anterior un **JButton** con el texto **Cambiar texto**". Al pulsarlo, el **JLabel** debe mostrar "¡Pulsado!". Usa un **ActionListener** definido con una expresión lambda. *Ver solución en la página 253.*

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 19.1



```

1 // Solución completa: calculadora con suma, resta, ↵
  multiplicación y división
2 import java.awt.GridLayout;
3 import javax.swing.JButton;
4 import javax.swing.JFrame;
5 import javax.swing.JLabel;
6 import javax.swing.JTextField;
7
8 public class CalculadoraCompleta {
9
10  public static void main(String[] args) {
11      JFrame ventana = new JFrame("Calculadora");
12      ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13      ventana.setLayout(new GridLayout(3, 1, 5, 5));
14
15      JTextField campo1 = new JTextField(8);
16      JTextField campo2 = new JTextField(8);
17      JLabel resultado = new JLabel("Resultado: -", JLabel.↵
        CENTER);
18
19      JButton btnSumar = new JButton("+");
20      JButton btnRestar = new JButton("-");
21      JButton btnMultiplicar = new JButton("×");
22      JButton btnDividir = new JButton("÷");
23
24      // Fila de campos de entrada
25      javax.swing.JPanel panelEntrada = new javax.swing.JPanel ↵
        ();
26      panelEntrada.add(campo1);
27      panelEntrada.add(new JLabel("op"));
28      panelEntrada.add(campo2);
29
30      // Fila de botones de operación
31      javax.swing.JPanel panelBotones = new javax.swing.JPanel ↵
        ();
32      panelBotones.add(btnSumar);
33      panelBotones.add(btnRestar);
34      panelBotones.add(btnMultiplicar);
35      panelBotones.add(btnDividir);
36
37      btnSumar.addActionListener(e -> calcular(campo1, campo2, ↵
        resultado, "+"));

```

```

38 btnRestar.addActionListener(e -> calcular(campo1, campo2, ↵
    resultado, "-"));
39 btnMultiplicar.addActionListener(e -> calcular(campo1, ↵
    campo2, resultado, "x"));
40 btnDividir.addActionListener(e -> calcular(campo1, campo2 ↵
    , resultado, "÷"));
41
42 ventana.add(panelEntrada);
43 ventana.add(panelBotones);
44 ventana.add(resultado);
45 ventana.pack();
46 ventana.setVisible(true);
47 }
48
49 static void calcular(JTextField c1, JTextField c2, JLabel ↵
    etiqueta, String op) {
50     try {
51         double a = Double.parseDouble(c1.getText());
52         double b = Double.parseDouble(c2.getText());
53         double res = switch (op) {
54             case "+" -> a + b;
55             case "-" -> a - b;
56             case "x" -> a * b;
57             case "÷" -> {
58                 if (b == 0) throw new ArithmeticException("División ↵
    por cero");
59                 yield a / b;
60             }
61             default -> throw new IllegalArgumentException("↵
    Operación desconocida");
62         };
63         etiqueta.setText(String.format("Resultado: %.2f", res));
64     } catch (NumberFormatException ex) {
65         etiqueta.setText("Error: introduce números válidos");
66     } catch (ArithmeticException ex) {
67         etiqueta.setText("Error: " + ex.getMessage());
68     }
69 }
70 }

```

Solución del ejercicio 19.2



```

1 // Programa completo: compilar y ejecutar ↵
  independiente
2 import javax.swing.JFrame;
3 import javax.swing.JLabel;

```

```

4 import javax.swing.SwingConstants;
5
6 public class HolaMundo {
7     public static void main(String[] args) {
8         JFrame ventana = new JFrame("Hola, mundo");
9         ventana.setSize(400, 300);
10        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11
12        JLabel etiqueta = new JLabel("¡Hola, mundo!", ↵
13        SwingConstants.CENTER);
14        ventana.add(etiqueta);
15
16        ventana.setVisible(true);
17    }
18 }

```

Solución del ejercicio 19.3



```

1 // Programa completo: compilar y ejecutar ↵
   independiente
2 import java.awt.FlowLayout;
3 import javax.swing.JButton;
4 import javax.swing.JFrame;
5 import javax.swing.JLabel;
6
7 public class BotonCambiaLabel {
8     public static void main(String[] args) {
9         JFrame ventana = new JFrame("Botón");
10        ventana.setSize(300, 200);
11        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
12        ventana.setLayout(new FlowLayout());
13
14        JLabel etiqueta = new JLabel("Texto original");
15        JButton boton = new JButton("Cambiar texto");
16        boton.addActionListener(e -> etiqueta.setText("¡Pulsado!"↵
17        ));
18
19        ventana.add(etiqueta);
20        ventana.add(boton);
21        ventana.setVisible(true);
22    }
23 }

```




Programación avanzada en Java

¿Cómo escribir código limpio?

En el desarrollo de software, escribir *Clean Code* (código limpio) es fundamental para garantizar la mantenibilidad, legibilidad y escalabilidad de los proyectos. El código limpio es fácil de entender, modificar y extender, lo que reduce la probabilidad de errores y facilita la colaboración entre desarrolladores.

20.1. Principios de Clean Code

El concepto de *Clean Code* va más allá de simplemente escribir código que funcione. Se trata de crear soluciones que sean fáciles de leer, entender y modificar por cualquier miembro del equipo, incluso mucho tiempo después de haber sido escritas. El código limpio minimiza la complejidad innecesaria, utiliza estructuras claras y evita ambigüedades. Además, fomenta la reutilización y la modularidad, lo que facilita la incorporación de nuevas funcionalidades y la corrección de errores.

Un código limpio suele tener las siguientes características:

- **Simplicidad:** Elimina todo lo innecesario y se enfoca en resolver el problema de la manera más directa posible.
- **Legibilidad:** Está escrito pensando en que otros desarrolladores lo leerán y lo mantendrán.
- **Consistencia:** Sigue convenciones y estilos definidos para todo el proyecto.
- **Testabilidad:** Facilita la escritura de pruebas automatizadas, lo que ayuda a garantizar su correcto funcionamiento.
- **Evolutividad:** Permite realizar cambios y mejoras sin afectar negativamente otras partes del sistema.

Adoptar una mentalidad de código limpio implica revisar y refactorizar constantemente, buscando siempre la mejor forma de expresar las ideas y soluciones en el código.

Algunos principios generales de Clean Code que puedes empezar a aplicar son:

- **Nombres significativos:** Utiliza nombres descriptivos para variables, funciones y clases, que reflejen claramente su propósito.
- **Funciones cortas y enfocadas:** Las funciones deben realizar una sola tarea y ser lo más breves posible.
- **Evita la duplicación:** El código duplicado dificulta el mantenimiento y aumenta el riesgo de errores.
- **Comentarios útiles:** Los comentarios deben explicar el *porqué* del código, no el *qué*, y evitar redundancias.
- **Formato consistente:** Mantén una estructura y estilo uniforme en todo el proyecto.
- **Manejo adecuado de errores:** Implementa mecanismos claros y robustos para el tratamiento de errores y excepciones.

20.2. Principios SOLID

Los principios SOLID son un conjunto de buenas prácticas orientadas a la programación orientada a objetos, que ayudan a crear sistemas flexibles y fáciles de mantener. A continuación se detallan cada uno de los principios.

20.2.1. Single Responsibility Principle (SRP)

Cada clase debe tener una única responsabilidad, es decir, debe estar encargada de una sola parte de la funcionalidad del programa. Esto facilita la comprensión y el mantenimiento del código, ya que los cambios en una responsabilidad no afectan otras partes del sistema.

Ejemplo: Una clase que gestiona la lógica de negocio no debería encargarse también de la persistencia de datos.

20.2.2. Open/Closed Principle (OCP)

Las entidades de software (clases, módulos, funciones) deben estar abiertas para extensión, pero cerradas para modificación. Esto significa que se debe poder agregar nueva funcionalidad sin alterar el código existente, generalmente mediante herencia o composición.

Ejemplo: Utilizar interfaces y clases abstractas para permitir la extensión sin modificar el código base.

20.2.3. Liskov Substitution Principle (LSP)

Las clases derivadas deben poder sustituir a sus clases base sin alterar el comportamiento esperado del programa. En otras palabras, los objetos de una subclase deben poder usarse en lugar de los de la clase base sin que el cliente note la diferencia.

Ejemplo: Si una función espera un objeto de tipo `Animal`, debería funcionar correctamente si se le pasa un objeto de tipo `Perro`, que hereda de `Animal`.

20.2.4. Interface Segregation Principle (ISP)

Los clientes no deben verse obligados a depender de interfaces que no utilizan. Es mejor tener varias interfaces específicas en lugar de una interfaz general con muchos métodos innecesarios.

Ejemplo: Separar una interfaz grande en varias interfaces pequeñas y especializadas.

20.2.5. Dependency Inversion Principle (DIP)

Los módulos de alto nivel no deben depender de módulos de bajo nivel, sino de abstracciones. Las abstracciones no deben depender de los detalles, sino los detalles de las abstracciones. Esto se logra mediante la inversión de dependencias, utilizando interfaces o clases abstractas.

Ejemplo: Inyectar dependencias a través de interfaces en lugar de instanciar clases concretas directamente.

20.3. *Code Calisthenics*

Los *Code Calisthenics* son ejercicios prácticos diseñados para mejorar la disciplina y la calidad del código que escribimos. Consisten en seguir reglas estrictas durante el desarrollo, con el objetivo de fomentar buenas prácticas y hábitos saludables en la programación. Algunas reglas comunes incluyen:

- Una sola instrucción por línea.
- Una sola variable declarada por línea.
- Métodos de máximo siete líneas.
- No utilizar más de dos niveles de indentación.
- No usar la palabra clave `else`.
- No utilizar números mágicos ni cadenas mágicas.

Estos ejercicios ayudan a identificar malas prácticas y a desarrollar un estilo de programación más limpio y estructurado. Aunque pueden parecer restrictivos, su objetivo es fomentar la reflexión sobre las decisiones de diseño y mejorar la calidad del código a largo plazo.

Ejercicio 20.1



Aplica estos principios en el siguiente código:

```

1 public class MessClass {
2     public static double convertirFarenheith(double x) {
3         return (x - 32) * 5 / 9;
4     }
5
6     public static double convertirCelsius(double x) {
7         return (x * 9 / 5) + 32;
8     }
9
10    public static int maxEnMatriz3(int[][][] matriz) {
11        int max = Integer.MIN_VALUE;
12        for (int[][] submatriz : matriz) {
13            for (int[] fila : submatriz) {
14                for (int elemento : fila) {
15                    if (elemento > max) {
16                        max = elemento;
17                    }
18                }
19            }
20        }
21        return max;
22    }
23
24    public static int mediana(int[] numeros) {
25        Arrays.sort(numeros);
26        if (numeros.length == 0) {
27            throw new IllegalArgumentException("El array está ↩
vacío");

```

```

28     } else if (numeros.length % 2 == 1) {
29         return numeros[numeros.length / 2];
30     } else {
31         return (numeros[numeros.length / 2 - 1] + numeros[↵
numeros.length / 2]) / 2;
32     }
33 }
34
35 public static double[] estadisticas(int[] numeros) {
36     if (numeros.length == 0) {
37         throw new IllegalArgumentException("El array está ↵
vacío");
38     }
39     return new double[] {
40         Arrays.stream(numeros).average().orElse(0),
41         mediana(numeros),
42         maxEnMatriz3(new int[][][] { numeros })
43     };
44 }
45 }
46

```

Ver solución en la página 263.

Resumen

Aplicar los principios de *Clean Code* y SOLID permite desarrollar sistemas robustos, escalables y fáciles de mantener. Adoptar estas buenas prácticas mejora la calidad del software y facilita el trabajo en equipo.

Para profundizar en el tema de *Clean Code* y buenas prácticas de desarrollo, se recomiendan los siguientes libros:

- **Clean Code: A Handbook of Agile Software Craftsmanship** – Robert C. Martin. Un libro fundamental que explora los principios y técnicas para escribir código limpio y mantenible.
- **The Pragmatic Programmer: Your Journey to Mastery** – Andrew Hunt y David Thomas. Ofrece consejos prácticos y estrategias para mejorar la calidad y profesionalismo en el desarrollo de software.

Ejercicios de refuerzo

Ejercicio 20.2



El siguiente fragmento usa nombres de variable poco descriptivos y un bucle `for` con índice en lugar de `for-each`. Reescríbelo aplicando las reglas de nombres significativos y extrae la lógica en un método con nombre descriptivo:

```
1 int c = 0;
2 for (int i = 0; i < l.size(); i++) {
3     if (l.get(i) > 0) c++;
4 }
```

Ver solución en la página 264.

Ejercicio 20.3



Una clase `GestorPedidos` realiza en un único método tres tareas: validar el pedido, calcular el precio total y guardarlo en base de datos. Explica qué principio SOLID viola este diseño y propón cómo dividirlo en clases separadas, indicando la responsabilidad de cada una. *Ver solución en la página 265.*

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 20.1



```

1 // Versión refactorizada de MessClass aplicando principios ↪
  de Clean Code
2 import java.util.Arrays;
3
4 /** Convierte temperaturas entre Fahrenheit y Celsius. */
5 public class ConversorTemperatura {
6
7     public static double fahrenheitACelsius(double grados) {
8         return (grados - 32) * 5.0 / 9.0;
9     }
10
11     public static double celsiusAFahrenheit(double grados) {
12         return (grados * 9.0 / 5.0) + 32;
13     }
14 }
15
16 /** Calcula estadísticas básicas sobre arrays de enteros. ↪
  */
17 class CalculadoraEstadisticas {
18
19     public static double media(int[] numeros) {
20         validarNoVacio(numeros);
21         return Arrays.stream(numeros).average().orElse(0);
22     }
23
24     public static int mediana(int[] numeros) {
25         validarNoVacio(numeros);
26         int[] ordenados = numeros.clone();
27         Arrays.sort(ordenados);
28         int mitad = ordenados.length / 2;
29         if (ordenados.length % 2 == 1) {
30             return ordenados[mitad];
31         }
32         return (ordenados[mitad - 1] + ordenados[mitad]) / 2;
33     }
34
35     public static int maximo(int[] numeros) {
36         validarNoVacio(numeros);
37         return Arrays.stream(numeros).max().orElseThrow();
38     }
39
40     /** Devuelve [media, mediana, máximo]. */
41     public static double[] estadisticas(int[] numeros) {

```

```

42     return new double[] { media(numeros), mediana(numeros), ↵
        maximo(numeros) };
43 }
44
45 private static void validarNoVacio(int[] numeros) {
46     if (numeros == null || numeros.length == 0) {
47         throw new IllegalArgumentException("El array no puede ↵
            estar vacío");
48     }
49 }
50 }
51
52 // Uso en main:
53 // int[] datos = {5, 2, 8, 1, 9};
54 // System.out.println(ConversorTemperatura.↵
    fahrenheitACelsius(98.6)); // 37.0
55 // double[] stats = CalculadoraEstadisticas.estadisticas(↵
    datos);
56 // System.out.printf("Media=%.1f Mediana=%.0f Máximo=%.0f%n↵
    ", stats[0], stats[1], stats[2]);

```

Solución del ejercicio 20.2



```

1 // Fragmento: versión refactorizada con nombres ↵
    significativos y método extraído
2
3 // Antes (nombres crípticos):
4 // int c = 0;
5 // for (int i = 0; i < l.size(); i++) { if (l.get(i) > 0) c↵
    ++; }
6
7 // Después:
8 int contarPositivos(List<Integer> numeros) {
9     int contador = 0;
10    for (int numero : numeros) {
11        if (numero > 0) {
12            contador++;
13        }
14    }
15    return contador;
16 }

```


Solución del ejercicio 20.3

```
1 // Fragmento: clases separadas para cumplir el Principio de ↵
   Responsabilidad Única
2
3 // Antes: una sola clase GestorPedidos hacía validación, ↵
   cálculo y guardado
4 // Después: tres clases con una única responsabilidad cada ↵
   una
5
6 class ValidadorPedido {
7     void validar(Pedido pedido) {
8         if (pedido.getProductos().isEmpty()) {
9             throw new IllegalArgumentException("El pedido no tiene ↵
   productos");
10        }
11    }
12 }
13
14 class CalculadorPrecio {
15     double calcularTotal(Pedido pedido) {
16         return pedido.getProductos().stream()
17             .mapToDouble(Producto::getPrecio)
18             .sum();
19     }
20 }
21
22 class RepositorioPedido {
23     void guardar(Pedido pedido) {
24         // lógica de persistencia
25         System.out.println("Pedido guardado: " + pedido);
26     }
27 }
```


Principios de programación funcional en programación orientada a objetos

La programación funcional es un paradigma que se centra en el uso de funciones como elementos fundamentales para la construcción de programas. A diferencia de otros enfoques, como la programación orientada a objetos, la programación funcional promueve la transparencia referencial y la inmutabilidad, lo que facilita la creación de código más predecible y fácil de mantener.

En este capítulo se explorarán los conceptos clave de la programación funcional y cómo pueden aplicarse dentro de la programación orientada a objetos. Se abordarán temas como el principio de transparencia referencial, la importancia de la inmutabilidad, el uso de métodos como elementos de primer orden, la definición y utilización de expresiones lambda e interfaces funcionales, así como el empleo de la clase `Optional` para manejar valores que pueden estar ausentes.

Estos conceptos permiten escribir código más expresivo, seguro y flexible, aprovechando las ventajas de ambos paradigmas para desarrollar aplicaciones robustas y modernas.

21.1. Transparencia referencial

La transparencia referencial es una característica de las funciones que garantiza que, para los mismos argumentos de entrada, siempre se obtiene el mismo resultado y no se producen efectos secundarios. Esto facilita la comprensión, el testeo y la depuración del código.

```
1 | int suma(int a, int b) {  
2 |     return a + b;  
3 | }
```

Ejemplo 21.1: Función con transparencia referencial

En el ejemplo anterior, la función `suma` es transparente referencial porque su resultado depende únicamente de los parámetros `a` y `b`, sin modificar ningún estado externo.

Por el contrario, una función que modifica variables externas o depende de ellas pierde esta propiedad:

```
1 private static int contador = 0;
2
3 int mediana(List<Integer> valores) {
4     contador++;
5     Collections.sort(valores);
6     int n = valores.size();
7     if (n % 2 == 0) {
8         return (valores.get(n/2 - 1) + valores.get(n/2)) / 2;
9     } else {
10        return valores.get(n/2);
11    }
12 }
```

Ejemplo 21.2: Función sin transparencia referencial

En este caso, la función `mediana` modifica el estado global (`contador`) y también altera la lista recibida como argumento, lo que afecta la transparencia referencial. El resultado puede variar si la lista cambia fuera de la función o si el estado global se utiliza en otros lugares.

21.2. Inmutabilidad

La inmutabilidad implica que los objetos no pueden cambiar su estado después de ser creados. Esto reduce errores y facilita la concurrencia. Para facilitar esto, podemos hacer uso de clases inmutables, que no permiten modificar sus atributos una vez instanciados, en particular, usando `record`.

```
1 public record Punto(int x, int y) { }
2
3 Punto p1 = new Punto(3, 5);
4 // No se pueden modificar los valores de x e y después de crear el objeto ↪
```

Ejemplo 21.3: Ejemplo de clase inmutable usando record

Si queremos modificar una objeto de esa clase, lo que hacemos es crear un nuevo objeto con los valores modificados:

```
1 Punto p2 = new Punto(p1.x() + 1, p1.y());
2 // p2 es un nuevo objeto, p1 permanece sin cambios
```

Ejemplo 21.4: Creación de un nuevo objeto inmutable

En este ejemplo, podemos simplificar el código, si creamos un método en la clase `Punto` que nos permita crear un nuevo objeto con un valor modificado:

```

1 public record Punto(int x, int y) {
2     public Punto mover(int dx, int dy) {
3         return new Punto(x + dx, y + dy);
4     }
5 }
6
7 Punto p1 = new Punto(3, 5);
8 Punto p2 = p1.mover(1, 0);
9 // p2 es un nuevo objeto, p1 permanece sin cambios

```

Ejemplo 21.5: Método para crear un nuevo objeto inmutable

Importante

En Java es fácil romper el principio de inmutabilidad, especialmente cuando se utilizan colecciones mutables, o se pasan arrays como argumentos. Por ejemplo, si una clase inmutable contiene una lista mutable, es posible modificar el contenido de la lista aunque el objeto en sí no cambie. Para evitar esto, se recomienda utilizar colecciones inmutables (como las de la clase `List.of(...)` en Java) o realizar copias defensivas de los datos en el constructor y los métodos de acceso.

21.3. Métodos como elementos de primer orden

En Java, los métodos pueden ser tratados como elementos de primer orden usando referencias a métodos y expresiones lambda.

```

1 List<String> nombres = Arrays.asList("Ana", "Luis", "Pedro");
2 nombres.forEach(System.out::println);

```

Ejemplo 21.6: Referencia a método

También podemos utilizar referencias a métodos de instancia (de objeto) y de clase (estáticos). Esto permite pasar métodos existentes como argumentos a funciones que esperan una interfaz funcional.

```

1 List<Integer> numeros = Arrays.asList(1, 2, 3, 4);
2 numeros.forEach(System.out::println); // System.out es un objeto ↩
   , println es su método

```

Ejemplo 21.7: Referencia a método de clase

En este ejemplo, `System.out::println` es una referencia a un método de objeto.

Para métodos de clase (estáticos), la sintaxis es similar:

```

1 class Utilidades {
2     public static void mostrar(String texto) {
3         System.out.println(texto);
4     }
5 }
6
7 List<String> mensajes = List.of("Hola", "Adiós");
8 mensajes.forEach(Utilidades::mostrar); // Utilidades::mostrar es →
    referencia a método de clase

```

Ejemplo 21.8: Referencia a método estático de clase

También podemos referenciar métodos de instancia de un objeto específico:

```

1 class Persona {
2     private final String nombre;
3     public Persona(String nombre) { this.nombre = nombre; }
4     public void saludar() { System.out.println("Hola, soy " + →
        nombre); }
5 }
6
7 List<Persona> personas = List.of(new Persona("Ana"), new Persona →
    ("Luis"));
8 personas.forEach(Persona::saludar); // Llama saludar() en cada →
    objeto Persona

```

Ejemplo 21.9: Referencia a método de instancia de objeto

Las referencias a métodos permiten escribir código más conciso y reutilizable, facilitando el uso de funciones como argumentos y mejorando la expresividad del lenguaje.

Consejo



Podemos pasar referencias a métodos de clases mutables si queremos acumular un estado interno. Por ejemplo, podríamos tener un acumulador que sume valores a medida que se procesan:

```

1 import java.util.concurrent.atomic.AtomicInteger;
2 import java.util.stream.Stream;
3
4 AtomicInteger acumulador = new AtomicInteger(0);
5 Stream.of(1, 2, 3, 4, 5).forEach(acumulador::addAndGet);
6 // acumulador ahora contiene la suma de los valores del →
    stream
7 System.out.println("Suma total: " + acumulador.get());
8

```

Ejemplo 21.10: Uso de AtomicInteger y Stream para acumular valores

21.4. Expresiones lambda e interfaces funcionales

Las expresiones lambda permiten definir funciones anónimas de manera concisa. Las interfaces funcionales son interfaces con un solo método abstracto. El uso de la anotación `@FunctionalInterface` ayuda a identificar estas interfaces, e impone una restricción de que solo habrá un único método no implementado.

```

1 @FunctionalInterface
2 interface Operacion {
3     int aplicar(int a, int b);
4 }
5
6 Operacion suma = (a, b) -> a + b;
7 System.out.println(suma.aplicar(3, 4)); // Imprime 7

```

Ejemplo 21.11: Uso de lambda e interfaz funcional

Al crear estas interfaces, podremos ajustarnos a nuestras propias necesidades. Sin embargo, Java incluye las siguientes interfaces funcionales, las cuales son suficiente para la mayoría de problemas.

Importante

Las interfaces anteriores no declaran el lanzamiento de excepciones, por eso, el compilador fallará si se intenta lanzar una excepción dentro de su implementación. Para ello es necesario crear interfaces funcionales que declaren las excepciones, o adaptar el código de manera que el control de errores se realice de otra manera, por ejemplo usando la clase `Optional`.

21.5. La clase `Optional`

La clase `Optional` ayuda a manejar valores que pueden estar ausentes, evitando errores de referencia nula. Es especialmente útil en librerías que gestionan colecciones, o peticiones a servidores externos, ya que puede encapsular un posible valor no presente dentro de la clase `Optional`.

```

1 Optional<String> nombre = Optional.ofNullable(obtenerNombre());
2 nombre.ifPresent(n -> System.out.println("Nombre: " + n));

```

Ejemplo 21.12: Uso de `Optional`

Al usar esta clase, es posible reducir el número de comprobaciones de nulos y hacer el código más legible. Por ejemplo, en lugar de verificar si un objeto es nulo antes de usarlo, podemos usar métodos como `ifPresent`, `orElse` o `map` para manejar el valor de manera más fluida.

```

1 // Este valor puede venir de una llamada a un método que ↪
  devuelve Optional<String>
2 Optional<String> texto = Optional.of(" Hola mundo ");
3
4 String resultado = texto
5   .map(String::trim)           // 1. Elimina espacios
6   .filter(t -> t.startsWith("Hola")) // 2. Filtra si empieza por ↪
  "Hola"
7   .map(String::toUpperCase)    // 3. Convierte a mayúsculas
8   .map(s -> s + "!")           // 4. Añade un signo de exclamación
9   .orElse("Valor alternativo"); // 5. Devuelve valor ↪
  alternativo si está ausente
10
11 System.out.println(resultado); // Imprime "HOLA MUNDO!"

```

Ejemplo 21.13: Ejemplo de llamadas encadenadas con Optional

Ejercicio 21.1



Toma uno de los ejercicios que has visto a lo largo de este libro, y reescríbelo haciendo uso de los principios de programación funcional. Usa la clase `Optional` para eliminar cualquier comprobación de nulos. *Ver solución en la página 281.*

21.6. La clase Stream

La clase `Stream` es una herramienta fundamental en la programación funcional de Java, ya que permite procesar colecciones de datos de manera declarativa y eficiente. Un `Stream` representa una secuencia de elementos sobre la que se pueden realizar operaciones como filtrado, transformación, agrupamiento y reducción, sin modificar la colección original.

Las operaciones sobre `Stream` suelen ser encadenadas y pueden ser de dos tipos: intermedias (como `filter`, `map`, `sorted`) y terminales (como `collect`, `forEach`, `reduce`). Esto facilita la escritura de código conciso y expresivo.

```

1 List<String> nombres = List.of("Ana", "Luis", "Pedro", "Lucía");
2 List<String> resultado = nombres.stream()
3   .filter(n -> n.length() > 4)
4   .map(String::toUpperCase)
5   .sorted()
6   .toList();
7
8 System.out.println(resultado); // Imprime [LUIS, LUCÍA, PEDRO]

```

Ejemplo 21.14: Uso básico de Stream

El uso de **Stream** junto con **Optional** es especialmente útil para manejar datos que pueden estar ausentes o para evitar errores de referencia nula. Por ejemplo, al buscar un elemento en una colección, podemos obtener un **Optional** que indique si el elemento está presente:

```
1 List<String> nombres = List.of("Ana", "Luis", "Pedro", "Lucía");
2 Optional<String> encontrado = nombres.stream()
3   .filter(n -> n.startsWith("L"))
4   .findFirst();
5
6 encontrado.ifPresent(n -> System.out.println("Encontrado: " + n) ↪
7   );
```

Ejemplo 21.15: Stream y Optional para buscar elementos

En este ejemplo, `findFirst` devuelve un `Optional<String>` que estará vacío si no se encuentra ningún elemento que cumpla la condición. Así, evitamos comprobaciones explícitas de nulos y hacemos el código más seguro y legible.

Además, podemos combinar operaciones de **Stream** y **Optional** para transformar, filtrar y procesar datos de manera funcional:

```
1 List<String> nombres = List.of("Ana", "Luis", "Pedro", "Lucía");
2 String resultado = nombres.stream()
3   .filter(n -> n.length() > 5)
4   .findFirst()
5   .map(String::toLowerCase)
6   .orElse("No encontrado");
7
8 System.out.println(resultado); // Imprime "lucía" o "No ↪
9   encontrado"
```

Ejemplo 21.16: Encadenamiento de Stream y Optional

El uso conjunto de **Stream** y **Optional** permite escribir código más robusto, evitando errores comunes y facilitando la programación funcional en Java.

Resumen

En este capítulo se han presentado los principios fundamentales de la programación funcional y su integración en la programación orientada a objetos. Se ha explicado la importancia de la transparencia referencial y la inmutabilidad para escribir código más seguro y predecible. Además, se ha mostrado cómo tratar métodos como elementos de primer orden mediante referencias y expresiones lambda, y el uso de interfaces funcionales para definir comportamientos flexibles. Finalmente, se ha destacado el papel de la clase **Optional** para gestionar valores ausentes de forma segura y expresiva. Estos conceptos permiten

combinar lo mejor de ambos paradigmas y desarrollar aplicaciones modernas, robustas y fáciles de mantener.

Cuadro 21.1: Principales interfaces funcionales de Java y su uso

Interfaz funcional	Descripción y uso principal
<code>Predicate<T></code>	Representa una función que recibe un argumento de tipo <code>T</code> y devuelve un valor booleano. Se usa para pruebas, filtros y condiciones.
<code>Consumer<T></code>	Recibe un argumento de tipo <code>T</code> y no devuelve nada. Se utiliza para realizar acciones sobre objetos, como imprimir o modificar estructuras externas.
<code>Supplier<T></code>	No recibe argumentos y devuelve un valor de tipo <code>T</code> . Se usa para generar o proveer valores, como fábricas o generadores.
<code>Function<T, R></code>	Recibe un argumento de tipo <code>T</code> y devuelve un resultado de tipo <code>R</code> . Se emplea para transformar o mapear valores.
<code>UnaryOperator<T></code>	Recibe y devuelve un valor del mismo tipo <code>T</code> . Es una especialización de <code>Function</code> para operaciones unarias.
<code>BinaryOperator<T></code>	Recibe dos argumentos del mismo tipo <code>T</code> y devuelve un resultado del mismo tipo. Se usa para operaciones binarias, como suma o concatenación.
<code>BiFunction<T, U, R></code>	Recibe dos argumentos de tipos <code>T</code> y <code>U</code> , y devuelve un resultado de tipo <code>R</code> . Se utiliza para combinar o transformar dos valores.
<code>BiConsumer<T, U></code>	Recibe dos argumentos de tipos <code>T</code> y <code>U</code> , y no devuelve nada. Se emplea para realizar acciones sobre pares de valores.
<code>BiPredicate<T, U></code>	Recibe dos argumentos de tipos <code>T</code> y <code>U</code> , y devuelve un valor booleano. Se usa para pruebas o condiciones sobre pares de valores.
<code>Runnable</code>	No recibe argumentos ni devuelve valores. Se utiliza para ejecutar código, especialmente en hilos o tareas asíncronas.

Cuadro 21.2: Principales métodos de la clase `Optional<T>`

Método	Descripción
<code>Optional.empty()</code>	Devuelve una instancia de <code>Optional</code> vacía, sin valor presente.
<code>Optional.of(T value)</code>	Devuelve un <code>Optional</code> que contiene el valor especificado, lanza excepción si el valor es nulo.
<code>Optional.ofNullable(T value)</code>	Devuelve un <code>Optional</code> que puede contener el valor especificado o estar vacío si es nulo.
<code>isPresent()</code>	Devuelve <code>true</code> si el valor está presente, <code>false</code> en caso contrario.
<code>ifPresent(Consumer<? super T> action)</code>	Ejecuta la acción dada si el valor está presente.
<code>get()</code>	Devuelve el valor si está presente, lanza excepción si está vacío.
<code>orElse(T other)</code>	Devuelve el valor si está presente, o el valor alternativo si está vacío.
<code>orElseGet(Supplier<? extends T> supplier)</code>	Devuelve el valor si está presente, o el resultado del proveedor si está vacío.
<code>orElseThrow()</code>	Devuelve el valor si está presente, o lanza <code>NoSuchElementException</code> si está vacío.
<code>orElseThrow(Supplier<? extends X> exceptionSupplier)</code>	Devuelve el valor si está presente, o lanza la excepción proporcionada si está vacío.
<code>map(Function<? super T, ? extends U> mapper)</code>	Si hay valor, aplica la función y devuelve un <code>Optional</code> con el resultado; si está vacío, devuelve vacío.
<code>flatMap(Function<? super T, Optional<U>> mapper)</code>	Similar a <code>map</code> , pero la función debe devolver un <code>Optional</code> .

Método	Descripción
<code>filter(Predicate<? super T> predicate)</code>	Si hay valor y cumple el predicado, devuelve el mismo <code>Optional</code> ; si no, devuelve vacío.

Cuadro 21.3: Principales métodos de la clase `Stream<T>`

Método	Descripción
<code>filter(Predicate<? super T> predicate)</code>	Devuelve un nuevo Stream con los elementos que cumplen el predicado.
<code>map(Function<? super T,? extends R> mapper)</code>	Transforma cada elemento usando la función dada y devuelve un Stream de los resultados.
<code>flatMap(Function<? super T,? extends Stream<? extends R>> mapper)</code>	Aplica la función a cada elemento y aplanar los resultados en un solo Stream .
<code>sorted()</code>	Devuelve un Stream con los elementos ordenados según el orden natural.
<code>sorted(Comparator<? super T> comparator)</code>	Devuelve un Stream con los elementos ordenados usando el comparador dado.
<code>distinct()</code>	Devuelve un Stream con elementos únicos, eliminando duplicados.
<code>limit(long maxSize)</code>	Devuelve un Stream con un máximo de <code>maxSize</code> elementos.
<code>skip(long n)</code>	Omite los primeros <code>n</code> elementos y devuelve el resto en un Stream .
<code>forEach(Consumer<? super T> action)</code>	Ejecuta la acción dada para cada elemento del Stream .
<code>collect(Collector<? super T, A, R> collector)</code>	Reduce el Stream a una colección o resultado usando el colector especificado.
<code>toList()</code>	Devuelve una lista con los elementos del Stream .
<code>reduce(BinaryOperator<T> accumulator)</code>	Reduce los elementos del Stream a un solo valor usando el acumulador.

Método	Descripción
<code>findFirst()</code>	Devuelve un <code>Optional</code> con el primer elemento del <code>Stream</code> , si existe.
<code>findAny()</code>	Devuelve un <code>Optional</code> con algún elemento del <code>Stream</code> , útil en procesamiento paralelo.
<code>anyMatch(Predicate<? super T> predicate)</code>	Devuelve <code>true</code> si algún elemento cumple el predicado.
<code>allMatch(Predicate<? super T> predicate)</code>	Devuelve <code>true</code> si todos los elementos cumplen el predicado.
<code>noneMatch(Predicate<? super T> predicate)</code>	Devuelve <code>true</code> si ningún elemento cumple el predicado.
<code>count()</code>	Devuelve el número de elementos en el <code>Stream</code> .

Ejercicios de refuerzo

Ejercicio 21.2



Dada la lista `List.of("java", "stream", "lambda", "yo", "él", "colección")`, usa la API de Streams para filtrar las palabras con más de 4 letras, convertirlas a mayúsculas e imprimirlas. Muestra también cuántas palabras cumplen el criterio. *Ver solución en la página 281.*

Ejercicio 21.3



Crea dos `Optional<String>`: uno con el valor `"Lucía"` y otro vacío. Para cada uno, obtén el nombre en mayúsculas usando `map` y `orElse`, de modo que si está vacío devuelva `"DESCONOCIDO"`. Imprime ambos resultados. *Ver solución en la página 282.*

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 21.1



A continuación se muestra la reescritura funcional del ejercicio de cálculo de la media de un array, usando `OptionalDouble`:

```

1 // Fragmento: reescritura funcional del ejercicio "media de
  un array" (cap. 9)
2 // Versión original usaba bucle for + validación manual de
  null/vacío.
3 // Versión funcional: Arrays.stream + Optional eliminan las
  comprobaciones explícitas.
4 import java.util.Arrays;
5 import java.util.OptionalDouble;
6
7 static OptionalDouble mediaFuncional(int[] arr) {
8     if (arr == null || arr.length == 0) {
9         return OptionalDouble.empty();
10    }
11    return Arrays.stream(arr).average();
12 }
13
14 // Uso en main:
15 int[] conDatos = {1, 2, 3, 4, 5};
16 int[] sinDatos = {};
17
18 mediaFuncional(conDatos).ifPresentOrElse(
19     m -> System.out.printf("Media: %.1f%n", m),
20     () -> System.out.println("Array vacío o nulo"));
21
22 mediaFuncional(sinDatos).ifPresentOrElse(
23     m -> System.out.printf("Media: %.1f%n", m),
24     () -> System.out.println("Array vacío o nulo"));
25
26 // También se puede encadenar con map/filter/orElse:
27 double resultado = mediaFuncional(conDatos).orElse(0.0);
28 System.out.println("Media (o 0 si vacío): " + resultado);

```

Solución del ejercicio 21.2



```

1 // Fragmento: incluir dentro de una clase
2 import java.util.List;
3
4 List<String> palabras = List.of("java", "stream", "lambda",
  "yo", "él", "colección");
5

```

```

6 long total = palabras.stream()
7   .filter(p -> p.length() > 4)
8   .map(String::toUpperCase)
9   .peek(System.out::println) // imprime cada elemento ↩
   procesado
10  .count();
11
12 System.out.println("Total de palabras con más de 4 letras: ↩
   " + total);

```

Solución del ejercicio 21.3



```

1 // Fragmento: incluir dentro de una clase
2 import java.util.Optional;
3
4 Optional<String> nombre = Optional.of("Lucía");
5 Optional<String> vacio = Optional.empty();
6
7 String resultado1 = nombre.map(String::toUpperCase).orElse(↩
   "DESCONOCIDO");
8 String resultado2 = vacio.map(String::toUpperCase).orElse("↩
   DESCONOCIDO");
9
10 System.out.println(resultado1); // LUCÍA
11 System.out.println(resultado2); // DESCONOCIDO

```

Patrones de diseño software

En este capítulo se estudian los patrones de diseño de software, que son soluciones reutilizables a problemas comunes en el desarrollo de aplicaciones. Los patrones de diseño ayudan a estructurar el código de manera eficiente, facilitando la mantenibilidad, la escalabilidad y la reutilización. Se dividen en tres grandes categorías: patrones creacionales, estructurales y de comportamiento. A lo largo del capítulo se explican los principales patrones de cada categoría, su propósito y ejemplos prácticos de implementación.

22.1. Patrones creacionales

Los patrones creacionales son aquellos que se centran en la manera de crear objetos en el software. Su objetivo principal es abstraer el proceso de instancia-ción, permitiendo que el sistema sea más flexible y desacoplado. Estos patrones ayudan a controlar cómo y cuándo se crean los objetos, facilitando la reutilización y la escalabilidad del código.

Entre los patrones creacionales más conocidos se encuentran:

- **Singleton:** Garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.
- **Factory Method:** Define una interfaz para crear un objeto, pero permite que las subclasses decidan qué clase instanciar.
- **Abstract Factory:** Proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.
- **Builder:** Separa la construcción de un objeto complejo de su representación, permitiendo crear diferentes representaciones con el mismo proceso de construcción.
- **Prototype:** Permite copiar objetos existentes para crear nuevos objetos, evitando la creación desde cero.

Estos patrones son fundamentales para diseñar sistemas robustos y mantenibles, especialmente cuando se requiere flexibilidad en la creación de objetos.

22.1.1. El patrón *Abstract Factory*

El patrón *Abstract Factory* es un patrón creacional que permite crear familias de objetos relacionados o dependientes sin especificar sus clases concretas. Este patrón proporciona una interfaz para crear objetos de diferentes tipos, pero deja en manos de las subclases la decisión de qué clase concreta instanciar. Es especialmente útil cuando el sistema debe ser independiente de cómo se crean, componen y representan los objetos.

Por ejemplo, supongamos que estamos desarrollando una interfaz gráfica que puede tener diferentes estilos visuales (por ejemplo, estilo clásico y estilo moderno). Cada estilo requiere un conjunto de componentes (botones, ventanas, menús) que deben ser compatibles entre sí. El patrón *Abstract Factory* permite definir una interfaz para crear estos componentes y luego implementar fábricas concretas para cada estilo.

A continuación se muestra un ejemplo de uso del patrón *Abstract Factory* en Java:

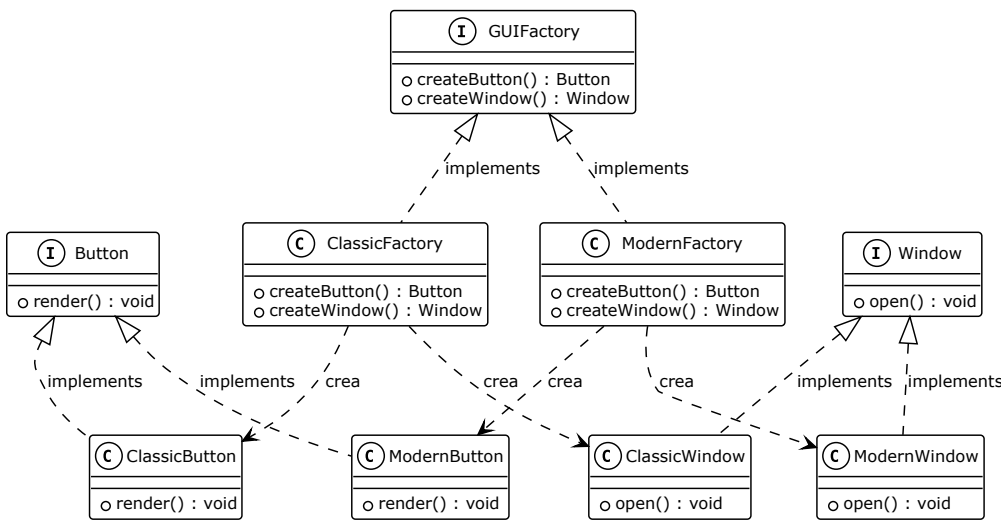


Figura 22.1: Diagrama UML del patrón Abstract Factory

```
1 interface Button {
2     void render();
3 }
4
5 interface Window {
```

```

6   void open();
7 }
8
9 // Productos concretos
10 class ClassicButton implements Button {
11     public void render() {
12         System.out.println("Renderizando botón clásico");
13     }
14 }
15
16 class ModernButton implements Button {
17     public void render() {
18         System.out.println("Renderizando botón moderno");
19     }
20 }
21
22 class ClassicWindow implements Window {
23     public void open() {
24         System.out.println("Abriendo ventana clásica");
25     }
26 }
27
28 class ModernWindow implements Window {
29     public void open() {
30         System.out.println("Abriendo ventana moderna");
31     }
32 }
33
34 // Abstract Factory
35 interface GUIFactory {
36     Button createButton();
37     Window createWindow();
38 }
39
40 // Fábricas concretas
41 class ClassicFactory implements GUIFactory {
42     public Button createButton() {
43         return new ClassicButton();
44     }
45     public Window createWindow() {
46         return new ClassicWindow();
47     }
48 }
49
50 class ModernFactory implements GUIFactory {
51     public Button createButton() {
52         return new ModernButton();
53     }
54     public Window createWindow() {

```

```

55     return new ModernWindow();
56 }
57 }
58
59 // Uso del patrón
60 public class Main {
61     public static void createGUI(GUIFactory factory) {
62         Button button = factory.createButton();
63         Window window = factory.createWindow();
64         button.render();
65         window.open();
66     }
67
68     public static void main(String[] args) {
69         GUIFactory factory = new ClassicFactory();
70         createGUI(factory);
71
72         factory = new ModernFactory();
73         createGUI(factory);
74     }
75 }

```

Ejemplo 22.1: Ejemplo de Abstract Factory en Java

En este ejemplo, el cliente puede cambiar el estilo de la interfaz simplemente cambiando la fábrica utilizada, sin modificar el código que utiliza los componentes.

Ejercicio 22.1



Implementa el patrón *Abstract Factory* para una aplicación que debe crear componentes de interfaz gráfica para dos sistemas operativos diferentes: Windows y Linux. Define las interfaces necesarias y las fábricas concretas para cada sistema operativo. Escribe un ejemplo de uso donde se creen botones y ventanas para ambos sistemas operativos y se muestre cómo el cliente puede cambiar el sistema operativo sin modificar el código que utiliza los componentes. *Ver solución en la página 292.*

22.2. Patrones estructurales

Los patrones estructurales se enfocan en la manera en que las clases y los objetos se organizan y se relacionan entre sí para formar estructuras más grandes y flexibles. Su objetivo es facilitar la composición y reutilización de clases y objetos, permitiendo que el sistema evolucione sin grandes modificaciones.

Algunos de los patrones estructurales más utilizados son:

- **Adapter:** Permite que la interfaz de una clase sea compatible con la que espera el cliente, facilitando la integración de clases con interfaces incompatibles.
- **Bridge:** Separa la abstracción de su implementación, permitiendo que ambas evolucionen independientemente.
- **Composite:** Permite tratar objetos individuales y composiciones de objetos de manera uniforme.
- **Decorator:** Añade funcionalidades adicionales a un objeto de manera dinámica, sin modificar su estructura original.
- **Facade:** Proporciona una interfaz simplificada para un conjunto de interfaces en un subsistema, facilitando su uso.
- **Flyweight:** Reduce el consumo de memoria compartiendo la mayor cantidad posible de datos entre objetos similares.
- **Proxy:** Proporciona un objeto sustituto que controla el acceso a otro objeto, permitiendo realizar acciones adicionales antes o después del acceso.

Estos patrones ayudan a construir sistemas más modulares y escalables, facilitando la gestión de la complejidad en el diseño de software.

22.2.1. El patrón *Adapter*

El patrón *Adapter* es un patrón estructural que permite que dos interfaces incompatibles trabajen juntas. Su objetivo es convertir la interfaz de una clase en otra interfaz que el cliente espera, facilitando la reutilización de código existente sin modificarlo. El adaptador actúa como un intermediario entre el cliente y el servicio, traduciendo las llamadas y datos entre ambos.

Este patrón es útil cuando se necesita integrar clases con interfaces diferentes, por ejemplo, al utilizar bibliotecas de terceros o al migrar sistemas antiguos.

A continuación se muestra un ejemplo de uso del patrón *Adapter* en Java:

```

1 interface MediaPlayer {
2     void play(String filename);
3 }
4
5 // Clase existente con interfaz incompatible
6 class AdvancedMediaPlayer {
7     public void playMp3(String filename) {
8         System.out.println("Reproduciendo MP3: " + filename);
9     }

```

```

10 public void playMp4(String filename) {
11     System.out.println("Reproduciendo MP4: " + filename);
12 }
13 }
14
15 // Adaptador que permite usar AdvancedMediaPlayer como ↪
16 // MediaPlayer
17 class MediaAdapter implements MediaPlayer {
18     private AdvancedMediaPlayer advancedPlayer;
19
20     public MediaAdapter() {
21         advancedPlayer = new AdvancedMediaPlayer();
22     }
23
24     public void play(String filename) {
25         if (filename.endsWith(".mp3")) {
26             advancedPlayer.playMp3(filename);
27         } else if (filename.endsWith(".mp4")) {
28             advancedPlayer.playMp4(filename);
29         } else {
30             System.out.println("Formato no soportado: " + filename);
31         }
32     }
33 }
34
35 // Uso del patrón Adapter
36 public class Main {
37     public static void main(String[] args) {
38         MediaPlayer player = new MediaAdapter();
39         player.play("cancion.mp3");
40         player.play("video.mp4");
41         player.play("documento.pdf");
42     }
43 }

```

Ejemplo 22.2: Ejemplo de Adapter en Java

En este ejemplo, `MediaAdapter` permite que el cliente utilice la interfaz `MediaPlayer` para reproducir archivos MP3 y MP4, adaptando las llamadas a los métodos de `AdvancedMediaPlayer` según el formato del archivo.

Ejercicio 22.2



Implementa el patrón *Adapter* para integrar dos sistemas de pago con interfaces diferentes: uno que utiliza el método `pagarConTarjeta(String numeroTarjeta, double monto)` y otro que utiliza el método `realizarPago(String cuenta, double cantidad)`. Define una interfaz común `Pago` con el método `pagar(String identificador,`

`double monto)` y crea los adaptadores necesarios para que ambos sistemas puedan ser utilizados de manera uniforme por el cliente. Escribe un ejemplo de uso donde se realicen pagos con ambos sistemas a través de la interfaz común. *Ver solución en la página 293.*

22.3. Patrones de comportamiento

Los patrones de comportamiento se centran en la manera en que los objetos interactúan y se comunican entre sí. Su propósito principal es definir cómo se distribuyen las responsabilidades y cómo se gestionan las interacciones entre los distintos componentes del sistema, promoviendo la flexibilidad y la reutilización del código.

Algunos de los patrones de comportamiento más conocidos son:

- **Chain of Responsibility:** Permite que varios objetos tengan la oportunidad de manejar una solicitud, evitando el acoplamiento entre el emisor y el receptor.
- **Command:** Encapsula una petición como un objeto, permitiendo parametrizar clientes con diferentes solicitudes y soportar operaciones como deshacer.
- **Interpreter:** Define una representación para un lenguaje y un intérprete para analizar sentencias de ese lenguaje.
- **Iterator:** Proporciona una manera de acceder secuencialmente a los elementos de un objeto agregado sin exponer su representación interna.
- **Mediator:** Define un objeto que encapsula la manera en que interactúan un conjunto de objetos, promoviendo el desacoplamiento.
- **Memento:** Permite capturar y restaurar el estado interno de un objeto sin violar la encapsulación.
- **Observer:** Define una dependencia uno-a-muchos entre objetos, de modo que cuando uno cambia su estado, todos sus dependientes son notificados automáticamente.
- **State:** Permite que un objeto altere su comportamiento cuando su estado interno cambia.

- **Strategy:** Define una familia de algoritmos, encapsula cada uno y los hace intercambiables.
- **Template Method:** Define el esqueleto de un algoritmo en una operación, dejando algunos pasos a las subclasses.
- **Visitor:** Permite definir nuevas operaciones sobre una estructura de objetos sin cambiar las clases de los objetos sobre los que opera.

Estos patrones son esenciales para organizar la lógica de interacción y el flujo de control en aplicaciones complejas, facilitando la extensión y el mantenimiento del software.

22.3.1. El patrón *Visitor*

El patrón *Visitor* es un patrón de comportamiento que permite definir nuevas operaciones sobre una estructura de objetos sin modificar las clases de los objetos sobre los que opera. Este patrón es útil cuando se necesita realizar distintas operaciones sobre una colección de objetos con diferentes tipos, y se desea evitar la proliferación de métodos en las clases de dichos objetos.

El patrón *Visitor* separa la lógica de las operaciones de la estructura de los objetos, facilitando la extensión de funcionalidades sin alterar el código existente. Para ello, se define una interfaz *Visitor* con métodos para cada tipo de objeto, y cada clase de objeto acepta un visitante mediante el método *accept*.

A continuación se muestra un ejemplo de uso del patrón *Visitor* en Java:

```

1 interface Visitor {
2     void visit(Book book);
3     void visit(Magazine magazine);
4 }
5
6 interface Element {
7     void accept(Visitor visitor);
8 }
9
10 class Book implements Element {
11     String title;
12     public Book(String title) { this.title = title; }
13     public void accept(Visitor visitor) { visitor.visit(this); }
14 }
15
16 class Magazine implements Element {
17     String name;
18     public Magazine(String name) { this.name = name; }
19     public void accept(Visitor visitor) { visitor.visit(this); }
20 }

```

```

21
22 // Visitor concreto
23 class PrintVisitor implements Visitor {
24     public void visit(Book book) {
25         System.out.println("Libro: " + book.title);
26     }
27     public void visit(Magazine magazine) {
28         System.out.println("Revista: " + magazine.name);
29     }
30 }
31
32 // Uso del patrón Visitor
33 public class Main {
34     public static void main(String[] args) {
35         Element[] items = { new Book("Patrones de Diseño"), new ↵
Magazine("Java Monthly") };
36         Visitor visitor = new PrintVisitor();
37         for (Element item : items) {
38             item.accept(visitor);
39         }
40     }
41 }

```

Ejemplo 22.3: Ejemplo de Visitor en Java

En este ejemplo, el visitante `PrintVisitor` implementa la lógica para imprimir información de libros y revistas, y puede añadirse fácilmente nuevos visitantes para realizar otras operaciones sin modificar las clases `Book` y `Magazine`.

Ejercicio 22.3



Implementa el patrón *Visitor* para una aplicación que gestiona diferentes tipos de figuras geométricas: círculo, cuadrado y triángulo. Define una interfaz `Visitor` con métodos para cada tipo de figura y una interfaz `Figura` con el método `accept`. Crea un visitante concreto que calcule el área de cada figura y muestra un ejemplo de uso donde se calcula el área de varias figuras utilizando el visitante. Crea otro visitante que calcule el perímetro de cada figura y muestra un ejemplo de uso similar. *Ver solución en la página 294.*

Soluciones de los ejercicios

Solución del ejercicio 22.1



```

1  // Programa completo: Abstract Factory para componentes GUI →
   Windows/Linux
2  interface Boton {
3      void renderizar();
4  }
5
6  interface Ventana {
7      void mostrar();
8  }
9
10 class BotonWindows implements Boton {
11     public void renderizar() { System.out.println("[Windows] →
        Botón renderizado."); }
12 }
13
14 class VentanaWindows implements Ventana {
15     public void mostrar() { System.out.println("[Windows] →
        Ventana mostrada."); }
16 }
17
18 class BotonLinux implements Boton {
19     public void renderizar() { System.out.println("[Linux] →
        Botón renderizado."); }
20 }
21
22 class VentanaLinux implements Ventana {
23     public void mostrar() { System.out.println("[Linux] →
        Ventana mostrada."); }
24 }
25
26 interface GUIFactory {
27     Boton crearBoton();
28     Ventana crearVentana();
29 }
30
31 class WindowsFactory implements GUIFactory {
32     public Boton crearBoton() { return new BotonWindows(); }
33     public Ventana crearVentana() { return new VentanaWindows →
        (); }
34 }
35
36 class LinuxFactory implements GUIFactory {
37     public Boton crearBoton() { return new BotonLinux(); }

```

```

38 public Ventana crearVentana() { return new VentanaLinux(); ↵
39 }
40
41 public class AplicacionGUI {
42     static void crearUI(GUIFactory factory) {
43         factory.crearBoton().renderizar();
44         factory.crearVentana().mostrar();
45     }
46
47     public static void main(String[] args) {
48         System.out.println("--- Windows ---");
49         crearUI(new WindowsFactory());
50         System.out.println("--- Linux ---");
51         crearUI(new LinuxFactory());
52     }
53 }

```

Solución del ejercicio 22.2



```

1 // Programa completo: Adapter para sistemas de pago con ↵
  interfaces incompatibles
2 interface Pago {
3     void pagar(String identificador, double monto);
4 }
5
6 class SistemaTarjeta {
7     public void pagarConTarjeta(String numeroTarjeta, double ↵
8         monto) {
9         System.out.printf("Pago con tarjeta %s: %.2f €%n", ↵
10             numeroTarjeta, monto);
11     }
12 }
13
14 class SistemaBancario {
15     public void realizarPago(String cuenta, double cantidad) {
16         System.out.printf("Transferencia desde %s: %.2f €%n", ↵
17             cuenta, cantidad);
18     }
19 }
20
21 class AdaptadorTarjeta implements Pago {
22     private final SistemaTarjeta sistema;
23
24     public AdaptadorTarjeta(SistemaTarjeta sistema) {
25         this.sistema = sistema;
26     }
27
28     public void pagar(String identificador, double monto) {
29         sistema.pagarConTarjeta(identificador, monto);
30     }
31 }

```

```

23 }
24
25 public void pagar(String identificador, double monto) {
26     sistema.pagarConTarjeta(identificador, monto);
27 }
28 }
29
30 class AdaptadorBancario implements Pago {
31     private final SistemaBancario sistema;
32
33     public AdaptadorBancario(SistemaBancario sistema) {
34         this.sistema = sistema;
35     }
36
37     public void pagar(String identificador, double monto) {
38         sistema.realizarPago(identificador, monto);
39     }
40 }
41
42 public class ClientePago {
43     public static void main(String[] args) {
44         Pago pago1 = new AdaptadorTarjeta(new SistemaTarjeta());
45         Pago pago2 = new AdaptadorBancario(new SistemaBancario()) ↪
46         ;
47         pago1.pagar("4111-1111-1111-1111", 49.99);
48         pago2.pagar("ES12 3456 7890 1234", 150.00);
49     }
50 }

```

Solución del ejercicio 22.3



```

1 // Programa completo: Visitor para calcular área y ↪
  perímetro de figuras geométricas
2 interface Visitor {
3     void visit(Circulo c);
4     void visit(Cuadrado c);
5     void visit(Triangulo t);
6 }
7
8 interface Figura {
9     void accept(Visitor v);
10 }
11
12 class Circulo implements Figura {
13     final double radio;
14     public Circulo(double radio) { this.radio = radio; }

```

```

15 public void accept(Visitor v) { v.visit(this); }
16 }
17
18 class Cuadrado implements Figura {
19     final double lado;
20     public Cuadrado(double lado) { this.lado = lado; }
21     public void accept(Visitor v) { v.visit(this); }
22 }
23
24 class Triangulo implements Figura {
25     final double base, altura, ladoA, ladoB, ladoC;
26     public Triangulo(double base, double altura, double a, ↵
27         double b, double c) {
28         this.base = base; this.altura = altura;
29         this.ladoA = a; this.ladoB = b; this.ladoC = c;
30     }
31     public void accept(Visitor v) { v.visit(this); }
32 }
33
34 class CalculadorArea implements Visitor {
35     public void visit(Circulo c) {
36         System.out.printf("Área círculo (r=%.0f): %.2f%n", c.↵
37             radio, Math.PI * c.radio * c.radio);
38     }
39     public void visit(Cuadrado c) {
40         System.out.printf("Área cuadrado (l=%.0f): %.2f%n", c.↵
41             lado, c.lado * c.lado);
42     }
43     public void visit(Triangulo t) {
44         System.out.printf("Área triángulo: %.2f%n", 0.5 * t.base ↵
45             * t.altura);
46     }
47 }
48
49 class CalculadorPerimetro implements Visitor {
50     public void visit(Circulo c) {
51         System.out.printf("Perímetro círculo (r=%.0f): %.2f%n", c↵
52             .radio, 2 * Math.PI * c.radio);
53     }
54     public void visit(Cuadrado c) {
55         System.out.printf("Perímetro cuadrado (l=%.0f): %.2f%n", ↵
56             c.lado, 4 * c.lado);
57     }
58     public void visit(Triangulo t) {
59         System.out.printf("Perímetro triángulo: %.2f%n", t.ladoA ↵
60             + t.ladoB + t.ladoC);
61     }
62 }

```

```

55 }
56
57 public class Main {
58     public static void main(String[] args) {
59         Figura[] figuras = {
60             new Circulo(5),
61             new Cuadrado(4),
62             new Triangulo(3, 4, 3, 4, 5)
63         };
64         Visitor areas = new CalculadorArea();
65         Visitor perimetros = new CalculadorPerimetro();
66         System.out.println("--- Áreas ---");
67         for (Figura f : figuras) { f.accept(areas); }
68         System.out.println("--- Perímetros ---");
69         for (Figura f : figuras) { f.accept(perimetros); }
70     }
71 }

```

Resumen

En este capítulo se han presentado los principales patrones de diseño de software, agrupados en tres categorías: creacionales, estructurales y de comportamiento. Estos patrones ofrecen soluciones probadas a problemas recurrentes en el desarrollo de aplicaciones, facilitando la creación de sistemas más flexibles, escalables y mantenibles. La correcta aplicación de los patrones de diseño permite reducir el acoplamiento, mejorar la reutilización del código y simplificar la gestión de la complejidad en proyectos de software. Comprender y utilizar estos patrones es fundamental para cualquier desarrollador que aspire a escribir código profesional y robusto. Para profundizar en el estudio de los patrones de diseño, se recomienda consultar la referencia: <https://refactoring.guru/design-patterns>.

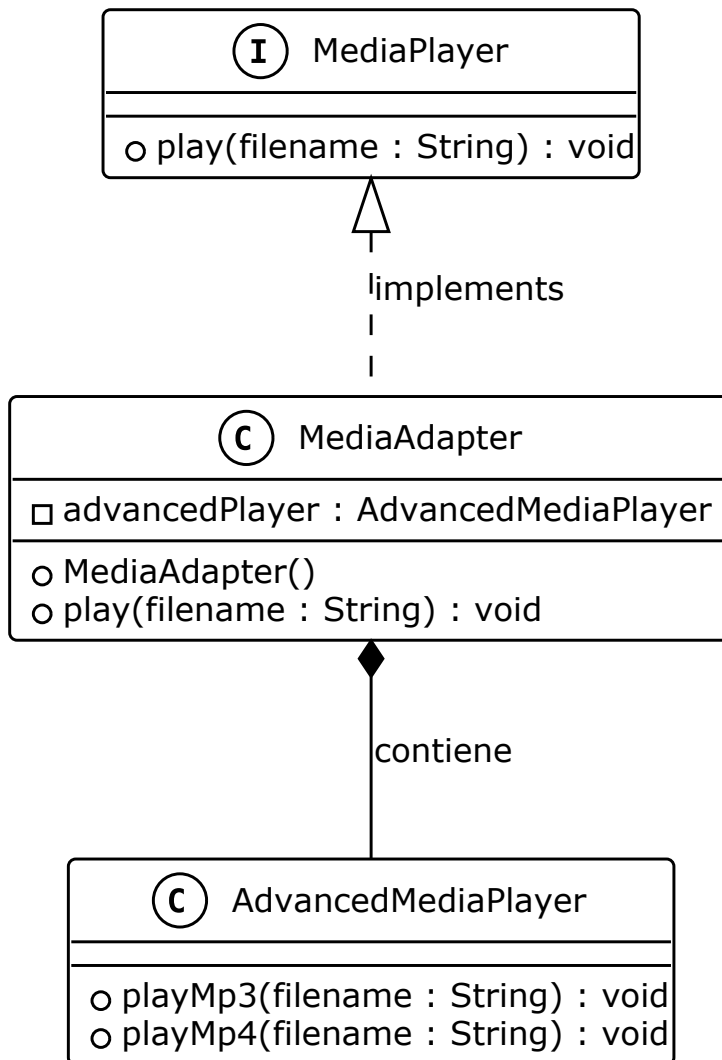


Figura 22.2: Diagrama UML del patrón Adapter

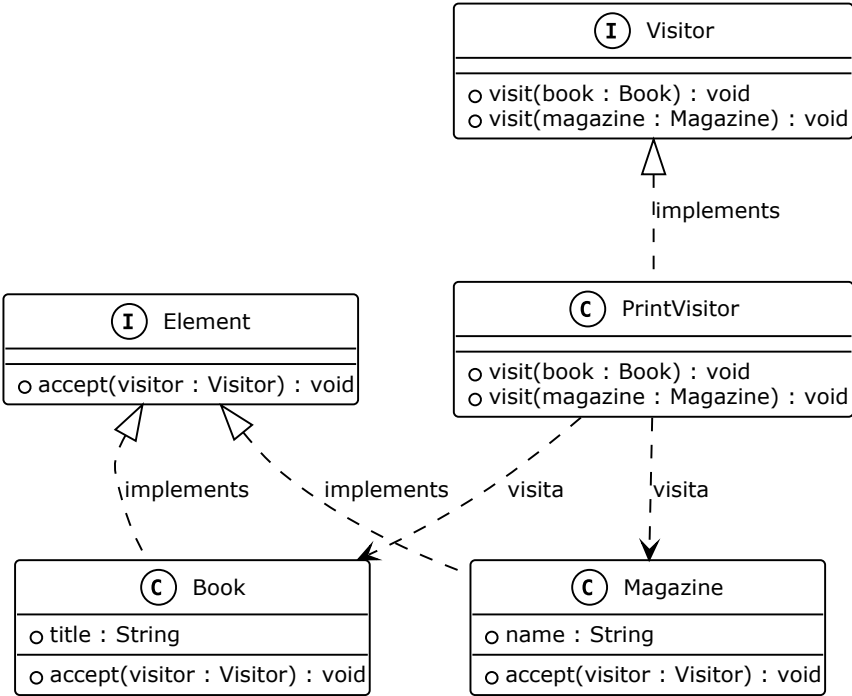


Figura 22.3: Diagrama UML del patrón Visitor

Programación paralela

En este capítulo se exploran los conceptos fundamentales de la programación paralela y concurrente, centrándose en el lenguaje Java. Aprenderás cómo crear y gestionar hilos, sincronizar el acceso a recursos compartidos, y utilizar herramientas como mutex, semáforos y clases atómicas para garantizar la seguridad y eficiencia en aplicaciones que requieren ejecutar múltiples tareas simultáneamente. Además, se presentan ejemplos prácticos y recomendaciones para evitar errores comunes en entornos concurrentes.

23.1. Introducción a la programación paralela

La programación paralela permite que un programa ejecute varias tareas al mismo tiempo, aprovechando los procesadores multinúcleo y mejorando el rendimiento en operaciones intensivas. En Java, la programación paralela se puede realizar mediante hilos (threads), el framework **Executor**, y las APIs de concurrencia y paralelismo.

Importante

La programación paralela requiere especial atención a la sincronización y la gestión de recursos compartidos para evitar errores como condiciones de carrera y bloqueos.

La figura 23.1 muestra los estados por los que puede pasar un hilo durante su ciclo de vida y las transiciones entre ellos.

23.2. Las clases **Thread**, y **Runnable**

Un hilo (*thread*) es una unidad de ejecución dentro de un proceso. Los programas pueden crear múltiples hilos para realizar tareas simultáneamente, como cálculos, acceso a archivos o comunicación en red.

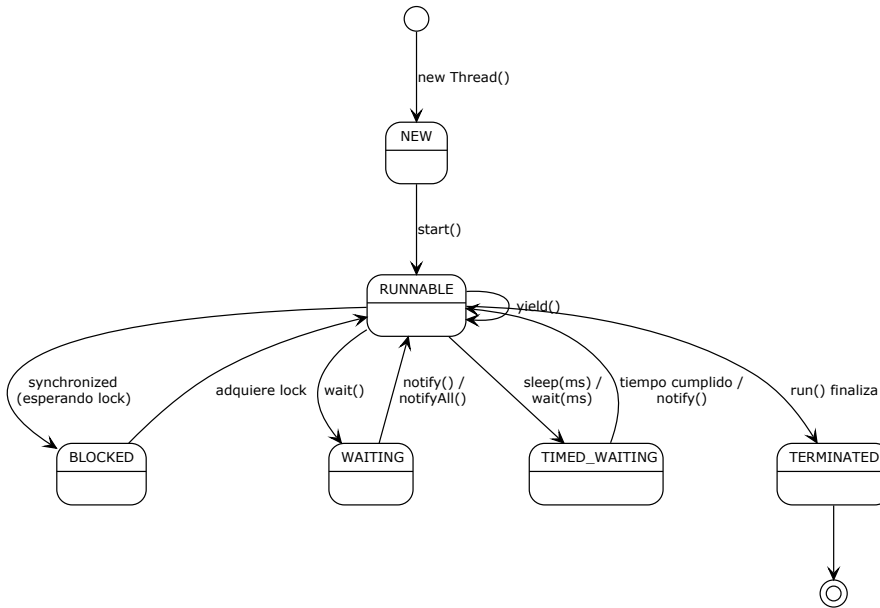


Figura 23.1: Ciclo de vida de un hilo en Java

- **Extender la clase Thread:** Consiste en crear una nueva clase que herede de `Thread` y sobrescriba el método `run()`, donde se define el código que ejecutará el hilo. Luego, se crea una instancia de la clase y se llama al método `start()` para iniciar el hilo. Esta forma es sencilla, pero limita la herencia, ya que Java no permite heredar de más de una clase.
- **Implementar la interfaz Runnable¹:** Se crea una clase que implementa la interfaz `Runnable` y define el método `run()`. Después, se crea un objeto `Thread` pasando una instancia de la clase `Runnable` como argumento y se invoca `start()`. Esta forma es más flexible, ya que permite que la clase implemente otras interfaces o extienda otras clases.

```

1 class MiHilo extends Thread {
2     public void run() {
3         System.out.println("Hola desde el hilo " + getName());
4     }
5 }
6
7 MiHilo hilo1 = new MiHilo();
8 MiHilo hilo2 = new MiHilo();
9 hilo1.start();

```

¹Esta es una interfaz funcional, así que es posible usar una expresión lambda en vez de crear una clase específica

```

10 hilo2.start();
11 hilo1.join();
12 hilo2.join();

```

Ejemplo 23.1: Hilo mediante extensión de Thread

Al crear un hilo mediante la extensión de **Thread**, se sobrescribe el método **run()** para definir la tarea que ejecutará el hilo. Luego, se crea una instancia de la clase y se llama al método **start()** para iniciar el hilo. Este hilo completará su ejecución hasta que termine el método **run()**. El hilo principal esperará a que acabe cada hilo, si se llama al método **join()**.

```

1 class Tarea implements Runnable {
2     public void run() {
3         System.out.println("Ejecutando tarea en hilo " + Thread.↵
4             currentThread().getName());
5     }
6 }
7 Thread hilo = new Thread(new Tarea());
8 hilo.start();

```

Ejemplo 23.2: Hilo mediante Runnable

Al usar la interfaz **Runnable**, se define el método **run()** en una clase que implementa la interfaz. Al pasar un objeto de esa clase al constructor de la clase **Thread**, se crea un hilo que ejecutará el código definido en **run()**, lo cual indica que el hilo delega el código a la clase **Runnable**.

```

1 Thread hiloLambda = new Thread(() -> {
2     System.out.println("Ejecutando hilo con lambda: " + Thread.↵
3         currentThread().getName());
4 });
5 hiloLambda.start();

```

Ejemplo 23.3: Hilo mediante expresión lambda

Una manera más compacta es usar una expresión lambda. De esta manera podemos reducir la cantidad de código necesario para crear un hilo. Sin embargo, al usar expresiones lambda, se pierde la capacidad de reutilizar el código en diferentes contextos, ya que no se puede instanciar la clase **Runnable** de nuevo.

En todos estos casos, el código de cada hilo se ejecutará independientemente del hilo principal, sin coordinación alguna. Por ello es importante introducir mecanismos de concurrencia.

23.3. Concurrencia

La concurrencia permite que varios hilos se ejecuten y compartan recursos. Es fundamental controlar el acceso a los recursos compartidos para evitar inconsis-

tencias y errores. Existen múltiples estrategias al respecto, pero aquí hablaremos de las más comunes.

23.3.1. Las clases Mutex, y Semaphore

En Java, los mutex y semáforos se utilizan para controlar el acceso a recursos compartidos.

■ Mutex (ReentrantLock):

- Permite bloquear y desbloquear explícitamente, lo que da mayor control sobre la sección crítica.
- Soporta bloqueos reentrantes, es decir, el mismo hilo puede adquirir el lock varias veces.
- Ofrece métodos avanzados como `tryLock()` y soporte para condiciones (`Condition`).

■ Semaphore:

- Permite controlar el número de hilos que acceden simultáneamente a un recurso.
- Es útil para limitar el acceso concurrente, por ejemplo, a un grupo de conexiones o recursos.
- Puede implementar sincronización de múltiples productores/consumidores.

```

1 public class EjemploConcurrencia {
2     private static ReentrantLock lock = new ReentrantLock();
3     private static Semaphore sem = new Semaphore(2);
4
5     public static void main(String[] args) {
6         Thread[] hilos = new Thread[5];
7         // Crear los hilos
8         for (int i = 0; i < 5; i++) {
9             final int id = i+1;
10            hilos[i] = new Thread(() -> metodoParalelo(id));
11        }
12        // Ejecutar cada hilo
13        for (Thread hilo : hilos) {
14            hilo.start();
15        }
16        // Esperar a que terminen todos los hilos
17        for (Thread hilo : hilos) {
18            try {

```

```

19     hilo.join();
20 } catch (InterruptedException e) {
21     Thread.currentThread().interrupt();
22 }
23 }
24 }
25
26 private static void metodoParalelo(int id){
27     System.out.println("Hilo " + id + " intentando acceder a la ↵
sección crítica");
28     seccionCritica();
29     System.out.println("Hilo " + id + " ha salido de la sección ↵
crítica");
30     System.out.println("Hilo " + id + " intentando acceder al ↵
recurso con semáforo");
31     usarSemaforo();
32     System.out.println("Hilo " + id + " ha salido del recurso ↵
con semáforo");
33 }
34
35 public static void seccionCritica() {
36     lock.lock();
37     try {
38         System.out.println("Accediendo a la sección crítica");
39         Thread.sleep(1000);
40     } catch (InterruptedException e) {
41         Thread.currentThread().interrupt();
42     } finally {
43         lock.unlock();
44     }
45 }
46
47 public static void usarSemaforo() {
48     try {
49         sem.acquire();
50         System.out.println("Accediendo al recurso con semáforo");
51         Thread.sleep(1000);
52     } catch (InterruptedException e) {
53         Thread.currentThread().interrupt();
54     } finally {
55         sem.release();
56     }
57 }
58 }

```

23.4. Atomicidad y consistencia

La atomicidad garantiza que una operación se realiza completamente o no se realiza en absoluto. La consistencia asegura que los datos permanezcan válidos tras las operaciones concurrentes. Como cada hilo puede ejecutarse en distinto orden a otros, o durante las instrucciones de otro hilo, es importante controlar que el acceso a los datos, y su modificación, ocurra de manera atómica.

23.4.1. La palabra clave `synchronized`

La palabra clave `synchronized` permite sincronizar métodos o bloques de código para evitar que varios hilos accedan simultáneamente a una sección crítica.

```
1 class Contador {  
2     private int valor = 0;  
3     public synchronized void incrementar() {  
4         valor++;  
5     }  
6     public int getValor() { return valor; }  
7 }
```

Ejemplo 23.4: Método sincronizado

La palabra clave `synchronized` es una herramienta sencilla y poderosa para controlar el acceso concurrente a recursos compartidos en Java. Permite que sólo un hilo acceda a una sección crítica a la vez, evitando condiciones de carrera y garantizando la consistencia de los datos.

Ventajas:

- **Facilidad de uso:** Es fácil de aplicar tanto en métodos como en bloques de código.
- **Legibilidad:** El código sincronizado es claro y explícito, lo que facilita su comprensión.
- **Integración:** Funciona bien con los mecanismos internos de Java, como los monitores de objetos.

Desventajas:

- **Rendimiento:** Puede causar bloqueos y reducir la concurrencia si se usa en exceso o en secciones de código extensas.
- **Deadlocks:** Un uso incorrecto puede provocar bloqueos mutuos entre hilos (deadlocks).

- **Falta de flexibilidad:** No permite intentar adquirir el bloqueo sin esperar, ni ofrece mecanismos avanzados como condiciones.
- **Dificultad de depuración:** Los errores de sincronización pueden ser difíciles de detectar y depurar.

Por ello, aunque `synchronized` es adecuado para casos simples, en aplicaciones complejas suele ser preferible usar clases de concurrencia avanzadas como `ReentrantLock` o herramientas específicas de la API de concurrencia de Java.

23.4.2. Las clases atómicas

Las clases atómicas de Java (como `AtomicInteger`, o `AtomicReference`) permiten realizar operaciones seguras sin necesidad de sincronización explícita.

```

1 import java.util.concurrent.atomic.AtomicInteger;
2
3 AtomicInteger contador = new AtomicInteger(0);
4 contador.incrementAndGet();
5 System.out.println(contador.get());

```

Ejemplo 23.5: Uso de `AtomicInteger`

El uso de clases atómicas es especialmente útil en situaciones donde se requiere un alto rendimiento y baja latencia, ya que minimizan la sobrecarga de la sincronización. Además, permite que código que se ejecuta en diferentes hilos modifique el mismo objeto de manera segura, sin necesidad de bloquearlo.

```

1 public class UsuariosRepository {
2     private final AtomicInteger contador = new AtomicInteger(0);
3     private final UsuariosApi usuariosApi;
4
5     // Constructor
6
7     public int agregarUsuario(String nombre) {
8         int id = contador.incrementAndGet();
9         usuariosApi.agregarUsuario(id, nombre);
10        return id;
11    }
12
13    public int getTotalUsuarios() {
14        return contador.get();
15    }
16
17    public String getUsuario(int id) {
18        return usuariosApi.getUsuario(id);
19    }
20 }

```

En el código anterior, la clase `UsuariosRepository` utiliza un `AtomicInteger` para llevar un conteo de usuarios. Cada vez que se agrega un usuario, se incrementa el contador de manera atómica, garantizando que no haya problemas de concurrencia al acceder al mismo recurso desde múltiples hilos. Esto asegura que cada usuario tiene un identificador distinto, único y secuencial.

Ejercicio 23.1



Crea un programa que sume los números del 1 al 100 de forma paralela utilizando `AtomicInteger`. Cada hilo debe sumar un rango de números y actualizar el contador atómico. Al final, imprime el resultado total. *Ver solución en la página 310.*

23.4.3. Colecciones concurrentes

Java proporciona colecciones concurrentes como `ConcurrentHashMap` y `CopyOnWriteArrayList` para trabajar de forma segura con múltiples hilos. Al usar este tipo de colecciones en nuestro código, permitiremos que en el futuro se puedan realizar operaciones concurrentes sin necesidad de bloqueos explícitos.

```
1 import java.util.concurrent.ConcurrentHashMap;
2
3 ConcurrentHashMap<String, Integer> mapa = new ConcurrentHashMap<>();
4 Thread hilo = new Thread(() -> {
5     mapa.put("c", 3);
6 });
7 mapa.put("a", 1);
8 hilo.start();
9 mapa.put("b", 2);
10 hilo.join();
11 System.out.println(mapa.get("c"));
```

Ejemplo 23.6: Uso de `ConcurrentHashMap`

Resumen

La programación paralela en Java permite ejecutar múltiples tareas simultáneamente, optimizando el uso de los procesadores y mejorando el rendimiento de las aplicaciones. Para lograrlo, se emplean hilos, mecanismos de sincronización como `synchronized`, locks y semáforos, así como clases atómicas y colecciones concurrentes que garantizan la seguridad y consistencia de los datos compartidos. El uso adecuado de estas herramientas es fundamental para evitar errores

como condiciones de carrera y bloqueos, y para desarrollar aplicaciones robustas y eficientes en entornos concurrentes.

Cuadro 23.1: Principales métodos de la clase `AtomicInteger`

Método	Descripción
<code>get()</code>	Obtiene el valor actual de manera atómica.
<code>set(int newValue)</code>	Establece el valor de manera atómica.
<code>getAndSet(int newValue)</code>	Establece el valor y devuelve el valor anterior, de forma atómica.
<code>incrementAndGet()</code>	Incrementa el valor en uno y devuelve el resultado, de forma atómica.
<code>getAndIncrement()</code>	Devuelve el valor actual y luego lo incrementa en uno, de forma atómica.
<code>decrementAndGet()</code>	Decrementa el valor en uno y devuelve el resultado, de forma atómica.
<code>getAndDecrement()</code>	Devuelve el valor actual y luego lo decrementa en uno, de forma atómica.
<code>addAndGet(int delta)</code>	Suma el valor indicado y devuelve el resultado, de forma atómica.
<code>getAndAdd(int delta)</code>	Devuelve el valor actual y luego suma el valor indicado, de forma atómica.
<code>compareAndSet(int expect, int update)</code>	Actualiza el valor si coincide con el esperado, de forma atómica. Devuelve <code>true</code> si la actualización fue exitosa.

Ejercicios de refuerzo

Ejercicio 23.2



Crea 3 hilos usando expresiones lambda. Cada hilo debe imprimir su número (1, 2 o 3) junto con la iteración actual, repitiendo 3 veces. Inicia los tres hilos y observa cómo se entrelazan sus salidas. *Ver solución en la página 310.*

Ejercicio 23.3



Usa un `ExecutorService` con un pool de 2 hilos para ejecutar una tarea `Callable<Integer>` que calcule la suma de los números del 1 al 100. Obtén el resultado mediante un `Future<Integer>` e imprímelo. Llama a `shutdown()` al terminar. *Ver solución en la página 311.*

Soluciones de los ejercicios de refuerzo

Solución del ejercicio 23.1



```

1 // Fragmento: suma paralela de 1..100 usando AtomicInteger
2 import java.util.concurrent.atomic.AtomicInteger;
3
4 AtomicInteger suma = new AtomicInteger(0);
5 int numHilos = 4;
6 int rango = 100 / numHilos; // 25 números por hilo
7 Thread[] hilos = new Thread[numHilos];
8
9 for (int t = 0; t < numHilos; t++) {
10     final int inicio = t * rango + 1;
11     final int fin = (t == numHilos - 1) ? 100 : inicio + rango - 1;
12     hilos[t] = new Thread(() -> {
13         for (int i = inicio; i <= fin; i++) {
14             suma.addAndGet(i);
15         }
16     });
17     hilos[t].start();
18 }
19
20 for (Thread hilo : hilos) {
21     try {
22         hilo.join(); // esperar a que todos terminen
23     } catch (InterruptedException e) {
24         Thread.currentThread().interrupt(); // restaurar el flag de interrupción
25     }
26 }
27
28 System.out.println("Suma de 1 a 100: " + suma.get()); // 5050

```

Solución del ejercicio 23.2



```

1 // Fragmento: incluir dentro de una clase
2 for (int i = 1; i <= 3; i++) {
3     final int numero = i;
4     Thread hilo = new Thread(() -> {
5         for (int iter = 1; iter <= 3; iter++) {
6             System.out.println("Hilo " + numero + " - iteración " + iter);
7         }
8     });
9 }

```

```

8   });
9   hilo.start();
10  }

```

Solución del ejercicio 23.3



```

1  // Fragmento: incluir dentro de una clase
2  import java.util.concurrent.Callable;
3  import java.util.concurrent.ExecutorService;
4  import java.util.concurrent.Executors;
5  import java.util.concurrent.Future;
6
7  ExecutorService pool = Executors.newFixedThreadPool(2);
8
9  Callable<Integer> tarea = () -> {
10     int resultado = 0;
11     for (int i = 1; i <= 100; i++) {
12         resultado += i;
13     }
14     return resultado;
15 };
16
17 Future<Integer> futuro = pool.submit(tarea);
18 try {
19     System.out.println("Suma 1..100 = " + futuro.get()); // ↵
20     5050
21 } catch (InterruptedException e) {
22     Thread.currentThread().interrupt();
23 } catch (java.util.concurrent.ExecutionException e) {
24     System.err.println("Error en la tarea: " + e.getCause());
25 } finally {
26     pool.shutdown();
27 }

```


Clases de utilidad

Estas clases forman parte del paquete estándar de Java y están diseñadas para resolver problemas frecuentes en el desarrollo de aplicaciones. Utilizarlas correctamente permite escribir código más limpio, seguro y eficiente. A lo largo de este capítulo, se explicarán las funcionalidades principales de cada clase, con ejemplos prácticos que ilustran su uso en situaciones reales. El dominio de estas utilidades es fundamental para cualquier programador Java, ya que facilitan tareas como cálculos matemáticos, manipulación de colecciones, manejo de fechas y horas, generación de identificadores únicos, procesamiento de cadenas y entrada/salida de datos.

24.1. La clase Math

La clase `Math` proporciona métodos estáticos para realizar operaciones matemáticas avanzadas, como cálculos aritméticos, trigonométricos, exponenciales y logarítmicos. No es necesario crear una instancia de `Math`, ya que todos sus métodos son accesibles directamente desde la clase. Además de los métodos básicos, `Math` incluye constantes útiles como `Math.PI` (el valor de π) y `Math.E` (el número de Euler).

Por ejemplo, para calcular el área de un círculo y el logaritmo natural de un número:

```
1 double radio = 5.0;
2 double area = Math.PI * Math.pow(radio, 2); // Área del círculo
3 double logaritmo = Math.log(10); // Logaritmo natural de 10
```

La clase también ofrece métodos para generar números aleatorios (`Math.random()`), redondear decimales (`Math.ceil`, `Math.floor`), y convertir entre grados y radianes (`Math.toRadians`, `Math.toDegrees`), facilitando cálculos precisos y seguros en cualquier aplicación Java.

Algunos métodos destacados de la clase `Math` son:

- `Math.sqrt(x)`: Devuelve la raíz cuadrada de `x`.

- `Math.pow(a, b)`: Calcula `a` elevado a la potencia `b`.
- `Math.abs(x)`: Retorna el valor absoluto de `x`.
- `Math.round(x)`: Redondea `x` al entero más cercano.
- `Math.max(a, b)` y `Math.min(a, b)`: Obtienen el máximo o mínimo entre dos valores.
- Funciones trigonométricas como `Math.sin(x)`, `Math.cos(x)`, `Math.tan(x)`.

Ejercicio 24.1



Crea un método en Java, que devuelva el valor de la siguiente función matemática:

$$f(x) = \sum_{i=1}^x \max(e^{-i}, \sin(i))$$

Ver solución en la página 323.

24.2. La clase Arrays

La clase `Arrays` proporciona métodos estáticos para trabajar eficientemente con arreglos en Java. Permite ordenarlos, buscarlos, copiarlos, rellenarlos, comparar su contenido y convertirlos en cadenas legibles, entre otras operaciones. Utilizar `Arrays` facilita tareas comunes y reduce errores al manipular datos en forma de array.

Algunos métodos destacados son:

- `Arrays.sort(array)`: Ordena el array de menor a mayor.
- `Arrays.binarySearch(array, valor)`: Busca un valor en un array ordenado y devuelve su posición.
- `Arrays.copyOf(array, nuevaLongitud)`: Crea una copia del array con la longitud indicada.
- `Arrays.fill(array, valor)`: Rellena todo el array con el valor especificado.
- `Arrays.equals(array1, array2)`: Compara si dos arrays son iguales.
- `Arrays.toString(array)`: Devuelve una representación en texto del array.

```

1 int[] datos = {5, 2, 8, 1};
2 Arrays.sort(datos); // Ordena el array
3 int pos = Arrays.binarySearch(datos, 8); // Busca el valor 8

```

Ejemplo 24.1: Ejemplo de uso de la clase Arrays

Ejercicio 24.2



Crea una función generar bingo, que devuelve 20 números aleatorios entre 1 y 100. Los números deben ser únicos y estar en un array ordenado. Ver solución en la página 323.

24.3. La clase Collections

La clase `Collections` es fundamental para trabajar con las colecciones del paquete `java.util`, como listas, conjuntos y mapas. Proporciona métodos estáticos que permiten realizar operaciones comunes de manera eficiente y segura, como ordenar, buscar, invertir, mezclar, sincronizar y crear colecciones inmutables.

Algunas utilidades destacadas de `Collections` son:

- `Collections.sort(lista)`: Ordena una lista según el orden natural de sus elementos o usando un comparador.
- `Collections.shuffle(lista)`: Mezcla aleatoriamente los elementos de una lista.
- `Collections.reverse(lista)`: Invierte el orden de los elementos de una lista.
- `Collections.max(coleccion)` y `Collections.min(coleccion)`: Obtienen el máximo o mínimo de una colección.
- `Collections.unmodifiableList(lista)`: Devuelve una vista inmutable de la lista.
- `Collections.synchronizedList(lista)`: Devuelve una lista sincronizada para uso seguro en entornos concurrentes.
- `Collections.frequency(coleccion, elemento)`: Cuenta cuántas veces aparece un elemento en una colección.

Estas utilidades permiten manipular colecciones de forma más sencilla y robusta, facilitando tareas como la gestión de datos, la concurrencia y la protección contra modificaciones accidentales.

```

1 List<String> nombres = Arrays.asList("Ana", "Luis", "Pedro");
2 Collections.sort(nombres); // Ordena la lista
3 Collections.shuffle(nombres); // Mezcla aleatoriamente

```

Ejemplo 24.2: Ejemplo de uso de la clase Collections

24.4. La clase Objects

La clase `Objects` es una utilidad esencial para trabajar con referencias de objetos en Java, especialmente en lo que respecta a la gestión de valores nulos y la comparación segura. Sus métodos estáticos ayudan a evitar errores comunes, como las excepciones `NullPointerException`, y facilitan la escritura de código más robusto y legible.

Algunas funciones importantes de `Objects` incluyen:

- `Objects.isNull(obj)` y `Objects.nonNull(obj)`: Verifican si una referencia es nula o no.
- `Objects.requireNonNull(obj)`: Lanza una excepción si el objeto es nulo, útil para validar argumentos.
- `Objects.requireNonNullElse(obj, default)`: Devuelve el objeto si no es nulo, o un valor por defecto si lo es.
- `Objects.equals(a, b)`: Compara dos objetos de forma segura, evitando errores si alguno es nulo.
- `Objects.hash(obj1, obj2, ...)`: Calcula el código hash de varios objetos, útil para implementar `hashCode()`.
- `Objects.toString(obj, default)`: Devuelve la representación en texto del objeto, o un valor por defecto si es nulo.

El uso de `Objects` mejora la seguridad y claridad del código, especialmente en métodos que reciben parámetros o manipulan datos que pueden ser nulos.

```

1 String nombre = null;
2 System.out.println(Objects.isNull(nombre)); // true
3 System.out.println(Objects.requireNonNullElse(nombre, "↩
  Desconocido")); // "Desconocido"
4
5 public class Persona {
6     private String nombre;
7     private int edad;
8
9     @Override

```

```

10 public boolean equals(Object obj) {
11     if (this == obj) return true;
12     if (!(obj instanceof Persona)) return false;
13     Persona otra = (Persona) obj;
14     return Objects.equals(nombre, otra.nombre) && edad == otra.↵
    edad;
15 }
16
17 @Override
18 public int hashCode() {
19     return Objects.hash(nombre, edad);
20 }
21 }

```

Ejemplo 24.3: Ejemplo de uso de la clase *Objects*

24.5. La clase *Random*, *ThreadLocalRandom* y *SecureRandom*

La clase *Random* es una utilidad fundamental para la generación de números aleatorios en Java. Permite obtener valores enteros, decimales, booleanos y otros tipos, facilitando tareas como simulaciones, juegos, pruebas y selección aleatoria de datos. Por ejemplo, el método `nextInt(n)` devuelve un entero entre 0 (inclusivo) y `n` (exclusivo), mientras que `nextDouble()` genera un valor decimal entre 0.0 y 1.0.

Para aplicaciones concurrentes, *ThreadLocalRandom* ofrece mayor rendimiento y seguridad al evitar la contención entre hilos, siendo ideal para entornos multihilo. Su uso es similar al de *Random*, pero se accede mediante `ThreadLocalRandom.current()`.

Ejemplo de uso de ambas clases:

```

1 Random rnd = new Random();
2 int valor = rnd.nextInt(100); // Número entre 0 y 99
3
4 int otroValor = ThreadLocalRandom.current().nextInt(100); // ↵
    Número entre 0 y 99 en entorno multihilo

```

Ejemplo 24.4: Uso de *Random* y *ThreadLocalRandom*

Por otro lado, *SecureRandom* está diseñada para aplicaciones que requieren mayor seguridad, como generación de contraseñas, claves criptográficas o tokens. Utiliza fuentes de entropía más robustas y algoritmos criptográficos para asegurar la imprevisibilidad de los valores generados.

Ejemplo de uso de ambas clases:

```

1 Random rnd = new Random();
2 int valor = rnd.nextInt(100); // Número entre 0 y 99
3
4 SecureRandom srnd = new SecureRandom();
5 byte[] bytes = new byte[16];
6 srnd.nextBytes(bytes); // Array de bytes aleatorios seguro

```

Ejemplo 24.5: Uso de Random y SecureRandom

Ejercicio 24.3

Crea una función en Java que calcule la media de n llamadas a otra función, la cual devuelve 4 si la suma del cuadrado de dos números aleatorios (distintos, comprendidos entre 0 y 1) es menor o igual a 1, si no, devuelve 0.

¿Qué resultado obtienes para un valor de n grande? Ver solución en la página 323.

24.6. Las clases BigInteger, y BigDecimal

Las clases `BigInteger` y `BigDecimal` permiten realizar cálculos numéricos con precisión arbitraria, superando las limitaciones de los tipos primitivos (`int`, `long`, `double`) en Java. Son esenciales cuando se requiere trabajar con números muy grandes, operaciones matemáticas exactas o cálculos financieros donde los errores de redondeo pueden ser críticos.

`BigInteger` representa enteros de cualquier tamaño, permitiendo operaciones como suma, resta, multiplicación, división, potenciación y cálculo de módulos, sin riesgo de desbordamiento. Por ejemplo, para calcular la secuencia de Fibonacci:

```

1 BigInteger a = BigInteger.ONE;
2 BigInteger b = BigInteger.ZERO;
3 for (int i = 2; i <= 1000; i++) {
4     BigInteger temp = a;
5     a = a.add(b);
6     b = temp;
7     System.out.println(a); // Imprime la secuencia de Fibonacci
8 }

```

Los principales métodos de `BigInteger` incluyen:

- `add(BigInteger val)`: Suma el valor especificado.
- `subtract(BigInteger val)`: Resta el valor especificado.
- `multiply(BigInteger val)`: Multiplica por el valor especificado.

- `divide(BigInteger val)`: Divide por el valor especificado.
- `mod(BigInteger val)`: Calcula el módulo respecto al valor especificado.
- `pow(int exponent)`: Eleva a la potencia indicada.
- `compareTo(BigInteger val)`: Compara dos valores.
- `gcd(BigInteger val)`: Calcula el máximo común divisor.
- `isProbablePrime(int certainty)`: Verifica si el número es probablemente primo.
- `toString()`: Convierte el valor a cadena.

`BigDecimal` se utiliza para representar números decimales con precisión exacta, ideal para aplicaciones financieras, científicas o cualquier contexto donde los errores de coma flotante sean inaceptables. Permite controlar el redondeo y la escala de los resultados:

```
1 BigDecimal precio = new BigDecimal("19.99");
2 BigDecimal cantidad = new BigDecimal("3");
3 BigDecimal total = precio.multiply(cantidad);
4 System.out.println(total); // 59.97
```

Los principales métodos de `BigDecimal` incluyen:

- `add(BigDecimal val)`: Suma el valor especificado.
- `subtract(BigDecimal val)`: Resta el valor especificado.
- `multiply(BigDecimal val)`: Multiplica por el valor especificado.
- `divide(BigDecimal val, MathContext mc)`: Divide por el valor especificado, controlando precisión y redondeo.
- `setScale(int newScale, RoundingMode mode)`: Ajusta la escala (número de decimales) y el modo de redondeo.
- `compareTo(BigDecimal val)`: Compara dos valores decimales.
- `abs()`: Devuelve el valor absoluto.
- `pow(int n)`: Eleva a la potencia indicada.
- `toPlainString()`: Convierte el valor a cadena sin notación científica.
- `stripTrailingZeros()`: Elimina ceros innecesarios al final de la parte decimal.

Ambas clases son inmutables y ofrecen métodos para comparar, convertir y formatear valores, así como para realizar operaciones matemáticas avanzadas. El uso de `BigInteger` y `BigDecimal` garantiza precisión y seguridad en cálculos que exceden las capacidades de los tipos numéricos estándar.

Ejercicio 24.4



Implementa un programa que calcule el factorial de un número grande utilizando `BigInteger`, y otro que lo haga usando la primitiva `long`.
¿Difiere el resultado? *Ver solución en la página 324.*

24.7. Clases para fechas y horas: `LocalDate`, `LocalTime`, `LocalDateTime`, `Duration`, `Period`

Las clases `LocalDate`, `LocalTime`, `LocalDateTime`, `Duration` y `Period` forman parte del paquete `java.time`, introducido en Java 8 para facilitar el manejo de fechas y horas de manera clara, segura y sin los problemas de la antigua API (`Date`, `Calendar`). Estas clases son inmutables y thread-safe, lo que evita errores en aplicaciones concurrentes.

- **`LocalDate`**: Representa una fecha sin hora (año, mes, día).
- **`LocalTime`**: Representa una hora sin fecha (hora, minuto, segundo).
- **`LocalDateTime`**: Combina fecha y hora.
- **`Duration`**: Modela una cantidad de tiempo en horas, minutos, segundos, útil para medir intervalos temporales.
- **`Period`**: Modela una cantidad de tiempo en años, meses y días, útil para calcular diferencias entre fechas.

Estas clases permiten realizar operaciones como sumar o restar días, comparar fechas, calcular diferencias, formatear y analizar cadenas de texto, y trabajar con zonas horarias mediante `ZonedDateTime`. Por ejemplo, para calcular cuántos días faltan para una fecha específica, se utiliza `Period.between`. Para medir la duración entre dos instantes, se emplea `Duration.between`.

Ejemplo de uso de `LocalDate`, `Period` y `Duration`:


```

1 LocalDate hoy = LocalDate.now();
2 LocalDate cumple = LocalDate.of(2025, 8, 18);
3 Period periodo = Period.between(hoy, cumple);
4 System.out.println(periodo.getDays()); // Días hasta el ↩
    cumpleaños
5
6 LocalTime inicio = LocalTime.of(9, 0);
7 LocalTime fin = LocalTime.of(17, 30);
8 Duration jornada = Duration.between(inicio, fin);
9 System.out.println(jornada.toHours()); // Horas trabajadas

```

El uso de estas clases mejora la precisión y legibilidad del código al trabajar con fechas y horas, evitando errores comunes y facilitando el desarrollo de aplicaciones que requieren cálculos temporales.

24.8. La clase UUID

La clase `UUID` (Universally Unique Identifier) permite generar identificadores únicos de 128 bits, útiles para distinguir objetos, registros o entidades en sistemas distribuidos, bases de datos y aplicaciones donde se requiere unicidad global. Los UUID se generan de forma aleatoria o basados en información como la hora y la dirección MAC, garantizando que no se repitan incluso en diferentes sistemas.

Algunos métodos importantes de la clase `UUID` son:

- `UUID.randomUUID()`: Genera un UUID aleatorio.
- `UUID.fromString(String uuid)`: Crea un UUID a partir de su representación textual.
- `uuid.toString()`: Devuelve el UUID en formato estándar de texto.
- `uuid.version()`: Indica la versión del UUID (por ejemplo, aleatorio, basado en tiempo, etc.).

El uso de `UUID` es común en la generación de claves primarias, tokens de sesión, nombres de archivos únicos y cualquier contexto donde la unicidad sea esencial, sin depender de un contador centralizado.

24.9. La clase Base64

La clase `Base64`, disponible en `java.util`, es una utilidad para la codificación y decodificación de datos en el formato Base64, ampliamente utilizado para transmitir información binaria como texto legible (por ejemplo, imágenes, archivos o credenciales en protocolos HTTP). Base64 convierte datos binarios en

una cadena de caracteres ASCII, facilitando su almacenamiento y transferencia en sistemas que solo admiten texto.

Las principales funcionalidades de la clase incluyen:

- `Base64.getEncoder()`: Obtiene un codificador para transformar datos binarios en texto Base64.
- `Base64.getDecoder()`: Obtiene un decodificador para revertir el proceso y recuperar los datos originales.
- `Base64.getUrlEncoder()`, `Base64.getMimeEncoder()`: Codificadores especializados para URLs y mensajes MIME.

El uso de `Base64` es común en el intercambio de datos entre sistemas, almacenamiento seguro de información y transmisión de archivos en APIs web. Por ejemplo, para codificar una cadena y luego decodificarla:

```
1 String texto = "Hola mundo";  
2 String codificado = Base64.getEncoder().encodeToString(texto.getBytes());  
3 byte[] decodificado = Base64.getDecoder().decode(codificado);  
4 System.out.println(new String(decodificado)); // "Hola mundo"
```

Ejemplo 24.6: Codificación y decodificación con Base64

Esta clase simplifica el manejo de datos binarios en aplicaciones Java, asegurando compatibilidad y seguridad en la transferencia de información.

Soluciones de los ejercicios

Solución del ejercicio 24.1



```

1 // Fragmento: incluir dentro de una clase
2 public static double f(int x) {
3     double suma = 0;
4     for (int i = 1; i <= x; i++) {
5         suma += Math.max(Math.exp(-i), Math.sin(i));
6     }
7     return suma;
8 }
9
10 // System.out.printf("f(10) = %.4f%n", f(10)); // Suma de ↵
    max(e^-i, sin(i)) para i=1..10

```

Solución del ejercicio 24.2



```

1 // Fragmento: incluir dentro de una clase
2 import java.util.ArrayList;
3 import java.util.Arrays;
4 import java.util.Collections;
5 import java.util.List;
6
7 public static int[] generarBingo() {
8     List<Integer> todos = new ArrayList<>();
9     for (int i = 1; i <= 100; i++) {
10         todos.add(i);
11     }
12     Collections.shuffle(todos);
13     int[] resultado = new int[20];
14     for (int i = 0; i < 20; i++) {
15         resultado[i] = todos.get(i);
16     }
17     Arrays.sort(resultado);
18     return resultado;
19 }
20
21 // System.out.println(Arrays.toString(generarBingo()));

```

Solución del ejercicio 24.3



Este ejercicio implementa el *método de Monte Carlo* para estimar π . La función interna comprueba si un punto aleatorio (x, y) cae dentro del

cuarto de círculo de radio 1. La proporción de puntos dentro multiplicada por 4 converge a $\pi \approx 3,14159$ conforme n crece.

```

1 // Fragmento: incluir dentro de una clase
2 // Estimación del número mediante el método de Monte Carlo →
3
4 // La función devuelve 4 cuando el punto (x,y) cae dentro →
   del círculo unidad,
5 // y 0 en caso contrario. La media de n llamadas converge a →
6
7 import java.util.Random;
8
9 public static double mediaMonteCarlo(int n) {
10     Random rnd = new Random();
11     double suma = 0;
12     for (int i = 0; i < n; i++) {
13         double x = rnd.nextDouble();
14         double y = rnd.nextDouble();
15         suma += (x * x + y * y <= 1.0) ? 4.0 : 0.0;
16     }
17     return suma / n;
18 }
19
20 // Para n grande, el resultado converge a 3.14159
21 // System.out.println(mediaMonteCarlo(1_000_000)); // →
   3.14159

```

Solución del ejercicio 24.4



Sí difieren: para valores como $n = 25$, **long** desborda y devuelve un resultado incorrecto (negativo o truncado), mientras que **BigInteger** calcula el valor exacto.

```

1 // Fragmento: incluir dentro de una clase
2 import java.math.BigInteger;
3
4 public static BigInteger factorialBig(int n) {
5     BigInteger resultado = BigInteger.ONE;
6     for (int i = 2; i <= n; i++) {
7         resultado = resultado.multiply(BigInteger.valueOf(i));
8     }
9     return resultado;
10 }
11
12 public static long factorialLong(int n) {
13     long resultado = 1;

```

```
14  for (int i = 2; i <= n; i++) {
15      resultado *= i;
16  }
17  return resultado;
18  }
19
20  // Comparación para n = 25:
21  // BigInteger: 15511210043330985984000000 (exacto)
22  // long:      7034535277573963776      (incorrecto, ↪
    desbordamiento)
23  // int n = 25;
24  // System.out.println("BigInteger: " + factorialBig(n));
25  // System.out.println("long:      " + factorialLong(n));
```




Ejercicios

Ejercicios de refuerzo

En este capítulo se presentan ejercicios de refuerzo por tema, con sus soluciones de referencia. Están diseñados para practicar conceptos específicos de forma progresiva.

25.1. Variables y operadores

25.1.1. Operaciones aritméticas

Ejercicio 25.1



Escribe un programa que declare dos variables enteras, les asigne los valores 8 y 12, y luego imprima la suma, la diferencia, el producto, el cociente y el resto de la división.

Una posible solución:

```
1 public class OperacionesBasicas {
2     public static void main(String[] args) {
3         int num1 = 8;
4         int num2 = 12;
5         System.out.println("Suma: " + (num1 + num2));
6         System.out.println("Diferencia: " + (num1 - num2));
7         System.out.println("Producto: " + (num1 * num2));
8         System.out.println("Cociente: " + (num1 / num2));
9         System.out.println("Resto: " + (num1 % num2));
10    }
11 }
```

Ejercicio 25.2



Escribe un programa que declare dos variables de tipo `double` con valores 8.5 y 2.3. Calcula la suma, la diferencia, el producto y el cociente, y muestra cada resultado por pantalla.

Una posible solución:

```

1 public class OperacionesDouble {
2     public static void main(String[] args) {
3         double x = 8.5;
4         double y = 2.3;
5         System.out.println("Suma: " + (x + y));
6         System.out.println("Diferencia: " + (x - y));
7         System.out.println("Producto: " + (x * y));
8         System.out.println("Cociente: " + (x / y));
9     }
10 }

```

Ejercicio 25.3



Escribe un programa que intercambie los valores de dos variables enteras (5 y 10) y muestre ambos valores antes y después del intercambio.

Una posible solución:

```

1 public class IntercambioValores {
2     public static void main(String[] args) {
3         int a = 5;
4         int b = 10;
5         System.out.println("Antes: a = " + a + ", b = " + b);
6         int temp = a;
7         a = b;
8         b = temp;
9         System.out.println("Después: a = " + a + ", b = " + b);
10    }
11 }

```

25.1.2. Operadores de asignación compuesta

Ejercicio 25.4



Escribe un programa que demuestre el uso de los operadores de asignación compuesta +=, -=, *=, /= y %= con una variable entera.

Una posible solución:

```

1 public class AsignacionCompuesta {
2     public static void main(String[] args) {
3         int numero = 20;
4         numero += 5;    System.out.println("+=5: " + numero);
5         numero -= 7;    System.out.println("-=7: " + numero);
6         numero *= 3;    System.out.println("*=3: " + numero);
7         numero /= 6;    System.out.println("/=6: " + numero);

```

```

8      numero %= 5; System.out.println("%=5: " + numero);
9  }
10 }
```

25.1.3. Ejercicios avanzados

Ejercicio 25.5



Escribe un programa que calcule el área de un círculo con radio 4.5, usando la fórmula $\text{área} = \pi \cdot r^2$.

Una posible solución:

```

1 public class AreaCirculo {
2     public static void main(String[] args) {
3         double radio = 4.5;
4         double area = Math.PI * radio * radio;
5         System.out.println("Área del círculo: " + area);
6     }
7 }
```

Ejercicio 25.6



Escribe un programa que convierta una temperatura de 25 grados Celsius a Fahrenheit usando la fórmula $F = C \times 9/5 + 32$.

Una posible solución:

```

1 public class CelsiusAFahrenheit {
2     public static void main(String[] args) {
3         double celsius = 25.0;
4         double fahrenheit = celsius * 9.0 / 5.0 + 32;
5         System.out.println("Celsius: " + celsius + ", Fahrenheit: " +
6         + fahrenheit);
7     }
8 }
```

25.2. Funciones

25.2.1. Funciones sin parámetros

Ejercicio 25.7



Escribe un programa que defina una función llamada `imprimirTablaCinco` que imprima la tabla de multiplicar del 5 (del 1 al 10). Llama a esta función desde `main`.

Una posible solución:

```
1 public class TablaCinco {
2     public static void main(String[] args) {
3         imprimirTablaCinco();
4     }
5
6     public static void imprimirTablaCinco() {
7         for (int i = 1; i <= 10; i++) {
8             System.out.println("5 x " + i + " = " + (5 * i));
9         }
10    }
11 }
```

25.2.2. Funciones con parámetros y valor de retorno

Ejercicio 25.8



Escribe una función `esPrimo` que reciba un número entero y devuelva `true` si es primo y `false` en caso contrario. Usa la función para imprimir todos los números primos del 1 al 50.

Una posible solución:

```
1 public class NumerosPrimos {
2     public static void main(String[] args) {
3         for (int i = 2; i <= 50; i++) {
4             if (esPrimo(i)) {
5                 System.out.println(i);
6             }
7         }
8     }
9
10    public static boolean esPrimo(int n) {
11        if (n < 2) return false;
12        for (int i = 2; i <= Math.sqrt(n); i++) {
13            if (n % i == 0) return false;
14        }
15        return true;
16    }
17 }
```

```

14     }
15     return true;
16 }
17 }

```

25.2.3. Recursividad

Ejercicio 25.9



Escribe una función recursiva `fibonacci` que calcule el n -ésimo número de la serie de Fibonacci. Imprime los primeros 10 números de la serie.

Una posible solución:

```

1 public class FibonacciRecursivo {
2     public static void main(String[] args) {
3         for (int i = 0; i < 10; i++) {
4             System.out.println(fibonacci(i));
5         }
6     }
7
8     public static int fibonacci(int n) {
9         if (n <= 1) return n;
10        return fibonacci(n - 1) + fibonacci(n - 2);
11    }
12 }

```

25.3. Estructuras de selección

25.3.1. Sentencia if-else

Ejercicio 25.10



Escribe un programa que determine si un número almacenado en una variable es positivo, negativo o cero.

Una posible solución:

```

1 public class SignoNumero {
2     public static void main(String[] args) {
3         int numero = -5;
4         if (numero > 0) {
5             System.out.println("El número es positivo.");
6         } else if (numero < 0) {
7             System.out.println("El número es negativo.");
8         } else {

```

```
9      System.out.println("El número es cero.");
10    }
11  }
12 }
```

Ejercicio 25.11



Escribe un programa que compare dos números almacenados en variables y muestre cuál es mayor, o si son iguales.

Una posible solución:

```
1 public class ComparacionNumeros {
2     public static void main(String[] args) {
3         int num1 = 10;
4         int num2 = 7;
5         if (num1 > num2) {
6             System.out.println("El primer número es mayor.");
7         } else if (num2 > num1) {
8             System.out.println("El segundo número es mayor.");
9         } else {
10            System.out.println("Ambos números son iguales.");
11        }
12    }
13 }
```

25.3.2. Sentencia switch-case

Ejercicio 25.12



Escribe un programa que reciba un número del 1 al 7 y muestre el nombre del día de la semana correspondiente usando switch-case.

Una posible solución:

```
1 public class DiaSemana {
2     public static void main(String[] args) {
3         int dia = 3;
4         switch (dia) {
5             case 1: System.out.println("Lunes"); break;
6             case 2: System.out.println("Martes"); break;
7             case 3: System.out.println("Miércoles"); break;
8             case 4: System.out.println("Jueves"); break;
9             case 5: System.out.println("Viernes"); break;
10            case 6: System.out.println("Sábado"); break;
11            case 7: System.out.println("Domingo"); break;
12            default: System.out.println("Número de día inválido.");
13        }
14    }
15 }
```

```

14 }
15 }

```

Ejercicio 25.13



Escribe el mismo programa usando la expresión `switch` con operador flecha (`->`) para devolver el nombre del día de la semana.

Una posible solución:

```

1 public class DiaSemanaFlecha {
2     public static void main(String[] args) {
3         int dia = 3;
4         String nombreDia = switch (dia) {
5             case 1 -> "Lunes";
6             case 2 -> "Martes";
7             case 3 -> "Miércoles";
8             case 4 -> "Jueves";
9             case 5 -> "Viernes";
10            case 6 -> "Sábado";
11            case 7 -> "Domingo";
12            default -> "Número de día inválido.";
13        };
14        System.out.println(nombreDia);
15    }
16 }

```

25.3.3. Operador ternario

Ejercicio 25.14



Escribe un programa que determine cuál de dos números es mayor usando el operador ternario.

Una posible solución:

```

1 public class MayorTernario {
2     public static void main(String[] args) {
3         int a = 15;
4         int b = 20;
5         int mayor = (a > b) ? a : b;
6         System.out.println("El número mayor es: " + mayor);
7     }
8 }

```

Ejercicio 25.15

Escribe un programa que determine si un número es positivo, negativo o cero usando el operador ternario anidado.

Una posible solución:

```

1 public class SignoTernario {
2     public static void main(String[] args) {
3         int numero = 0;
4         String resultado = (numero > 0) ? "positivo" : (numero < 0) ↩
? "negativo" : "cero";
5         System.out.println("El número es " + resultado + ".");
6     }
7 }

```

25.4. Estructuras de repetición

25.4.1. Bucle for

Ejercicio 25.16

Escribe un programa que calcule la suma de los números pares entre 1 y 100 usando un bucle for.

Una posible solución:

```

1 int suma = 0;
2 for (int i = 2; i <= 100; i += 2) {
3     suma += i;
4 }
5 System.out.println(suma);

```

Ejercicio 25.17

Escribe un programa que calcule el factorial de un número entero positivo usando un bucle for.

Una posible solución:

```

1 int numero = 5;
2 int factorial = 1;
3 for (int i = 1; i <= numero; i++) {
4     factorial *= i;
5 }
6 System.out.println(factorial);

```


Ejercicio 25.18

Escribe un programa que imprima el siguiente triángulo de números usando bucles anidados:

```
1
12
123
1234
12345
```

Una posible solución:

```
1 for (int i = 1; i <= 5; i++) {
2   for (int j = 1; j <= i; j++) {
3     System.out.print(j);
4   }
5   System.out.println();
6 }
```

25.4.2. Bucle while**Ejercicio 25.19**

Escribe un programa que imprima los primeros 10 números de la serie Fibonacci usando un bucle **while**.

Una posible solución:

```
1 int n1 = 0, n2 = 1, contador = 0;
2 while (contador < 10) {
3   System.out.println(n1);
4   int suma = n1 + n2;
5   n1 = n2;
6   n2 = suma;
7   contador++;
8 }
```

Ejercicio 25.20

Escribe un programa que encuentre el primer múltiplo de 13 mayor que 100 usando un bucle **while**.

Una posible solución:

```
1 int numero = 101;
```

```
2 while (numero % 13 != 0) {  
3     numero++;  
4 }  
5 System.out.println(numero);
```

25.4.3. Sentencias break y continue

Ejercicio 25.21



Escribe un programa que busque el primer par de números (i, j) tal que $i + j = 10$, donde i va de 1 a 5 y j va de 1 a 10, usando **break** con etiqueta para salir de ambos bucles al encontrar la solución.

Una posible solución:

```
1 outer:  
2 for (int i = 1; i <= 5; i++) {  
3     for (int j = 1; j <= 10; j++) {  
4         if (i + j == 10) {  
5             System.out.println("i = " + i + ", j = " + j);  
6             break outer;  
7         }  
8     }  
9 }
```

25.5. Introducción a la POO

Ejercicio 25.22



Gestión de una biblioteca. Crea las siguientes clases:

- **Autor:** con atributos `nombre` y `apellidos` (ambos con valor por defecto "Desconocido/a" si se pasan cadenas vacías o nulas).
- **Libro:** con atributos `titulo`, `autor` (de tipo `Autor`), `anioPublicacion` e `isbn`.
- **Prestamo:** con atributos `libro` (de tipo `Libro`), `fechaPrestamo` y `fechaDevolucion`.
- **Biblioteca:** que gestione una lista de libros y préstamos. Debe permitir añadir y buscar libros, y registrar y listar préstamos activos.

Una posible solución:

```

1 public class Autor {
2     private String nombre;
3     private String apellidos;
4
5     public Autor(String nombre, String apellidos) {
6         this.nombre = (nombre == null || nombre.isEmpty()) ? "↩
Desconocido/a" : nombre;
7         this.apellidos = (apellidos == null || apellidos.isEmpty()) ↩
? "Desconocido/a" : apellidos;
8     }
9
10    @Override
11    public String toString() { return nombre + " " + apellidos; }
12 }

```

```

1 public class Libro {
2     private String titulo;
3     private Autor autor;
4     private int anioPublicacion;
5     private String isbn;
6
7     public Libro(String titulo, Autor autor, int anioPublicacion, ↩
String isbn) {
8         this.titulo = titulo;
9         this.autor = autor;
10        this.anioPublicacion = anioPublicacion;
11        this.isbn = isbn;
12    }
13
14    public String getIsbn() { return isbn; }
15
16    @Override
17    public String toString() {
18        return titulo + " (" + autor + ", " + anioPublicacion + ")";
19    }
20 }

```

```

1 import java.time.LocalDate;
2
3 public class Prestamo {
4     private Libro libro;
5     private LocalDate fechaPrestamo;
6     private LocalDate fechaDevolucion;
7
8     public Prestamo(Libro libro, LocalDate fechaPrestamo, ↩
LocalDate fechaDevolucion) {
9         this.libro = libro;
10        this.fechaPrestamo = fechaPrestamo;
11        this.fechaDevolucion = fechaDevolucion;
12    }

```

```

13
14 public boolean estaActivo() { return fechaDevolucion == null; ↵
15 }
16
17 @Override
18 public String toString() {
19     return libro + " - prestado: " + fechaPrestamo
20     + (estaActivo() ? " (activo)" : " - devuelto: " + ↵
21     fechaDevolucion);
22 }
23 }

```

```

1 import java.time.LocalDate;
2 import java.util.ArrayList;
3 import java.util.List;
4
5 public class Biblioteca {
6     private List<Libro> libros = new ArrayList<>();
7     private List<Prestamo> prestamos = new ArrayList<>();
8
9     public void añadirLibro(Libro libro) { libros.add(libro); }
10
11     public Libro buscarPorIsbn(String isbn) {
12         return libros.stream()
13             .filter(l -> l.getIsbn().equals(isbn))
14             .findFirst().orElse(null);
15     }
16
17     public void registrarPrestamo(Libro libro) {
18         prestamos.add(new Prestamo(libro, LocalDate.now(), null));
19     }
20
21     public void listarPrestamosActivos() {
22         prestamos.stream()
23             .filter(Prestamo::estaActivo)
24             .forEach(System.out::println);
25     }
26 }

```

Ejercicio 25.23



Implementa una clase `NumeroComplejo` que represente un número complejo con parte real e imaginaria. La clase debe incluir métodos para sumar, restar, multiplicar y calcular el módulo de dos números complejos.

Una posible solución:

```

1 public class NumeroComplejo {
2     private double real;

```

```

3  private double imaginario;
4
5  public NumeroComplejo(double real, double imaginario) {
6      this.real = real;
7      this.imaginario = imaginario;
8  }
9
10 public NumeroComplejo sumar(NumeroComplejo otro) {
11     return new NumeroComplejo(real + otro.real, imaginario + ↵
    otro.imaginario);
12 }
13
14 public NumeroComplejo restar(NumeroComplejo otro) {
15     return new NumeroComplejo(real - otro.real, imaginario - ↵
    otro.imaginario);
16 }
17
18 public NumeroComplejo multiplicar(NumeroComplejo otro) {
19     double r = real * otro.real - imaginario * otro.imaginario;
20     double i = real * otro.imaginario + imaginario * otro.real;
21     return new NumeroComplejo(r, i);
22 }
23
24 public double modulo() {
25     return Math.sqrt(real * real + imaginario * imaginario);
26 }
27
28 @Override
29 public String toString() {
30     return real + (imaginario >= 0 ? " + " : " - ") + Math.abs(↵
    imaginario) + "i";
31 }
32
33 public static void main(String[] args) {
34     NumeroComplejo a = new NumeroComplejo(3, 4);
35     NumeroComplejo b = new NumeroComplejo(1, -2);
36     System.out.println("a + b = " + a.sumar(b));
37     System.out.println("a - b = " + a.restar(b));
38     System.out.println("a * b = " + a.multiplicar(b));
39     System.out.println("|a| = " + a.modulo());
40 }
41 }

```

25.6. Estructuras de almacenamiento

Ejercicio 25.24



Crea un programa que gestione un array de 10 números enteros aleatorios (entre 1 y 100). El programa debe:

- Calcular e imprimir la suma y la media de los elementos.
- Encontrar e imprimir el valor máximo y el mínimo.
- Imprimir los elementos del array en orden inverso.

Una posible solución:

```

1 import java.util.Random;
2
3 public class ArrayAleatorio {
4     public static void main(String[] args) {
5         Random random = new Random();
6         int[] array = new int[10];
7         for (int i = 0; i < array.length; i++) {
8             array[i] = random.nextInt(100) + 1;
9         }
10        int suma = 0, max = array[0], min = array[0];
11        for (int n : array) {
12            suma += n;
13            if (n > max) max = n;
14            if (n < min) min = n;
15        }
16        System.out.println("Suma: " + suma);
17        System.out.println("Media: " + (double) suma / array.length)↵
18    ;
19        System.out.println("Máximo: " + max);
20        System.out.println("Mínimo: " + min);
21        System.out.print("Orden inverso: ");
22        for (int i = array.length - 1; i >= 0; i--) {
23            System.out.print(array[i] + " ");
24        }
25        System.out.println();
26    }
27 }
```

Ejercicio 25.25



Crea un programa que implemente una matriz de 5×5 con números aleatorios. El programa debe calcular la suma de cada fila, la suma de

cada columna, y la suma de la diagonal principal.

Una posible solución:

```

1 import java.util.Random;
2
3 public class Matriz5x5 {
4     public static void main(String[] args) {
5         Random random = new Random();
6         int[][] m = new int[5][5];
7         for (int i = 0; i < 5; i++)
8             for (int j = 0; j < 5; j++)
9                 m[i][j] = random.nextInt(100) + 1;
10
11         for (int i = 0; i < 5; i++) {
12             int sumaFila = 0;
13             for (int j = 0; j < 5; j++) sumaFila += m[i][j];
14             System.out.println("Suma fila " + i + ": " + sumaFila);
15         }
16         for (int j = 0; j < 5; j++) {
17             int sumaCol = 0;
18             for (int i = 0; i < 5; i++) sumaCol += m[i][j];
19             System.out.println("Suma columna " + j + ": " + sumaCol);
20         }
21         int sumaDiag = 0;
22         for (int i = 0; i < 5; i++) sumaDiag += m[i][i];
23         System.out.println("Suma diagonal principal: " + sumaDiag);
24     }
25 }

```

Ejercicio 25.26



Implementa una versión simplificada del algoritmo de ordenación por burbuja (*bubble sort*) para ordenar un array de enteros de menor a mayor.

Una posible solución:

```

1 public class BubbleSort {
2     public static void main(String[] args) {
3         int[] array = {64, 34, 25, 12, 22, 11, 90};
4         int n = array.length;
5         for (int i = 0; i < n - 1; i++) {
6             for (int j = 0; j < n - i - 1; j++) {
7                 if (array[j] > array[j + 1]) {
8                     int temp = array[j];
9                     array[j] = array[j + 1];
10                    array[j + 1] = temp;
11                }
12            }
13        }
14    }
15 }

```

```

12     }
13 }
14 for (int x : array) System.out.print(x + " ");
15 System.out.println();
16 }
17 }

```

25.7. Colecciones

Ejercicio 25.27



Implementa una lista de la compra usando `ArrayList<String>`. El programa debe permitir añadir artículos, eliminar artículos por nombre, marcar artículos como comprados y mostrar los artículos pendientes.

Una posible solución:

```

1 import java.util.ArrayList;
2 import java.util.List;
3 import java.util.Scanner;
4
5 public class ListaCompra {
6     public static void main(String[] args) {
7         List<String> pendientes = new ArrayList<>();
8         List<String> comprados = new ArrayList<>();
9         Scanner sc = new Scanner(System.in);
10        while (true) {
11            System.out.println("1. Añadir 2. Eliminar 3. Marcar ↩
comprado"
12                + " 4. Ver pendientes 0. Salir");
13            int op = Integer.parseInt(sc.nextLine());
14            if (op == 0) break;
15            System.out.print("Artículo: ");
16            String art = sc.nextLine();
17            if (op == 1) pendientes.add(art);
18            else if (op == 2) pendientes.remove(art);
19            else if (op == 3) {
20                if (pendientes.remove(art)) comprados.add(art);
21            } else if (op == 4) System.out.println("Pendientes: " + ↩
pendientes);
22        }
23    }
24 }

```


Ejercicio 25.28

Escribe un programa que reciba una lista de palabras con posibles duplicados y las muestre ordenadas alfabéticamente y sin duplicados, usando las colecciones de Java apropiadas.

Una posible solución:

```

1 import java.util.Arrays;
2 import java.util.List;
3 import java.util.TreeSet;
4
5 public class PalabrasSinDuplicados {
6     public static void main(String[] args) {
7         List<String> palabras = Arrays.asList(
8             "manzana", "pera", "manzana", "uva", "pera", "kiwi");
9         System.out.println(new TreeSet<>(palabras));
10    }
11 }

```

Ejercicio 25.29

Implementa una agenda telefónica usando `HashMap<String, String>` donde la clave sea el nombre y el valor sea el número de teléfono. El programa debe permitir añadir, buscar y eliminar contactos, y mostrar todos los contactos ordenados por nombre.

Una posible solución:

```

1 import java.util.HashMap;
2 import java.util.Map;
3 import java.util.Scanner;
4 import java.util.TreeMap;
5
6 public class AgendaTelefonica {
7     public static void main(String[] args) {
8         Map<String, String> agenda = new HashMap<>();
9         Scanner sc = new Scanner(System.in);
10        while (true) {
11            System.out.println("1. Añadir 2. Buscar 3. Eliminar"
12                + " 4. Listar 0. Salir");
13            int op = Integer.parseInt(sc.nextLine());
14            if (op == 0) break;
15            System.out.print("Nombre: ");
16            String nombre = sc.nextLine();
17            if (op == 1) {
18                System.out.print("Teléfono: ");
19                agenda.put(nombre, sc.nextLine());

```

```

20     } else if (op == 2) {
21         System.out.println(agenda.getDefault(nombre, "No ↩
encontrado"));
22     } else if (op == 3) {
23         agenda.remove(nombre);
24     } else if (op == 4) {
25         new TreeMap<>(agenda).forEach(
26             (k, v) -> System.out.println(k + ": " + v));
27     }
28 }
29 }
30 }

```

25.8. Ficheros

Ejercicio 25.30



Escribe un programa que lea un fichero de texto e imprima el número de palabras, líneas y caracteres que contiene.

Una posible solución:

```

1 import java.io.File;
2 import java.io.IOException;
3 import java.util.Scanner;
4
5 public class ContadorFichero {
6     public static void main(String[] args) throws IOException {
7         File fichero = new File("texto.txt");
8         int lineas = 0, palabras = 0, caracteres = 0;
9         try (Scanner sc = new Scanner(fichero)) {
10             while (sc.hasNextLine()) {
11                 String linea = sc.nextLine();
12                 lineas++;
13                 caracteres += linea.length();
14                 if (!linea.isBlank()) {
15                     palabras += linea.trim().split("\\s+").length;
16                 }
17             }
18         }
19         System.out.println("Líneas: " + lineas);
20         System.out.println("Palabras: " + palabras);
21         System.out.println("Caracteres: " + caracteres);
22     }
23 }

```

Ejercicio 25.31

Implementa una agenda de contactos persistente en un fichero de texto con el formato `nombre;telefono;email` (un contacto por línea). El programa debe permitir añadir contactos, buscar por nombre y listar todos los contactos.

Una posible solución:

```

1 import java.io.*;
2 import java.util.Scanner;
3
4 public class AgendaFichero {
5     private static final String FICHERO = "contactos.txt";
6
7     public static void main(String[] args) throws IOException {
8         Scanner sc = new Scanner(System.in);
9         while (true) {
10             System.out.println("1. Añadir 2. Buscar 3. Listar 0. Salir↵");
11             int opcion = Integer.parseInt(sc.nextLine());
12             if (opcion == 0) break;
13             if (opcion == 1) {
14                 System.out.print("Nombre: "); String nombre = sc.↵
15                 nextLine();
16                 System.out.print("Teléfono: "); String tel = sc.nextLine↵
17                 ();
18                 System.out.print("Email: "); String email = sc.nextLine↵
19                 ();
20                 try (PrintWriter pw = new PrintWriter(new FileWriter(↵
21                     FICHERO, true))) {
22                     pw.println(nombre + ";" + tel + ";" + email);
23                 }
24             } else if (opcion == 2) {
25                 System.out.print("Nombre a buscar: ");
26                 String buscar = sc.nextLine();
27                 try (Scanner fs = new Scanner(new File(FICHERO))) {
28                     while (fs.hasNextLine()) {
29                         String linea = fs.nextLine();
30                         if (linea.startsWith(buscar + ";"))
31                             System.out.println(linea);
32                     }
33                 }
34             } else if (opcion == 3) {
35                 try (Scanner fs = new Scanner(new File(FICHERO))) {
36                     while (fs.hasNextLine()) System.out.println(fs.↵
37                     nextLine());
38                 }
39             }
40         }
41     }
42 }

```

```

35     }
36 }
37 }

```

Ejercicio 25.32



Escribe un programa que registre temperaturas diarias en un fichero binario usando `DataOutputStream`. El programa debe luego leer el fichero y calcular la temperatura media, máxima y mínima del período registrado.

Una posible solución:

```

1  import java.io.*;
2  import java.util.Random;
3
4  public class TemperaturasBinario {
5      private static final String FICHERO = "temperaturas.dat";
6
7      public static void main(String[] args) throws IOException {
8          // Escribir temperaturas aleatorias
9          Random rand = new Random();
10         try (DataOutputStream dos = new DataOutputStream(
11             new FileOutputStream(FICHERO))) {
12             for (int i = 0; i < 7; i++) {
13                 dos.writeDouble(10 + rand.nextDouble() * 25);
14             }
15         }
16         // Leer y calcular estadísticas
17         double suma = 0, max = Double.MIN_VALUE, min = Double.↩
MAX_VALUE;
18         int count = 0;
19         try (DataInputStream dis = new DataInputStream(
20             new FileInputStream(FICHERO))) {
21             while (dis.available() > 0) {
22                 double temp = dis.readDouble();
23                 suma += temp;
24                 if (temp > max) max = temp;
25                 if (temp < min) min = temp;
26                 count++;
27             }
28         }
29         System.out.printf("Media: %.2f%n", suma / count);
30         System.out.printf("Máxima: %.2f%n", max);
31         System.out.printf("Mínima: %.2f%n", min);
32     }
33 }

```

25.9. POO avanzada

Ejercicio 25.33



Crea una jerarquía de figuras geométricas con una interfaz `Figura` que declare los métodos `area()` y `perimetro()`. Implementa las clases `Circulo`, `Rectangulo` y `TrianguloEquilatero`. Crea un programa que cree una lista con figuras de diferentes tipos y calcule el área total.

Una posible solución:

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 interface Figura {
5     double area();
6     double perimetro();
7 }
8
9 class Circulo implements Figura {
10     private double radio;
11     public Circulo(double radio) { this.radio = radio; }
12     public double area() { return Math.PI * radio * radio; }
13     public double perimetro() { return 2 * Math.PI * radio; }
14 }
15
16 class Rectangulo implements Figura {
17     private double ancho, alto;
18     public Rectangulo(double ancho, double alto) { this.ancho = ↵
19     ancho; this.alto = alto; }
20     public double area() { return ancho * alto; }
21     public double perimetro() { return 2 * (ancho + alto); }
22 }
23
24 class TrianguloEquilatero implements Figura {
25     private double lado;
26     public TrianguloEquilatero(double lado) { this.lado = lado; }
27     public double area() { return (Math.sqrt(3) / 4) * lado * lado ↵
28     ; }
29     public double perimetro() { return 3 * lado; }
30 }
31
32 public class Figuras {
33     public static void main(String[] args) {
34         List<Figura> figuras = new ArrayList<>();
35         figuras.add(new Circulo(5));
36         figuras.add(new Rectangulo(4, 6));
37         figuras.add(new TrianguloEquilatero(3));
38         double areaTotal = figuras.stream().mapToDouble(Figura::area ↵

```

```

    ).sum();
37     System.out.printf("Área total: %.2f%n", areaTotal);
38 }
39 }

```

Ejercicio 25.34



Implementa un enumerado `Mes` que represente los meses del año. Cada mes debe tener un atributo con el número de días que tiene (teniendo en cuenta los años bisiestos para febrero). El enumerado debe incluir un método `diasEnAnio(int anio)` que devuelva los días que tiene un mes en un año concreto.

Una posible solución:

```

1 public enum Mes {
2     ENERO(31), FEBRERO(28), MARZO(31), ABRIL(30), MAYO(31), JUNIO↵
3     (30),
4     JULIO(31), AGOSTO(31), SEPTIEMBRE(30), OCTUBRE(31), NOVIEMBRE↵
5     (30), DICIEMBRE(31);
6
7     private final int dias;
8
9     Mes(int dias) { this.dias = dias; }
10
11     public int diasEnAnio(int anio) {
12         if (this == FEBRERO) {
13             boolean bisiesto = (anio % 4 == 0 && anio % 100 != 0) || (↵
14             anio % 400 == 0);
15             return bisiesto ? 29 : 28;
16         }
17         return dias;
18     }
19
20     public static void main(String[] args) {
21         System.out.println("Febrero 2024: " + FEBRERO.diasEnAnio↵
22         (2024)); // bisiesto
23         System.out.println("Febrero 2023: " + FEBRERO.diasEnAnio↵
24         (2023)); // no bisiesto
25         System.out.println("Enero: " + ENERO.diasEnAnio(2024));
26     }
27 }

```

Ejercicios de Programación

En este capítulo se presentan ejercicios prácticos para aplicar los conceptos aprendidos en programación. Cada ejercicio está diseñado para reforzar el conocimiento y fomentar la práctica. Deberás aplicar todos los conocimientos adquiridos en los capítulos anteriores.

26.1. Criba de Eratóstenes

La criba de Eratóstenes es un algoritmo clásico para encontrar todos los números primos menores o iguales a un número dado n . El método consiste en ir eliminando los múltiplos de cada número primo comenzando desde el 2, de modo que al finalizar el proceso, los números que no han sido eliminados son los primos.

Ejercicio 26.1



Implementa la criba de Eratóstenes en Java. El programa debe solicitar al usuario un número n y mostrar todos los números primos menores o iguales a n .

Conocimientos requeridos: Estructuras de control, arreglos y algoritmos de búsqueda.

26.2. Balanceo de paréntesis

El balanceo de paréntesis es un problema común en la programación, que consiste en verificar si los paréntesis en una expresión están correctamente balanceados. Esto significa que cada paréntesis de apertura tiene un correspondiente paréntesis de cierre y que están en el orden correcto.

Ejercicio 26.2

Implementa un programa en Java que verifique si una expresión matemática tiene los paréntesis balanceados. El programa debe solicitar al usuario una expresión y mostrar si está balanceada o no.

Una segunda versión de este ejercicio consiste en comprobar el balanceo de otros tipos de paréntesis, como corchetes y llaves.

Ejercicio 26.3

Implementa un programa en Java que verifique si una expresión matemática tiene los paréntesis balanceados, incluyendo corchetes y llaves. El programa debe solicitar al usuario una expresión y mostrar si está balanceada o no.

Conocimientos requeridos: Estructuras de control, manejo de cadenas y pilas.

26.3. Pirateo de contraseñas MD5

El algoritmo de cifrado MD5 es famoso por su rapidez y por generar un *hash* de 128 bits a partir de una entrada de cualquier longitud. Sin embargo, se ha descubierto que es vulnerable a ataques de colisión, lo que significa que es posible encontrar dos entradas diferentes que produzcan el mismo *hash* MD5.

Para generar un *hash* MD5 en Java, puedes utilizar la clase ‘MessageDigest’ del paquete ‘java.security’. Aquí tienes un ejemplo de cómo hacerlo:

```

1 import java.security.MessageDigest;
2 import java.security.NoSuchAlgorithmException;
3 import java.util.Scanner;
4
5 public class MD5Example {
6     public static void main(String[] args) throws ↩
7         NoSuchAlgorithmException {
8         Scanner scanner = new Scanner(System.in);
9         System.out.print("Introduce la contraseña: ");
10        String input = scanner.nextLine();
11
12        MessageDigest md = MessageDigest.getInstance("MD5");
13        md.update(input.getBytes());
14        byte[] digest = md.digest();
15
16        StringBuilder sb = new StringBuilder();
17        for (byte b : digest) {
18            sb.append(String.format("%02x", b));
19        }
20    }
21 }

```



```

19 System.out.println("Hash MD5: " + sb.toString());
20 }
21 }

```

Ejemplo 26.1: Generar un hash MD5 en Java

En este caso, el *hash* en MD5 tiene 16 bytes. Para poder atacar este algoritmo se usan las conocidas como *rainbow tables*, que son tablas pre-computadas de *hashes* y sus correspondientes entradas. Estas tablas permiten realizar ataques de colisión de manera más eficiente, ya que evitan la necesidad de calcular el *hash* para cada posible entrada en tiempo real. Estas tablas se guardan en disco en formato binario, y en cada posición se almacena el valor de la contraseña que genera ese valor de *hash*. P.e.: en la posición 0 del fichero está la contraseña cuyo MD5 es 0x0, en la posición 1 aquella que genera el valor 0x1, etc.

Ejercicio 26.4



Crea dos programas.

El primero de ellos deberá generar la tabla *rainbow*, guardando después del espacio de *hashes* cuál fue la última contraseña que se usó para rellenar la tabla. Esto permitirá continuar rellenando la tabla en consecutivas ejecuciones del programa.

El segundo programa, recibirá como entrada un *hash* MD5 y deberá buscar en la tabla *rainbow* la contraseña correspondiente. Si la encuentra, la mostrará; si no, informará que no se encontró.

Consejo: Si cada contraseña puede tener hasta 8 caracteres, al buscar en la tabla la posición n leeremos 8 bytes desde la posición $(n - 1) * 8$.

Conocimientos requeridos: Estructuras de control, manejo de cadenas y ficheros de acceso aleatorio.

26.4. Agenda personal

Una agenda personal es una aplicación que permite gestionar contactos, incluyendo su nombre, número de teléfono y correo electrónico. El programa debe permitir al usuario agregar, eliminar y buscar contactos. Además, debe ofrecer la opción de listar todos los contactos almacenados.

Ejercicio 26.5



Implementa la agenda personal en Java, teniendo en cuenta las siguientes funcionalidades:

- El usuario podrá acceder a la agenda por consola, o usando una interfaz gráfica.
- La agenda deberá utilizar alguna técnica de persistencia de datos.
- Deberá disponer de una opción para importar, y exportar, contactos desde un fichero de texto, o un fichero XML.

Conocimientos requeridos: Clases avanzadas, manejo de ficheros, persistencia de datos, interfaces gráficas.

26.5. Juego de adivinanza de números

El juego de adivinanza de números es una aplicación que permite a un jugador adivinar un número secreto generado aleatoriamente por la computadora. El programa debe proporcionar pistas al jugador sobre si su suposición es demasiado alta o demasiado baja.

Ejercicio 26.6



Implementa el juego de adivinanza de números en Java, teniendo en cuenta las siguientes funcionalidades:

- El número secreto debe ser generado aleatoriamente en un rango específico (por ejemplo, 1-100).
- El jugador debe poder hacer múltiples intentos para adivinar el número.
- Después de cada intento, el programa debe indicar si la suposición es demasiado alta, demasiado baja o correcta.
- Cuando el jugador adivina el número, el programa debe mostrar un mensaje de felicitación y el número de intentos realizados.

Conocimientos requeridos: Estructuras de control, generación de números aleatorios, manejo de entradas y salidas.

26.6. Juego de ahorcado

El ahorcado es un juego clásico en el que el jugador debe adivinar una palabra secreta letra por letra. El jugador tiene un número limitado de intentos para adivinar la palabra antes de que se complete el dibujo del ahorcado.

Ejercicio 26.7



Implementa el juego de ahorcado en Java, teniendo en cuenta las siguientes funcionalidades:

- La palabra secreta debe ser elegida aleatoriamente de un fichero de texto que contenga una lista predefinida de palabras.
- El jugador debe poder hacer múltiples intentos para adivinar la palabra.
- Después de cada intento, el programa debe mostrar el estado actual de la palabra (letras adivinadas y letras faltantes).
- Cuando el jugador adivina la palabra, el programa debe mostrar un mensaje de felicitación y el número de intentos realizados.

Conocimientos requeridos: Estructuras de control, manejo de cadenas y ficheros de texto.

26.7. Calculadora IRPF

Los impuestos a las personas físicas (IRPF) son un tema importante en la programación financiera. Una calculadora IRPF permite a los usuarios calcular el importe de su impuesto sobre la renta en función de sus ingresos y deducciones.

En este caso, vamos a basarnos en un caso simplificado, ya que existen distintas deducciones, y cada comunidad autónoma puede tener sus propias reglas. Consideraremos que los tramos de IRPF son los siguientes:

- Hasta 12.450 €: 19 %
- De 12.450 € a 20.200 €: 24 %
- De 20.200 € a 35.200 €: 30 %
- De 35.200 € a 60.000 €: 37 %
- Más de 60.000 €: 45 %

Ejercicio 26.8



Crea un programa en Java que calcule el IRPF de un usuario en función de su salario. El programa debe solicitar al usuario que introduzca su salario bruto anual y, a continuación, calcular y mostrar el importe del IRPF a pagar según los tramos establecidos.

Además, mostrará un desglose de cómo se ha calculado el impuesto, indicando el tramo correspondiente y el porcentaje aplicado.

Por último, mostrará cuál es el salario neto del usuario, es decir, el salario bruto menos el IRPF, tanto su valor anual, como su valor para 12 y 14 pagas.

Recuerda que deberás usar precisión de 1 céntimo.

Conocimientos requeridos: Estructuras de control, manejo de entradas y salidas.

Índice de figuras

1.1	Orden de lectura recomendado.	15
10.1	Diagrama de clases: herencia de Vehiculo a Moto	97
13.1	Jerarquía de excepciones en Java	138
14.1	Jerarquía del Java Collections Framework	148
16.1	Diagrama de composición: Auto contiene un Motor	183
16.2	Polimorfismo: Perro y Gato sobrescriben hacerSonido()	185
16.3	Herencia e implementación de interfaces: Pato extiende Animal e implementa Volador y Nadador	193
22.1	Diagrama UML del patrón Abstract Factory	284
22.2	Diagrama UML del patrón Adapter	297
22.3	Diagrama UML del patrón Visitor	298
23.1	Ciclo de vida de un hilo en Java	300

Índice de cuadros

4.1	Mapeado de Resultados de Aprendizaje del módulo 0485 a los capítulos del libro.	26
10.1	Principales métodos de la clase <code>Object</code>	98
11.1	Principales métodos de aserción en JUnit	112
12.1	Principales métodos de la interfaz <code>Collection<E></code>	130
12.2	Principales métodos de la interfaz <code>Map<K, V></code>	131
15.1	Principales métodos de la clase <code>Scanner</code>	178
17.1	Comparativa de sistemas de gestión de bases de datos populares	204
17.2	Principales métodos de la clase <code>ResultSet</code> en JDBC	212
18.1	Elementos del lenguaje HQL	238
19.1	Componentes gráficos comunes en Swing	248
21.1	Principales interfaces funcionales de Java y su uso	275
21.2	Principales métodos de la clase <code>Optional<T></code>	276
21.3	Principales métodos de la clase <code>Stream<T></code>	278
23.1	Principales métodos de la clase <code>AtomicInteger</code>	308